

Обобщения по глави

Изследване на възможностите за приложение на изкуствения интелект в компютърни
игри

Александър Андреев

Юли 2026

Съдържание

Предговор	3
Въведение	3
1 Глава 1 – Основи на обучението с подкрепление	7
1.1 Как обучението с подкрепление се утвърди като самостоятелна дисциплина	7
1.2 Основни принципи – агент, действие, среда	8
1.3 Формулировка на MDP	9
1.4 Информационни множества – когато не можете да видите всичко	10
1.5 Динамично програмиране – при наличие на модел	11
1.6 Обучение с темпорална разлика – обучение без модел	11
1.7 DQN – Дълбоки Q-мрежи	12
1.8 PPO – оптимизация на стратегията с ограничение на близостта	14
1.9 Практическа проверка – собствени реализации срещу Stable-Baselines3	15
2 Глава 2 – Основи на теорията на игрите и CFR	19
2.1 От обучение с подкрепление с един агент към многоагентно стратегическо взаимодействие	19
2.2 Игри в разгърната форма и информационни множества	19
2.3 Теорема за минимакс	20
2.4 Равновесие на Наш	20
2.5 Напасване на съжалението – градивният елемент	22
2.6 Минимизиране на контрафактичното съжаление (CFR)	22
2.7 Математиката на покера	24
2.8 Кун покер – най-простото изпитателно поле	24
2.9 Експлоатируемост – мярка за отклонението от равновесието на Наш	25
2.10 Връзки с Глава 1 и бъдещи насоки	26
2.11 Емпирични визуализации	27
3 Глава 3 – Варианти на CFR и методи на Монте Карло	30
3.1 Защо обикновеният вариант на CFR се проваля при голям мащаб	30
3.2 Търсене с метод Монте Карло в дърво като вдъхновение	31
3.3 Марковски вериги и законът за големите числа	32
3.4 Ледюк покер – по-голяма еталонна игра	33
3.5 CFR+ – Трикът с нулирането на отрицателните съжаления	34
3.6 MCCFR – Извадка от дървото на играта	36
3.7 Компромисът между дисперсия и скорост	39
3.8 Кога MCCFR надделява – точката на пресичане	42
3.9 Връзки с Глава 2 и бъдещи насоки	44
4 Глава 4 – Абстракция на играта и мащабиране на игри с непълна информация	45
4.1 Защо е необходима абстракцията	45
4.2 Пътища към абстракцията	46
4.3 Видове оси на абстракцията	48
4.4 Разлика в експлоатируемостта	49
4.5 Разстояние на Васерщайн: Въведение	50
4.6 Критерият за сливане	51
4.7 Бюджетът на грешките	53
4.8 Избор на абстракция на практика	54
4.9 Решения по време на компилация	55

4.10	Корекция по време на изпълнение	57
4.11	Архитектура: план и корекции в реално време	58
4.12	Практическо валидиране	59
4.13	Аналози на дълбокото обучение с подкрепление	63
4.14	Връзки и препратки напред	63
5	Глава 5 – Невронни мрежи за игри с непълна информация	65
	Защо невронни мрежи	65
5.1	Основни положения	66
5.2	Архитектура и композиция	69
5.3	Невронни мрежи в CFR	76
5.4	Други невронни приложения	81
6	Глава 6 – Архитектури на игрови изкуствен интелект от край до край	84
6.1	Въведение	84
6.2	DeepStack (2017)	86
6.3	Libratus (2017/2018)	93
6.4	Pluribus (2019)	103
6.5	ReBeL (2020)	111
6.6	Student of Games / SoG (2023)	121
6.7	Синтез	132
7	Глава 7 – Моделиране на противника в игри с непълна информация	140
7.1	Защо е необходимо моделиране на противника	140
7.2	Байесовото ядро – убеждение, наблюдения и реакция	142
7.3	Виждане през ключалка – проблемът с частичната наблюдаемост	145
7.4	Три модела с единен интерфейс	146
7.5	Проблемът на състоятелността и решението в последователна форма	148
7.6	Когато моделът е уверен, но греши	151
7.7	От модела до печалбата – най-добрият отговор, таванът и самонанесената уязвимост	153
7.8	Адаптиране към промяна – нестационарни опоненти	155
7.9	Връзки и бъдещи насоки	157
8	Глава 8 – Безопасна експлоатация в игри с непълна информация	159
8.1	Защо безопасната експлоатация – другата половина на скалата	159
8.2	Експлоатацията като оптимизация с ограничения	161
8.3	Какво означава „безопасен“ – три определения с нарастваща степен на реализъм	163
8.4	Линейно програмиране в последователна форма и генериране на ограничения	165
8.5	Ограничен отговор по Наш и скокообразното превключване	166
8.6	Най-доброто равновесие в Кун – безопасно, но само малко по-печелившо (основният резултат)	169
8.7	Безопасност в реално време – решаване на под-игри (SES)	172
8.8	В мащаб – Кун работи; Ледюк се разпада глобално, но се задържа локално	174
8.9	Връзки и бъдещи насоки	178
9	Глава 9 – Многоагентно обучение с подкрепление: координация, конкуренция и комуникация	179
9.1	Защо многоагентното обучение с подкрепление е различен проблем	179
9.2	Марковски игри – мостът между глави 2–8 и MARL	181
9.3	Семейството от методи	182
9.4	Независимо обучение и начините, по които то се проваля	186

9.5	Централизирано обучение, децентрализирано изпълнение (CTDE)	189
9.6	PSRO – теория на игрите върху популация от стратегии	193
9.7	Научена комуникация (CommNet)	197
9.8	LOLA – моделиране на противника като учещ се агент	198
9.9	Честни бележки, ограничения и връзка със следващите глави	200
9.10	Основни изводи за синтеза на дисертацията	201
10	Глава 10 – Обучение на базата на популации и еволюционна теория на игрите	203
10.1	Защо популации – и защо структурата определя всичко	203
10.2	Семейството от методи	204
10.3	Репликаторна динамика – къде една популация може да се установи	205
10.4	„Пумпалът“ – транзитивна и циклична структура	208
10.5	PBT лига в стила на AlphaStar	211
10.6	Разнообразие – наистина ли популацията е разнообразна?	214
10.7	EGTA – оценка на популацията и неочаквано откритие	216
10.8	Честни бележки, ограничения и предаване на щафетата	218
10.9	Основни изводи за синтеза на дисертацията	219
11	Глава 11 – Формиране и засичане на коалиции	221
11.1	Защо появата на трети играч е повратна точка	221
11.2	Детекторът на коалиции – извличане на съюзи от ходовете	222
11.3	Принос по Шапли в състезателна игра	224
11.4	MAPPO с отчитане на коалициите – и кога всъщност расте коалиционният резултат	228
11.5	EGTA и въртящият се пумпал – SLS колело ли е, или стълба?	231
11.6	Бележки, ограничения и връзка със следващите глави	232
11.7	Основни изводи за синтеза на дисертацията	233
12	Глава 12 – Модели на последователности и агенти с голям езиков модел в стратегически среди	235
12.1	Защо да се разглежда обучението с подкрепление като прогнозиране на последователност	235
12.2	Капанът между късмета и умението	235
12.3	ARDT – обуславяне по гарантираната възвръщаемост	236
12.4	Какво всъщност прави обуславянето по възвръщаемостта в покера	237
12.5	Езикови модели като стратегически агенти	239
12.6	Експлоатация, адаптация и ограниченията на опростената игра	242
12.7	Основни изводи за синтеза на дисертацията	243

Предговор

Въведение

Изследователски контекст и значимост

Многоагентните системи, опериращи в условията на непълна информация (imperfect information), представляват една от най-предизвикателните и практически значими области на изследване в сферата на изкуствения интелект. В подобни системи автономните агенти трябва да вземат стратегически решения без пълно знание за състоянието на средата или намеренията на другите учас-

тници — постановка, която възниква естествено в киберсигурността (противоборстваща защита на мрежи), финансовите пазари (конкуриращи се алгоритмични трейдъри), координацията на автономни системи и разпознаването на измами.

Математическата рамка за анализ на подобни взаимодействия се предоставя от игрите в разгърната форма (extensive-form games) с непълна информация — клон на теорията на игрите, който моделира последователно вземане на решения, когато играчите разполагат с частна (скрита) информация. През изминалото десетилетие беше постигнат значителен напредък в изчисляването на равновесни стратегии за мащабни игри с непълна информация. Знакови системи в тази област са Libratus (Brown and Sandholm, 2017) и Pluribus (Brown and Sandholm, 2019), които победиха професионални човешки играчи съответно в покер за двама и шестима участници, както и по-обща архитектура като ReBeL (Brown et al., 2020) и Student of Games (Schmid et al., 2023), които унифицират подходите за решаване на игри както в среди с пълна, така и с непълна информация.

Въпреки този напредък, продължава да съществува едно критично ограничение: системите, представляващи водещия стандарт, изчисляват фиксирани равновесни стратегии, които не се адаптират спрямо специфичните поведенчески модели, демонстрирани от срещаните опоненти. В реалната практика противниците рядко играят оптимално и способността за откриване и експлоатация на систематични отклонения от равновесието — като същевременно се запазват гаранциите срещу бъдеща експлоатация (в отговор) — представлява фундаментален нерешен проблем. Този проблем, известен като безопасна експлоатация на противника (safe opponent exploitation), е изследван предимно в среди за двама играчи с нулева сума. Разширяването му към среди с множество участници, където е възможно динамично формиране и разпадане на коалиции, остава слабо изследвано в литературата.

Настоящият план за обучение е организиран като прогресивна програма от петнадесет обучителни стъпки, групирани в седем тематични етапа:

- **Основи** (Глави 1–2): Фундаменти от областта на обучението с подкрепление и теорията на игрите, необходими за цялата последваща работа.
- **Мащабиране на инструментариума** (Глави 3–4): Монте Карло варианти на CFR и техники за абстракция на игри, предназначени за мащабиране към по-големи инстанции.
- **Невронни методи за игри** (Глави 5–6): Невронни мрежови апроксимации на равновесни стратегии и архитектури за изкуствен интелект „от край до край“ в игри.
- **Моделиране и експлоатация на противника** (Глави 7–8): Изводи от поведенчески следи и алгоритми за безопасна експлоатация — същинското ядро на дисертацията.
- **Многоагентна динамика** (Глави 9–11): Многоагентно обучение с подкрепление (MARL), базирано на популации обучение (PBT) и динамика на коалициите в конкурентни среди.
- **Подходи, основани на данни** (Глави 12–13): Секвенционни модели и конвейери за поведенчески анализ, свързващи теорията с реални данни.
- **Интеграция** (Глави 14–15): Изграждане на рамка за оценяване и картографиране на изследователската граница (research frontier).

В рамките на плана за обучение игрите Кун покер и Ледюк холдем служат като основна тестова среда за имплементациите. Те са избрани умишлено поради своята педагогическа стойност: техните малки пространства на състоянията (съответно 12 и приблизително 936 информационни множества) и познатите аналитични равновесия позволяват точна верификация на всеки алгоритъм, докато структурата на непълна информация запазва стратегическата комплексност, която същността на дисертацията изисква. Работейки в рамките на напълно изучени и добре разбрани области, всяка стъпка се концентрира върху алгоритмичните концепции, вместо върху софтуерното инженерство, специфично за конкретната област, което максимизира обема на теоретичния материал, покрит в определеното време. В по-късните етапи планът за обучение валидира обобщимостта отвъд покера, като прилага разработените методи към матрични игри, Goofspiel, So Long Sucker и анонимизирани поведенчески данни от реалния свят.

Планираните приноси на дисертацията са формулирани в термини, независими от конкретната предметна област — нито един от тях не е специфичен изключително за покер или друга единична игра. Рамката за поведенческа адаптация (Принос 1) е насочена към произволни игри с непълна информация; Евристиките за многоагентна безопасна експлоатация (Принос 2) ще бъдат дефинирани върху общи N-играчески структури в разгърнатата форма; а Методологията за оценяване (Принос 3) е предназначена да бъде приложима във всякакви игрови среди.

Планът за обучение е съобразен с ключовите етапи от индивидуалния план към университета. След завършването на всеки тематичен етап, съответният материал ще бъде включен в Глава I на дисертацията — обхващаща анализ на водещия стандарт, както и формулиране на актуалността, целите, задачите и тезисите. По този начин обзорът на литературата и полагането на основите се натрупват постепенно чрез инкрементално изграждане, вместо да бъдат съставяни ретроспективно. Глава 15 изготвя подробен план на Глава I и план за публикации.

Изследователска цел и очаквани приноси

Основната цел на това докторантско изследване е да се разработи адаптивен AI агент за използване в различни компютърни игри. Агентът ще стартира от безопасна стратегия, но ще бъде способен да подобрява резултата си срещу субоптимални противници, като по този начин премахва ограниченията на предходни системи, които рядко адаптират поведението си динамично. Изследователската програма е структурирана около три планирани приноса:

Принос 1 — Рамка за поведенческа адаптация. Целта е да се разработи общ метод за извличане на изводи относно стратегиите на опонента и адаптиране към тях въз основа на наблюдавани последователности от действия в реално време, приложим към произволни игри с непълна информация. Този принос се стреми да преодолее празнината между изчисляването на статични равновесия и динамичната игра, съобразена с противника.

Принос 2 — Многоагентна безопасна експлоатация. Този принос ще изследва възможни и изчислително реализуеми евристични подходи — включително KL-регуляризирана експлоатация (ограничаваща експлоатиращата политика да остане близка до сигурна базова стратегия чрез на-

лагането на санкция за отклонение посредством дивергенция на Kullback-Leibler) и базови линии за запазване на равен дял — за безопасна експлоатация в малки N-играчески игри (разновидности на Кун и Ледюк за трима играчи), опитвайки се да разшири съществуващите резултати от случаи за двама играчи с нулева сума. Обхватът се ограничава до емпирично валидиране на практически евристики, а не изграждането на генерална теорема за безопасност за произволни среди с N-играчи.

Принос 3 — Методология за оценяване. Целта е да се проектира рамка, независима от конкретната среда, за измерване на адаптивността и устойчивостта на агента в различни игрови условия и противникови популации, с акцент върху идентифицирането на възможни режими на отказ в съществуващи методики на оценяване и демонстриране на ситуации, където стандартните метрики могат да предоставят подвеждащи оценки.

1 Глава 1 – Основи на обучението с подкрепление

1.1 Как обучението с подкрепление се утвърди като самостоятелна дисциплина

Обучението с подкрепление не възниква изведнъж като завършена дисциплина. Неговите корени се простират в множество области, които дълги десетилетия почти не са били свързани помежду си.

Най-ранната нишка идва от животинската психология в началото на XX век – „Законът за ефекта“ на Торндайк (1911)¹ предполага, че действията, последвани от удовлетворяващи резултати, стават по-вероятни. Това всъщност е идеята за сигнал за награда и началото на нишката на обучението чрез проба и грешка. Успоредна нишка преминава през теорията на оптималното управление през 50-те години на миналия век, където Белман разработва динамично програмиране за решаване на последователни проблеми на вземане на решение.² Той въвежда функцията на стойността и принципа на оптималност, които и двете са в ядрото на съвременното обучение с подкрепление. Трета, по-слабо обособена нишка са методите на темпоралната разлика: програмата за дама на Самюел (1959)³ се учи, като играе срещу себе си, и приближава оценката на всяка позиция към оценките на следващите позиции.

Тези нишки се сливат в края на 80-те години на миналия век. Сътън (1988)⁴, продължавайки докторската си работа от 1984 г., отделя обучението с темпорална разлика от задачата за управление и го представя като общ метод за прогнозиране. Q-обучението на Уоткинс (1989)⁵ окончателно обединява нишките на темпоралната разлика и на оптималното управление и дава на областта първия ясен безмоделен алгоритъм за управление; сходимостта му е доказана през 1992 г. Към момента, когато Сътън и Барто публикуват своя учебник през 1998 г. (актуализиран през 2018 г.),⁶ обучението с подкрепление вече се е утвърдило като разпознаваема дисциплина със собствена нотация, таксономия на проблемите и алгоритмичен инструментариум.

Революцията в дълбокото обучение достига до обучението с подкрепление през 2013–2015 г., когато DQN на DeepMind се научава да играе игри на Atari само по суровите пиксели на екрана.⁷ Тази демонстрация ясно показва, че алгоритмите за обучение с подкрепление, комбинирани с неврон-

¹Thorndike, E.L. (1911). *Animal Intelligence: Experimental Studies*. Macmillan.

²Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

³Samuel, A.L. (1959). "Some Studies in Machine Learning Using the Game of Checkers." *IBM Journal of Research and Development*, 3(3), 210-229.

⁴Sutton, R.S. (1988). "Learning to Predict by the Methods of Temporal Differences." *Machine Learning*, 3(1), 9-44. Докторска дисертация: Sutton, R.S. (1984). *Temporal Credit Assignment in Reinforcement Learning*. University of Massachusetts Amherst.

⁵Watkins, C.J.C.H. (1989). *Learning from Delayed Rewards*. Докторска дисертация, University of Cambridge. Сходимостта е доказана в Watkins, C.J.C.H. & Dayan, P. (1992). "Q-learning." *Machine Learning*, 8(3-4), 279-292.

⁶Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, второ издание. MIT Press. Гл. 1 "Introduction"; гл. 3 "Finite Markov Decision Processes"; гл. 4 "Dynamic Programming"; гл. 5 "Monte Carlo Methods"; гл. 6 "Temporal-Difference Learning"; §11.3 "The Deadly Triad"; гл. 13 "Policy Gradient Methods". <http://incompleteideas.net/book/the-book-2nd.html>

⁷Mnih, V. et al. (2013). "Playing Atari with Deep Reinforcement Learning." arXiv:1312.5602; Mnih, V. et al. (2015). "Human-level control through deep reinforcement learning." *Nature*, 518(7540), 529-533. DOI 10.1038/nature14236

ни апроксиматори на функции, могат да решават високоизмерни проблеми от реалния свят. PPO се появява през 2017 г. като практичен метод на градиента на стратегията⁸ и оттогава обучението с подкрепление се разширява в областите на роботиката, игровия изкуствен интелект, системите за препоръки и съгласуването на езиковите модели с човешките предпочитания (RLHF).

Обучението с подкрепление не е просто „машинно обучение с награди“, а самостоятелна рамка за последователно вземане на решения при несигурност, със собствени теоретични основи, произтичащи от теорията на управлението, психологията и статистиката.⁶

1.2 Основни принципи – агент, действие, среда

Цялата рамка на обучението с подкрепление се основава на прост цикъл. **Агентът** наблюдава текущото **състояние** на **средата**, избира **действие** и получава сигнал за **награда** заедно с ново **състояние**. След това процесът се повтаря. Целта на агента е да научи **стратегия** – съпоставяне между състояния и действия – която максимизира натрупаната награда във времето.

Няколко уточнения относно терминологията:

- **Състояние** (s): описание на средата в даден момент. В CartPole това включва позицията на количката, скоростта на количката, ъгъла на пръта и ъгловата скорост на пръта. В покера биха били раздадените карти и историята на залозите.
- **Действие** (a): възможните действия на агента. Те могат да бъдат дискретни (например бутане наляво или надясно) или непрекъснати (например прилагане на конкретен въртящ момент).
- **Награда** (r): скаларна величина за обратна връзка, която агентът получава след всяко действие. Наградите могат да бъдат редки (само в края на играта) или чести (на всеки времеви интервал).
- **Стратегия** (π): правилото, по което агентът избира действията си. Тя може да бъде детерминистична ($\pi(s) = a$) или стохастична, като $\pi(a|s)$ дава разпределение на вероятностите върху възможните действия.
- **Функция на стойността** ($V^\pi(s)$): очакваната натрупана награда от състояние s нататък, ако агентът следва стратегия π . Тя показва на агента колко добро е едно състояние в дългосрочен план, а не само непосредствено.
- **Коефициент на дисконтиране** (γ): число между 0 и 1, което определя доколко агентът отчита бъдещите награди спрямо непосредствените такива. Висока стойност на $\gamma = 0,99$ означава, че агентът е търпелив; ниска стойност на $\gamma = 0,5$ показва, че е късоглед.

Критичното предизвикателство при проектирането на системи за обучение с подкрепление е **компромисът изследване–експлоатация**. Агентът трябва да изследва непознати действия, за да открие потенциално по-добри стратегии, но същевременно и да експлоатира наличните си

⁸Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.

знания, за да събира награда. Прекомерното изследване води до загуба на време, а прекалената експлоатация може да доведе до заседналост в локален оптимум.

Съществуват два основни класа алгоритми. **Методите, базирани на стойност** (като DQN⁷) научават стойността на състоянията или двойките състояние-действие и извеждат стратегия въз основа на тези стойности. **Методите, базирани на стратегията** (като PPO⁸), научават самата стратегия директно.⁶ И двата подхода имат своите предимства; значителна част от областта на обучението с подкрепление е посветена на тяхното ефективно комбиниране.

Прочетете повече: OpenAI Spinning Up – „Part 1: Key Concepts in RL“
https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

1.3 Формулировка на MDP

Марковски процес на вземане на решения (Markov decision process, MDP) представлява формалната математическа рамка, върху която са изградени повечето алгоритми за обучение с подкрепление. Той включва пет основни компонента:

- S : множество от състояния
- A : множество от действия
- $P(s'|s, a)$: функция на преход – вероятност за преминаване в състояние s' при дадено състояние s и действие a
- $R(s, a, s')$: функция на наградата
- $\gamma \in [0, 1)$: коефициент на дисконтиране

Частта „Марков“ означава, че бъдещето зависи единствено от текущото **състояние**, а не от историята на пътя до него. Това е силно предположение – и такова, което не е валидно в много интересни ситуации (като **покер**, където историята на залозите има значение). Разделът „Информационни множества – когато не можете да видите всичко“ по-долу разглежда този въпрос.

Целта е да се намери стратегия π^* , която максимизира очакваната дисконтирана възвръщаемост.

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Ключовата рекурсивна зависимост е **уравнението на Белман**:

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi}(s')]$$

Това означава, че стойността на едно състояние е равна на очакваната непосредствена награда плюс дисконтираната стойност на следващото състояние. Динамичното програмиране, обучени-

ето с темпорална разлика (temporal-difference learning, TD) и дълбокото обучение с подкрепление се основават на вариации на това уравнение.⁶

1.4 Информационни множества – когато не можете да видите всичко

Стандартните MDP предполагат, че агентът може напълно да наблюдава състоянието. В действителност обаче често това не е така. В покера например не виждате картите на опонента, а в игрите със стратегия в реално време действа т.нар. **мъгла на войната**. Това са **частично наблюдаеми** среди.

Формалното разширение е **частично наблюдаемият марковски процес на вземане на решения** (Partially Observable MDP, POMDP)⁹, който включва функция за наблюдение – агентът получава наблюдения, представляващи шумни или непълни изгледи на истинското състояние. Но POMDP са печално известни с това, че оптималното им решаване е изключително трудно.

В теорията на игрите същата идея се изразява чрез **информационни множества**. Информационно множество обединява всички състояния на играта, които играчът не може да различи едно от друго въз основа на направените наблюдения. В Кун покер (3 карти, 2 играчи), когато държите Вале и все още не е имало залагания, вие се намирате в информационно множество – знаете своята карта, но не и тази на опонента си, така че не можете да определите дали действителното състояние е „Аз имам Вале, опонентът има Дама“ или „Аз имам Вале, опонентът има Поп“.

Алгоритмите, които работят в тези среди – като CFR (минимизиране на контрафактичното съжаление, разгледано в глава 2) – оперират върху информационни множества, а не върху отделни състояния. Това представлява фундаментално различен изчислителен проблем от стандартното RL и е причината игровият изкуствен интелект да се нуждае от собствен набор инструменти, различни от тези, които DQN или PPO предоставят по подразбиране.

Усвояването на информационните множества е от значение, защото главите от 2 до 8 се занимават с игри с непълна информация. Алгоритмите се променят, но **ядрото** на концепцията остава: вие вземате решения въз основа на това, което можете да наблюдавате, като разсъждавате върху това, което не можете.¹⁰

⁹Kaelbling, L.P., Littman, M.L. & Cassandra, A.R. (1998). “Planning and acting in partially observable stochastic domains.” *Artificial Intelligence*, 101(1-2), 99-134. DOI 10.1016/S0004-3702(98)00023-X

¹⁰Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press. DOI 10.1017/CBO9780511811654. Гл. 3 “Introduction to Noncooperative Game Theory: Games in Normal Form”; гл. 4 “Computing Solution Concepts of Normal-Form Games” (§4.1 – линейната програма за игри на двама с нулева сума); гл. 5 “Games with Sequential Actions: Reasoning and Computing with the Extensive Form” (§5.2 – игри с непълна информация и информационни множества; §5.2.3 – последователната форма, механизмът под всяка линейна програма в Глава 8); гл. 7 “Learning and Teaching” – рамката за учене в повтарящи се игри, включително напрежението, че действията едновременно *експлоатират* и *обучават* опонента. Свободно достъпна: <http://www.masfoundations.org/download.html>

1.5 Динамично програмиране – при наличие на модел

Динамичното програмиране (DP) представлява най-ранният подход за решаване на MDP. То е приложимо, когато разполагате с пълна информация за средата: вероятностите за преход $P(s'|s, a)$ и функцията на наградата $R(s, a, s')$. На практика това означава, че правилата на играта са ви напълно известни.

Двата основни алгоритъма при динамичното програмиране са:

Итерация по стратегии редува два етапа. Първо се извършва *оценка на стратегия*: при дадена фиксирана стратегия се изчислява стойността на всяко състояние чрез многократно прилагане на уравнението на Белман, докато не се постигне сходимост. След това следва *подобрене на стратегията*: стратегията се актуализира така, че да бъде алчна спрямо новоизчислените стойности. Процесът се повтаря, докато стратегията престане да се променя. Гарантирано е, че процесът достига оптималната стратегия.

Итерация на стойностите е съкратен вариант. Вместо да се оцени напълно една **стратегия**, преди тя да бъде подобрена, тя комбинира оценката и подобренето в една-единствена актуализация. На всяка стъпка, за всяко **състояние**, се избира **действието**, което максимизира очакваната едностъпкова възвръщаемост плюс дисконтираната стойност на следващото **състояние**. Тя клони към оптималната функция на стойността, от която се извлича оптималната **стратегия**.

DP е полезен, защото осигурява точни решения и има ясни гаранции за сходимост. Недостатъкът е очевиден: необходим е пълният модел, а изчислението нараства с броя на състоянията. За MDP с около дванадесет състояния DP е тривиален. За играта Го със 10^{170} състояния това е невъзможно.

Причината DP да има значение, въпреки че рядко го използваме директно, е, че всеки следващ метод – методът Монте Карло, TD обучението, DQN, PPO – може да се разглежда като приближение към решението на DP, когато моделът е неизвестен или пространството на състоянията е твърде голямо. DP представлява теоретичния таван, към който практическите алгоритми се стремят да се доближат.⁶

1.6 Обучение с темпорална разлика – обучение без модел

Обучението с темпорална разлика (TD) е пробивът, който прави обучението с подкрепление приложимо на практика. За разлика от динамичното програмиране, то не изисква модел на средата. За разлика от методите на Монте Карло, то не е необходимо да чака края на един епизод, за да се учи. То актуализира стойностните оценки след всяка стъпка, използвайки самоподкрепяне: текущата оценка на стойността на следващото състояние замества истинската бъдеща възвръщаемост.

Основната актуализация на TD(0) за функцията на стойността е:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

Изразът в скобите е **TD грешката** – разликата между настъпилото (награда плюс оценена бъдеща стойност) и предвиденото. Ако TD грешката е положителна, състоянието се е оказало по-добро от очакваното; ако е отрицателна, то е било по-лошо.

Вариантът на тази идея за задачата за управление е **Q-обучението** (Watkins, 1989):⁵

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Q-обучението научава стойността на двойки състояние-действие и използва оператора max, затова се обучава **извън текущата стратегия** (off-policy): научава оптималната стратегия дори докато следва друга, експериментална стратегия. Това го прави много практично.

TD обучението има предимство пред динамичното програмиране, защото не изисква модел, а пред метода Монте Карло – защото се учи онлайн (на всяка стъпка, а не на всеки **епизод**) и има по-ниска **дисперсия**. Самоподкрепянето въвежда известно отместване (bias), но на практика този компромис е силно изгоден. TD методите са основата, върху която се градят DQN и повечето съвременни **алгоритми, базирани на стойности**.

SARSA е аналогът на Q-обучението, който се обучава по текущата стратегия (on-policy) – той актуализира стойностите, използвайки действието, което агентът действително предприема след това, а не най-доброто възможно действие. Това го прави по-консервативен и понякога по-безопасен на практика, но не му позволява да научи оптимална стратегия, докато следва неоптимална изследователска стратегия.⁶

1.7 DQN – Дълбоки Q-мрежи

DQN (Mnih и др., 2015)⁷ е алгоритъмът, който доказва възможността дълбоките невронни мрежи да се използват като апроксиматори в обучението с подкрепление в голям мащаб. Преди DQN обучението с Q-стойности беше ограничено до таблична среда или ръчно извлечени признаци. DQN показва, че може да се научи да играе 49 игри на Atari директно по суровите пиксели на екрана, като надмина човешкото представяне в много от тях.

Как работи. DQN заменя Q-таблицата с невронна мрежа, която приема състояние като вход и извежда Q-стойности за всички действия. Обучението на мрежата се извършва чрез минимизиране на квадратичната TD грешка – разликата между предсказаната Q-стойност и целта (награда плюс дисконтираната максимална Q-стойност на следващото състояние). На всяка стъпка агентът избира действието с най-висока Q-стойност (експлоатация) или произволно действие с вероятност ϵ (изследване).

Два ключови похвата осигуряват достатъчна стабилност, за да работи този подход:

1. **Повторение на натрупан опит.** Вместо да се обучава върху последователни преходи (които са силно корелирани), преходите се съхраняват в голям буфер и се извличат на случаен принцип в минипартиди. Това нарушава корелацията, намалява дисперсията и позволява всеки опит да бъде използван многократно. Принципът е аналогичен на разбъркването на тренировъчните данни при обучение с учител.
2. **Целева мрежа.** Поддържа се отделно копие на Q-мрежата, което се обновява само периодически (на всеки няколко хиляди стъпки или епизода). Целевата стойност на TD се изчислява с помощта на това замразено копие. Без него целта се измества при всяка тренировъчна стъпка – вие преследвате подвижна цел, което води до колебания и дори до разходимост на обучението. Целевата мрежа разделя процеса на обновяване и стабилизира обучението.

Защо DQN е важен в сравнение с предишните методи: табличното Q-обучение не може да се справи с високоизмерни състояния (изображения, непрекъснати наблюдения); апроксимацията на функция само с Q-обучение беше известна като нестабилна („смъртоносната триада“⁶: обучение извън текущата стратегия + апроксимация на функции + самоподкрепяне). Двата похвата на DQN – повторението на натрупан опит и целевата мрежа – ограничиха нестабилността и за първи път направиха практически приложимо дълбокото обучение с подкрепление, базирано на стойности.

Ограничения. DQN работи единствено с дискретни пространства на действията. Той може да надценява Q-стойностите (проблем, който Double DQN¹¹ решава). Тъй като се обучава извън текущата стратегия, той използва данните ефективно, но това може да доведе до остаряла информация в буфера за повторение на опита. И най-важното – той научава алчна детерминистична стратегия, без естествен начин за изразяване на стохастични стратегии, което е от значение в игрите, където смесените стратегии са оптимални.

В нашата реализация от Глава 1 DQN реши CartPole-v1 (средна награда за последните 100 епизода 477,5 при цел 475) след приблизително 1000 епизода обучение. Най-влиятелните хиперпараметри бяха размерът на мрежата, епсилон минимумът и честотата на синхронизация на целевата мрежа.

Бележка относно невронните мрежи в тази глава. Q-мрежата в DQN е стандартен многослоен перцептрон (MLP), използван тук като практичен инструмент – функция, която съпоставя на състоянията Q-стойности. За Глава 1 вътрешните механизми на невронната мрежа (инициализация на теглата, потокът на градиента, функциите на активиране) бяха разглеждани като черна кутия: използвахме настройките по подразбиране на PyTorch и се фокусирахме върху самия RL алгоритъм. По-задълбочено разглеждане на архитектурите на невронните мрежи и тяхното взаимодействие с методите за намиране на равновесие ще бъде представено в Глава 5 (Невронни мрежи за игри с непълна информация).

¹¹van Hasselt, H., Guez, A. & Silver, D. (2016). “Deep Reinforcement Learning with Double Q-Learning.” *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1). DOI 10.1609/aaai.v30i1.10295. arXiv:1509.06461

1.8 PPO – оптимизация на стратегията с ограничение на близостта

PPO (Schulman и др., 2017)⁸ прилага фундаментално различен подход към RL в сравнение с DQN. Вместо да изчислява стойността на действията и да извежда стратегия въз основа на тези стойности, PPO **научава стратегията директно** – невронна мрежа генерира разпределение на вероятностите върху действията, а мрежата се обучава да увеличава вероятността на действията, които водят до висока възвръщаемост.

Проблемът, който PPO решава. Методите на градиента на стратегията са мощни в теорията, но са печално известни с нестабилността си на практика. Основният алгоритъм REINFORCE¹² страда от висока дисперсия и може да предприема разрушително големи стъпки, които провалят добра стратегия. Оптимизацията на стратегията в доверителна област (Trust Region Policy Optimization, TRPO; Schulman и др., 2015)¹³ се справи с това, като ограничи колко може да се промени стратегията при всяка актуализация, но TRPO трябва да решава ограничена задача от втори ред (спрегнати градиенти върху матрицата на Фишер за ограничението по KL и последващо линейно търсене), което прави реализацията му сложна.

Как работи PPO. PPO запазва идеята за стабилност от TRPO, но заменя ограничението с много по-прост механизъм: **изрязване**. Ключовото количество е отношението на вероятностите $r_t(\theta) = \pi_{\theta}(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ – колко по-вероятно (или по-малко вероятно) е новата стратегия да предприеме същото действие, което старата стратегия е предприела. PPO изрязва това отношение до диапазона $[1 - \epsilon, 1 + \epsilon]$ (обикновено $\epsilon = 0,2$), което означава:

- Ако едно **действие** се окаже добро (положително **предимство**) и отношението на вероятностите надхвърли $1 + \epsilon$, **градиентът** се анулира – **стратегията** не трябва да става прекалено „алчна“ за това **действие**.
- Ако едно **действие** се окаже лошо (отрицателно **предимство**) и отношението на вероятностите падне под $1 - \epsilon$, **градиентът** също се анулира – така се предотвратява прекомерна корекция на **стратегията**.

Това е **изрязаната сурогатна цел** и именно тя осигурява работата на PPO. Тя постига стабилност, сходна с тази на TRPO, само с обикновени стохастични градиентни стъпки от първи ред, което значително опростява реализацията.

Архитектура. PPO е актьор-оценител: **мрежа на стратегията** (актьор) извежда вероятности за действия, а **мрежа за стойности** (оценител) оценява стойностите на състоянията. Мрежата за стойности позволява прилагането на обобщената оценка на предимството (generalized advantage

¹²Williams, R.J. (1992). “Simple statistical gradient-following algorithms for connectionist reinforcement learning.” *Machine Learning*, 8(3-4), 229-256. DOI 10.1007/BF00992696

¹³Schulman, J., Levine, S., Abbeel, P., Jordan, M. & Moritz, P. (2015). “Trust Region Policy Optimization.” *Proceedings of the 32nd International Conference on Machine Learning*, 1889-1897.

estimation, GAE)¹⁴, която дава оценки на предимството с ниска дисперсия за **градиента на стратегията**. И двете мрежи се обучават едновременно.

Защо PPO е важен в сравнение с DQN: PPO естествено работи както с дискретни, така и с непрекъснати пространства на **действията**. Той може да усвоява **стохастични стратегии**, което е от решаващо значение в игрови среди, където прилагането на **смесени стратегии** е необходимо. Тъй като се обучава **по текущата си стратегия** (on-policy) и използва само пресни данни, събрани с нея, той избягва проблемите с остарели данни. А изрязаната цел го прави забележително устойчив – PPO се превърна в един от най-широко използваните алгоритми на дълбокото обучение с подкрепление; с него например е реализиран първоначалният RLHF процес за езикови модели, които следват инструкции.¹⁵

Ограничения. Тъй като се обучава по текущата си стратегия, PPO използва данните по-малко ефективно от методите, които се обучават извън текущата стратегия (off-policy), като DQN (всеки опит се използва само за няколко **епохи на обновяване**, преди да бъде отхвърлен). Той изисква внимателна настройка на оценката на **предимството** (параметър λ на GAE), **бонуса за ентропия** и архитектурата на **мрежата**. Освен това все още може да среща трудности при редки награди – ако **агентът** рядко постига успех, наличният сигнал за учене е недостатъчен.

В нашата реализация от Глава 1 PPO успя да реши LunarLander-v3 (средна награда за последните 100 епизода 202,2 при цел 200) след приблизително 264 хиляди стъпки на средата. Решаващата промяна между двете записани изпълнения беше капацитетът на мрежата (удвоен от [64,64] на [128,128], при по-голям бюджет); нормализацията на предимството беше използвана и в двете.

Бележка относно невронните мрежи в тази глава. Както и при DQN, мрежите за **стратегия** и **стойност** тук са стандартни многослойни перцептрони (MLP), чиито вътрешни механизми не бяха във фокуса на тази глава. Приехме прагматичен подход (използвайки налични настройки за тях) към невронните компоненти и се концентрирахме върху разбирането на логиката на алгоритъма PPO (изрязване, GAE, структура на разгръщането). По-задълбоченото взаимодействие между дизайна на **невронната мрежа** и **сходимостта** в обучението с подкрепление – особено в контекста на **агенти за игри** – е отложено за Глава 5.

1.9 Практическа проверка – собствени реализации срещу Stable-Baselines3

За да проверим дали нашите реализации от нулата функционират коректно, ги сравнихме със Stable-Baselines3 (SB3)¹⁶ – широко използвана и добре тествана RL библиотека. При PPO на SB3 бяха зададени нашите хиперпараметри и същият бюджет от 500 хил. стъпки. При DQN показаното

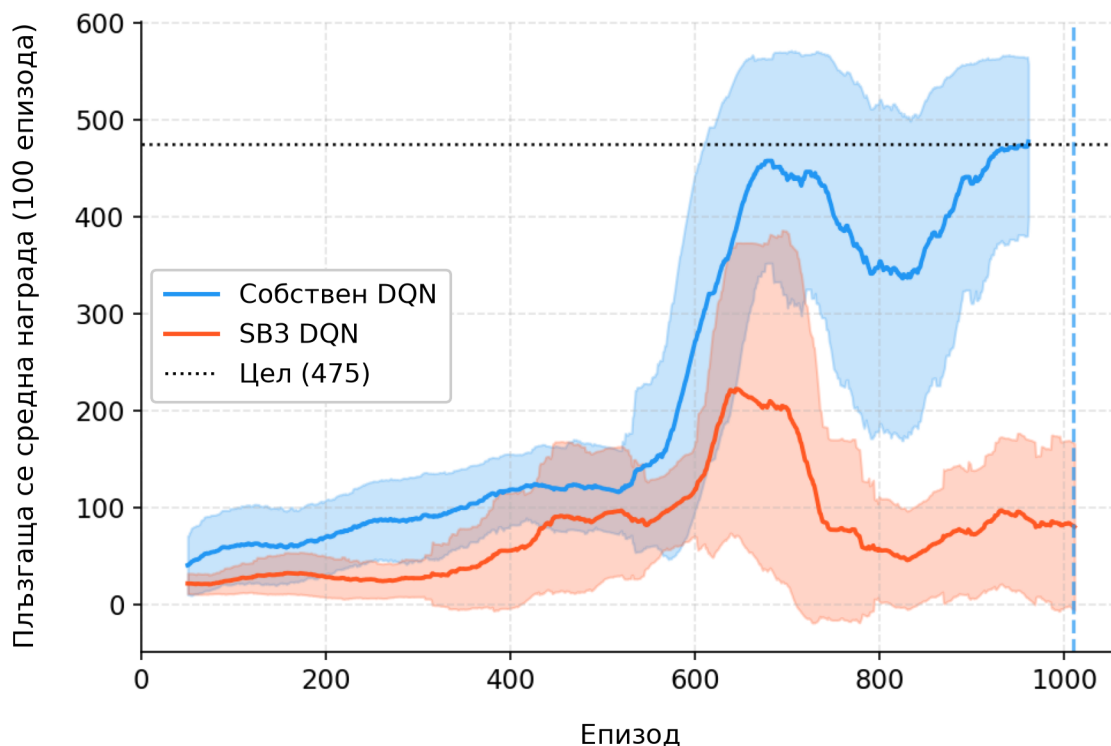
¹⁴Schulman, J., Moritz, P., Levine, S., Jordan, M. & Abbeel, P. (2016). “High-Dimensional Continuous Control Using Generalized Advantage Estimation.” *ICLR 2016*. arXiv:1506.02438

¹⁵Ouyang, L., Wu, J., Jiang, X. et al. (2022). “Training language models to follow instructions with human feedback.” arXiv:2203.02155.

¹⁶Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. & Dormann, N. (2021). “Stable-Baselines3: Reliable Reinforcement Learning Implementations.” *Journal of Machine Learning Research*, 22(268), 1-8.

изпълнение на SB3 използва настроените за CartPole хиперпараметри от RL Baselines3 Zoo¹⁷ и бюджет от 100 хил. стъпки, така че там сравнението е с настроен базов агент, а не при еднакви настройки.

1.9.1 DQN върху CartPole-v1



Фигура 1: DQN върху CartPole-v1: собствената реализация срещу SB3 (изпълнение от 6 април 2026 г.; показани са първите 1 061 от 1 255 епизода на SB3). Плъзгаща се средна на наградата за 100 епизода (оцветено – $\pm$ едно стандартно отклонение в прозореца); пунктирната линия е целта 475, вертикалната – край на обучението на собствения агент.

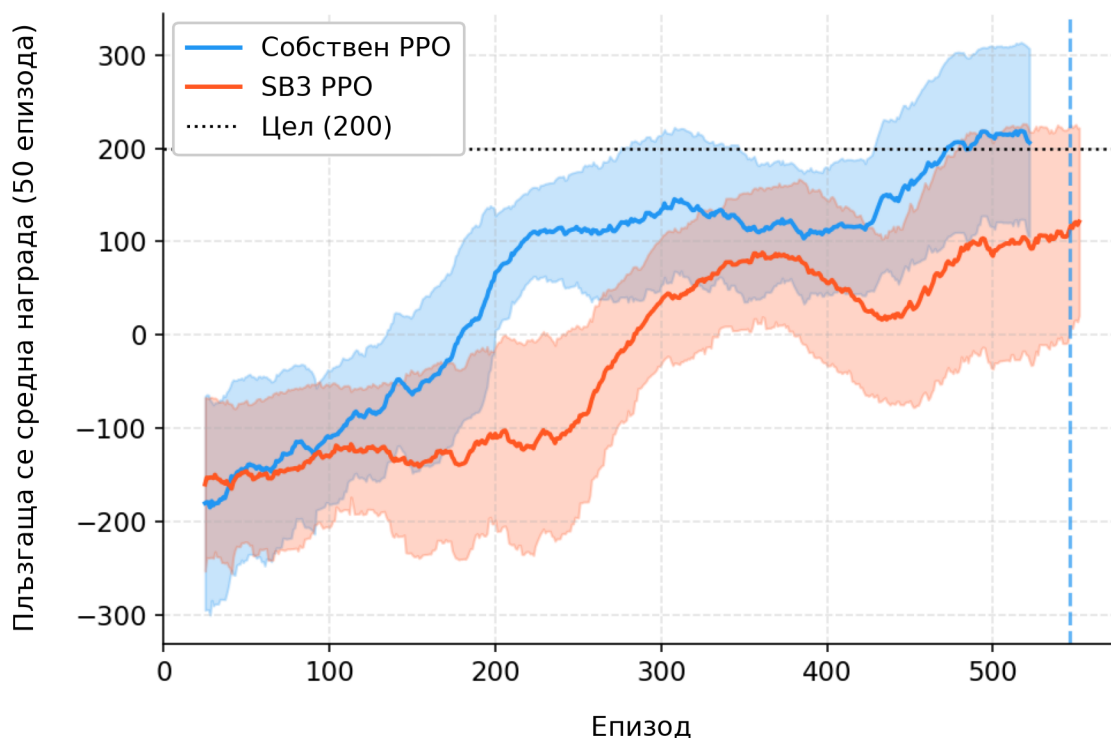
Нашият собствен DQN решава CartPole около епизод 1011, като достига плъзгаща се средна за 100 епизода 477,5 (над целта от 475). Показаното изпълнение на DQN на SB3 е от 6 април 2026 г. и наградите му по епизоди са в запазения кеш с резултати (`sb3_results_cache.json`). То използва настройките на Zoo (скорост на обучение 0,0023, мрежа [256,256], синхронизация на целевата мрежа на всеки 10 стъпки, ϵ намалява до 0,04 през първите 16 хил. стъпки) с бюджет от 100 хил. стъпки – два пъти повече от 50-те хиляди, за които Zoo ги обучава. През своите 1 255 епизода плъзгащата му се средна достига най-много 222,4 при епизод 695 (около 51 хил. стъпки) и завършва на 134,3, без да се доближи до целта.

Тъй като SB3 използва настроени хиперпараметри, а не същите като нашите, сравнението не е

¹⁷Raffin, A. (2020). *RL Baselines3 Zoo*. Хранилище в GitHub. <https://github.com/DLR-RM/rl-baselines3-zoo>; настройките за CartPole-v1 са в `hyperparams/dqn.yml`.

равностойно и причината за разликата не е установена. Две различия не са проверени. Първо, показаните награди са от епизоди на обучение, в които SB3 все още избира случайно действие в 4% от случаите (крайно $\epsilon = 0,04$), докато нашият ϵ е спаднал под 1% към момента на решаването на задачата. Второ, SB3 прави 128 градиентни стъпки на всеки 256 стъпки на средата, т.е. наполовина по-малко обновявания на стъпка от нашия агент, който обновява мрежата при всяка стъпка. Всяка реализация е изпълнена веднъж, нашата – без фиксирано начално число, така че неуспехът на SB3 при бюджет, два пъти по-голям от този на Zoo, трябва да се провери с второ начално число, преди да се тълкува като свойство на SB3, а не на това изпълнение.

1.9.2 PPO върху LunarLander-v3



Фигура 2: PPO върху LunarLander-v3: собствената реализация (окончателното изпълнение, спряно при епизод 547) срещу SB3 (показани са първите 577 от 1 224 епизода). Плъзгаща се средна наградата за 50 епизода (оцветено – $\pm$ едно стандартно отклонение в прозореца); пунктирната линия е целта 200, вертикалната – крайт на обучението на собствения агент.

Нашият собствен PPO преминава целта от 200 около епизод 543 (след 264 хил. стъпки). PPO на SB3 със същия бюджет от 500 хил. стъпки достига най-добра плъзгаща се средна за 100 епизода 131,2 – все още се покачва и не е достигнал сходимост.

Разликата тук е по-малка, отколкото при DQN. И двата използват идентични хиперпараметри на PPO. Оставащата разлика може да се дължи на ортогоналната инициализация на теглата в SB3 и на това, че SB3 нормализира предимството във всяка мини-партида, а не в цялото разгръщане; при

едно-единствено изпълнение на всяка реализация тя може да е и случайна. При предоставяне на по-голям обучителен бюджет (1–2 млн. стъпки), PPO на SB3 вероятно би достигнал целта.

1.9.3 Извод

Чиста реализация от нулата обхваща около 300–500 реда код, докато тази на SB3 достига приблизително 10 000. Компромисът е по-малко функции срещу пълна прозрачност и лесна възможност за отстраняване на грешки. За изследователски цели – особено при планираната работа върху експлоатацията на противника в глави 7–8 – собствените реализации дават възможност за прецизни модификации (например добавяне на наблюдения на състоянието на убежденията), които биха изисквали значително подкласиране в SB3. Именно тази практическа гъвкавост беше причината да изградим решение от нулата.

2 Глава 2 – Основи на теорията на игрите и CFR

2.1 От обучение с подкрепление с един агент към многоагентно стратегическо взаимодействие

Глава 1 разглеждаше средите като пасивни – количката с прът не оказва съпротива, а кацащият модул не се опитва да предизвика катастрофа. Агентът оптимизира спрямо фиксирана динамика. Глава 2 въвежда фундаментално различен проблем: средата включва друг *стратегически* вземащ решения участник, чиито действия зависят от първия.

Тази промяна нарушава MDP рамката. В едноагентна среда оптималната стратегия е фиксирана – съществува един-единствен най-добър начин за действие, независимо от това каква стратегия сте обмисляли преди. В многоагентна среда обаче „най-добрата“ стратегия зависи от действията на противника, а неговата най-добра стратегия – от вашите действия. Тази взаимна зависимост е в основата на предизвикателството, което поставя теорията на игрите.

Формализацията започва с **игрите в нормална форма** (едновременни ходове, като камък-ножица-хартия) и стига до **игрите в разгърната форма** (последователни ходове с дървовидна структура, като покер). И в двата случая концепцията за решение се измества от „оптимална стратегия“ към **равновесие на Наш** – стратегически профил, при който никой играч не може да подобри резултата си чрез едностранно променяне на собствената си стратегия.¹⁸

2.2 Игри в разгърната форма и информационни множества

Игра в разгърната форма представя стратегическото взаимодействие под формата на дърво. Всеки възел е точка за вземане на решение от даден играч (или „случайност“ при случайни събития като раздаване на карти). Ребрата обозначават действия, а крайните възли съдържат печалбите на всеки играч.

Ключовото разграничение е между **пълна** и **непълна** информация:

- **Пълна информация** (шах, Го): всеки играч има достъп до пълното състояние на играта. В дървото на играта няма групирани възли – всеки възел представлява отделно информационно множество. Тези игри се решават чрез минимакс и алфа-бета подрязване.
- **Непълна информация** (покер, много реални проблеми): играчите не могат да наблюдават всички аспекти на състоянието. В покера виждате само собствените си карти, но не и тези на

¹⁸Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Гл. 3 (игри в нормална форма; §3.3 – най-добър отговор и равновесие на Наш, §3.4.1 – максиминни и минимаксни стратегии); §4.1 (изчисляване на равновесия на Наш в игри за двама играчи с нулева сума); гл. 5 (игри в разгърната форма; §5.2 – непълна информация и информационни множества, §5.2.3 – последователната форма); §7.5 “No-regret learning and universal consistency”. Свободно достъпна: <http://www.masfoundations.org/download.html>

опонента. Това означава, че множество състояния на играта са *неразличими* за действащия играч.

Информационно множество обединява всички състояния на играта, които играчът не може да различи едно от друго. В рамките на такова множество играчът трябва да приложи една и съща стратегия към всички състояния в него (тъй като те са неразличими за него). Именно това ограничение прави игрите с **непълна информация** фундаментално по-сложни от тези с **пълна информация** – вместо да се избира най-доброто действие за всяко отделно състояние, трябва да се подбере едно действие, което да работи добре *средно* за всички състояния в информационното множество.

В Кун покер информационното множество "2pb" включва две състояния на играта: „Аз държа Дама, опонентът държи Вале, историята е пас–залог“ и „Аз държа Дама, опонентът държи Поп, историята е пас–залог“. Играчът с Дама не може да различи тези състояния и трябва да използва една и съща стратегия (вероятност за плащане) и в двата случая.¹⁸

2.3 Теорема за минимакс

Теоремата за минимакс, установена от Джон фон Нойман през 1928 г.,¹⁹ представлява основна концепция за решение на игрите за двама играчи с нулева сума. Тя гласи, че за всяка крайна игра за двама играчи с нулева сума съществува стратегия за всеки от двамата, при която максималната очаквана загуба е минимизирана. С други думи, всеки играч може да гарантира конкретна очаквана печалба, известна като „стойност на играта“, независимо от стратегията на противника. Това създава ситуация, в която оптималната стратегия на единия играч е да максимизира своята минимална награда (максимин), докато противникът се стреми да минимизира максималната награда на първия играч (минимакс). Теоремата твърди, че при смесени стратегии тези две стойности винаги са равни; общата им стойност е стойността на играта. Теоремата е пряко свързана с равновесието на Наш: в частния случай на игрите за двама играчи с нулева сума минимаксният стратегически профил е еквивалентен на равновесие на Наш. Откриването на това минимаксно решение гарантира защита срещу всяка експлоатираща стратегия, която противникът може да приложи.

2.4 Равновесие на Наш

Равновесие на Наш представлява стратегически профил, при който стратегията на всеки играч е най-добър отговор на стратегиите на всички останали играчи. Формално, за всеки играч i :

¹⁹von Neumann, J. (1928). "Zur Theorie der Gesellschaftsspiele." *Mathematische Annalen*, 100(1), 295-320. Превод на английски: "On the Theory of Games of Strategy," in *Contributions to the Theory of Games*, Vol. 4 (1959), 13-42.

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*) \quad \forall \sigma_i$$

Нито един играч не може да подобри своята очаквана печалба чрез едностранно отклоняване. Това не означава, че всички са доволни от резултата (дилемата на затворника), а само че никой не може да постигне по-добър резултат, като промени единствено собствената си стратегия.

В игрите, разширени до повече от двама играчи (игри с N играчи) или игри с обща сума, свойствата на равновесието на Нaш стават значително по-сложни. В тези условия изчисляването на равновесие на Нaш е PPAD-пълна задача – дори при игри за двама играчи с обща сума – и затова се смята за изчислително непосилно.²⁰²¹ Освен това равновесията губят своята гаранция за безопасност: играта по равновесна стратегия в игра с трима играчи не предпазва от двама опоненти, които могат да се отклоняват от равновесие, потенциално формирайки временни коалиции, които експлоатират третия играч.²² Тази динамика изисква изцяло различни обучителни рамки и методи за оценка (като тези, обсъдени в глави 9 и 11), тъй като пряката еквивалентност между равновесието на Нaш и минимакс оптималността вече не важи.

Ключови свойства:

- **Съществуване:** Нaш (1950)²³ доказва, че всяка крайна игра има поне едно равновесие на Нaш (възможно в смесени стратегии). Това е основополагаща теорема, но не помага при изчисленията.
- **Единственост:** Игрите могат да имат множество равновесия. Кун покер има еднопараметрично семейство от равновесия на Нaш, индексирани от $\alpha \in [0, 1/3]$.
- **Смесени стратегии:** В много игри равновесието изисква рандомизация. В Кун покер равновесието на Нaш изисква Играч 1 (вторият по ред) да блъфира с Вале в точно 1/3 от случаите – всяка друга честота е експлоатируема.

За игри за двама с нулева сума (където печалбата на единия играч е загуба за другия), равновесията на Нaш имат специално свойство: те са **оптимални по минимакс**. Равновесната стратегия гарантира поне стойността на играта, независимо от действията на опонента. Това е гаранцията за безопасност, която прави равновесието на Нaш базова линия за изкуствен интелект за покер.²³

²⁰Daskalakis, C., Goldberg, P. W. & Papadimitriou, C. H. (2009). “The Complexity of Computing a Nash Equilibrium.” *SIAM Journal on Computing*, 39(1), 195–259.

²¹Chen, X., Deng, X. & Teng, S.-H. (2009). “Settling the Complexity of Computing Two-Player Nash Equilibria.” *Journal of the ACM*, 56(3), 1–57.

²²Szafron, D., Gibson, R. & Sturtevant, N. (2013). “A Parameterized Family of Equilibrium Profiles for Three-Player Kuhn Poker.” *Proc. AAMAS 2013*, 247–254. В тройния Кун покер един играч може да прехвърля печалба от единия противник към другия, без да напуска семейството от равновесия.

²³Nash, J.F. (1950). “Equilibrium Points in N-Person Games.” *Proceedings of the National Academy of Sciences*, 36(1), 48–49.

2.5 Напасване на съжалението – градивният елемент

Преди CFR да може да минимизира съжалението в дърво на играта, е необходим механизъм за минимизиране на съжалението в отделна точка на вземане на решение. **Напасването на съжалението** изпълнява тази функция.

Идеята е следната: след множество изиграни рундове, за всяко **действие** a се изчислява **натрупаното съжаление** – с колко по-добър резултат бихте се справили, ако винаги бяхте избирали a вместо действително използваната **смесена стратегия**. След това следващата **стратегия** се определя пропорционално на *положителните* стойности на **съжалението**:

$$\sigma^{T+1}(a) = \begin{cases} \frac{R^{T,+}(a)}{\sum_{a'} R^{T,+}(a')} & \text{ако } \sum > 0 \\ \frac{1}{|A|} & \text{в противен случай} \end{cases}$$

Гаранция за сходимост: Блекуел (1956)²⁴ и Харт и Мас-Колел (2000)²⁵ доказват, че при използване на **напасване на съжалението** средното съжаление клони към нула със скорост $O(1/\sqrt{T})$ (натрупаното съжаление расте не по-бързо от $O(\sqrt{T})$). В игра за двама играчи с нулева сума, ако и двамата играчи прилагат **напасване на съжалението** в своите точки на вземане на решение, средният **стратегически профил** се сближава до **равновесие на Наш**.

Прост пример е **камък-ножица-хартия**. Ако прилагате **напасване на съжалението** срещу постоянен противник, който винаги избира **камък**, вашето натрупано съжаление за **хартия** нараства (тъй като хартията побеждава камъка), докато съжалението за ножица става отрицателно и не влияе на стратегията. Стратегията бързо преминава към хартия – правилният най-добър отговор.²⁶

2.6 Минимизиране на контрафактичното съжаление (CFR)

CFR (Zinkevich и др., 2007)²⁷ разширява **напасването на съжалението** към игрите в разгърната форма. Ключовата идея е да се разложи общото **съжаление** на стратегията на играта на **локални съжаления** във всяко **информационно множество** и след това всяко локално **съжаление** да бъде минимизирано независимо чрез **напасване на съжалението**. Ако средното съжаление във всяко информационно множество клони към нула, цялостната *средна* стратегия клони към равновесие на Наш.

„**Контрафактичната**“ част е от решаващо значение. Във всяко информационно множество I съжалението за действие a не е просто „колко по-добро би било a “. То представлява *контрафактич-*

²⁴Blackwell, D. (1956). “An Analog of the Minimax Theorem for Vector Payoffs.” *Pacific Journal of Mathematics*, 6(1), 1-8.

²⁵Hart, S. & Mas-Colell, A. (2000). “A Simple Adaptive Procedure Leading to Correlated Equilibrium.” *Econometrica*, 68(5), 1127-1150.

²⁶Neller, T.W. & Lanctot, M. (2013). “An Introduction to Counterfactual Regret Minimization,” §2–3 (версия от 9 юли 2013 г.).

²⁷Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2007). “Regret Minimization in Games with Incomplete Information.” *Advances in Neural Information Processing Systems* 20, 1729-1736.

ното съжаление – подобрението, което би настъпило, ако играчът умишлено бе избрал да достигне до I (вероятност за достигане = 1 за играча), като останалите условия (стратегия на противника и случайни събития) останат непроменени. Формално:

$$R^T(I, a) = \sum_{t=1}^T \pi_{-i}^t \cdot (v^t(I, a) - v^t(I))$$

където π_{-i}^t е вероятността за достигане на противника, а $v^t(I, a)$ представлява контрафактичната стойност на действието a в I при итерация t .

Защо вероятността за достигане на противника? Съжалението в дадено информационно множество има по-голяма тежест, когато съществува по-голяма вероятност противникът да е изиграл така, че да се достигне до него. Претеглянето с π_{-i} прави съжалението пропорционално на честотата, с която информационното множество е „от значение“ за крайния резултат от играта.

Алгоритъм (CFR с извадка на случайните събития, по Neller и Lanctot, 2013²⁶):

1. Инициализирайте всички кумулативни съжаления и суми на стратегиите с нула.
2. За T итерации:
 - а) Изтеглете случайно раздаване на картите (извадка на случайните събития).
 - б) Рекурсивно обходите дървото на играта, започвайки от корена.
 - в) Във всяко информационно множество изчислете стратегия чрез напасване на съжалението.
 - г) За всяко действие рекурсивно обходите поддървото (като смените знака на полезността, тъй като играта е с нулева сума).
 - д) Обновете кумулативните съжаления, претеглени с вероятността за достигане на противника.
 - е) Натрупайте стратегия, претеглена със собствената вероятност за достигане на играча.
3. Изведете **средната** стратегия (а не стратегията от последната итерация).

Сходимост: Средният профил от стратегии се сближава до равновесие на Наш със скорост $O(1/\sqrt{T})$ по отношение на експлоатируемостта, където T е броят на итерациите (Теорема 4, Zinkevich и др. 2007). Границата на сходимост е:

$$\text{exploit}(\bar{\sigma}^T) \leq \frac{\Delta |\mathcal{J}| \sqrt{|\mathcal{A}|}}{\sqrt{T}}$$

където Δ е обхватът на печалбите, $|\mathcal{J}|$ - броят на информационните множества на двамата играчи, а $|\mathcal{A}|$ - най-големият брой действия в едно информационно множество.²⁷ Границата е линейна по $|\mathcal{J}|$; по-късно е доказана и по-тясна граница.²⁸

²⁸Lanctot, M., Waugh, K., Zinkevich, M. & Bowling, M. (2009). "Monte Carlo Sampling for Regret Minimization in Extensive Games." *Advances in Neural Information Processing Systems* 22, 1078–1086.

2.7 Математиката на покера

Покерът е каноничният тестов полигон за теорията на игрите с непълна информация, тъй като съчетава три източника на сложност, които рядко се срещат в комбинация:

1. **Скрита информация** – всеки играч вижда само своите карти. Едно и също наблюдавано състояние на играта (информационно множество) може да съответства на много различни действителни състояния на играта.
2. **Стохастични елементи** – раздаването на картите въвежда случайни възли в дървото на играта. Оптималната стратегия трябва да отчита всички възможни раздавания, а не само наблюдаваното.
3. **Стратегическо заблуждение** – за разлика от игрите с пълна информация, покерът възнаграждава смесените стратегии. Играч, който винаги залага със силни ръце и пасува със слаби, е тривиално експлоатируем. Равновесието на Наш изисква **рандомизирано блъфиране с математически точно определени честоти**.

Чен и Анкенман формализират тези концепции чрез решения на опростени игрови модели, при които честотите на блъфиране в равновесие на Наш могат да бъдат изведени аналитично. Техните модели с половин и с пълен рунд на залагане (half-street и full-street) изграждат интуиция защо изходните стратегии на CFR съдържат точните съотношения между блъф и плащане – балансиращият играч трябва да блъфира с честота, пропорционална на шансовете на пота, които предлага.²⁹

2.8 Кун покер – най-простото изпитателно поле

Кун покер (Kuhn, 1950) е стандартният минимален пример за алгоритми в игри с непълна информация. С едва три карти, двама играчи и 12 информационни множества той е достатъчно малък, за да бъде решен аналитично, но същевременно достатъчно богат, за да демонстрира ключовите явления: блъфиране, информационна асиметрия и смесени равновесия.

Правила: Три карти {J, Q, K}; всеки играч внася начален залог от 1 жетон и получава по една тайна карта. Играч 0 действа пръв – пас или залог. Играч 1 отговаря – пас или залог. Ако Играч 0 е пасувал, а Играч 1 е направил залог, Играч 0 има право да отговори отново. Победител е играчът с по-висока карта при разкриването.

Равновесие на Наш (параметризирано с $\alpha \in [0, 1/3]$):

- **Играч 0 с J:** Залог (блъф) с вероятност α . Ако след пас срещне залог, винаги се отказва.

²⁹Chen, B. & Ankenman, J. (2006). *The Mathematics of Poker*. ConJelCo – аналитично извеждане на равновесия на Наш за опростени варианти на покер.

- **Играч 0 с Q:** Винаги пас. Ако след пас срещне залог, плаща с вероятност $1/3 + \alpha$.
- **Играч 0 с K:** Залог с вероятност 3α . Ако след пас срещне залог, винаги плаща.
- **Играч 1 с J:** След пас – залог (блъф) с вероятност $1/3$. След залог – винаги се отказва.
- **Играч 1 с Q:** След пас – винаги пас. След залог – плаща с вероятност $1/3$.
- **Играч 1 с K:** Винаги залага и винаги плаща.

(Стратегията на Играч 1 е една и съща във всички равновесия; от α зависи само стратегията на Играч 0. Стратегията, до която стига CFR, е показана на последната фигура в раздела „Емпирични визуализации“.)

Стойност на играта: При това равновесие очакваната печалба на Играч 0 е $-1/18 \approx -0,0556$. Играч 0 е в структурно неблагоприятна позиция, тъй като действа пръв. Как обучението се доближава до тази стойност, показва първата фигура в раздела „Емпирични визуализации“.

Защо Кун е важен: Всяка ключова концепция при решаването на игри с непълна информация се проявява тук в умален вид – блъфиране (с J се залага, въпреки че е най-слабата карта), информационна асиметрия (Играч 1 вижда действието на Играч 0, но не и картата му), смесени стратегии (точно определена честота на блъф $1/3$), безразличие (с Q играчът е безразличен между плащане и отказ в определени информационни множества). Ако вашият алгоритъм не може да реши Кун коректно, той няма да може да реши нищо друго.³⁰

2.9 Експлоатируемост – мярка за отклонението от равновесието на Наш

Експлоатируемостта е основният оценителен показател за стратегии в игри с непълна информация. Тя измерва доколко една стратегия може да бъде експлоатирана от противник в най-лошия случай:

$$\text{exploit}(\sigma) = BR_0(\sigma_1) + BR_1(\sigma_0)$$

където $BR_i(\sigma_{-i})$ представлява очакваната печалба на играч i при прилагането на неговия най-добър отговор спрямо стратегията на противника. В игра за двама играчи с нулева сума тази сума се нарича още NashConv; OpenSpiel отчита като „експлоатируемост“ половината от нея, така че стойностите по двете конвенции се различават два пъти.

В равновесие на Наш експлоатируемостта е точно нула – никой играч не може да подобри резултата си чрез отклонение от стратегията. За приблизителни равновесия на Наш експлоатируемостта измерва качеството на приближението.

Изчисляването на най-добър отговор в игри с непълна информация е деликатно. Най-добре реагиращият играч трябва да избере едно и също действие във всички състояния в рамките на

³⁰Kuhn, H.W. (1950). “Simplified Two-Person Poker.” *Contributions to the Theory of Games*, Vol. 1, Annals of Mathematics Studies 24, Princeton University Press, 97–103.

едно информационно множество (тъй като той не може да ги различава). Наивен подход, който избира най-доброто действие за всяко състояние на играта (използвайки знанието за тайната карта на противника), изчислява приблизителна стойност, а не истински най-добър отговор. За Кун покер е достатъчно да се изброят всички $2^6 = 64$ чисти стратегии и да се оцени всяка от тях; при по-големи игри най-добрият отговор се изчислява с едно обхождане отдолу нагоре, което обединява стойностите по информационни множества.

Скорост на сходимост: Теоретично експлоатируемостта на средната стратегия на CFR намалява като $O(1/\sqrt{T})$ ²⁷ (вж. втората фигура в раздела „Емпирични визуализации“). В двулогаритмична диаграма това се проявява като права линия с наклон $\approx -0,5$. При пет начални числа и контролни точки от 100 до 100 000 итерации измереният наклон е $-0,52 \pm 0,02$, което съответства на границата $O(1/\sqrt{T})$; тя е горна граница, а не точна прогноза за скоростта.

Защо не просто стойност на играта? Една стратегия може да постигне правилната стойност на играта, но въпреки това да бъде експлоатируема. Представете си стратегия за Кун покер, при която Играч 1 винаги блъфира с Вале (вместо в $1/3$ от случаите). Средната стойност на играта може да е близо до $-1/18$, но стратегията е силно експлоатируема – Играч 0 може винаги да плаща с Дама и да печели. Експлоатируемостта улавя това; самата стойност на играта не го прави.

Тази метрика се появява многократно в дисертацията. Глави 7–8 (моделиране на противника, безопасна експлоатация) използват експлоатируемостта като основна мярка. Тя е и отправната точка на методологията за оценяване в дисертацията (Принос №3).

2.10 Връзки с Глава 1 и бъдещи насоки

Локална оптимизация → глобална сходимост. В Глава 1 Q-обучението намалява TD грешката за всяко състояние поотделно, но в табличния случай Q-функцията доказано се сближава до оптималната; DQN запазва правилото за обновяване, но с невронна мрежа губи тази гаранция. В Глава 2 CFR минимизира съжалението за всяко информационно множество поотделно, но цялостната *средна* стратегия се сближава до равновесие на Наш. И двете демонстрират един и същ принцип: локалните актуализации, когато са правилно структурирани, постигат глобални цели.

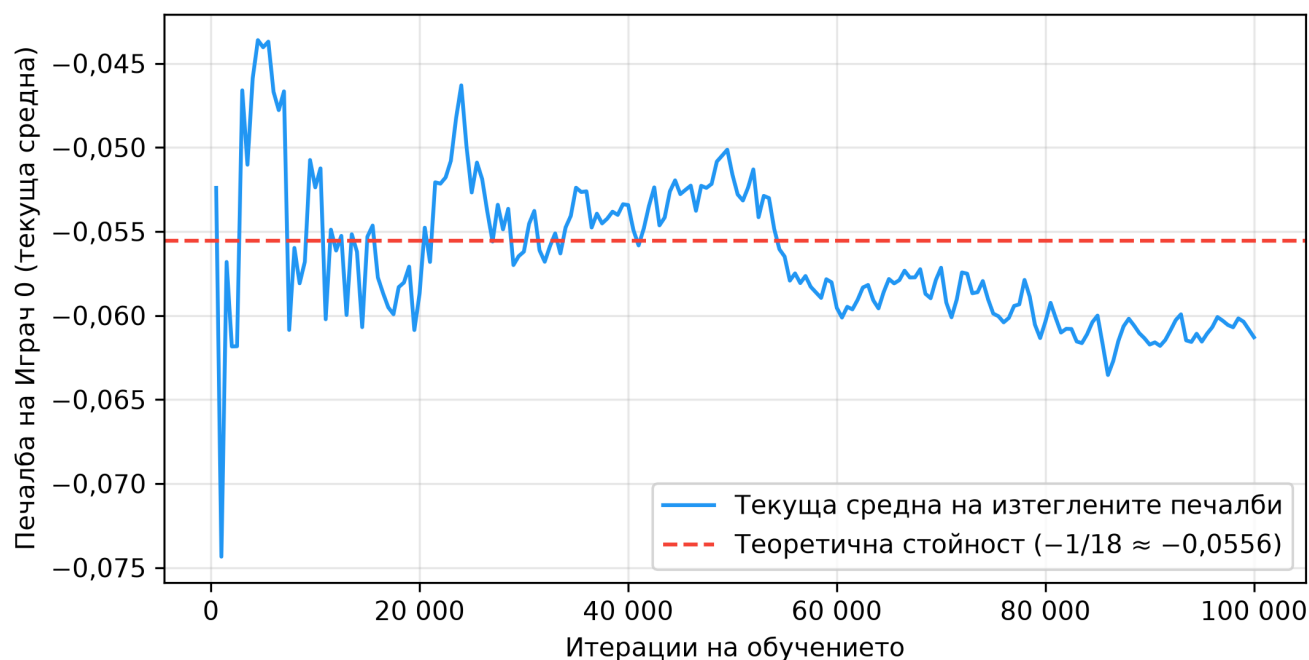
Натрупване на стратегии $\approx$ повторение на натрупан опит. CFR извежда *средната* стратегия, а не стратегията от последната итерация. Това осредняване стабилизира сходимостта, по същия начин както повторението на натрупан опит в DQN стабилизира обучението, като предотвратява преобучението на агента върху последните преходи. И двете представляват механизми за памет, които изглаждат шумовите колебания.

Напред: Реализацията в тази глава изтегля по едно раздаване на картите на итерация (извадка на случайните събития), но обхожда всички последователности от действия за него, така че всяка итерация пак засяга голяма част от дървото. При по-големи игри дори това е изчислително непо-

силно.³¹ В глава 3 се въвежда методът Монте Карло CFR (MCCFR), който обхожда само извадка от дървото, като заменя точността с мащабируемост. Глава 4 добавя абстракция на играта, намалявайки размера ѝ. Глава 5 заменя табличните стратегии с невронни мрежи.

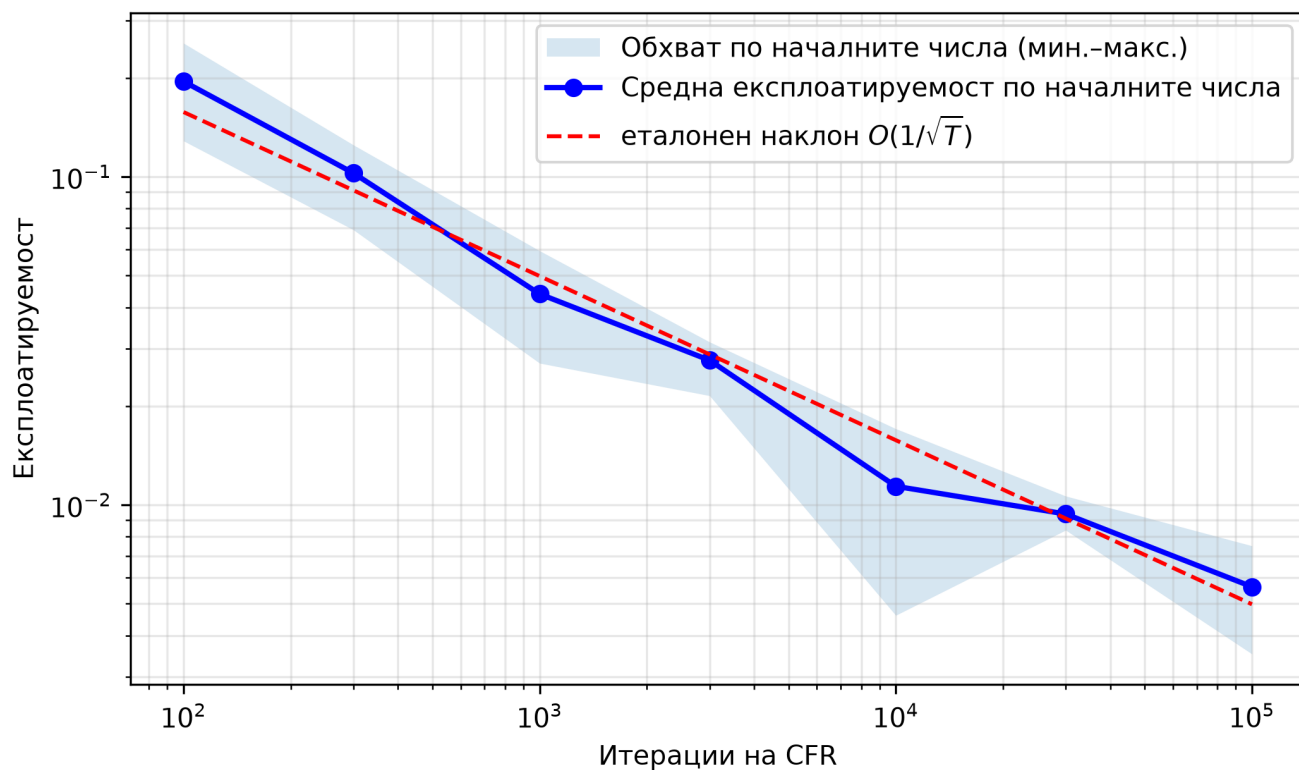
2.11 Емпирични визуализации

За да се потвърдят **теоретичните гаранции** на CFR и приложението му към **Кун покер**, в хода на обучението бяха наблюдавани няколко показателя.

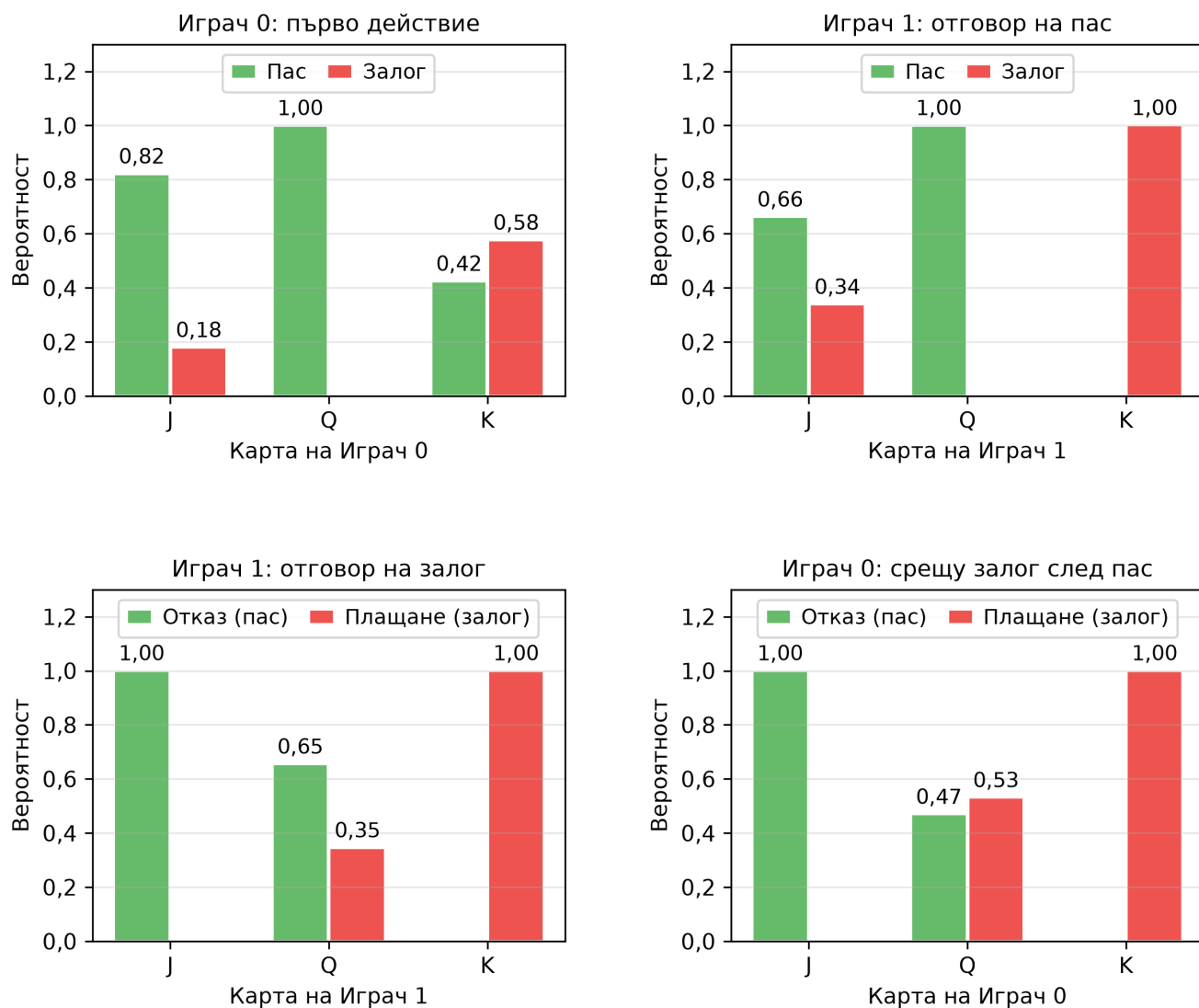


Фигура 3: Средната печалба на Играч 0 при изтеглените раздавания, усреднена от началото на обучението, се колебае около теоретичната стойност $-1/18$ в рамките на шума от извадката. Точната стойност на средната стратегия след 100 000 итерации е $-0,0555$.

³¹Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold’em poker is solved.” *Science*, 347(6218), 145–149. С CFR+ хедс-ъп лимит тексаски холдем е решен по същество (в слаб смисъл) – първата нетривиална игра с непълна информация, играна състезателно от хора, която е решена.



Фигура 4: Експлоатируемост на средната стратегия спрямо броя на итерациите в двулогаритмичен мащаб, средно за пет начални числа; оцветената ивица показва обхвата между тях. Измерените точки следват приблизително еталонния наклон $O(1/\sqrt{T})$ (прекъснатата линия), което съответства на теоретичната граница.



Фигура 5: Средната стратегия, получена с CFR за Кун покер след 100 000 итерации: вероятностите за действие във всяко информационно множество. Честотите на блъф с Вале и на плащане с Дама са близки до семейството от равновесия на Наш ($\alpha \approx 0,18$).

3 Глава 3 – Варианти на CFR и методи на Монте Карло

3.1 Защо обикновеният вариант на CFR се проваля при голям мащаб

В Глава 2 беше създаден работещ решавач с обикновения вариант на CFR за Кун покер – 12 информационни множества, 58 възела (30 от тях крайни), решим за милисекунди. Това създава грешно впечатление. Обикновеният вариант на CFR изисква **пълно обхождане на дървото на играта при всяка итерация**, а истинските игри не са като Кун покер. Ледюк покер има 936 информационни множества (~78 пъти повече), а лимитният тексаски холдем – над 10^{14} , като едно-единствено обхождане на дървото му е отнело около час на 4800 процесорни ядра (вж. раздел 3.8).

Глава 3 показва двата взаимно допълващи се механизма за преодоляване на проблема с мащаба, разработени независимо в литературата по теория на игрите и по-късно комбинирани на практика:

1. **CFR+ (Tammelin, 2014)**³² – модификация на обикновения вариант на CFR, която постига значително по-бърза сходимост чрез нулиране на отрицателните съжаления, линейно претеглено усредняване на стратегиите и редуващи се актуализации. CFR+ е алгоритъмът, който „реши“ лимитния тексаски холдем за двама играчи през 2015 г. – първият нетривиален вариант на покер, който по същество беше решен.
2. **Монте Карло CFR (Lanctot и др., 2009)**³³ – вместо да извършва пълно обхождане на дървото на играта при всяка итерация, MCCFR взема проба от малка част от него. Всяка итерация е стотици до хиляди пъти по-евтина, за сметка на по-шумни актуализации. Това е основният подход за игри, при които пълното обхождане е физически невъзможно; другият – абстракцията на играта – е предмет на Глава 4.

За дисертацията тези решавачи изчисляват базовата стратегия на равновесие на Наш – опорната точка „играй на сигурно“, от която адаптивният агент се отклонява въз основа на наблюденията върху опонента (Принос №1). В игрите с размера на Кун и Ледюк това е CFR+ (вж. раздел 3.7), а MCCFR става необходим при по-големите игри. CFR+ прави това изчисление осъществимо върху междинните еталонни игри, използвани в глави 5–8.³⁴

³²Tammelin, O. (2014). “Solving Large Imperfect Information Games Using CFR+.” arXiv:1407.5042. Резултатът за лимитния тексаски холдем за двама играчи е Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold’em poker is solved.” *Science*, 347(6218), 145-149.

³³Lanctot, M., Waugh, K., Zinkevich, M. & Bowling, M. (2009). “Monte Carlo Sampling for Regret Minimization in Extensive Games.” *Advances in Neural Information Processing Systems* 22, 1078-1086.

³⁴Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold’em poker is solved.” *Science*, 347(6218), 145–149. DOI 10.1126/science.1259433. Използва CFR+ за решаването на лимитния тексаски холдем за двама играчи – първата нетривиална игра с непълна информация, играна състезателно от хора, която по същество е решена.

3.2 Търсене с метод Монте Карло в дърво като вдъхновение

Търсене с метод Монте Карло в дърво (MCTS) представлява каноничното семейство алгоритми, които използват **случайна извадка** за оценка на позиции в **дърво на играта**, без да прибегват до изчерпателно изброяване. Философията на проектирането му – „оценявай някои клонове до пълна дълбочина вместо всички клонове до фиксирана дълбочина“ – е пряка мотивация за MCCFR.

MCTS изгражда асиметрично **дърво на търсенето** чрез четири повтарящи се фази:

1. **Подбор** – слизване от корена чрез стратегия (обикновено UCB1, горна граница на доверителен интервал), която балансира експлоатацията на известни добри ходове и изследването на недостатъчно посетени такива.
2. **Разширяване** – във възел се добавят един или повече дъщерни възли към дървото.
3. **Симулация (разгръщане)** – от новия възел се играе случайна игра до нейния край.
4. **Обратно разпространение** – разпространяване на крайния резултат обратно по посещения път, като се актуализират броят посещения и кумулативните награди.

След множество итерации дъщерният възел на корена с най-голям брой посещения се избира за най-добър ход. Насочвани от стратегията за подбор, разгръщанията, обобщени в хиляди итерации, дават все по-точни оценки на стойността – дори без никакви експертни познания извън правилата на играта.

Класически алтернативи като **минимакс с алфа-бета подрязване** оценяват всеки релевантен възел до фиксирана дълбочина и след това прилагат евристика в крайните възли. И двата подхода имат своите области на приложение:

Таблица 1: Минимакс с алфа-бета отсичане срещу MCTS по пет характеристики.

Свойство	Минимакс + α - β	MCTS
Покритие на дървото	Всички разклонения до дълбочина d	Подбрани разклонения до крайно състояние
Оценка	Евристика при границата на дълбочината	Точна (крайна печалба) или чрез невронна мрежа
Чувствителност към коефициента на разклоняване	Експоненциална: $O(b^d)$	Умерена: съсредоточава се върху обещаващи разклонения
Необходими експертни познания	Силна оценъчна функция	Никакви (случайно разгръщане) или придобити чрез обучение
Най-подходящ за	Умерен коефициент на разклоняване, добри евристики	Висок коефициент на разклоняване, слаби/никакви евристики

За Го (коефициент на разклоняване ~250, без добра оценъчна функция преди невронните мрежи) минимакс е непрактичен. MCTS беше пробивът, който направи изкуствения интелект за Го конкурентоспособен; същият принцип „извадка вместо изброяване“ се пренася чрез MCCFR към дърветата на игрите с непълна информация.³⁵

3.3 Марковски вериги и законът за големите числа

„Метод Монте Карло“ в MCCFR обозначава широко семейство изчислителни методи, които използват **повтаряща се случайна извадка** за получаване на числени резултати. Методът е създаден в Лос Аламос през 1946 г., където Станислав Улам и Джон фон Нойман го прилагат за симулиране на дифузията на неутрони – задача, твърде сложна за аналитично решаване; наименованието, предложено от Никълъс Метрополис, идва от казиното в Монте Карло³⁶.

Математическата основа се крепи на **марковски вериги** – стохастични процеси, при които следващото състояние зависи единствено от текущото състояние и не от историята на достигането му (марковско свойство). Обхождането на дърво на играта представлява марковска верига: от всяко **състояние на играта** преходът зависи само от текущото състояние и избраното действие.

Ключовата теоретична гаранция е **законът за големите числа**: ако се изтеглят независимо достатъчно много траектории, средната стойност на извадката клони към истинската очаквана стойност. За MCCFR, при фиксиран профил от стратегии:

$$\frac{1}{T} \sum_{t=1}^T \hat{v}_I^{(t)} \xrightarrow{T \rightarrow \infty} v_I$$

където $\hat{v}_I^{(t)}$ представлява извадената контрафактична стойност в информационно множество I на итерация t , а v_I е истинската стойност, която обикновеният вариант на CFR изчислява точно. Тези извадкови стойности са **неизместени оценки** – очакваната им стойност съвпада с истинската. Заедно с ограничение отдолу на вероятностите за извадка това осигурява, с вероятност поне $1 - p$, същото намаляване на средното съжаление от порядъка $O(1/\sqrt{T})$ като при CFR с пълно обхождане и следователно сходимост на средната стратегия към приблизително равновесие на Наш³³.

Цената е **дисперсия**. Отделните извадки могат значително да се отклоняват от истинската стойност, което налага повече итерации за постигане на същата прецизност. Този компромис между дисперсия и скорост е основната тема на главата.³⁷

³⁵Browne, C. et al. (2012). “A Survey of Monte Carlo Tree Search Methods.” *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43.

³⁶Metropolis, N. (1987). “The Beginning of the Monte Carlo Method.” *Los Alamos Science*, 15, 125ff. Първата публикация е Metropolis, N. & Ulam, S. (1949). “The Monte Carlo Method.” *JASA*, 44(247), 335–341.

³⁷Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, второ издание. MIT Press. Гл. 1 (областта); гл. 3 (крайни марковски процеси на вземане на решения); гл. 4 (динамично програмиране); гл. 5 (методи на Монте Карло);

3.4 Ледюк покер – по-голяма еталонна игра

Кун покер улавя същността на непълната информация в най-малкото възможно дърво на играта. Ледюк покер увеличава мащаба с приблизително два порядъка – все още решим точно, но достатъчно голям, за да станат значими разликите между алгоритмите:

Таблица 2: Сравнение между Кун и Ледюк покер: карти, кръгове на залагане, информационни множества и размер на дървото.

Свойство	Кун покер	Ледюк покер
Карти	3 (J, Q, K)	6 ($\{J, Q, K\} \times \{\spadesuit, \heartsuit\}$)
Кръгове	1	2 (префлоп + обща карта)
Обща карта	Няма	1 разкрита между кръговете
Класиране на ръка	Само висока карта	Чифт (частна = обща) бие висока карта
Размери на залози	Фиксирани (1 жетон)	Променливи (2 в първи кръг, 4 във втори кръг)
Максимален брой вдигания в кръг	1	2
Информационни множества	12	936
Възли на игрово дърво	58	10 200
Случайни изходи	6	120

Появяват се три качествено нови характеристики:

- **Многорундов строеж** – информацията се променя между кръговете (разкриване на обща карта), което изисква стратегии, адаптиращи се към новата информация.
- **Чифтове** – класирането на ръката вече зависи от общата карта; вале може да стане най-силната ръка, ако и общата карта е вале.
- **По-големи залози в по-късните кръгове** – вдигането с 4 жетона във втория кръг означава, че решенията в по-късните кръгове имат по-голямо тегло.

Въпреки че е приблизително 78 пъти по-голям от **Кун**, **Ледюк** остава достатъчно малък за точно изчисление (пълно обхождане на дърво отнема около 50 ms). Това го прави идеална еталонна игра: достатъчно голяма, за да бъдат значими разликите в производителността, и достатъчно малка, така че всички алгоритми да могат да достигнат почти пълна сходимост в рамките на минути.³⁸

гл. 6 (обучение с темпорална разлика). <http://incompleteideas.net/book/the-book-2nd.html>

³⁸Southey, F. et al. (2005). "Bayes' Bluff: Opponent Modelling in Poker." *UAI*. arXiv:1207.1411.

3.5 CFR+ – Трикът с нулирането на отрицателните съжаления

CFR+ въвежда три малки изменения в обикновения вариант на CFR, всяко от които е лесно за реализиране, но води до съществени резултати.

Нулиране на отрицателните съжаления. При обикновения вариант на CFR кумулативното съжаление може да става произволно отрицателно:

$$R^{T+1}(I, a) = R^T(I, a) + r^T(I, a)$$

CFR+ задава долна граница на съжаленията при нула след всяка итерация:

$$R^{T+1}(I, a) = \max(R^T(I, a) + r^T(I, a), 0)$$

Това предотвратява натрупването на големи отрицателни съжаления за дадено действие, които биха отнели множество итерации, за да се „изплатят“, преди действието отново да бъде разглеждано. Аналогията с активационната функция ReLU в невронните мрежи е пряка: и двете занулават отрицателните стойности.

Линейно претеглено усредняване на стратегиите. Обикновеният вариант на CFR включва стратегията от всяка итерация с еднакво тегло в текущата средна. При CFR+ стратегията от итерация t получава тегло t :

$$\bar{\sigma}^T(I, a) = \frac{\sum_{t=1}^T t \cdot \sigma^t(I, a)}{\sum_{t=1}^T t}$$

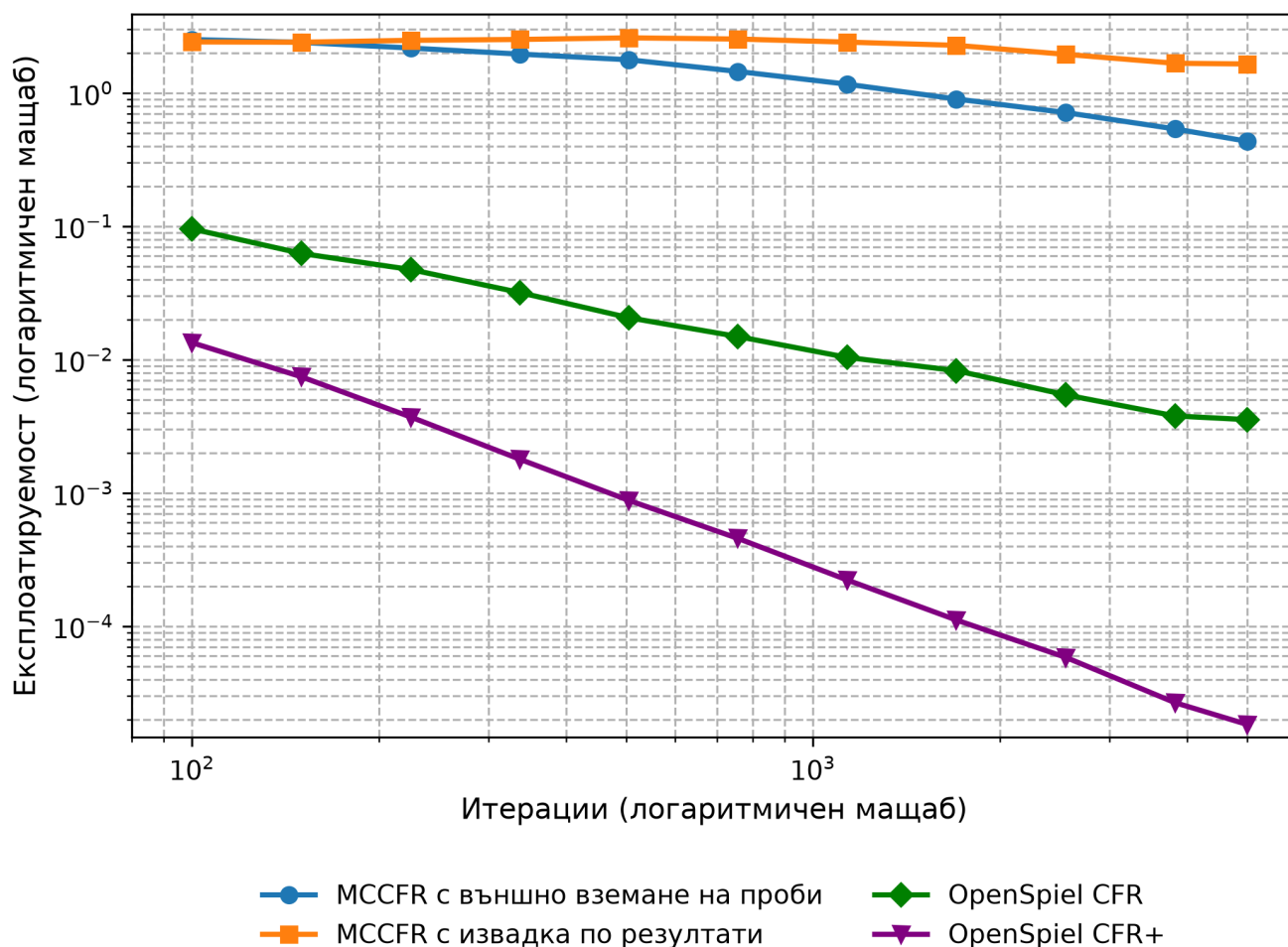
По-късните итерации – които се характеризират с по-малко съжаление и по-добри стратегии – оказват по-голямо влияние върху средната стойност. Ранните, шумни итерации получават по-ниска тежест.

Редуващи се актуализации. Вместо двамата играчи да се обновяват едновременно спрямо един и същ профил от стратегии, CFR+ ги обновява последователно в рамките на всяка итерация, така че обновяването на втория играч вече отчита новата стратегия на първия. Работата за една итерация не се променя; ползата е по-бързата сходимост.

Комбиниран ефект: емпиричната сходимост се подобрява от $O(1/\sqrt{T})$ до приблизително $O(1/T)$. При Ледюк CFR+ достига $2,0 \times 10^{-4}$ след около 1100 итерации; при измерената скорост на обикновения вариант на CFR за същата стойност биха били нужни от порядъка на два милиона итерации (екстраполация). Именно това направи възможно решаването на лимитния тексаски холдем за двама играчи през 2015 г. Забележка: скоростта $O(1/T)$ се наблюдава последователно,

но не е доказана; доказана е само общата за семейството горна граница $O(1/\sqrt{T})$ ³⁹, чието първоначално доказателство е коригирано от Burch и др.⁴⁰

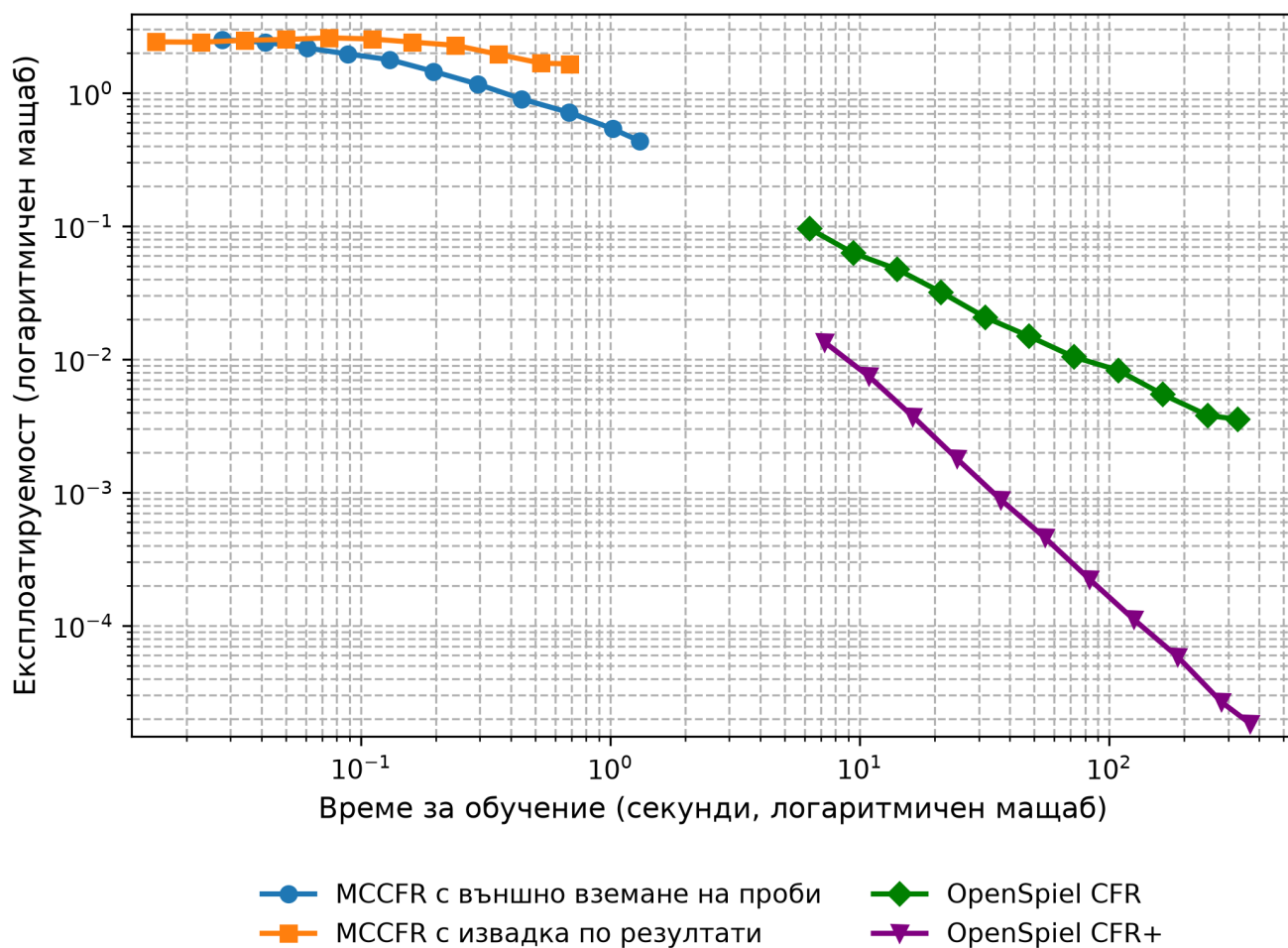
Тук експлоатируемост означава средната печалба на най-добрия отговор срещу двамата играчи, $(BR_0 + BR_1)/2$; NashConv е сумата им, т.е. двойно по-голяма. Емпирично, при Ледюк след 5000 итерации реализацията на CFR+ в OpenSpiel достига експлоатируемост $\approx 1,8 \times 10^{-5}$, докато обикновеният вариант на CFR все още е около $3,6 \times 10^{-3}$ – подобрение приблизително 190 пъти, постигнато с две от трите модификации (и двата решавача на OpenSpiel редуват актуализациите).³²



Фигура 6: Ледюк покер: експлоатируемост спрямо броя на итерациите при четирите решавача на OpenSpiel

³⁹Tammelin, O., Burch, N., Johanson, M. & Bowling, M. (2015). "Solving Heads-Up Limit Texas Hold'em." *Proc. IJCAI*, 645–652.

⁴⁰Burch, N., Moravčík, M. & Schmid, M. (2019). "Revisiting CFR+ and Alternating Updates." *Journal of Artificial Intelligence Research*, 64, 429–443. DOI 10.1613/jair.1.11370.



Фигура 7: Ледюк покер: експлоатируемост спрямо времето за обучение при четирите решаваща на OpenSpiel

3.6 MCCFR – Извадка от дървото на играта

MCCFR заменя пълното обхождане на дърво с частично извадково обхождане. Два варианта се разполагат върху една и съща крива дисперсия–скорост в различни точки:

Таблица 3: Външно вземане на проби и извадка по резултати: от какво взема извадка всеки вариант при всеки тип възел и каква е цената на една итерация.

Вариант	Възли на случайността	Възли на обхождащия играч	Възли на опонента	Цена на итерация
Външно вземане на проби	Извадка от едно раздаване	Разглеждане на всички действия	Извадка от едно действие	~42 възела (~945× по-бързо от пълното)
Извадка по резултати	Извадка от едно раздаване	Извадка от едно действие (ε-микс с текущата стратегия)	Извадка от едно действие	~5,5 възела (~2 228× по-бързо)

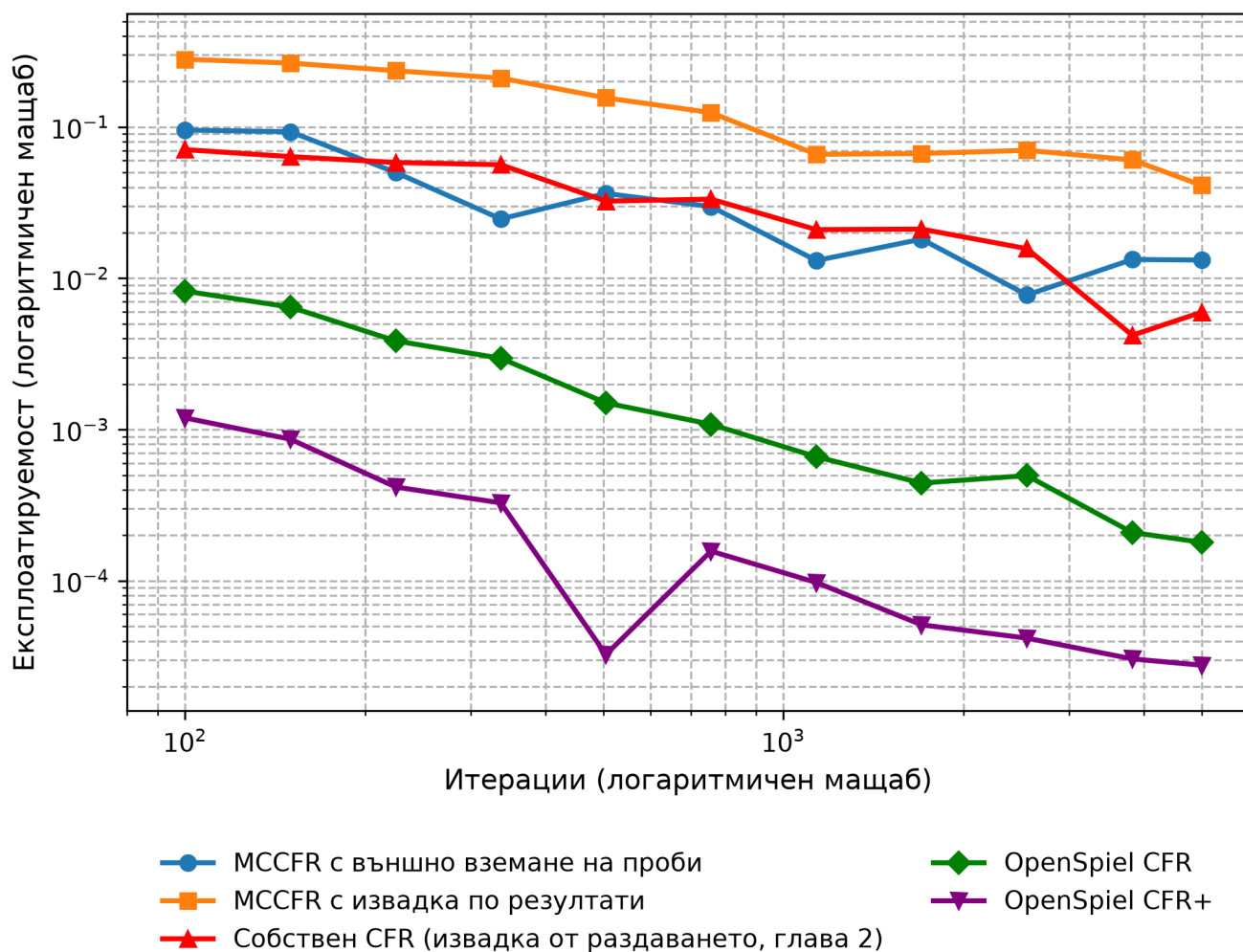
Външно вземане на проби разглежда всички възможни действия във възлите на играча, който актуализира стратегията си, но взема извадка във възлите на случайността и във възлите на противника. Съжаленията на обхождащия играч се обновяват въз основа на поддървото от извадката. Тъй като се посещава само едно раздаване и един път на опонента, всяка итерация обхваща малка част от дървото.

Извадката по резултати отива още по-далеч: тегли се една-единствена траектория до край. Тъй като действията във възлите на играча, който актуализира стратегията си, също се изваждат, актуализацията на съжалението трябва да бъде коригирана чрез **извадка по важност** – отношението между истинската вероятност за достигане и вероятността за извадка. Един ε-микс (с вероятност ε действието се избира равномерно, а в противен случай – според текущата стратегия) гарантира, че всички действия се разглеждат, дори когато текущата стратегия им присвоява нулева вероятност.

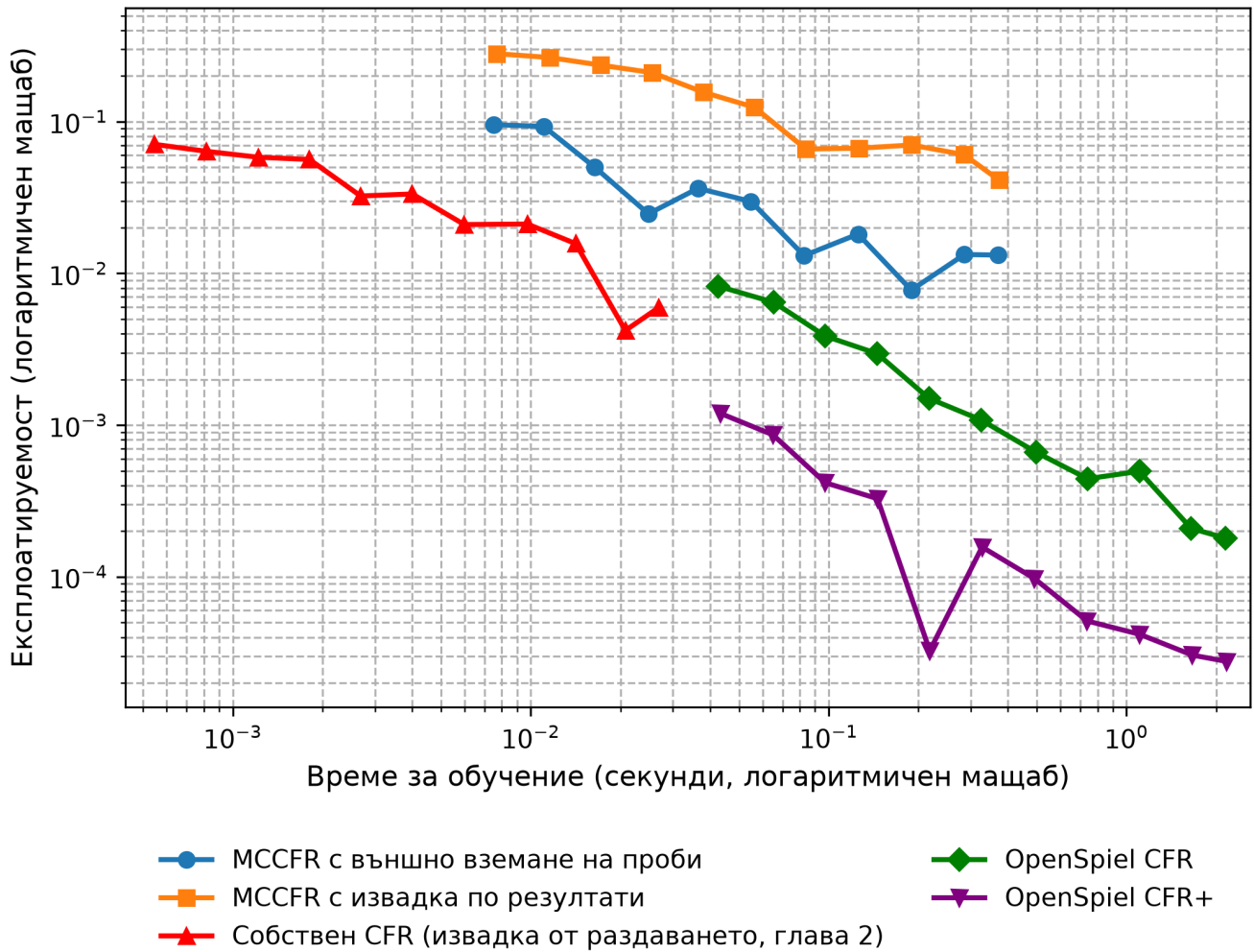
Критичното теоретично свойство и за двата подхода е, че извадковите контрафактни стойности са **неизместени оценки** на действителните стойности (Lanctot и др., лема 1). Заедно с техните граници за съжалението (теореми 4 и 5) това гарантира сходимост към приблизително равновесие на Наш въпреки шума от извадката. Цената, както винаги при метода Монте Карло, е дисперсията. Всяка извадкова актуализация се отклонява от действителната контрафактична стойност, тъй като отразява само едно възможно раздаване, а не очакваната стойност за всички раздавания.

Върху 12-те информационни множества на Кун покер всички алгоритми се изпълняват за секунди, а подредбата им вече е същата като при Ледюк: след 5000 итерации CFR+ достига експлоатируемост $\approx 3 \times 10^{-5}$, обикновеният CFR – $\approx 2 \times 10^{-4}$, а външното вземане на проби и извадката по резултати са едва при $\approx 0,013$ и $\approx 0,04$ (решавачът от Глава 2, който тегли по едно раздаване на

итерация, е между обикновения CFR и външното вземане на проби с $\approx 0,006$):



Фигура 8: Кун покер: експлоатируемост спрямо броя на итерациите



Фигура 9: Кун покер: експлоатируемост спрямо времето за обучение

За да се прояви компромисът, дървото на играта трябва да бъде достатъчно голямо, така че разходът на итерация да има значение. Именно затова е необходим **Ледюк**.³³

3.7 Компромисът между дисперсия и скорост

И четирите алгоритъма имат гарантирана горна граница на експлоатируемостта от порядъка $O(1/\sqrt{T})$ - за CFR⁴¹, за MCCFR с вероятност поне $1 - p$ ³³ и за CFR+^{39,40}. Измерената скорост обаче се различава: двата варианта на MCCFR следват наклон около $-0,4$ до $-0,5$, обикновеният CFR - около $-0,8$, а CFR+ - около $-1,8$. Затова моделът $\epsilon(T) = C/\sqrt{T}$ по-долу е приближение (при обикновения CFR произведението $\epsilon\sqrt{T}$ намалява от 0,94 до 0,27), а за CFR+ не е приложим. Преобразуването към време за изпълнение става чрез $C_w = C/\sqrt{\text{скорост}}$:

⁴¹Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2007). "Regret Minimization in Games with Incomplete Information." *Advances in Neural Information Processing Systems* 20.

Таблица 4: Измерени константи на сходимост за четирите решавача – на итерация и по време на изпълнение.

Алгоритъм	$C_{\text{итер}}$	Скорост (ите- рации/сек)	C_w (по време на изпълнение)	C_w спрямо обикновения CFR
Обикновен вариант на CFR	0,34	20,6	0,075	1,0×
CFR+	–	20,6	–	не следва $C/\sqrt{T}$ (вж. текста)
MCCFR, външно вземане на проби	107	19 470	0,767	10,2× ($\approx 105 \times$ повече време)
MCCFR, извадка по резултати	245	45 901	1,143	15,3× ($\approx 233 \times$ повече време)

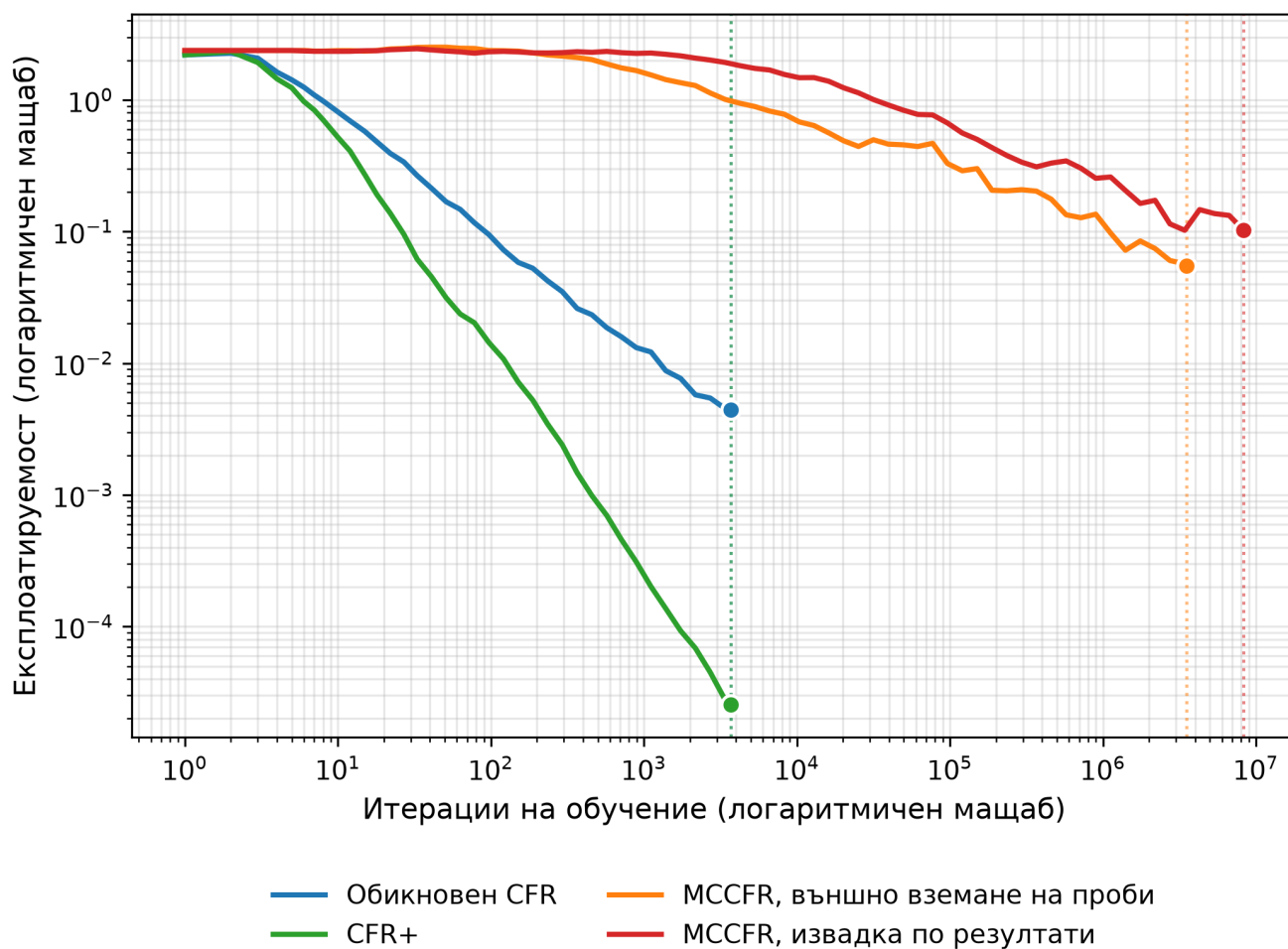
Константата по време на изпълнение определя представянето на практика. Въпреки че външно-то вземане на проби е 945 пъти по-бързо на итерация, неговата константа на дисперсията (107 спрямо 0,34) е 315 пъти по-голяма. Предимството в скоростта **не компенсира наказанието от дисперсията**:

$$\frac{99,000 \times \text{повече итерации}}{945 \times \text{по-бърза итерация}} \approx 105 \times \text{повече време за изпълнение}$$

Защо се наблюдава дисперсия? Обикновеният вариант на CFR изчислява **точни** контрафактични стойности чрез изброяване на всички 120 раздавания. Неговото обновяване на съжалението има нулева дисперсия. MCCFR взема проба от едно раздаване на итерация. Дори ако обходи цялото поддърво за това раздаване, пак ще се наблюдава дисперсия между различните раздавания – следствие от извадката на случайното събитие (раздаването), а не от начина на обхождане в рамките на едно раздаване:

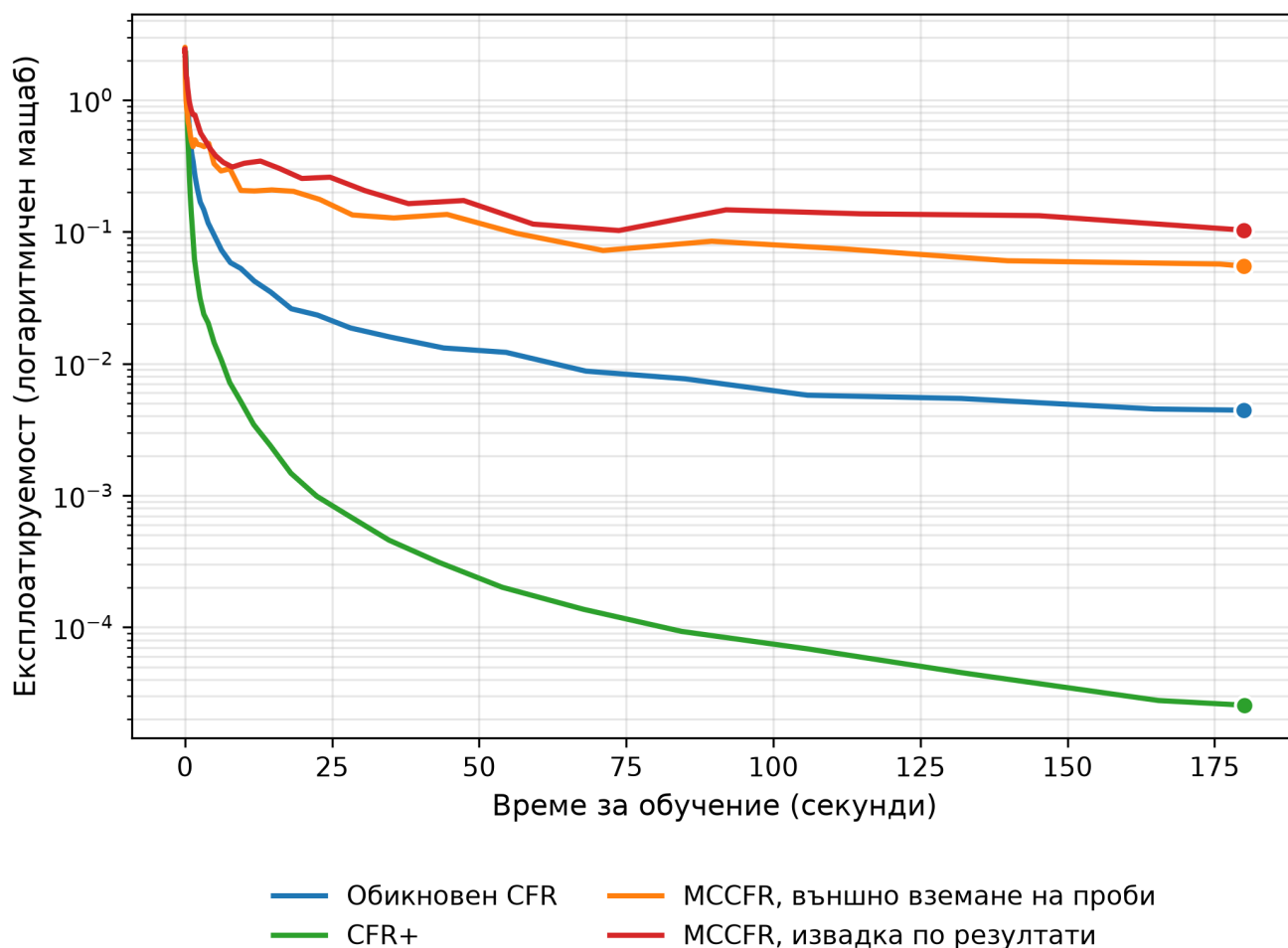
$$\text{Var}[\hat{v}_I] = \mathbb{E}_{\text{раздаване}} [(\hat{v}_I - v_I)^2]$$

Сравнителен тест с бюджет от 180 секунди за всеки от четирите варианта върху Ледюк прави сравнението по време на изпълнение конкретно. Изглед по итерации (вариантите с извадка извършват милиони обновявания, а вариантите с пълно обхождане – само няколко хиляди):



Фигура 10: Ледюк покер, собствени реализации с бюджет от 180 s за всеки алгоритъм: експлоатируемост спрямо броя на итерациите

Изглед по време на изпълнение – справедливото сравнение (при еднакви изчислителни разходи кой алгоритъм дава най-добрата стратегия?):



Фигура 11: Ледюк покер, собствени реализации с бюджет от 180 s за всеки алгоритъм: експлоатируемост спрямо времето за изпълнение

CFR+ достига експлоатируемост от $2,6 \times 10^{-5}$ – практически точно равновесие на Наш – за 3 минути. Обикновеният вариант на CFR следва с $4,4 \times 10^{-3}$. Двата варианта на MCCFR изостават 12–23 пъти от обикновения CFR и три до четири порядъка от CFR+ при този размер на играта.

3.8 Кога MCCFR надделява – точката на пресичане

Тази закономерност поставя очевиден въпрос: ако MCCFR винаги е по-слаб в Ледюк, защо тогава беше изобретен? Отговорът се крие във факта, че разходът на итерация при обикновения вариант на CFR нараства линейно с размера на дървото $|N|$, докато този на MCCFR расте много по-бавно (при външното вземане на проби – приблизително като $\sqrt{|N|}$ ³³). При достатъчно голям мащаб линейната цена надделява. Критичният размер на дървото е:

$$|N|_{\text{пресичане}} = |N|_{\text{Ледюк}} \cdot \left(\frac{C_{mc}}{C_v} \right)^2 \cdot \frac{\text{скорост}_v}{\text{скорост}_{mc}}$$

При заместване на измерените стойности:

Таблица 5: Размер на играта, при който MCCFR изпреварва обикновения вариант на CFR, по вариант на извадка-та.

Вариант	$ N _{\text{пресичане}}$
Външно вземане на проби	~1,1 милиона възела
Извадка по резултати	~2,4 милиона възела

Сравнение с реални игри:

Таблица 6: Прагът на пресичане, приложен към три реални игри.

Игра	$ N $	Печели ли външното вземане на проби?	Печели ли извадката по резултати?
Ледюк покер	10 200	Не (105 пъти под прага)	Не (233 пъти под прага)
Лимитен тексаски холдем за двама играчи	$\sim 3 \times 10^{17}$	Да по модела ($\sim 10^{11}$ пъти над прага), но е решен с CFR+ с пълно обхождане	Да по модела
Безлимитен тексаски холдем	до $\sim 6 \times 10^{164}$	Да	Да

Ледюк е 105 пъти под прага за външното вземане на проби и 233 пъти под прага за извадката по резултати. Пълното обхождане доминира, тъй като цялото дърво на играта се побира в паметта и може да бъде изброено за милисекунди. Оценката е от порядъка на величината и важи за тази реализация: константите са измерени само върху Ледюк, обикновеният CFR е моделиран като $C/\sqrt{T}$, въпреки че се сближава по-бързо, а ако цената на итерация на MCCFR расте като $\sqrt{|N|}$, същите данни дават праг около 10^8 възела.

При реалните мащаби на покера заключението се обръща, но не навсякъде. Лимитният тексаски холдем за двама играчи ($3,16 \times 10^{17}$ състояния) все пак е решен с CFR+ с пълно обхождане – 1579 итерации по около 61 минути на 4800 процесорни ядра⁴⁴ – като авторите отчитат, че CFR+ изисква значително по-малко изчисления от вариантите на CFR с извадка. Безлимитният тексаски холдем (до $6,31 \times 10^{164}$ състояния във форматите на ACPC⁴²) е недостижим за пълно обхождане; там прак-

⁴²Johanson, M. (2013). “Measuring the Size of Large No-Limit Poker Games.” arXiv:1302.7008.

тическият път е извадката – MCCFR в комбинация с абстракция и с разширения за намаляване на дисперсията⁴³.

3.9 Връзки с Глава 2 и бъдещи насоки

Същият принцип, различен режим. В Глава 2 беше реализиран CFR с пълно обхождане за Кун и беше показано, че минимизирането на съжалението локално във всяко информационно множество води глобалната стратегия към равновесие на Наш. Глава 3 запазва този принцип, но въвежда две независими усъвършенствания: CFR+ променя начина на акумулиране на съжаленията (нулиране на отрицателните съжаления + линейно претегляне), а MCCFR модифицира начина на измерването им (извадкова оценка вместо точна очаквана стойност). И двете все още извеждат **средната** стратегия, а не последната итерация – същият механизъм за усредняване, който стабилизира обикновения вариант на CFR, стабилизира всеки негов вариант.

Скорости на сходимост. Глава 2 измери наклон в лог-лог мащаб, близък до $-0,5$, за експлоатируемостта на играта Кун, потвърждавайки $O(1/\sqrt{T})$. В Глава 3 се вижда, че $O(1/\sqrt{T})$ е горна граница за цялото семейство, а не наблюдаваната скорост: върху Ледюк двата варианта на MCCFR следват наклон около $-0,4$ до $-0,5$, обикновеният CFR се сближава по-бързо (около $-0,8$), а CFR+ – значително по-бързо (около $-1,8$), което не е формално доказано. MCCFR остава със скорост $O(1/\sqrt{T})$, но с драстично по-голяма константа.

Напред:

- **Глава 4 (Абстракция на играта и мащабиране)** – атакува проблема с мащабируемостта от противоположната посока: вместо да се прави извадка от дървото, самото дърво се редуцира чрез абстракция на състоянията и действията. В комбинация с MCCFR прави решаването на реалния покер изчислително постижимо.
- **Глава 5 (Невронни мрежи за игри с непълна информация)** – заменя табличното съхранение на стратегията с невронни апроксиматори на функции, използвайки рамката за извадка на MCCFR като двигател за генериране на данни. Свойствата на дисперсията, установени тук, обясняват защо Деер CFR използва външно вземане на проби и защо неговият наследник DREAM, който преминава към извадка по резултати, добавя обучена базова стойност за намаляване на дисперсията.
- **Глава 6 (Архитектури на игрови изкуствен интелект от край до край)** – надгражда както върху CFR+, така и върху MCCFR за създаване на пълноценни агенти. Реализацията на Ледюк, разработена за тази глава, остава еталонната среда в следващите глави.⁴⁴

⁴³Schmid, M., Burch, N., Lanctot, M., Moravčík, M., Kadlec, R. & Bowling, M. (2019). “Variance Reduction in Monte Carlo Counterfactual Regret Minimization (VR-MCCFR) for Extensive Form Games Using Baselines.” *Proc. AAAI*, 33(01), 2157–2164.

⁴⁴Brown, N. & Sandholm, T. (2017). “Safe and Nested Subgame Solving for Imperfect-Information Games.” *NeurIPS* – свързва табличния CFR с декомпозицията на под-игра за реалния покер.

4 Глава 4 — Абстракция на играта и мащабиране на игри с непълна информация

4.1 Защо е необходима абстракцията

Глави 1–3 изградиха алгоритмите (DQN/PPO; обикновен вариант на CFR; CFR+ и MCCFR външен/изходен). Всеки от тях беше демонстриран върху игра с достатъчно малък размер, за да може дървото на играта ѝ да бъде изброено точно: Кун (12 информационни множества) и Ледюк (936). Този набор от инструменти е завършен, но той функционира единствено когато цялото дърво на играта се побира в паметта.

Размерът на дървото на играта е произведение от три фактора: (а) броят на различните скрити състояния, които възелът на случайността може да генерира, (б) коефициентът на разклоняване във всяка точка за вземане на решение и (в) дълбочината (решения до терминал). Всеки от тези три фактора нараства независимо:

- **Скрити състояния** — В тексаски холдем се раздават по 2 скрити карти от 52, след което до 5 общи карти. Само комбинациите от скрити и общи карти са около $2,8 \times 10^9$, още преди да се отчете историята на залозите; с нея лимитният вариант за двама играчи достига $3,16 \times 10^{17}$ състояния.⁴⁵
- **Коефициент на разклоняване** — Безлимитният покер позволява всякакъв размер на залога от „минимално вдигане“ до „ол-ин“. Дори дискретизирането до няколко размера увеличава разклоняването на възел от 3 (отказ/плащане/вдигане в лимитния вариант) до 5–10.
- **Дълбочина** — Множество рундове на залагане, всеки от които потенциално с няколко вдигания, се умножават.

Броят на информационните множества в безлимитния тексаски холдем за двама играчи е $\sim 10^{161}$ — около 10^{80} пъти повече от броя на атомите в наблюдаемата вселена. Нито един от алгоритмите от глава 3 не може да работи върху това дърво.⁴⁶

Глава 4 представлява мостът между „игри, които можем да изброим изцяло“ и „игри, които не можем“. Механизмът е **абстракция**: умишлено свеждане на части от играта до по-опростен вид, така че същите алгоритми да могат да се прилагат върху по-малък, структурно опростен заместител и да се измери цената на това в качеството на стратегията. В края на тази глава резултатът включва количествен отговор на централния въпрос на всеки практически изкуствен интелект за покер след 2007 г.:

⁴⁵Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold’em poker is solved.” *Science*, 347(6218), 145–149. DOI 10.1126/science.1259433. Използва CFR+ за решаването на лимитния тексаски холдем за двама играчи – първата нетривиална игра с непълна информация, играна състезателно от хора, която по същество е решена.

⁴⁶Johanson, M. (2013). “Measuring the Size of Large No-Limit Poker Games.” Технически доклад, University of Alberta; arXiv:1302.7008.

До каква степен може да се намали размерът на играта, преди равновесната стратегия на абстрактната игра да престане да бъде добра стратегия в реалната игра?

Този отговор се представя под формата на крива на Парето: по едната ос е размерът на **абстрахираната** игра, а по другата – **разликата в експлоатируемостта**, т.е. с колко абстрактната стратегия е по-лоша от точното равновесие на Наш в реалната игра. И двете метрики са въведени формално по-долу.

Рецептата е една и съща за всеки подход: *изгражда се по-малка игра, чиито стратегии могат да бъдат трансформирани в приложими стратегии за реалната игра, изпълняват се алгоритмите от глава 3 върху тази по-малка игра и след това се оценяват загубите*. Тази рецепта може да се следва по два пътя.

4.2 Пътища към абстракцията

В тази глава думата „абстракция“ се използва в два оперативно различни смисъла; в литературата невронното приближение обикновено се представя като алтернатива на абстракцията, а не като неин вид.⁴⁷ И двете намират приложение в съвременния игрови изкуствен интелект и ще бъдат разгледани в настоящата дисертация; смесването им представлява категориална грешка. Настоящият раздел очертава тази граница. Всичко, което следва, реализира **явния** път от край до край; неявните съответствия заемат същите концептуални позиции, но тъй като в тази глава не се извършва работа с тях, те са събрани в отделен раздел към края на главата („Аналози на дълбокото обучение с подкрепление“), вместо да бъдат разпределени между останалите.

И двата пътя споделят една и съща **интуиция** — компресиране със запазване на стойността — и тази интуиция се изразява най-добре чрез целевата функция на информационното стеснение (по аналогия с едноименния метод⁴⁸), като β е обменният курс между паметта и стойността:

$$\mathcal{L} = \underbrace{\text{Сложност}(Z)}_{\text{цена в памет}} - \beta \cdot \underbrace{\text{Стойност}(\pi_Z)}_{\text{стратегическа стойност}}$$

Когато $\beta = 0$, алгоритъмът компресираща цялата игра до едно състояние и играе изключително слабо. Когато $\beta \rightarrow \infty$, той отказва всякакво сливане и използва пълната игра. Всеки алгоритъм в конвейера за абстракции представлява конкретна настройка на този регулатор:

- **Беззагубно сливане** — граничният случай $\beta \rightarrow \infty$: извършва сливане само когато $\text{Стойност}(\pi_Z) = \text{Стойност}(\pi)$ е точно.
- **Ограничена загуба** — средата на кривата с гаранция: избира се β така, че да бъде изпълнено условието $|\text{Стойност}(\pi_Z) - \text{Стойност}(\pi)| \leq \varepsilon_{\text{abs}}$.

⁴⁷Brown, N., Lerer, A., Gross, S. & Sandholm, T. (2019). “Deep Counterfactual Regret Minimization.” *ICML*.

⁴⁸Tishby, N., Pereira, F. C. & Bialek, W. (1999). “The Information Bottleneck Method.” arXiv:physics/0004057.

- **Емпирична сходност (HSD + разстояние на Васерщайн)** — измерва самата крива: разстоянието на Васерщайн между абстрактното и пълното разпределение служи като заместител на члена за загуба на стойност.
- **Корекция по време на изпълнение** — ортогонален подход: приема се загубата на стойност, произтичаща от избраното β , след което тя се компенсира по време на изпълнение чрез решаване на под-игра.

Съпоставени един до друг, двата пътя се различават по всеки практически показател:

Таблица 7: Явният и неявният път към абстракцията, сравнени по всеки практически показател.

Аспект	Явен / табличен	Неявен / в стил IB (дълбоко обучение с подкрепление)
Къде се намира компресията	Разбиване на информационните множества / краен избран набор от действия	Непрекъснат латентен вектор $z = f_{\theta}(s)$
Кой я извършва	Правило, зададено от човек, или клъстеризация по признаци на ръката	Оптимизаторът, чрез градиентен спуск върху загуба
Фактор за настройка	k в k -средни, наборът от размери на залога, правилото за изоморфизъм на боите	β умножено по $I(S; Z)$ в загубата
Гаранция	Формална граница на разликата в експлоатируемостта само за беззагубни и ограничени абстракции; за емпиричните (k -средни + EMD) – никаква	Информационнотеоретична граница върху $I(S; Z)$; няма теорема за запазване на равновесието на Наш
Кога се изчислява	Преди решаването — фиксиран вход към CFR/MCCFR	По време на решаването — мрежата е стратегията
Тип на изхода	Дискретен идентификатор на клъстер за всяко информационно множество	Реален вектор
Къде се появява в тази дисертация	Тази глава (4)	Глава 5 (Деер CFR), глава 6 (от край до край), глава 12 (модели на последователности)

4.3 Видове оси на абстракцията

Всяка конкретна техника за абстракция се отнася към една от двете взаимно ортогонални оси — *кои състояния агентът разглежда като идентични* (информационна абстракция) и *кои действия агентът обмисля* (абстракция на действията). Трета ос, *усъвършенстване по време на изпълнение*, е независима от двете и има собствен раздел (Корекция по време на изпълнение, по-долу).

4.3.1 Информационна абстракция

Групират се информационните множества, които агентът ще разглежда като едно и също състояние. Това е централната операция в конвейера — всяка фаза – от критерия за сливане до решенията по време на компилация – е посветена на *как* и *кога* да се извършват сливания на информация.

Затова тази подсекция е умишлено кратка: тук се дава само *наименованието* на информационната абстракция, а нейното подробно разглеждане следва в следващите раздели („Критерият за сливане“, „Бюджетът на грешките“, „Решения по време на компилация“). За разлика от нея, абстракцията на действията е разгледана изцяло по-долу.

Бележка относно режимите. При явна информационна абстракция в обикновения вариант на CFR съществува тънко разграничение между абстрахирането (а) на **картата на възлите** (само памет) и (б) на **обхождането** (време за изпълнение на итерация). Само второто скъсява времето за изпълнение.

Запомнете: информационната абстракция определя кои скрити ситуации агентът разглежда като еднакви.

4.3.2 Абстракция на действията и проблемът с трансляцията

Ограничава се множеството от действия, които решаващът разглежда, изпълнява се CFR върху получената ограничена игра и след това се обработва всичко, което реалният опонент направи извън това ограничение. Разграничават се два режима:

- **Дискретни множества действия.** Базовото множество вече е крайно. Абстракцията означава *подрязване* (премахване на доминирани или незначителни действия преди решаването) или *макродействия* (групиране на последователности от примитиви в едно абстрактно действие, т.е. времева абстракция). Транслацията обикновено е тривиална: ако абстрактното множество действия е подмножество на допустимите действия, собствените ходове на агента винаги са в рамките на абстракцията.
- **Непрекъснати множества действия.** Базовото множество е неизброимо (размери на залозите в безлимитния покер, въртящи моменти в ставите при непрекъснато управление). За табличните методи абстракцията на действията е *задължителна* — CFR не може да се изпълнява върху непрекъснатото дърво, без първо да бъде сведено до краен заместител (типичен на-

бор от действия в покера: {отказ, плащане, $0,5 \times \text{пот}$, $1 \times \text{пот}$, $2 \times \text{пот}$, ол-ин}). Алтернативата на дълбокото обучение с подкрепление е параметризиран актьор, който директно извежда непрекъснато разпределение върху действията (PPO/SAC), елиминирайки проблема с транслацията за сметка на формални гаранции.

Проблемът с транслацията е **значителен** при явната дискретизация: когато опонентът направи залог от $0,7 \times \text{пот}$, а абстракцията съдържа само $0,5 \times \text{пот}$ и $1 \times \text{пот}$, агентът трябва да транслира този залог към възел, върху който е обучен. Най-често се използват три транслатора:

1. **Най-близко действие** — закръгляне до най-близкия абстрактен залог по линейния мащаб. Най-голяма загуба на стойност, когато залозите се групират между абстрактните размери.
2. **Вероятностно разделяне (линейно)** — присвояване на маса към двата най-близки абстрактни залога обратно пропорционално на разстоянието им от реалния залог.
3. **Псевдохармонично преобразуване** – вероятностно преобразуване, изведено от равновесието на опростена покерна игра: залог x (в части от пота) между абстрактните размери A и B се свързва с A с вероятност $\frac{(B-x)(1+A)}{(B-A)(1+x)}$. В сравненията на авторите то е значително по-малко експлоатируемо от по-ранните евристични преобразувания в три малки покерни игри, сред които Кун и Ледюк, и е водещият статичен транслатор преди появата на вложеното решаване на под-игри (по-долу).⁴⁹

Грешките при транслацията се натрупват в отделните рундове на залагане — опонент, който открие закръглянето, може систематично да залага точно под или над точките на решетката, за да предизвика отговори с грешен размер. Такава уязвимост е наблюдавана на практика (агентът Tartanian1, 2007 г.)⁴⁹, и това е оперативната причина за съществуването на корекция по време на изпълнение (по-долу): реалната под-игра се преизчислява с реалния залог, вместо да се разчита на транслатора.

Запомнете: абстракцията на действията определя кои ходове агентът може да обмисля преди транслацията или повторното решаване.

4.4 Разлика в експлоатируемостта

За стратегия σ , изиграна в игра G , експлоатируемостта е стандартният показател от глава 3:

$$\text{exploit}_G(\sigma) = \frac{1}{2}[v_G(\text{BR}(\sigma_1), \sigma_1) + v_G(\sigma_0, \text{BR}(\sigma_0))]$$

където σ_0 и σ_1 са стратегиите на двамата играчи, а $\text{BR}(\sigma)$ (най-добър отговор) е стратегията, която оптимално максимизира печалбата именно срещу стратегията σ .

Глава 4 въвежда производна метрика – **разлика в експлоатируемостта**: с колко една абстрактна

⁴⁹Ganzfried, S. & Sandholm, T. (2013). “Action Translation in Extensive-Form Games with Large Action Spaces: Axioms, Paradoxes, and the Pseudo-Harmonic Mapping.” *IJCAI*, 120–127.

стратегия е по-лоша от точното равновесие на Нaш на реалната игра, измерено в самата реална игра.

Нека G е реалната игра, а $\hat{G}$ – нейната абстракция. Нека $\hat{\sigma}^*$ е равновесие на Нaш на $\hat{G}$, а $T(\hat{\sigma}^*)$ – неговата трансляция обратно в играема стратегия за G (търждественото преобразуване, ако $\hat{G}$ е само информационна абстракция, и нетривиално преобразуване, ако включва и абстракция на действията — вж. „Видове оси на абстракцията“ по-горе).

$$\Delta_{\text{abs}}(\hat{G}) = \text{exploit}_G(T(\hat{\sigma}^*)) - \text{exploit}_G(\sigma_G^*)$$

където σ_G^* е точното равновесие на Нaш на G . По дефиниция $\Delta_{\text{abs}} \geq 0$, като равенство има, когато абстракцията е беззагубна (разгледано в критерия за сливане по-долу).

Това е основната величина в Глава 4. Всяка Парето диаграма в тази глава съдържа Δ_{abs} по една от осите. Всяко твърдение относно „качество на абстракцията“ се проверява чрез изчисляването ѝ — а когато директното изчисление е твърде скъпо, бюджетът на грешките по-долу предоставя граница и заместител вместо него.

Запомнете: разликата в експлоатируемостта е цената, платена за решаване на по-малката игра вместо реалната.

4.5 Разстояние на Васерщайн: Въведение

Разстоянието на Васерщайн се среща в почти всеки от следващите раздели, затова му е посветено кратко самостоятелно въведение.

Какво измерва. За две вероятностни разпределения върху един и същ носител разстоянието на Васерщайн е минималното количество „работа“, необходимо за трансформирането на едното в другото, където *работа* = *преместена маса* × *преместено разстояние*. Може да си представим всяко разпределение като купчина пясък върху числова ос — тогава разстоянието на Васерщайн е най-малкото общо усилие за преоформяне на купчина А в купчина Б.

Произход и други наименования. Идеята води началото си от проблема за *оптималния транспорт*, формулиран от Монж през 1781 г. (преместване на купчини пръст с цел запълване на дупки при минимално усилие), а по-късно формализиран от Канторович през 1942 г. като линейна програма. В машинното обучение същото понятие се среща под няколко имена — **разстояние на Васерщайн-1**, **разстояние на Канторович–Рубинщайн** или просто **разстояние на Васерщайн**. Името Earth Mover’s Distance (EMD) е популяризирано от Rubner, Tomasi и Guibas (2000)⁵⁰ в компютърното зрение при търсенето на изображения по съдържание. В по-ново време то стои в основата на генеративно-състезателните мрежи на Васерщайн (WGAN) и се използва широко

⁵⁰Rubner, Y., Tomasi, C. & Guibas, L. J. (2000). “The Earth Mover’s Distance as a Metric for Image Retrieval.” *International Journal of Computer Vision*, 40(2), 99–121.

при сходството на документи, адаптацията на домейни и всички задачи, при които е от значение сравняването на *формата* на разпределенията.

Защо превъзхожда сравненията, базирани на средна стойност. Разстоянието на Васерщайн отчита *геометрията* на разположението на масата, а не само средните стойности. Две разпределения, концентрирани близо едно до друго, са близки; две разпределения, концентрирани в противоположните краища, са далеч едно от друго, независимо къде техните средни стойности съвпадат случайно. Сравненията на очакваната стойност игнорират цялата тази структура.

В контекста на тази глава. Всяко информационно множество има *разпределение на силата на ръката (HSD)* — хистограма от вероятностите за победа в края на играта, изчислени чрез разиграване на невидяни карти. Две HSD с една и съща средна стойност могат да имат напълно различна форма: тясно разпределение с един връх („стабилна средна ръка“) или бимодално („успех или провал“). Клъстеризацията по очаквана сила на ръката ги обединява, докато разстоянието на Васерщайн не го прави. Ето защо всеки качествен критерий, всеки заместител на грешка и всеки клъстеризационен конвейер по-долу използва разстоянието на Васерщайн между HSD като основен сигнал за сходство.

Ефективно при 1-D хистограми. Когато носителят е едномерен и подреден (вероятността за победа $\in [0, 1]$, разделена на интервали), разстоянието на Васерщайн се свежда до **разстоянието на Манхатън** между кумулативните разпределения — едно линейно преминаване през интервалите. Ето защо HSD + разстояние на Васерщайн е достатъчно бързо, за да клъстеризира милиони **информационни множества**.

4.6 Критерият за сливане

Допълнително четиво: <https://www.cs.cmu.edu/~gilpin/papers/extensive.JACM.pdf> · https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf>

Кога могат две информационни множества да се **обединят** в едно? Три вложени нива на строгост, подредени от най-строгото към най-хлабавото.

4.6.1 Ниво 1 — Беззагубен

Сливане на две информационни множества се допуска единствено когато те са *стратегически еквивалентни*: имат еднаква вероятност за достигане, еднаква рекурсивна структура и водят до еднакви последствия по отношение на полезността срещу всяко възможно продължение от страна на опонента. Последното условие е решаващо — ако дори едно действие на опонента може да ги различи, те не могат да бъдат сляти без загуба.⁵¹

⁵¹Gilpin, A. & Sandholm, T. (2007). “Lossless Abstraction of Imperfect Information Games.” *Journal of the ACM*, 54(5), art. 25 – GameShrink и резултатът за Род Айлънд холдем.

Когато това условие е изпълнено, сливането е безплатно: всяка оптимална стратегия в абстрактната игра може да бъде директно приложена към оригиналната игра с **нула** разход за експлоатируемост.

Пример: Червен вале и черен вале в игра, в която боите нямат значение (невъзможни са **флъшове**), могат да се слоят без загуба. Червен вале в игра с **флъшове** не може — боята мълчаливо влияе върху вероятността на опонента да събере **флъш** чрез премахване на карти от тестето.

Запомнете: беззагубната абстракция е безплатна компресия: същото стратегическо значение, по-малко съхранени информационни множества.

4.6.2 Ниво 2 — Ограничена загуба

Изискването за „идентични полезности“ се отпуска до „полезностите се различават с най-много ϵ “; същата идея важи и за вероятности на случайността. Цената е количествено определима грешка — всяко сливане добавя своето допустимо отклонение към общия бюджет на грешката, който в крайна сметка определя граница върху общата експлоатируемост (използвана в следващия раздел). Последователностите от действия по слетите пътища все още трябва да съвпадат, така че това представлява *контролирано* забравяне, а не произволно.

Пример: Обединяването на вале и дама в един „клъстер с ниски карти“ принуждава агента да ги играе еднакво, въпреки че оптимално те се различават леко. Системата поема тази грешка в замяна на значително опростяване на играта.

Запомнете: абстракция с ограничена загуба е контролирано забравяне: играта се свива, но всяко сливане получава цена за грешка.

4.6.3 Ниво 3 — Емпирична сходност

Когато аналитичната граница е твърде песимистична или входовете са твърде високодименсионни, за да бъдат изброени (в тексаски холдем на последния рунд има около $2,4 \times 10^9$ канонични комбинации от скрити и общи карти⁵²), границата за сливане се премахва и се заменя с *научена функция за разстояние*:

- Изчислява се вектор на характеристики за всяко информационно множество — обикновено **разпределението на силата на ръката (HSD)**, дефинирано в увода по-горе.
- Използва се **разстояние на Васерщайн (EMD)** между векторите на характеристики като метрика за сходство.⁵²⁵³
- Групира се информационните множества с близки разпределения на признаци и се извършва сливане в рамките на групите.

⁵²Johanson, M., Burch, N., Valenzano, R. & Bowling, M. (2013). “Evaluating State-Space Abstractions in Extensive-Form Games.” AAMAS, 271–278.

⁵³Ganzfried, S. & Sandholm, T. (2014). “Potential-Aware Imperfect-Recall Abstraction with Earth Mover’s Distance in Imperfect-Information Games.” AAAI, 28(1).

Няма формална гаранция за експлоатируемост тук — качеството се проверява *след* решаването, като експлоатируемостта на получената стратегия се измерва. Нещо повече, дори по-фината абстракция не гарантира по-ниска експлоатируемост.⁵⁴

Запомнете: емпиричната абстракция е приближение в рамките на бюджета, чието качество трябва да се измери след решаването.

4.7 Бюджетът на грешките

Допълнително четиво: https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf>

Три инструмента за количествена оценка, подредени така, че точността на оценката расте, а формалната строгост намалява. Първите два съответстват на нивата на строгост по-горе — аналитичната граница оценява сливане със загуби от Ниво 2 (с ограничено отклонение), а заместващата оценка чрез разстоянието на Васерщайн оценява емпирично сливане от Ниво 3 — докато третият измерва завършената абстракция, независимо как е била конструирана.

4.7.1 Инструмент 1 — Аналитична граница

За абстракция със загуби допустимите отклонения на отделните сливания се сумират, претеглени с вероятността всяко информационно множество действително да бъде достигнато в игра, и така се получава горна граница на разликата в експлоатируемостта.⁵⁵

Ключовата идея е претеглянето по вероятността за достигане. Редките информационни множества могат да бъдат агресивно сливани, без това да влошава общата експлоатируемост; небрежните сливания в гъсти и често посещавани области са катастрофални. Небрежното сливане в рядък сценарий от крайната игра почти не оказва влияние върху общия резултат, което позволява на една практична абстракция да бъде агресивна, без средно да води до загуби.

Запомнете: аналитичната граница е строга, но обикновено хлабава; най-важната ѝ идея е претеглянето по вероятността за достигане.

4.7.2 Инструмент 2 — Заместваща оценка чрез разстоянието на Васерщайн

Разстоянието на Васерщайн между хистограмите на силата на ръката на две информационни множества, измерено без изброяване на листата (вижте въведението по-горе за механиката). То е *заместваща оценка*, а не граница, и не носи формална гаранция.⁵² В експериментите на тази глава

⁵⁴Waugh, K., Schnizlein, D., Bowling, M. & Szafron, D. (2009). “Abstraction Pathologies in Extensive Games.” *AAMAS*, 781–788.

⁵⁵Kroer, C. & Sandholm, T. (2014). “Extensive-Form Game Abstraction with Bounds.” *ACM EC*, 621–638; и Kroer, C. & Sandholm, T. (2016). “Imperfect-Recall Abstractions with Bounds in Games.” *ACM EC*, 459–476.

то не проследява експлоатируемостта: за абстракциите с $k = 3$ и $k = 5$ е нула, а тяхната експлоатируемост остава 0,38–0.57.

4.7.3 Инструмент 3 — Пряка оценка чрез CFR-BR

Най-силната мярка: за абстракции с пълна памет тя намира стратегията в абстракцията с най-ниска експлоатируемост в пълната игра, като по този начин *изолира* грешката на абстракцията от грешката при решаване.

В експериментите стратегиите на CFR-BR показват експлоатируемост, толкова ниска, колкото $1/3$ от съответните обикновени CFR стратегии при *същата* абстракция. Изводът е, че част от измерената „грешка на абстракцията“ не се дължи на самата абстракция, а на избора на *равновесието на абстрактната игра* за цел: CFR клони към него, но то не е най-малко експлоатируемата стратегия, която абстракцията може да изрази.⁵²

Запомнете: CFR-BR изследва какво може да изрази абстракцията, а не доколко добре се е обучил конкретен решавач.

4.8 Избор на абстракция на практика

Трите критериални нива и трите измервателни инструмента оставят един въпрос отворен: към какво всъщност се обръщаме в реална игра? Три открития дават отговор на този въпрос.

Редът на вземане на решения. Първо се използват всички налични беззагубни сливания; след това се приемат сливания с ограничени загуби само когато бюджетът на грешките е приемлив; когато точните проверки са прекалено песимистични или твърде скъпи, се извършва клъстеризация с HSD + разстояние на Васерщайн и получената абстракция се валидира емпирично.

Глобалната оптимизация е практически неприложима. Изборът на разделяне, което минимизира аналитичната граница, е **NP-пълна** задача дори за игра с един играч и дърво с височина две. Затова никой не минимизира границата точно — практическите конвейери я приближават ниво по ниво (един рунд наведнъж), а не глобално. При разумни условия, едно ниво се свежда до клъстеризация около центрове в метрично пространство, което разполага с приблизителни алгоритми с полиномиално време с гаранции с постоянен коефициент. Това съответства на практиката: абстракциите в покера се строят ниво по ниво чрез клъстеризация, а не чрез глобална оптимизация.⁵⁵

Две надеждни сравнения при еднакъв брой информационни множества.

- **Отчитащи разпределението срещу основани на очакването.** При достатъчно голяма абстракция HSD + разстояние на Васерщайн превъзхожда клъстеризацията по очаквана сила на ръката както по експлоатируемост, така и в директни срещи; при много малки абстракции очакването може да е по-добро.⁵²

- **Непълна срещу пълна памет.** В тексаски холдем абстракциите с непълна памет превъзхождат тези с пълна памет при еднакъв брой информационни множества – свободата да се преразпределят клъстерите се оказва по-ценна от изгубената непрекъснатост.⁵²

И двете са причина конвейерът по време на компилация по подразбиране да използва непълна памет, HSD и разстоянието на Васерщайн.

4.9 Решения по време на компилация

Допълнително четиво: <https://www.cs.cmu.edu/~gilpin/papers/extensive.JACM.pdf> · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf> · https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf

Два допълващи се конвейера, по един на ниво строгост на критерия.

4.9.1 Конвейер 1 — GameShrink (беззагубен)

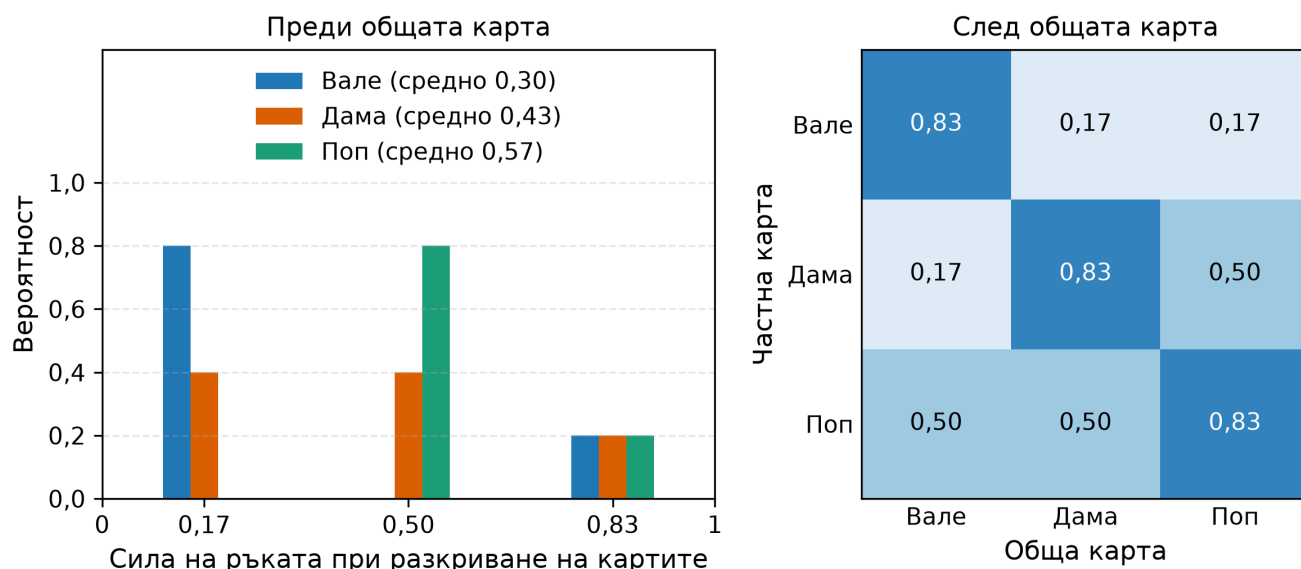
Изчерпателно сливане, което обхожда *сигналното дърво* — структура, по-малка от дървото на играта, изброяваща само последователности от публични и частни сигнали — отдолу-нагоре. На всяко ниво се проверяват двойки поддървета братя за стратегическа еквивалентност и всяка преминала двойка се слива.

Защо сигналното дърво е по-малко: дървото на играта умножава всеки отделен сигнал от скрито състояние по всяка отделна последователност от залози, която може да доведе до него, докато сигналното дърво колабира всички тези пътища на залагане в един сигнален възел. В Род Айлънд холдем (Rhode Island Hold'em) сигналното дърво е около 6,6 милиона възела спрямо приблизително 3,1 милиарда възела на дървото на играта — това представлява компресия от около 500 пъти преди да се приложи каквато и да е стъпка със загуби. GameShrink е *пълен* по отношение на своя критерий за сливане: всяко беззагубно сливане, изразимо чрез този критерий, се открива.

Този конвейер, комбиниран с линейно програмиране, решава Род Айлънд холдем през 2007 г., четири поредъка отвъд всяка покер игра, решавана дотогава.⁵¹

Запомнете: GameShrink търси в *сигналното дърво* всяко сливане без загуба, преди да бъде разгледана каквато и да е компресия със загуби.

4.9.2 Конвейер 2 — HSD + разстояние на Васерщайн + k-средни + непълна памет (със загуби)



Фигура 12: Разпределения на силата на ръката (HSD) в Ледюк. Вляво: преди откриването на общата карта всяка частна карта има разпределение върху три стойности на силата при разкриване на картите, а не една стойност. Вдясно: след общата карта деветте комбинации (частна, обща карта) дават само три различни HSD; цветът е клъстерът, който k-средни с разстоянието на Васерщайн им определя, и при $k = 5$ той е същият като при $k = 3$.

Когато беззагубното сливане достигне сходимост и играта все още е твърде голяма, се пристъпва към клъстеризация:

Конвейерът изчислява HSD за всяко информационно множество, групира получените разпределения чрез разстоянието на Васерщайн като метрика на разстояние и съпоставя всяко информационно множество към идентификатор на клъстер. Възможни са два режима на паметта:

- **Пълна памет** — идентификаторът на клъстера включва предишната траектория на клъстерите.
- **Непълна памет** — клъстерът в по-късен рунд може да забрави по-ранната траектория, като по този начин освобождава повече клъстери за рунда, в който новата информация има най-голямо значение.

В тексаски холдем непълната памет надделява при фиксиран бюджет на клъстерите – капацитетът се изразходва за рундовете, които имат значение, вместо за запомняне на историята.⁵² В малкия Ледюк от тази глава предимството не се потвърждава: при сходен размер (108 срещу 132 информационни множества) абстракцията с непълна памет не е по-добра (0,574 срещу 0,571; вж. фигурата за Ледюк с лимит в „Практическо валидиране“).

Запомнете: HSD + разстояние на Васерщайн определя какво изглежда стратегически

подобно; непълна памет определя къде да се използва бюджетът на клъстерите.

Резултатът. Замразеният резултат от двата конвейера — стратегията, получена при решаването на абстрактната игра, и началната точка за всичко, което се случва по време на изпълнение — в останалата част от главата се обозначава като **план**.

4.10 Корекция по време на изпълнение

За допълнително четене: <https://arxiv.org/pdf/1705.02955v3>

Когато играта премине в под-игра, която абстрактният план покрива твърде грубо, под-играта се преизчислява с по-висока точност *в реално време*. Две корекции са от значение — заедно те бяха носещите елементи на Libratus, първия изкуствен интелект, който победи най-добрите професионалисти в безлимитния тексаски холдем за двама играчи.⁵⁶

4.10.1 Защо решаването на под-игра не може да се извършва изолирано (хвърляне на монета)

Прост контрапример, наречен *хвърляне на монета*: монета пада с ези или тура с еднаква вероятност, като само P_1 вижда изхода. Играчът P_1 избира между „Продай“ (Sell; с печалба, зависеща от резултата на монетата) и „Играй“ (Play; при която играчът P_2 се опитва да познае коя страна е паднала). Оптималната P_2 стратегия в под-играта „Играй“ **не е** функция единствено на самата тази под-игра — тя зависи от стойността, която P_1 би получил, ако вместо това беше избрал „Продай“. Ако печалбата от „Продай“ се промени, оптималната стратегия за „Играй“ също се променя, въпреки че самата под-игра „Играй“ остава непроменена.⁵⁷

Това е централната патология, в която попада всяко наивно решаване на под-игра с непълна информация. Решението: решаване на *разширена под-игра*, включваща оригиналната под-игра и допълнителни възли „алтернативна печалба“, кодиращи какво всеки играч би могъл да постигне, ако *не беше влязъл* в тази под-игра.⁵⁸

4.10.2 Корекция 1 — Безопасно решаване на под-игри

Разширената под-игра се основава на план-стойности: всяко информационно множество в началото на под-играта получава алтернативна печалба, равна на обещаното от плана за този играч в съответния момент от играта. Решаването на разширената игра води до усъвършенствана стратегия с гаранция за безопасност — експлоатируемостта е доказано не по-висока от тази на плана.⁵⁷

⁵⁶Brown, N. & Sandholm, T. (2018). “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals.” *Science*, 359(6374), 418-424.

⁵⁷Brown, N. & Sandholm, T. (2017). “Safe and Nested Subgame Solving for Imperfect-Information Games.” *NeurIPS*.

⁵⁸Burch, N., Johanson, M. & Bowling, M. (2014). “Solving Imperfect Information Games Using Decomposition.” *AAAI* – пре-решаване и разширената под-игра.

Практическо усъвършенстване (Reach) пренася напред „подаръци“ — разлики в стойността от по-ранни точки по траекторията, където играчът е могъл да постигне строго по-добър резултат — за допълнително усъвършенстване.

На практика консервативните план-печалби могат да бъдат заменени с *оценки* на равновесната стойност. Така строгата гаранция отпада, но замяната обикновено води до по-ниска експлоатируемост в реална игра, тъй като самите консервативни план-печалби са неточни.

Запомнете: безопасното решаване на под-игри работи, тъй като под-играта не се решава самостоятелно; тя е закотвена чрез план-стойности.

4.10.3 Корекция 2 — Вложено решаване на под-игри (заместител на транслацията на действия)

Това е директният отговор на проблема с транслацията, изложен по-горе в раздела „Видове оси на абстракцията“. Когато *противникът* извърши действие *a* извън зададената абстракция, *a* не се закръгля до познато действие чрез транслация — а *преизчислява се нова под-игра, която включва a*.

По-евтиният вариант изгражда под-игра непосредствено след **действие извън дървото**, преизчислява я по схемата на безопасното решаване на под-игри и добавя новата подстратегия към плана. Ако по-късно се появи друго **действие извън дървото**, процесът се повтаря — **планът** нараства единствено там, където действително протича играта.

Емпирично въздействие. В опростен безлимитен вариант (холдем, игран само до флопа) вложеното решаване на под-игри срещу залози на противника извън дървото има около 10 пъти по-ниска експлоатируемост от псевдохармоничната транслация (119–150 срещу 1465 хилядни от големия блайнд на раздаване, mbb/g).⁵⁷

Рекурсията е плътка на практика. Повечето реални действия извън дървото не образуват вериги — противникът прави един необичаен залог, агентът преизчислява и новото абстрактно дърво го абсорбира. Статичните транслатори на действия остават подходящият избор само когато закъснението не позволява решаване с CFR в реално време (онлайн игра, вградени приложения).

Запомнете: транслацията на действия закръгля хода на противника; вложеното решаване запазва хода и решава около него.

4.11 Архитектура: план и корекции в реално време

Целият конвейер се свежда до един архитектурен модел, използван от водещите системи за безлимитен покер след 2017 г. – Libratus⁵⁶ и Modicum при двама играчи и Pluribus⁵⁹ при шестима

⁵⁹Brown, N. & Sandholm, T. (2019). “Superhuman AI for multiplayer poker.” *Science*, 365(6456), 885-890.

(DeepStack⁶⁰ е изключение: той не изчислява предварително стратегия за цялата игра):

1. **Време на компилация** — прилагат се беззагубна абстракция и абстракция със загуби, за да се свие играта; получената абстрактна игра се решава с CFR / CFR+ / MCCFR; получената стратегия се замразява като план.
2. **Време на изпълнение** — когато играта премине в под-игра, която планът покрива грубо, тя се преизчислява чрез безопасно решаване на под-игри (корекция 1). Когато противникът извърши действие извън абстракцията, се преизчислява нова под-игра, включваща това действие (корекция 2).

Двете части се допълват взаимно: чрез абстракцията задачата по време на компилация става изчислително поносима; корекциите в реално време възстановяват точността, изгубена вследствие на абстракцията в онези моменти от играта, където това е от съществено значение.

Запомнете: архитектурата на реалните системи е наполовина предварително изчислен план и наполовина повторно решаване в реално време — нито едната половина не може да съществува самостоятелно.

4.12 Практическо валидиране

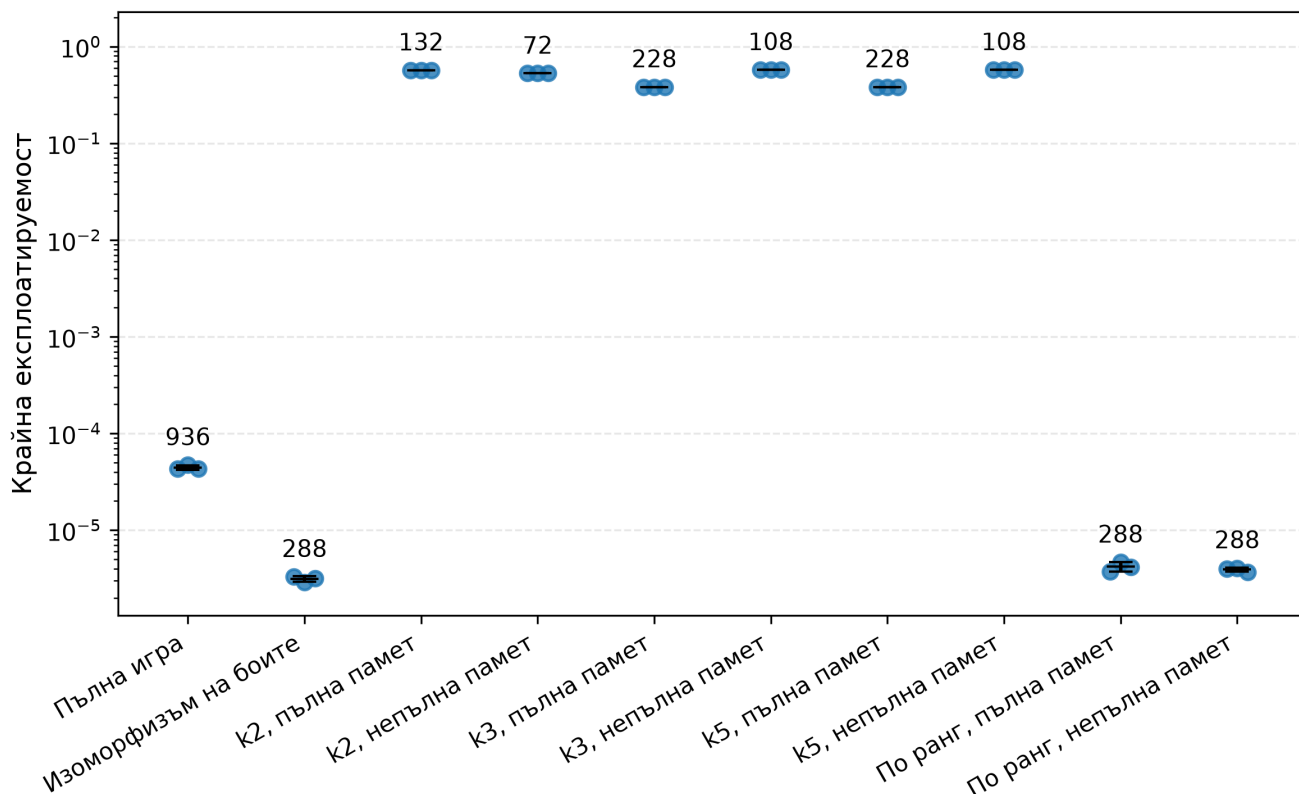
Фазата на имплементация превърна абстракциите от резюмето в малък, но цялостен конвейер от типа „Ледюк“:

- **Беззагубен изоморфизъм на боите** върху Ледюк с лимит и разширения Ледюк (Extended Leduc).
- **Групиране на карти със загуби**, използващо характеристики за силата на ръката, разстояния от тип Васерщайн и конфигурируем брой клъстери за карти.
- **Абстракция на действията** върху Mini-NL Ледюк с реализирани транслатори „най-близко действие“, „вероятностно разделяне“ и „псевдохармонично преобразуване“.
- **Разширен Ледюк** с четири ранга и две бои, предоставящ по-голяма тестова среда с непълна информация.
- **Комбинирана абстракция**, съставена от редуциране на боите, редуциране на действията и групиране на карти.
- **Оценка на качеството** чрез разлики в експлоатируемостта, заместващи оценки чрез EMD, кръстосана проверка в OpenSpiel и Парето граница.

Основните емпирични изводи съответстват на теорията: **беззагубната абстракция** е почти безплатна стратегически и много полезна изчислително, докато информационната абстракция със загуби и абстракцията на действията въвеждат трайна долна граница на експлоатируемостта. При бюджети от 180 секунди за CFR+, **изоморфизмът на боите** намали броя на информационните

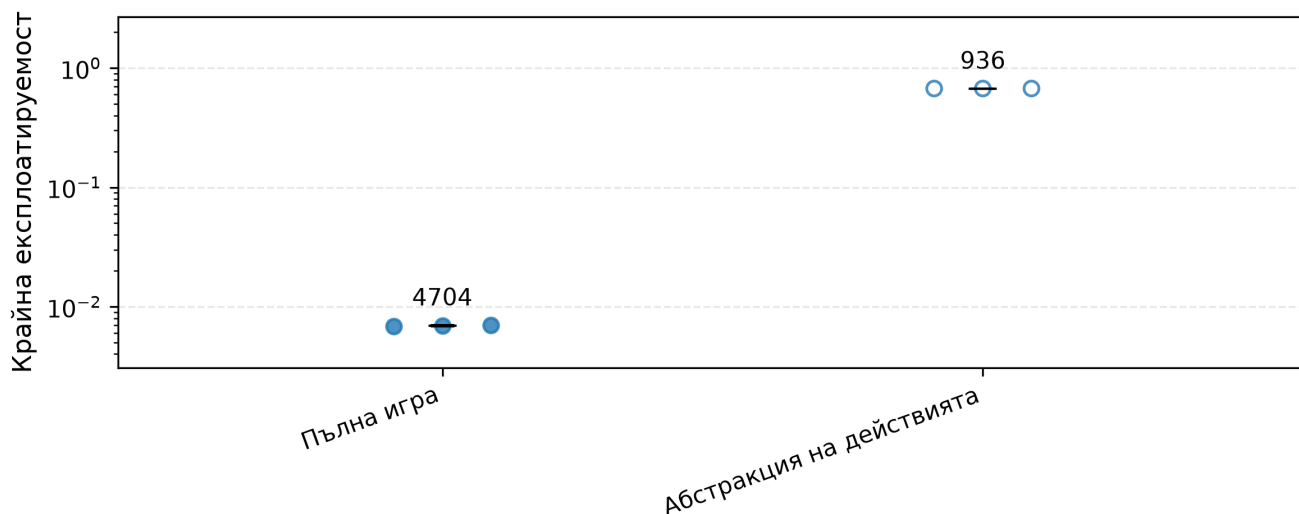
⁶⁰Moravčík, M. et al. (2017). “DeepStack: Expert-level artificial intelligence in heads-up no-limit poker.” *Science*, 356(6337), 508-513.

множества в Ледюк с лимит от 936 до 288 и достигна по-ниска експлоатируемост от пълната игра, тъй като завърши значително повече **итерации**. В разширения Ледюк същата идея намали броя на информационните множества от 10 304 до 2 968 и подобри крайната експлоатируемост от около $2,7 \times 10^{-2}$ до около $1,3 \times 10^{-3}$.

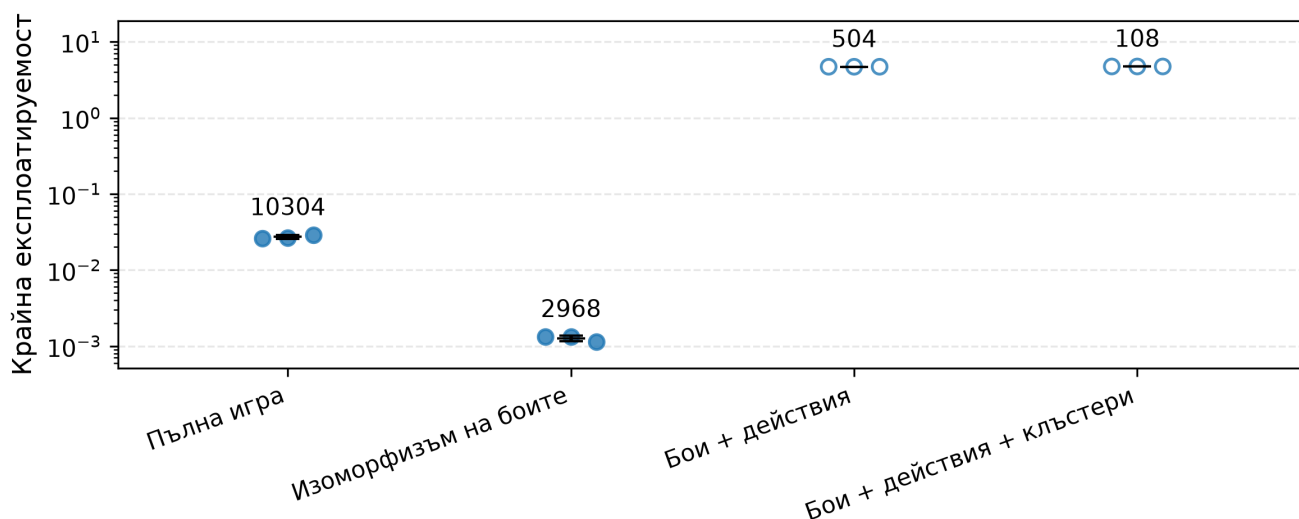


Фигура 13: Ледюк с лимит: крайна експлоатируемост след 180 s CFR+ за всяка конфигурация на абстракцията (три изпълнения, логаритмична скала); числото над всяка група е броят информационни множества.

Експериментите с клъстери със загуби показват другата страна на компромиса. По-малките игри с клъстери се обучават по-бързо, но грешката не изчезва при повече **итерации** на CFR+, тъй като стратегията решава грешната игра. В Ледюк с лимит грубите абстракции с клъстери останаха на експлоатируемост около 0,38–0,57, въпреки че завършиха повече **итерации** от пълния CFR+. Това е практическото значение на разликата в експлоатируемостта: грешката на абстракцията не е грешка на оптимизатора.



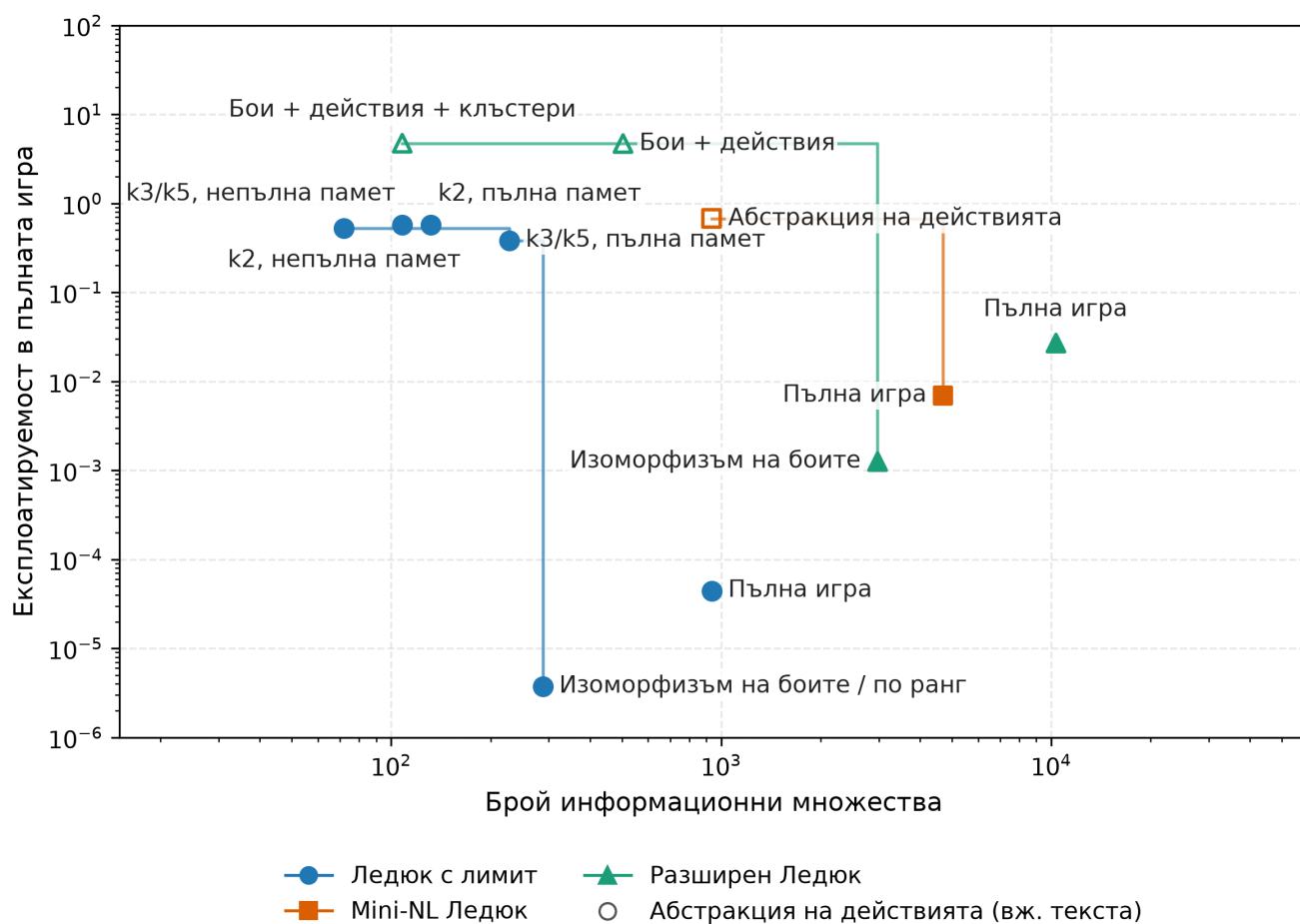
Фигура 14: Mini-NL Ледюк: крайна експлоатируемост след 180 s CFR+ – пълната игра срещу абстракцията на действията; числото над всяка група е броят информационни множества. Незапълнените маркери отбелязват абстракцията на действията, чиято стойност отразява текущото правило за внедряване (вж. текста).



Фигура 15: Разширен Ледюк: крайна експлоатируемост след 180 s CFR+ за пълната игра, изоморфизма на боите и комбинираните абстракции; числото над всяка група е броят информационни множества. Незапълнените маркери отбелязват конфигурациите с абстракция на действията (вж. текста).

Абстракцията на действията беше най-рисковата част от главата. В Mini-NL Ледюк ограничаването на множеството действия намали броя информационни множества от 4704 до 936 и позволи извършването на значително повече **итерации** на CFR+ в рамките на същия времеви бюджет, но експлоатируемостта остана висока. В разширения Ледюк добавянето на абстракция на дейс-

твията върху изоморфизма на боите доведе до компактно дърво, но внедрената стратегия беше силно експлоатируема. Тези стойности обаче измерват текущото правило за внедряване – всеки абстрактен малък залог се изиграва като голям, а големите залози на противника се четат като малки – а не качеството на трите транслятора, които в тази реализация не се задействат и дават еднакви резултати. Те показват колко може да струва наивното съпоставяне на абстрактното и реалното множество действия; отговорът на литературата за действията извън дървото е вложеното решаване на под-игри.⁵⁷



Фигура 16: Граница на Парето: брой информационни множества срещу експлоатируемост в пълната игра след 180 s CFR+ за всяка конфигурация на абстракцията (средно от три изпълнения). Линиите свързват недоминираните точки на всяка игра; незапълнените маркери са конфигурации с абстракция на действията, чиито стойности отразяват текущото правило за внедряване.

Изгледът на Парето е правилната крайна проверка. Всяка точка показва: с колко е намалена играта и каква цена или полза по отношение на експлоатируемостта е постигната чрез тази компресия. Беззагубният изоморфизъм на боите се намира в атрактивната част от границата. Грубите клъстери и абстракцията на действията могат да намалят играта още повече, но това води до преминаване към различен режим, при който по-малкият размер се постига за сметка на качеството

на стратегията.

4.13 Аналози на дълбокото обучение с подкрепление

Всяка фаза на явния конвейер има аналог в дълбокото обучение с подкрепление, който заема същото концептуално място, но заменя формалните гаранции с обучение от край до край. Това са само препратки напред – реализациите са в глави 5 и 6.

- **Критерий за сливане** — трите нива на строгост съответстват на различни мрежови архитектури: беззагубен $\leftrightarrow$ мрежи, еквивариантни спрямо пермутации (Deep Sets), ограничена загуба $\leftrightarrow$ информационно стеснение в стратегии с рекурентни мрежи или трансформъри, емпирична сходност $\leftrightarrow$ латентни вграждания, научени от край до край (както при Deep CFR).
 - **Бюджет на грешките** — не се задават изрични граници за всяко сливане. Качеството се проследява чрез криви скорост–изкривяване (информационно стеснение) и наблюдение на тестовата загуба, като вместо алгоритмични гаранции се приема асимптотична сходимост.
 - **Конвейер по време на компилация** — отделна фаза на компилация не съществува. Латентното пространство на мрежата изгражда собствено абстрактно представяне, оформяно директно чрез градиентен спуск върху загубата за минимизиране на съжалението.
 - **Корекция по време на изпълнение** — точните методи за изчисляване на равновесие се заменят от евристично търсене с ограничена дълбочина, подкрепено от невронни мрежови функции на стойността (DeepStack, ReBeL).
-

4.14 Връзки и препратки напред

Глава 4 продължава прогресията, започната в глави 2 и 3. Глава 2 въвежда CFR като локално минимизиране на съжалението върху информационни множества. Глава 3 прави този решаващ по-бърз и по-евтин чрез CFR+ и MCCFR. Глава 4 променя самата входна игра: преди решаването, тя определя кои състояния и действия могат да бъдат обединени, без да се унищожи стратегическият сигнал.

Мостът към глава 5 е току-що описаният неявен път: Деер CFR и невронното приближение на равновесие заменят ръчно изградените и изградени чрез клъстеризация дялове от глава 4 с научени функционални апроксиматори. Представянето остава компресирано, но компресията вече е реализирана в мрежа, вместо във фиксирана карта на клъстери.

Мостът към глава 6 е архитектурният план. Съвременните покер агенти първо решават груба абстрактна игра, а след това закръпват слабостите онлайн чрез решаване на под-игра. Глава 4 предоставя речника: план, трансляция на действия, разлика в експлоатируемостта, Парето граница и безопасно/вложено усъвършенстване. Глава 6 превръща тези елементи в пълни системи за игра на игри.

За дисертацията абстракцията е от значение, защото адаптацията към съперника е ефективна единствено в границите на резолюцията, която представянето на играта запазва. Ако абстракцията обедини две стратегически различни състояния, обърнати към противника, нито един последващ модел на противника не би могъл да възстанови това разграничение. Обратно, представяне, което е прекалено детайлно, може да бъде твърде скъпо за решаване или оценяване. Парето границата от глава 4 следователно се включва в методологията за оценяване: качеството на стратегията трябва да бъде докладвано заедно с размера и детайлността на представянето на играта, което я е породило.

5 Глава 5 – Невронни мрежи за игри с непълна информация

Това е съкратен обзор на невронни методи за игри с непълна информация – материалът, който Глава 5 разширява от първоначалния си фокус върху Deer CFR до по-широко разглеждане на изграждането, обучението и внедряването на невронни мрежи в игри, при които играчите не могат да възприемат пълното състояние. Той служи като кратък справочник при четенето на следващите глави. Глава 5 представлява умишлено еднократно отклонение от обичайния учебен цикъл – фазата на реализация от нулата се отлага, а вместо нея се разширява теорията – поради което този документ е карта на областта в ширина, а не запис на код. Математическите постановки са представени опростено: ключовите уравнения се посочват там, където изострят интуицията, докато извежданията и доказателствата остават в цитираните източници. Разделите за научената абстракция, оценяването на невронни стратегии, експерименталните архитектури и обобщаващата карта на решенията са планирани, но не са включени в тази версия.

Защо невронни мрежи

Табличният CFR и неговите варианти на Монте Карло (Глави 2–4) съхраняват стойност на съжалението и стойност на стратегията във всяко информационно множество, така че разходът им за памет е пропорционален на броя на информационните множества, който нараства с размера на играта.⁶¹ Експериментите в тази глава определиха количествено тази граница в Ледюк покер – игра, достатъчно малка, за да се побере в таблица: табличният MCCFR достигна експлоатируемост 0,097, докато Deer CFR при сравнимо време за изпълнение остана над 1,1 при всички изпробвани размери на мрежата. В този мащаб табличната методика е за предпочитане; невронната методика не предлага никакво предимство, тъй като таблицата вече се побира изцяло в паметта.

Невронна мрежа заменя таблицата с апроксимация на функция f_θ , която съпоставя характеристиките на едно състояние към стойност или разпределение на действията и обобщава към състояния, които не са били посетени по време на обучението. Това обръща структурата на разходите: паметта става независима от броя на информационните множества, за сметка на по-скъпи итерации и по-слаби гаранции за сходимост. Двата режима се разделят от праг в размера на играта: под него изброяването е по-евтино и точно, а над него таблицата не може да бъде построена и апроксимацията на функция е единствената възможност. Игрите, към които е насочена тази работа, са над този праг, поради което останалата част от главата се посвещава на невронни методи.

⁶¹Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2007). “Regret Minimization in Games with Incomplete Information.” *Advances in Neural Information Processing Systems* 20, 1729-1736.

5.1 Основни положения

5.1.1 Основи на невронните мрежи

Мрежа с директно разпространение представлява композиция от слоеве, като всеки слой прилага афинно преобразуване, последвано от нелинейност по елементи:

$$h = \sigma(Wx + b),$$

където x е входът на слоя, W и b са съответно обучаемата матрица на теглата и векторът на отместванията, а σ представлява фиксирана нелинейност, като например изправителя $\text{ReLU}(z) = \max(0, z)$. Подреждането на L такива слоя формира функцията на мрежата f_θ , при която θ обединява всички W и b . Именно нелинейностите позволяват на тази композиция да представя функции, които не могат да бъдат изразени чрез едно-единствено афинно преобразуване.

ReLU (rectified linear unit): стандартната функция за активация – тя пропуска положителните стойности непроменени и ограничава отрицателните до нула, което е изчислително евтино и избягва проблема с изчезващите градиенти, характерен за по-старите насищащи се функции за активация.

Обучението изисква **скаларна величина** за измерване на грешката. При задача за **регресия** – например при оценка на **контрафактична стойност** – стандартно се използва **средноквадратичната грешка**:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (f_\theta(x_i) - y_i)^2,$$

където N е броят на примерите в партида, x_i - вход, а y_i - неговата цел. Когато целта представлява разпределение на действията, вместо това се използва загубата на кръстосана ентропия $-\sum_a y_a \log f_\theta(x)_a$, като y_a е целевата вероятност за действие a .

Обучението представлява **градиентен спуск** върху тази **загуба**. Всеки параметър се измества с малка стъпка в посоката, която води до най-голямо намаление на **загубата**:

$$\theta \leftarrow \theta - \eta \nabla_\theta L,$$

където η е скоростта на обучение, а $\nabla_\theta L$ - градиентът на загубата спрямо параметрите. *Обратно разпространение на грешката* изчислява този градиент, като прилага правилото за верижно диференциране слой по слой от изхода към входа и използва повторно междинните стойности, съхранени във всеки слой; така изчисляването на един градиент струва приблизително колкото едно преминаване напред. Автоматичното диференциране извършва тази процедура механично.

На практика градиентът се оценява от малка случайна партида, а не от целия набор от данни (*стохастичен градиентен спуск*) – по-шумен, но значително по-евтин на стъпка. Импулсът намалява този шум, като извършва стъпките с текуща средна на градиентите, $v \leftarrow \beta v + \nabla_{\theta} L$ и $\theta \leftarrow \theta - \eta v$, където $\beta \in [0, 1)$ определя степента, в която се запазват миналите градиенти. Adam, оптимизаторът по подразбиране в методите на тази глава, освен това разделя стъпката за всеки параметър на текуща оценка на големината на неговия градиент, което осигурява стабилен напредък с минимално ръчно настройване на η .

Adam (адаптивна оценка на моментите): комбинира импулс с размери на стъпките за всеки параметър, извлечени от текущи статистики на градиента, поради което се обучава надеждно с малко настройване.

Мрежа с достатъчен капацитет може да напасне точно една тренировъчна партида, включително и нейния шум, и след това да се представи слабо при невиджани състояния – *преобучение* (overfitting). Регуляризацията противодейства на това. Отслабването на теглата добавя наказание $\lambda \|\theta\|^2$ към загубата, като λ контролира степента, в която се обезкуражават големите тегла; отпадането произволно занулява част p от активациите по време на обучението, така че мрежата да не може да разчита на нито един отделен елемент; и ранното спиране прекратява обучението, след като грешката върху задържаните данни започне да нараства.

Една идея от обучението с подкрепление се повтаря през цялата глава и си струва да бъде изразена изрично: *самоподкрепяне*. Актуализация чрез темпорални разлики оценява стойността на едно състояние въз основа на собствената оценка на мрежата за следващото състояние, а не от краен резултат:

$$V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)],$$

където s и s' са последователни състояния, r е наградата, $\gamma \in [0, 1]$ е коефициентът на дисконтиране, а α е размерът на стъпката; величината в скобите представлява грешка при темпорална разлика – разликата между текущата оценка и целта с бутстрапиране. Самоподкрепянето прави обучението ефективно по отношение на данните, а както показва следващият раздел, също така го прави и по-малко стабилно. Основният материал е представен в глави 9 и 11 от Sutton & Barto (2-ро издание); всяка стандартна литература за дълбоко обучение разглежда подробно обратното разпространение на грешката и оптимизацията.

5.1.2 Обединяване на основите в Ледюк покера

Тези елементи се комбинират в най-простия **невронен решаващ** за игра. Вземете Ледюк покер, тестовата среда от глави 3–4: неговото информационно състояние – кой играч е на ход, тайната карта, общата карта и историята на залозите – се кодира като вектор от 30 числа. Малка мрежа съпоставя на този вектор стойност за всяко действие; обратното разпространение на грешката с Adam я настройва спрямо целевите стойности, генерирани по време на игра, а леката **регуляриза-**

ция предотвратява запаметяването на шума в тези цели. Използвана правилно, мрежата предоставя две възможности, които таблицата няма: тя обобщава, така че **стойността**, научена в едно **информационно множество**, информира подобни невиджани такива, а цената на паметта ѝ е фиксирана от броя на параметрите, а не от броя на информационните множества.

Същата техника се влошава, когато бъде претоварена, а **изследването** направи режимите на отказ конкретни. Капацитетът е най-ясният пример. При **Ледюк** най-малката тествана **мрежа** – два скрити слоя с тридесет и две единици – достигна по-ниска **експлоатируемост** от мрежа, няколко пъти по-голяма, вероятно защото при толкова малък бюджет по-големите мрежи не получават достатъчно различни извадки, за да се обучат (резултатът е от едно начално число при почти хоризонтални криви, затова обяснението е хипотеза). **Скоростта на обучение**, зададена твърде висока, дестабилизира спускането; прекомерната **регуляризация** изглажда острите, почти **детерминистични** стратегии, към които CFR се сближава, тъй като силното **отслабване на теглата** или **отпадане** дърпат изхода към **равномерното разпределение**. Общият принцип е, че капацитетът и регуляризацията трябва да бъдат съобразени с количеството и качеството на данните, а не да се максимизират по подразбиране – точка, която раздел 5.2 развива в изрични насоки за оразмеряване.

5.1.3 Нестационарност и смъртоносната троица

Комбинирането на горните компоненти въвежда нестабилност. Сътон и Барто идентифицират една *смъртоносна троица*: когато **апроксимация на функция**, **самоподкрепяне** и **обучение извън текущата стратегия (off-policy)** се използват заедно, параметрите могат да се отдалечат неограничено, вместо да клонят към решение, тъй като **целевите стойности по метода „bootstrap“** за всяка актуализация зависят от същите параметри, които се променят. Всеки един компонент е **безопасен** самостоятелно; комбинацията не е. Средствата за защита са практични, а не теоретични. Една *целева мрежа* – забавено копие на параметрите, използвано само за изчисляване на **целеви стойности по метода „bootstrap“** – и един *буфер за възпроизвеждане*, който декорелира последователните проби, заедно правят дълбоките Q-мрежи обучаеми, и двата се появяват отново в невронните решаващи на игри от раздели 5.3 и 5.4.

Игрите въвеждат втори източник на нестабилност, който липсва при обучението на един агент. При обучението с учител разпределението на данните е фиксирано, а при обучението с подкрепление с един агент то се стабилизира, след като стратегията достигне сходимост. В една игра противникът също се адаптира, така че разпределението на ситуациите, които мрежата среща, се променя по време на обучението; функцията, която се апроксимира, сама по себе си е подвижна цел. Тази нестационарност не е случайна – тя е централната трудност на средата, към която това изследване е насочено. Няколко метода в следващите раздели се разбират най-добре като отговор на нея: самообучението срещу замразени или осреднени във времето противници възстановява стационарна цел, популационните методи заменят подвижната цел с изричен набор от фиксирани противници, а техниките за моделиране на противника и бърза адаптация (глави 7 и 12) се опитват да я проследят пряко.

5.2 Архитектура и композиция

5.2.1 Индуктивно пристрастие: съгласуване на архитектурата със структурата на играта

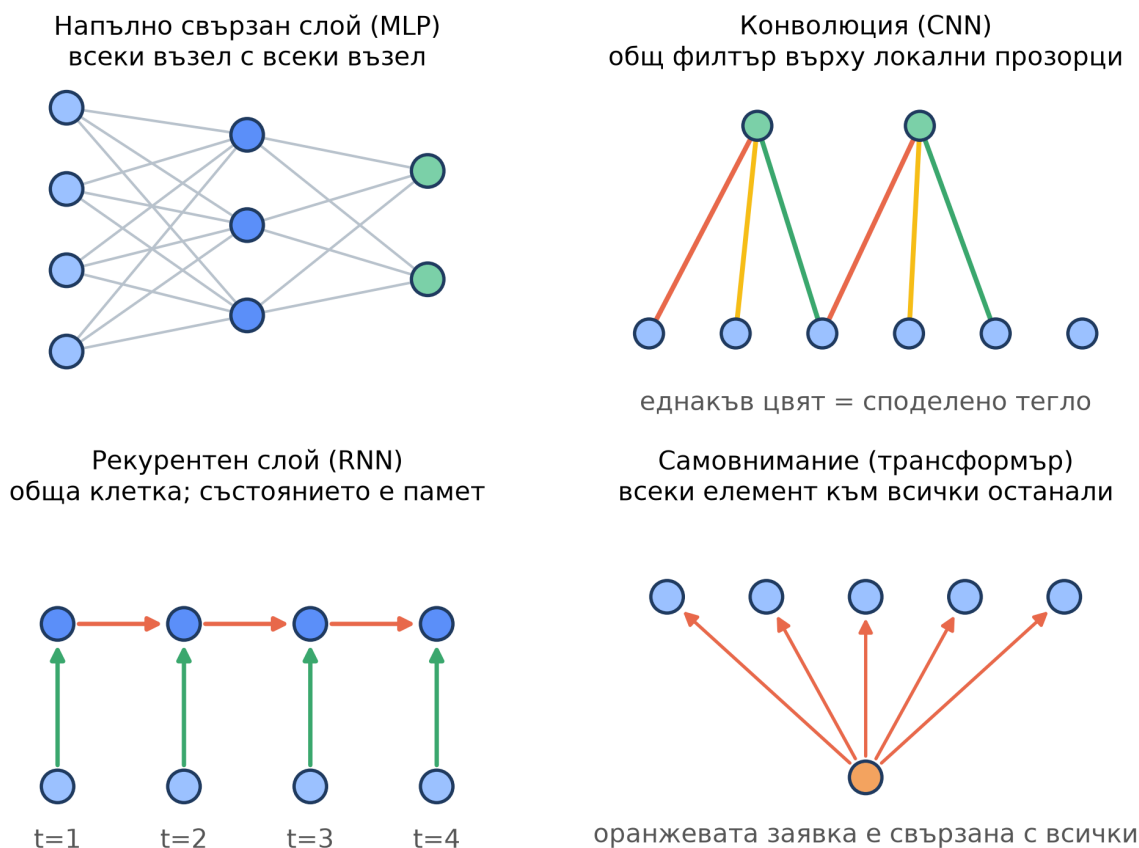
Всяка архитектура кодира предположения относно структурата на своя вход още преди да види каквито и да е данни; тези вградени предположения представляват нейното *индуктивно пристрастие*. Напълно свързаната мрежа не предполага нищо – всяка входна характеристика може да взаимодейства с всяка друга – което я прави обща, но изисква голям обем данни, тъй като мрежата трябва да научи структурата от нулата. Изборът на архитектура, чието пристрастие съответства на структурата на играта, намалява броя на параметрите и количеството необходими данни за постигане на определена точност, както и подобрява обобщаването към състояния, които не са били част от обучението. Решенията за архитектура в останалата част от раздел 5.2 по същество представляват избор на индуктивно пристрастие.

Три съответствия покриват повечето случаи в игри с **непълна информация**. Когато **състоянието** е пространствено – дъска или решетка – **конволюционната мрежа** предполага, че един и същ локален модел е смислен, независимо къде се появи, така че характеристика, научена в една област, може да бъде приложена към други; тази *еквивариантност спрямо трансляция* обяснява защо конволюциите са стандартният избор за дъски и решетки.⁶² Когато част от **състоянието** е неподредена съвкупност – картите в ръката или множеството от останалите играчи – **мрежата** трябва да бъде *инвариантна спрямо пермутации*, като произвежда един и същ изход, независимо от реда, в който тези елементи са представени, за да не се използва капацитет за научаване на това, че редът е без значение. Когато **състоянието** представлява история, разкрита с течение на времето, както при **частична наблюдаемост**, **рекурентна мрежа** или **механизъм за внимание** позволяват на **мрежата** да обобщи последователността и да пренесе напред изводите от по-ранните наблюдения. Съгласуването на тези пристрастия със структурата на играта е разликата между **мрежа**, която се учи ефективно, и такава, която само би могла по принцип.

5.2.2 Типове слоеве на практика: MLP, CNN, RNN, внимание/трансформър

Семействата се различават по своята *свързаност* – кои единици са свързани с кои – и дали свързващите *тегла* се *споделят* между позиции или във времето. Всичко останало (невронът, нелинейността, обратното разпространение на грешката) е идентично. В панела на конволюцията цветът на връзката обозначава теглото: еднаквият цвят означава споделено тегло, а в останалите панели цветът само различава видовете връзки. Следващите параграфи описват всеки панел.

⁶²LeCun, Y., Bengio, Y. & Hinton, G. (2015). “Deep learning.” *Nature* 521, 436–444. DOI 10.1038/nature14539.



Фигура 17: Свързаност на четирите семейства слоеве: напълно свързан слой (MLP), конволюция, която прилага един и същ филтър към локални прозорци, рекурентна клетка, използвана многократно във времето, и самовнимание.

Напълно свързан слой – многослоен перцептрон, или MLP – свързва всяка единица с всяка единица в следващия слой, като всяка връзка носи свое собствено тегло. Той не предполага никаква структура и може да представи всяка зависимост по принцип, но броят на теглата е произведение от размерите на двата слоя и цялата структура трябва да бъде научена от данните. Това е правилният избор за плосък вектор на характеристики без пространствено или последователно значение, като например информационно състояние в покер.

Конволюционният слой (CNN) свързва всяка изходна единица само с малък локален прозорец от входа – неговото *приемно поле* – и прилага един и същ малък набор от тегла, *филтърът*, на всяка позиция. От тази една промяна в свързаността следват две последствия: броят на теглата зависи от размера на филтъра, а не от размера на входа, и модел, научен на едно място, се разпознава на всяко друго място (еквивариантност спрямо трансляция). Наслагването на няколко конволюционни слоя увеличава ефективното приемно поле, така че по-късните слоеве комбинират локални модели в по-големи. Това е стандартният избор за дъски и решетки.

Споделяне на тегла: повторно използване на едни и същи тегла на много позиции (CNN) или времеви стъпки (RNN). Това е, което прави тези подредби параметърно-

фективни и е механизмът зад тяхната еквивариантност.

Рекурентният слой (RNN) прилага една и съща клетка последователно по дължината на последователността: на всяка стъпка той приема текущия вход заедно със *скритото състояние* от предишната стъпка и произвежда ново скрито състояние, което се предава напред. Тъй като клетката е споделена между стъпките, броят на теглата не зависи от дължината на последователността, а скритото състояние функционира като памет за всичко видяно до момента – свойството, необходимо когато състоянието представлява история, разкривана с течение на времето, както при частична наблюдаемост. Вариантите с гейтове (LSTM, GRU) добавят научени гейтове, които контролират какво да се запази и какво да се изхвърли, което им позволява да задържат информация за по-дълги интервали от време.

Слой за **самовнимание** (трансформър) позволява на всеки елемент да взаимодейства директно с всеки друг. Всеки елемент излъчва *заявка*, *ключ* и *стойност*; изходът за даден елемент представлява претеглена средна стойност на стойностите на всички елементи, като теглата се определят от сходството между неговата заявка и всеки ключ. Тъй като операцията не отчита реда на елементите, тя е еквивариантна спрямо пермутации (пренареждането на входа пренарежда изхода по същия начин), а редът трябва да бъде подаден изрично чрез **позиционни кодировки**, когато има значение. Самовниманието улавя **дългосрочни зависимости**, до които рекурентният слой достига само косвено, за сметка на изчисления, чиято сложност нараства квадратично спрямо броя на елементите. То е подходящо за истории с променлива дължина и входове със стойности от множества.

Вход, който е наистина неподреден – например картите в ръката или множеството от останалите играчи – изисква **инвариантност спрямо пермутации**, която не се постига чрез нов тип слой, а чрез подредба: прилага се една и съща малка мрежа към всеки елемент, след което резултатите се комбинират с операция, независима от реда, като сума, **средна стойност** или максимум. Това е принципът зад Deep Sets и представлява естественият **кодировчик** за колекция, чийто ред няма значение.

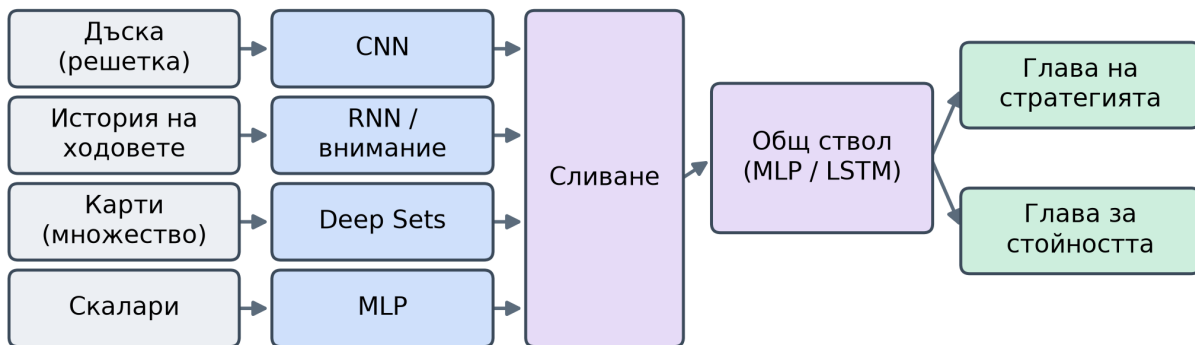
На практика една мрежа рядко използва само едно семейство архитектури. Обичайният дизайн е конвейер: отделен **кодировчик**, съобразен с всяка част от входа, изходите на кодировчика се *обединяват* чрез конкатенация в общ *ствол*, и една или повече *глави*, които извличат прогнозите от ствола – обикновено глава на стратегията и стойностна глава. Deep Recurrent Q-Network (DRQN) комбинира конволюционен кодировчик, LSTM и изходен MLP, като рекурентността осигурява памет, която едно преминаване напред няма при частична наблюдаемост – естественият шаблон за агент в решетъчна среда с „мъгла на войната“. ⁶³ AlphaStar отива по-далеч: кодира множеството от единици с трансформър, пространствената миникарта с CNN и скаларните величини с MLP, след което обединява трите представяния и ги подава през LSTM ядро към изходните глави за действия. ⁶⁴ Практическият урок за останалата част от раздел 5.2 е, че „коя архитектура“ обикновено

⁶³Hausknecht, M. & Stone, P. (2015). “Deep Recurrent Q-Learning for Partially Observable MDPs.” *arXiv:1507.06527*.

⁶⁴Vinyals, O. et al. (2019). “Grandmaster level in StarCraft II using multi-agent reinforcement learning.” *Nature* 575, 350–354. DOI 10.1038/s41586-019-1724-z.

означава „кой кодировчик за всеки вход и как да ги комбинираме“, а не един-единствен избор.

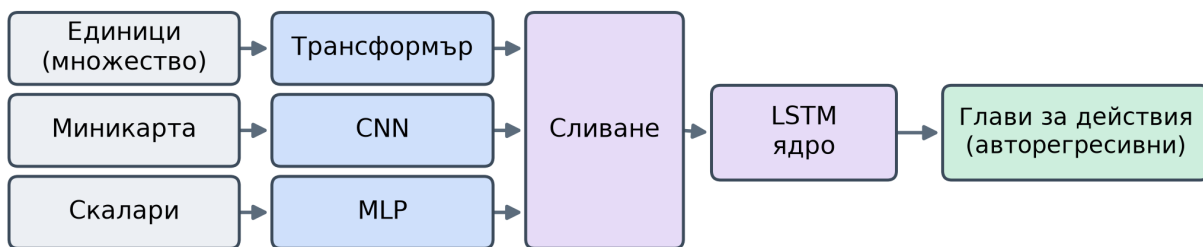
Обща схема: кодировчици → сливане → общ ствол → изходни глави



Пример 1 — DRQN (памет при частична наблюдаемост)



Пример 2 — AlphaStar



Фигура 18: Общата схема с кодировчици, общ ствол и изходни глави (горе) и два нейни примера (долу): DRQN и AlphaStar.

5.2.3 Кодирание на състоянието на играта и историята

Преди някоя от тези архитектури да може да работи, състоянието на играта трябва да бъде превърнато в числа. Информационното състояние – всичко, което действащият играч може законно да наблюдава – трябва да стане тензор с фиксирана форма, а начинът, по който е подредено, определя коя архитектура може да го използва. Категориалните факти се кодират или като *one-hot* вектори – вектор с дължина k , съдържащ една единствена единица, която маркира коя от k категории е валидна – или, когато k е голямо, като *вграждания*: кратки научени вектори, които се извличат за всяка категория и се обучават заедно с мрежата. Ледюк покер е достатъчно малък, за да използва *one-hot* навсякъде: играчът на ход, тайната карта, общата карта и историята на залозите стават *one-hot* полета, което дава трийсетмерния вектор на OpenSpiel; размерът на пота не се подава отделно, защото следва от историята на залозите.

One-hot срещу вграждане: *one-hot* е точен, но неговата дължина нараства с броя на

категориите; едно вграждане компресира голям речник – всяка карта или всеки елемент от дъската – в няколко научени измерения и позволява на подобни категории да споделят структура.

По-богатите входове изискват по-богати подредби. Пространственото състояние се кодира като поредица от наслоени решетки, по една *канална* равнина за всеки тип обект, така че конволюцията да може да го обработи пряко. Pommerman – игра в стил Bomberman върху решетка, в която агентите се движат по дъска, поставяйки бомби, за да разрушават стени и да прихващат противници – е ясен пример: неговата дъска 11×11 е разделена на отделна двоична равнина за всеки тип обект (проходи, дървени стени, твърди стени, бомби, пламъци, бонуси, действащият агент и всеки противник), подредени в тензор (11, 11, канали); скаларни величини като оставащите боеприпаси и обхватът на взрива след това се добавят като кратък вектор или се излъчват като постоянни равнини. История, чийто ред има значение, се кодира като *последователност* от вектори за всяка стъпка за рекурентен слой или трансформър, вместо да бъде сплескана в един вектор, който пренебрегва реда. Непоредена колекция, като текущия набор от противници, се кодира елемент по елемент и се обединява. Във всеки случай кодирането е мястото, където индуктивното приращие от предишните раздели всъщност се прилага: решетката се подава като решетка именно за да са в сила предположенията на конволюцията.

Една дисциплина е специфична за игри с **непълна информация** и лесно може да бъде нарушена: кодирането трябва да съдържа само това, което играчът може да наблюдава, и нищо скрито. Ако каквато и да е **привилегирована информация** – **тайната карта** на противника или истинското съдържание на клетка, покрита с мъгла – попадне в тензора за вход, мрежата ще се научи да я използва и ефективно ще „вижда“ през мъглата, което води до стратегия, която не може да бъде приложена срещу реален противник, и до показател за **експлоатируемост**, който е безсмислен. Ето защо рамки като OpenSpiel дефинират изрично представяне на **информационно състояние** за всеки играч; правилното получаване на това представяне е изискване за коректност, а не избор за настройка.⁶⁵

Остават две по-дребни практически бележки. Непрекъснати признаци като **пота** или **размера на стака** се нормализират до сравним мащаб, преди да постъпят в мрежата, така че нито един признак да не доминира градиента. А кодировчикът не е задължително да бъде проектиран ръчно от край до край: една мрежа може да се научи да компресира дълга история в компактен обобщен вектор – идея, която стои в основата на научените абстракции и на научените състояния на убеждение (глава 6).

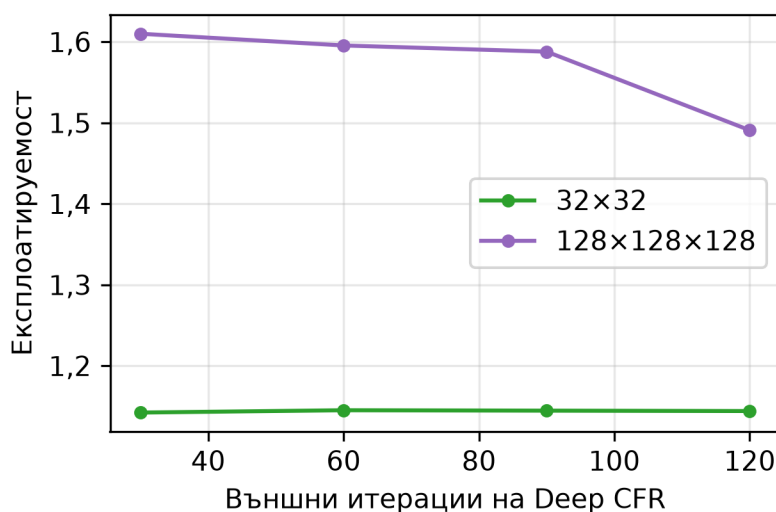
5.2.4 Оразмеряване и капацитет

Капацитетът на една *мрежа* – определен от нейната ширина, броя на единиците във всеки слой, и нейната дълбочина, броя на слоевете – е размерът на семейството от функции, които тя може да представи. При твърде малък капацитет мрежата остава *недообучена* (underfitting) и не може

⁶⁵ Lanctot, M. et al. (2019). “OpenSpiel: A Framework for Reinforcement Learning in Games.” *arXiv:1908.09453*.

да изрази целевата стратегия; при твърде голям капацитет тя се *преобучава* (overfitting), като напасва шума в ограничения набор от данни вместо основната структура. Правилният капацитет е този, който съответства на количеството информативни данни, което процедурата за обучение действително предоставя – а при решаващ на игра това често е значително по-малко от суровия брой състояния, тъй като самите стойности, които се пасват, представляват шумни оценки.

Експериментите в тази глава показаха това пряко. Прилагайки Deep CFR върху Ледюк при фиксиран бюджет, но с различни размери на мрежата, най-малката мрежа – два скрити слоя от тридесет и две единици – постигна по-ниска експлоатируемост (1,14) от трислойната мрежа, четири пъти по-широка (1,49). Вероятното обяснение е, че няколкото хиляди обхождания не стигат, за да се обучат по-големите мрежи; при едно начално число и почти хоризонтални криви това е хипотеза, а не измерен ефект. Трети опит, с два слоя от по шестдесет и четири единици, е изключен: той остава на 1,70 във всяка контролна точка – стойността, която решаващът даваше преди поправката на грешката му (раздел 5.2.5), затова изчаква повторно изпълнение. Изводът не е, че по-малкият размер винаги е по-добър, а че капацитетът трябва да съответства на бюджета за данни: при по-голяма игра или с много повече обхождания класирането вероятно би се обърнало.

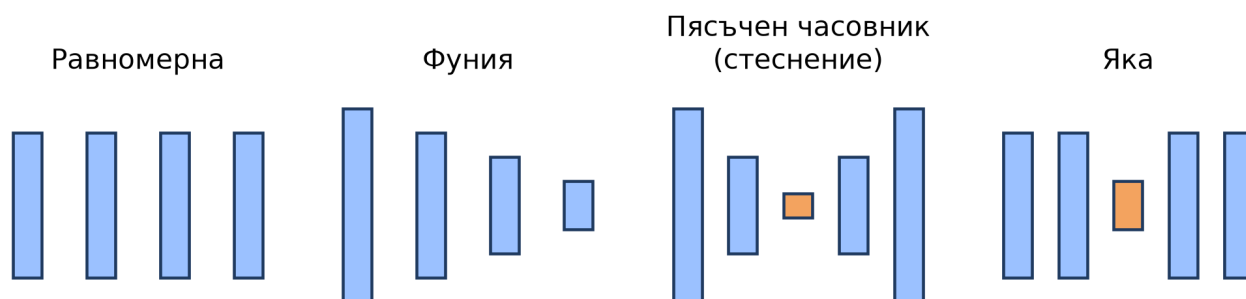


Фигура 19: Експлоатируемост на Deep CFR в Ледюк при два размера на мрежата (32×32 и 128×128×128; едно начално число). Опитът с 64×64 е изключен до повторното му изпълнение.

Това е аргумент в полза на съзнателното оразмеряване на мрежата, вместо да се използва по подразбиране най-голямата, която се побира в паметта. Работеща рецепта е да се започне с малък размер и ширината и дълбочината да се увеличават само докато задържан показател – загуба при валидиране или експлоатируемост за решаващ – продължава да се подобрява, като се спира, когато достигне плато. Ширината позволява на мрежата да представя повече различни модели на дадено ниво на абстракция; дълбочината ѝ позволява да съставя прости модели в по-абстрактни, поради което пространствените и последователните проблеми се възнаграждават с няколко слоя, докато плосък вектор на характеристики често не. Капацитетът трябва да се мащабира спрямо

размера и богатството на играта, а не да бъде максимизиран по подразбиране.

Ширината на мрежата не е задължително да остава постоянна във всичките ѝ слоеве. Те могат да се *свиват* към изхода, като компресират входа до малко обобщение (кодировчик или *фуния*), или да се *разширяват* от малък вход, за да реконструират структуриран изход (декодер). Комбинирането на двете формира *пясъчен часовник*, чиято тясна среда – *стеснение* – принуждава цялата информация да премине през нискоразмерен код; тъй като мрежата трябва да действа или реконструира въз основа на този код, тя запазва единствено най-същественото, което регулира модела и води до компактно, многократно използвано представяне – зародишът на научена абстракция или на състояние на убеждението. Същото стесняване, вградено в иначе равномерна купчина от слоеве, представлява *яка*. Фигурата по-долу илюстрира тези профили.



Фигура 20: Типични форми на мрежата: равномерна поредица от слоеве, стесняваща се фуния, пясъчен часовник с централно стеснение (в оранжево) и равномерна поредица с една тясна „яка“ (в оранжево). Височината на всеки правоъгълник показва ширината на слоя.

Стеснението ограничава *ширината*, а *остатъчните (residual) връзки* решават проблема с *дълбочината*, като добавят входа на даден слой към неговия изход, така че градиентите да достигат до дълбоките слоеве и много дълбоките мрежи да стават обучаеми – принципът, залегнал в основата на ResNet и на дълбоките мрежи за стойности от глава 6. И двете допълват отпадането, отслабването на теглата и нормализацията, споменати по-рано: всяка от тях определя начина, по който се използва капацитетът, а не само неговия обем.⁶⁶

Остатъчна (residual) връзка: пряк път, който добавя входа на слой към неговия изход, така че слой трябва да научи само корекция; тя поддържа градиентите силни през дълбоки мрежи и е това, което прави много дълбоките модели обучаеми.

5.2.5 Стабилност на обучението и диагностиката

Две стабилизиращи средства от обсъждането на „смъртоносната тройца“ – целевата мрежа и буферът за възпроизвеждане – вършат по-голямата част от работата по предотвратяване на разходимостта при обучението, основано на оценка на стойностите. Още две се срещат специално в

⁶⁶He, K., Zhang, X., Ren, S. & Sun, J. (2016). “Deep Residual Learning for Image Recognition.” *CVPR*, 770–778. DOI 10.1109/CVPR.2016.90.

невронните решаващи на игри по-нататък. *Извадка от резервоар* поддържа фиксиран по размер, равномерно случаен образец от всички данни, видени досега, което е важно, когато дадено количество трябва да бъде осреднено върху цялата история на обучението, а не само върху последните данни; Деер CFR го използва, така че неговата стратегическа мрежа да се обучава върху неизместена извадка през всички итерации (раздел 5.3).⁶⁷ *Намаляване на дисперсията* – чрез изваждане на базова линия от взетите извадкови стойности или използване на контролни променливи⁶⁸ – намалява шума, който извадката внася в градиента, така че мрежата да се сближава с по-малко извадки. Нормализиране на входа и разумна инициализация на теглата, споменати по-рано, допълват списъка: и двете поддържат активациите и градиентите в използваем диапазон още от първата стъпка.

Извадка от резервоар: метод, който поддържа извадка с фиксиран размер, при която всеки елемент, видян досега, има еднаква вероятност да бъде запазен, така че текущата средна остава неизместена, без да се съхраняват всички данни.

Дори и с тези мерки на място, невронният решаващ се проваля по-често тихо, отколкото шумно, така че четенето на диагностиката е от значение. Загуба, която се променя, докато оценителният показател – експлоатируемост за решаващ – застива, е признак, че мрежата *се обучава, но още не е достигнала сходимост*: обучението работи, но бюджетът е твърде малък, за да може средната стратегия да се установи, което показаха мрежите от експериментите в тази глава при Ледюк на няколко хиляди обхождания. Обратният и по-опасен случай е, когато обучението изобщо не протича, въпреки че изглежда така: при изследването едносимволна грешка в рутина на OpenSpiel остави мрежите за предимство необучени, но програмата завърши изпълнението си и произведе правдоподобен резултат, като експлоатируемостта просто замръзна около 1,69 независимо от броя итерации (равномерно случайната стратегия има 2,37). Дори след поправката опитът с мрежа 64×64 остава на 1,70 във всяка контролна точка, т.е. на същото ниво, затова тази глава не разчита на неговите числа до повторното му изпълнение. Урокът е да се потвърди, че загубата действително се променя и че оценителният показател реагира, преди да се вярва на който и да е резултат – обучение, което тихо не прави нищо, изглежда точно като труден проблем, освен ако кривите не бъдат проверени.

5.3 Невронни мрежи в CFR

5.3.1 От табличен CFR към апроксимация на функция

Табличният CFR поддържа по две стойности за всяко информационно множество: кумулативно контрафактично съжаление за всяко действие, от което се извежда текущата стратегия чрез напасване на съжалението, както и кумулативна стратегия, чиято текуща средна представлява приближението на равновесието на Наш, което алгоритъмът в крайна сметка връща (Глави 2–4). И двете

⁶⁷Brown, N., Lerer, A., Gross, S. & Sandholm, T. (2019). “Deep Counterfactual Regret Minimization.” *ICML*.

⁶⁸Schmid, M., Burch, N., Lanctot, M., Moravčík, M., Kadlec, R. & Bowling, M. (2019). “Variance Reduction in Monte Carlo Counterfactual Regret Minimization (VR-MCCFR) for Extensive Form Games Using Baselines.” *AAAI* 33, 2157–2164.

таблицы се разрастват с увеличаването на броя на информационните множества – стената, описана в началото на тази глава.

Замяната им с невронни мрежи е концептуално проста: апроксиматор на функции приема тензор на информационно състояние като вход и предсказва това, което таблицата би съхранявала. Една мрежа може да научи съжаленията – еквивалентно, контрафактичните предимства – които определят текущата стратегия; друга може да научи средната стратегия директно. Тъй като една мрежа обобщава подобни информационни състояния, тя никога не трябва да ги изброява и нейният размер е фиксиран от параметрите ѝ, а не от играта. Методите по-долу се различават основно по това *коя* от двете таблици апроксимират и *как* генерират данните за обучението ѝ: Deep CFR научава предимства от извадкови обхождания на дървото, докато NFSP научава средната стратегия от самообучение без изобщо да обхожда дървото.⁶⁷

5.3.2 Deep CFR и неговите варианти с една мрежа

Deep CFR (Brown и др., 2019) е прекият невронен аналог на MCCFR. При всяка итерация той обхожда дървото на играта чрез външно вземане на проби и във всяко информационно множество, което посещава, изчислява извадковото контрафактично предимство на всяко действие – колко по-добро е било това действие от текущата стратегия средно. Тези двойки (информационно състояние, предимство) се записват в резервоарен буфер и *мрежа за предимства*, по една за всеки играч, се обучава да ги предсказва; текущата стратегия във всяко информационно множество след това се извежда от предсказаните предимства на мрежата чрез напасване на съжалението, точно както би била използвана таблицата. Отделна *стратегическа мрежа*, обучавана от собствен резервоарен буфер, научава средната стратегия през всички итерации – приближението на равновесието на Наш, което се връща в края.

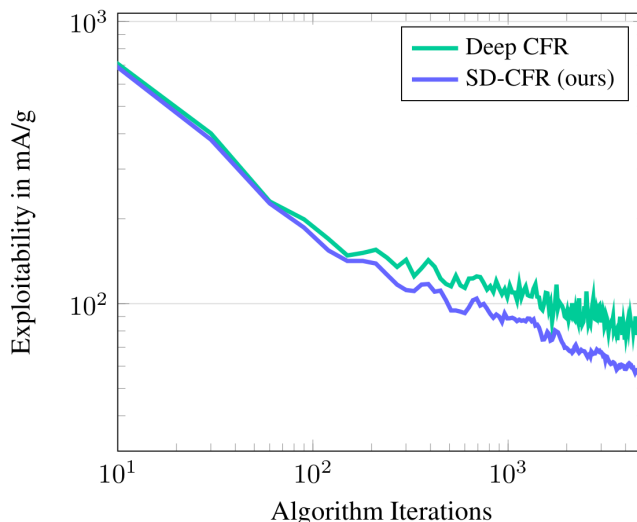
Единственото уравнение, което отличава Deep CFR от табличния MCCFR, е целевата функция на мрежата за предимства: средноквадратична грешка между нейната прогноза и извадковите предимства,

$$L(\theta) = \frac{1}{|B|} \sum_{(I, \tilde{a}) \in B} \|f_{\theta}(I) - \tilde{a}\|^2,$$

където B е партида, изтеглена от резервоара, I тензор на информационно състояние, $\tilde{a}$ векторът на извадковите предимства за неговите действия и f_{θ} мрежата за предимства (на практика пробите се претеглят според итерацията, отразявайки линейния CFR). Всичко останало – обхождането на дървото, стъпката на напасване на съжалението, осредняването – се запазва непроменено спрямо табличния алгоритъм.

Последваха две усъвършенствания. *Single Deep CFR* премахва отделната стратегическа мрежа: пази мрежата за предимства от всяка итерация и възстановява средната стратегия пряко от тях, с което отпада грешката от приближаването на средната стратегия. В Ледюк то достига същата или

малко по-ниска експлоатируемост от Deep CFR, а в пряк двубой в по-голямата игра 5-Flor Hold'em го побеждава.⁶⁹ DREAM прави метода безмоделен, т.е. той вече не изисква перфектен симулатор на играта: вместо външно вземане на проби използва извадка по резултати (по една траектория на обхождане) и добавя научена базова линия като контролна променлива, която поема допълнителната дисперсия – същата идея за намаляване на дисперсията от раздел 5.2.⁷⁰ ESCHER отива още по-далеч: премахва изцяло претеглянето по значимост (importance sampling), като оценява съжалението чрез научена функция на стойността на историята, и в пряк двубой в „тъмен шах“ (dark chess) побеждава DREAM и NFSP в над 90% от игрите.⁷¹



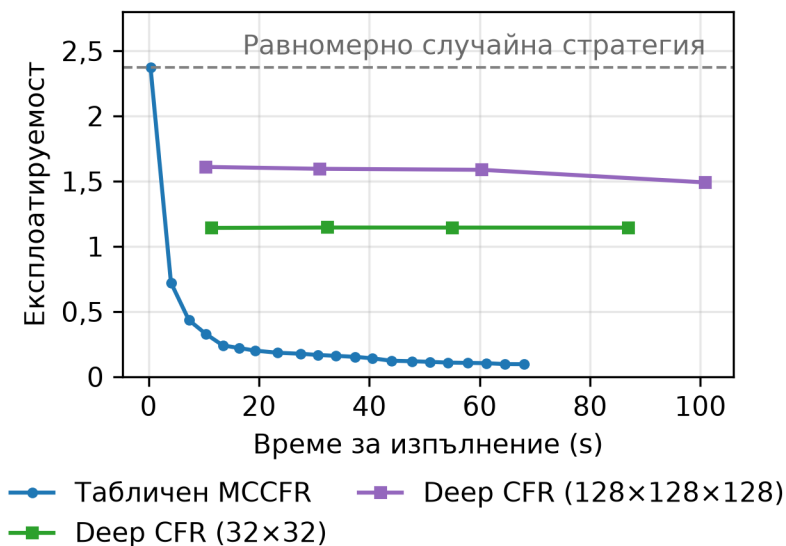
Фигура 21: Single Deep CFR (SD-CFR) достига същата или малко по-ниска експлоатируемост от Deep CFR в Ледюк (в хилядни от анте на игра, mA/g), без да обучава мрежа за средната стратегия. Фигурата е от статията на Steinberger (2019), фиг. 1a, и не представя експеримент от настоящата работа.

В Ледюк изследването потвърди както обещанието, така и уловката. Deep CFR се обучаваше – загубите на мрежите за предимства се променяха (и растяха, защото целите се претеглят с номера на итерацията) – но на 120 итерации експлоатируемостта му остана над 1,1 при всички размери на мрежата (1,14 в най-добрия случай), срещу 0,097 за табличния MCCFR при сравнимо време за изпълнение. Това само по себе си не е дефект: Ледюк е достатъчно малък, за да може табличният подход да спечели категорично, а на Deep CFR е необходим много по-голям бюджет от извадки, за да се доближи до равновесие на Наш (тук с 40 обхождания на итерация срещу 1500 в настройката на Steinberger за Ледюк). Неговото предимство се проявява едва когато играта е твърде голяма, за да съществува таблица – режимът, който раздел 5.3.4 уточнява.

⁶⁹Steinberger, E. (2019). “Single Deep Counterfactual Regret Minimization.” *arXiv:1901.07621*.

⁷⁰Steinberger, E., Lerer, A. & Brown, N. (2020). “DREAM: Deep Regret Minimization with Advantage Baselines and Model-free Learning.” *arXiv:2006.10410*.

⁷¹McAleer, S., Farina, G., Lanctot, M. & Sandholm, T. (2023). “ESCHER: Eschewing Importance Sampling in Games by Computing a History Value Function to Estimate Regret.” *ICLR*; *arXiv:2206.04122*.



Фигура 22: Експлоатируемост на Deep CFR (32×32 и 128×128×128) и на табличния MCCFR в Ледюк спрямо времето за изпълнение; пунктирната линия е равномерно случайната стратегия.

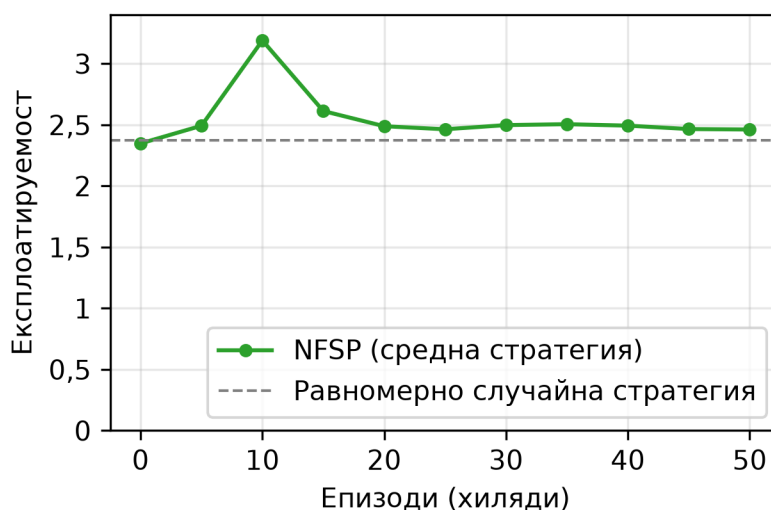
5.3.3 NFSP – Neural Fictitious Self-Play

NFSP – Neural Fictitious Self-Play (Heinrich & Silver, 2016) – достига приблизително равновесие на Наш от страната на обучението с подкрепление, без никога да обхожда дървото на играта. Всеки играч поддържа две мрежи: *мрежа за най-добър отговор* (DQN), която научава алчен най-добър отговор на текущото поведение на опонента, и *мрежа на средната стратегия*, която чрез обучение с учител върху собствените минали действия на играча при игра с най-добрия отговор научава средното по времето на тези най-добри отговори. Средната стратегия е тази, която клони към равновесие на Наш – същото осредняване, което CFR извършва, но постигнато чрез игра, а не чрез съжаление.

Двете мрежи са свързани чрез *антиципиращия параметър* η : в началото на всеки епизод агентът избира с вероятност η да играе с мрежата за най-добър отговор, а в останалите случаи – с мрежата на средната стратегия. Играта с най-добър отговор продължава да генерира свежи данни от разпределението, които средната стратегия усвоява, докато средната стратегия осигурява стабилния опонент, спрямо когото се изчислява най-добрият отговор. Не е необходим резервоар от обхождания на дървото – данните идват единствено от епизоди на самообучение.

Цената е ниската ефективност по отношение на данните. В играта Ледюк изследването стартира NFSP за десетки хиляди епизоди и наблюдава колебание на експлоатируемостта около 2,5, близо до нивото на равномерно случайната стратегия (2,37), вместо спад, тъй като достигането на използваемо равновесие дори в тази малка игра изисква порядъка на един милион епизоди или повече – собственият пример на OpenSpiel за Ледюк използва още повече епизоди. NFSP се мащабира до големи игри, където безмоделната му простота е предимство, но в учебна игра като

Ледюк той се сближава значително по-бавно от табличния CFR или Deep CFR.⁷²



Фигура 23: Експлоатируемост на NFSP в Ледюк в хода на 50 000 епизода обучение: кривата остава около нивото на равномерно случайната стратегия (пунктир, $\approx 2,37$).

5.3.4 Компромиси: кога невронният CFR е от полза

В обобщение: каква е реалната полза от невронните мрежи в CFR и каква е цената им? Ползата е обобщаване и ограничена памет – една мрежа споделя структура между сходни информационни множества и съхранява стратегия във фиксиран брой параметри, така че изчисляването на равновесие вече не изисква изброяване или дори посещение на всяко информационно множество. Цената е висок разход на итерация и по-слаби гаранции: всяка итерация обучава мрежи вместо да увеличава броячи, сходимостта е по-шумна, а чистото монотонно поведение на табличен CFR става приблизително. Резултатите от изследването в Ледюк показват тази страна на компромиса при малки игри: когато таблицата се побира в паметта, тя е по-ефективна.⁶⁷

Компромисът се променя при увеличаване на мащаба. С разрастването на играта табличните таблици в крайна сметка изобщо не се побират в паметта, докато отпечатъкът на мрежата остава непроменен; след този преход невронната методика не е просто конкурентна, а единствената възможна. Затова Deep CFR и неговите разновидности са важни за пълномащабен покер, въпреки че губят в Ледюк.

Едно структурно ограничение определя къде се прилагат невронните методи, базирани на съжаление. CFR изчислява контрафактни стойности чрез разсъждения върху цялото поддърво под едно информационно множество, което е евтино, когато дървото е *широко, но плитко* – покерът има огромен брой **раздавания**, но завършва след няколко рунда на залагане. В игри, които са *дълбоки* – със стотици последователни хода, както в една решетъчна среда – това разсъждение

⁷²Heinrich, J. & Silver, D. (2016). “Deep Reinforcement Learning from Self-Play in Imperfect-Information Games.” *arXiv:1603.01121*.

става непосилно скъпо и безмоделните методи за самообучение от раздел 5.4 се превръщат в по-естественото решение. Семейството, базирано на съжаление, продължава да расте в своята ниша – NeuRD, например, променя с един ред обновяването по градиента на стратегията така, че то да следва репликаторната динамика; в табличния случай методът е еквивалентен на softmax CFR⁷³ – но неговата най-добра област на приложение остават широките, плитки игри с непълна информация.

5.4 Други невронни приложения

5.4.1 Четирите водещи фамилии алгоритми

Раздел 5.3 представи един от начините за прилагане на невронни мрежи в игри с непълна информация – чрез приближаване на CFR. Това е едно от четирите големи семейства, по които в тази глава се подреждат съвременните методи, като изборът между тях до голяма степен зависи от съответствието им с мащаба на играта, структурата на дървото и броя на участниците.⁷⁴ Четирите са методи, *базирани на съжаление* (Deer CFR, DREAM, NeuRD), от раздел 5.3; *популационни* методи (PSRO и неговите разновидности, включително NFSP); методи, *съчетаващи търсене и обучение с подкрепление* (ReBeL, Student of Games); и *самообучение без използване на модел* (PPO, DQN, MAPPO, QMIX). Таблицата ги представя; следващите раздели разглеждат трите, които все още не са обхванати.

Таблица 8: Семейства невронни методи за игри с непълна информация: основна идея, приложимост и цена.

Семейство	Основна идея	Най-подходящо за	Цена
Базиран на съжаление (Deer CFR, DREAM, NeuRD)	Приближаване на CFR с невронни мрежи	Широки, плитки игри; неексплоатируемост	Висока
Популационен (PSRO, NFSP, XFP)	Най-добър отговор на нарастваща популация от противници	С n играчи; моделиране на противника	Средна
Търсене + RL (ReBeL, Student of Games)	Научена функция на стойността плюс търсене по време на игра	Игри с високи залози; изчисления по време на игра	Висока
Самообучение без използване на модел (PPO, DQN, MAPPO, QMIX)	Оптимизиране на награда чрез самообучение	Пространствени, с дълъг хоризонт (решетки)	Ниска–средна

⁷³Hennes, D. et al. (2020). “Neural Replicator Dynamics: Multiagent Learning via Hedging Policy Gradients.” AAMAS, 492–501; arXiv:1906.00190.

⁷⁴Rudolph, M. et al. (2026). “Reevaluating Policy Gradient Methods for Imperfect-Information Games.” ICLR; arXiv:2502.08938.

5.4.2 Популационни методи: PSRO, NFSP, XFP

Популационните методи преформулират **намирането на равновесие** като итеративен турнир. Започвайки от малък набор от **стратегии**, се формира *мета-игра* чрез изиграване на членовете на набора един срещу друг; след това се обучава нова **стратегия** като приблизителен **най-добър отговор** на текущата **популация**, добавя се обратно към набора и процесът се повтаря. Това е идеята зад PSRO (Policy-Space Response Oracles) и неговите мащабируеми варианти като Pipeline PSRO; фиктивната **самоигра**, включително NFSP от раздел 5.3, е специалният случай, при който **най-добрият отговор** е насочен към средното по времето на набора. Тъй като **популацията** представлява изричен набор от противници, а не една **подвижна цел**, тези методи естествено излизат извън рамките на игрите с нулева сума за двама играчи и обхващат сценариите с n играчи и моделиране на противника, които са предмет на тази дисертация; популационно обучение от този вид е в основата и на „лигата“ от агенти, с която AlphaStar, първоначално обучен чрез подражание на човешки игри, достига нивото на гротмайстор в StarCraft II.⁶⁴ Цената е поддържането и съхраняването на множество агенти, а гаранциите за **равновесие** са по-слаби от тези на CFR.⁷⁵

5.4.3 Търсене + RL: ReBeL и Student of Games

Методите търсене плюс обучение с подкрепление представляват идеята на AlphaZero, адаптирана към скрита информация. В игра с пълна информация може да се извършва търсене напред от текущото състояние, тъй като то е известно; при непълна информация търсещият играч не знае действителното състояние, затова търсенето трябва да обхваща разпределение на вероятностите върху възможни състояния. ReBeL (Brown и др., 2020)⁷⁶ прави това прецизно чрез *публичното състояние на убежденията* (PBS) – разпределение на вероятностите върху частната информация на играчите, предвид всичко публично – и изпълнява CFR върху малка, ограничена по дълбочина под-игра по време на игра, като научена мрежа за стойности предоставя стойностите на листата. Student of Games (Schmid и др., 2023; в предварителната версия – Player of Games)⁷⁷ обобщава този подход в един алгоритъм, който обработва както пълна, така и непълна информация. Тези системи са в основата на най-силните резултати в покера (ReBeL достига свръхчовешко ниво в безлимитния тексаски холдем за двама играчи), но са най-сложни за изграждане и зависят от изчисления, извършвани *по време* на игра, а не само по време на обучение; те са предмет на Глава 6, така че тази секция само ги позиционира в общата картина.

5.4.4 Самообучение без използване на модел: PPO, DQN, MAPPO, QMIX

Методите за самообучение без използване на модел изцяло игнорират дървото на играта и оптимизират директно сигнала за награда, усъвършенствайки стратегията от епизоди на игра срещу

⁷⁵Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO).

⁷⁶Brown, N., Bakhtin, A., Lerer, A. & Gong, Q. (2020). “Combining Deep Reinforcement Learning and Search for Imperfect-Information Games.” *NeurIPS* 33; arXiv:2007.13544.

⁷⁷Schmid, M. et al. (2023). “Student of Games: A unified learning algorithm for both perfect and imperfect information games.” *Science Advances* 9(46), eadg3256. DOI 10.1126/sciadv.adg3256.

собствени копия. Те са сред най-използваните методи в многоагентното обучение с подкрепление: PPO⁷⁸ и DQN⁷⁹ за едноагентното ядро и многоагентните разширения MAPPO⁸⁰ и QMIX⁸¹ за кооперативни сценарии. Тъй като никога не обмислят поддържането под дадено състояние, те не се затрудняват от дълбочината, която пречи на методите, основани на съжалението, което ги прави естествения избор за пространствени игри с дълъг хоризонт – решетъчни среди, Pommerman и подобни. За да се справят с частичната наблюдаемост, те използват рекурентните архитектури и архитектурите с внимание от раздел 5.2, предоставяйки на агента памет за това, което мъглата е скрила. Наивното самообучение с алгоритъм за един агент може да се върти в цикъл и да остане силно експлоатируемо. Затова по-новите безмоделни методи променят динамиката на обучението (NeuRD,⁷³ R-NaD в DeepNash,⁸² магнитно огледално спускане, MMD⁸³), така че играта да се доближава до равновесие. Масово сравнение в пет игри с нулева сума за двама играчи показва, че специализираните методи NFSP, PSRO, ESCHER и R-NaD не превъзхождат по експлоатируемост общите методи с градиент на стратегията като PPO и MMD.⁷⁴ Затова експлоатируемостта на всеки безмоделен агент трябва да се измерва, а не да се предполага.

⁷⁸Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.

⁷⁹Mnih, V. et al. (2013). “Playing Atari with Deep Reinforcement Learning.” arXiv:1312.5602; Mnih, V. et al. (2015). “Human-level control through deep reinforcement learning.” *Nature*, 518(7540), 529–533.

⁸⁰Yu, C. et al. (2022). “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games.” *NeurIPS Datasets and Benchmarks*; arXiv:2103.01955.

⁸¹Rashid, T. et al. (2018). “QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning.” *ICML*; arXiv:1803.11485.

⁸²Perolat, J. et al. (2022). “Mastering the game of Stratego with model-free multiagent reinforcement learning.” *Science* 378, 990–996. DOI 10.1126/science.add4679.

⁸³Sokota, S. et al. (2023). “A Unified Approach to Reinforcement Learning, Quantal Response Equilibria, and Two-Player Zero-Sum Games.” *ICLR*; arXiv:2206.05825.

6 Глава 6 – Архитектури на игрови изкуствен интелект от край до край

6.1 Въведение

Глава 6 е крайъгълният камък на този изследователски план. Глави 1–5 изграждат поотделно съставните елементи – игровитеоретичния речник на игрите в разгърната форма и равновесията на Наш, минимизирането на контрафактичното съжаление (CFR) и неговите **варианти на Монте Карло**, абстракцията на играта и невронните апроксиматори на функции, които заменят табличното съхранение, когато една игра го надрасне – и тази глава е мястото, където тези елементи се сглобяват в пълноценни системи от състезателно ниво. Подходът е умишлено **архитектурен**: вместо да се извеждат отново математическите зависимости, които са представени в техническите глави и цитираните статии, всяка система се разглежда на нивото на това как е *изградена* – какво изчислява **офлайн**, какво изчислява в **реално време**, къде е обучението и какво всъщност може да гарантира. Всеки раздел започва с кратка обобщаваща таблица по едни и същи девет измерения, така че системите да се сравняват с един поглед, а текстът след това проследява дизайнските решения, инженерните компромиси и изоставените подходи зад тях.

Петте системи обхващат седем години и, прочетени в ред, проследяват еволюцията на свръхчовешкия игрови изкуствен интелект: **DeepStack** (2017), **Libratus** (2017/2018), **Pluribus** (2019), **ReBeL** (2020) и **Student of Games** (2023). Изкушаващо е да се прочете такъв списък като класация – всяка следваща по-силна от предишната – но това нито се е случило, нито така е организирана тази глава. Прогресията е последователност от *умишлени компромиси*: всяка система придобива нова способност, като се отказва от нещо друго, а най-показателните моменти са тези, в които областта прави странична стъпка или дори отстъпление. **DeepStack** и **Libratus** са почти съвременници, които предлагат *противоположни решения* на едно и също заболяване – **DeepStack** изоставя парадигмата „абстракция и план“, която доминираше в компютърния покер близо две десетилетия, докато **Libratus** я запазва и лекува единствения ѝ фатален недостатък с решаване на под-игри в реално време, което е доказуемо безопасно. **Pluribus** след това пробива *барьерата на броя играчи*, достигайки свръхчовешка игра с шестима участници, но само като се отказва изцяло от гаранции за безопасност и остава изцяло табличен. **ReBeL** се връща към двама играчи, но прави огромен скок в обобщаването – научавайки стойности на състоянието на убеждението със самообучение в стила на **AlphaZero**, премахвайки както абстракцията, така и плана, и *възстановявайки* безопасността, от която **Pluribus** се беше отказал. **Student of Games**, като завършек, обединява игра с пълна и непълна информация в един алгоритъм – и плаща за тази широта с пикова сила, губейки решително от специализиран **AlphaZero** в Го. Прочетена по този начин, главата е изследване на цената на всеки напредък, а не класация на победители.

Под отделните компромиси три оси на движение преминават през всичките пет системи и придават гръбнака на главата. Първата ос е **на представянето**: преходът от ръчно изградена *абстракция* – групиране на подобни ръце и допускане само на няколко размера залози – към *научено невронно приближение*, при което мрежата обобщава ситуации, които таблица никога не би могла да

изброи. Втората е **времева**: преходът от *решаване на цялата игра офлайн* в съхранена стратегия към *научаване на компактен модел и търсене с него в реално време*, така че изчислението се извършва в момента на вземане на решение, а не е предварително заложено. Третата ос се появява едва в самия край, със Student of Games: обединяването на игра с **пълна и непълна информация** в един коректен алгоритъм, съчетаващ двете големи традиции на игровия изкуствен интелект – линията минимакс / търсене в дърво по метода Монте Карло / AlphaZero и линията CFR / покер – които са се развивали по отделни пътища в продължение на седемдесет години. Първите две оси описват четирите покер системи; третата е това, което прави петата завършек на главата.

Една идея свързва тези нишки и заслужава да бъде назована преди самите системи, именно защото тя *не е* една от тях: **ограничено по дълбочина решаване**. Формализирано от Brown, Sandholm & Amos (2018)⁸⁴, то е принципът, че може да се търси само малко напред в игра с непълна информация и остатъкът да се замести с *научена или предварително изчислена стойност* – при условие че заместването се извършва така, че скритата информация да не го прави невалидно. Това е теорията, която поставя в обща рамка непрекъснатото пререшаване на **DeepStack** и вложеното решаване на под-игри на **Libratus**, която стои в основата на стратегиите за продължение на **Pluribus** и която, във формата на състояние на убеждението, се превръща във вътрешния цикъл на **ReBeL** и **Student of Games**. Ние го разглеждаме като свързваща тъкан – споменавано навсякъде, където една система го реализира – а не като шеста точка, защото ограниченото по дълбочина решаване е механизмът, *чрез който* всъщност действа оста „от офлайн изчисление към търсене в реално време“.

Накрая, тъй като тази глава е връзката между основите на глави 1–5 и изследванията върху моделирането на противника и експлоатацията в глави 7–12, си струва още в началото да се посочи единствената нишка, към която синтезът се връща. Всяка система тук е, по умишлен дизайн, **„сляпа“ за противника**: всяка изчислява стратегия, която е трудна за побеждаване *в най-лош случай* и след това я прилага без да обръща внимание кой всъщност седи срещу нея – Pluribus дори не знае самоличността на своите опоненти, а и той, и Libratus съзнателно избягват да ги моделират или да се адаптират към тях – защото отклонението с цел експлоатация може да бъде контраексплоатирано и, по думите на авторите на Pluribus, защото съществуващите техники за експлоатация на противника „изискват твърде много наблюдения, за да се конкурират с човешките способности извън малките игри“; и двете статии посочват консервативната (безопасна) експлоатация като единствено изключение.⁸⁵⁸⁶ Този подход, поставящ устойчивостта на първо място, е голямата сила на областта и, за дисертацията относно *адаптивна* игра, нейното определящо ограничение: именно отчитането на конкретния противник, което тези системи пропускат, глави 7–12 се стремят да добавят. Затова главата завършва не с победител, а със синтез – карта на това, което петте системи споделят, какво всяка от тях е жертвала и къде всъщност се намират откритите проблеми,

⁸⁴Brown, N., Sandholm, T. & Amos, B. (2018). “Depth-Limited Solving for Imperfect-Information Games.” *NeurIPS* 31; arXiv:1805.08195.

⁸⁵Brown, N. & Sandholm, T. (2019). “Superhuman AI for multiplayer poker.” *Science*, 365(6456), 885-890.

⁸⁶Brown, N. & Sandholm, T. (2018). “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals.” *Science*, 359(6374), 418-424.

които мотивират останалата част от тази работа.

6.2 DeepStack (2017)

DeepStack (Moravčík et al., 2017)⁸⁷, от компютърно-покер групата на Университета на Алберта с колаборатори в Прага, беше първата програма, която побеждава професионални покер играчи в безлимитния тексаски холдем за двама (HUNL) – професионалисти, макар и не специалисти по HUNL – със статистическа значимост и първата, която поставя евристичното търсене – двигателят зад шахмата и Го – на теоретична основа в игра с непълна информация. В рамките на четириседмично проучване той победи група от 33 професионалисти с 492 хилядни от големия блайнд на раздаване (mbb/g, стандартната единица за темп на печалба в покера; 50 mbb/g представлява значително професионално предимство) в 44 852 раздавания и нито една известна техника не успя да открие пропуск в играта му. Неговият водещ *какво-ако* въпрос е този, който рамкира цялата тази глава: **може ли програма да играе покер по начина, по който AlphaGo играе Го – търсейки локално от текущата ситуация и доверявайки се на научена функция на стойността за всичко извън хоризонта – дори когато в покера самата ситуация е частично скрита?**

Таблица 9: DeepStack (2017) накратко.

С един поглед	DeepStack (2017)
Играчи	2 (един срещу един)
Тип игра	HUNL – безлимитен тексаски холдем за двама (игра за двама с нулева сума)
План (офлайн)?	Не – офлайн обучението е за мрежите за стойност, а не за съхранена стратегия
Невронна компонента	Дълбоки мрежи за контрафактични стойности (за флопа, търна и спомагателна); само стойности
Механизъм за търсене	Непрекъснато пререшаване (ограничено по дълбочина предвиждане с CFR при всяко решение)
Абстракция?	Никаква абстракция не ограничава играта; групиране в 1000 клъстера само на входа на мрежата, плюс разреден набор от залози и групиране на действията на ривъра при предвиждането
Също и с пълна информация?	Не (само непълна информация)
Изчислителни разходи	Предимно офлайн (~175 процесорни ядро-години за генериране на целите на мрежата за търна); по време на игра работи на един GPU, < 5 s/решение

⁸⁷Moravčík, M. et al. (2017). “DeepStack: Expert-level artificial intelligence in heads-up no-limit poker.” *Science*, 356(6337), 508-513.

С един поглед	DeepStack (2017)
Ключова иновация	Непрекъснато пререшаване + научени контрафактични стойности: първото <i>коректно</i> евристично търсене за игри с непълна информация

6.2.1 Пропускът, който запълни

Всеки предишен етап в развитието на изкуствения интелект за игри – табла, шах, Го – се основаваше на *локално търсене*: от текущата позиция се разглеждат няколко хода напред и останалото се замества с евристична стойност. Тази рецепта предполага, че позицията е известна, а покерът нарушава това предположение – правилното действие зависи от вероятностното разпределение на скритите карти на опонента, което се разкрива единствено чрез неговите залози, а те от своя страна зависят рекурсивно от това какво той предполага за вашите карти. Отвореният въпрос, който DeepStack се опита да реши, е дали евристичното търсене може да бъде направено *коректно* при наличие на скрита информация в мащаба на HUNL – игра с приблизително 10^{160} точки на решение – сравнима по обем с Го, но с непълна информация.

Почти две десетилетия доминиращият отговор беше нещо съвсем различно: **абстракция плюс офлайн равновесие плюс транслация**. Свийте 10^{160} ситуациите на HUNL до приблизително 10^{14} абстрактни такива чрез групиране на подобни ръце (*абстракция на карти*) и позволяване само на няколко размера залози (*абстракция на действията*), решете тази по-малка игра офлайн с минимизиране на контрафактичното съжаление (CFR), за да произведете пълна стратегия – *план* – съхранете я и в момента на играта *транслирайте* всяка реална ситуация и залог на опонента към най-близката абстрактна. Компресията е със загуби и загубата се проявява като експлоатируемост – колко може да спечели опонент в най-лошия случай, основният показател за качество в областта, нула при равновесие на Наш. През 2015 г. основаната на абстракция програма Claudico загуби от професионалисти с 91 mbb/g,⁸⁷ а проверка с локален най-добър отговор (LBR, ефективно изчислима долна граница на експлоатируемостта) по-късно показва, че топ състезателните ботове са експлоатируеми с повече от 3000 mbb/g – четири пъти по-зле, отколкото ако се откажеш от всяка ръка.⁸⁸ Плановите също бяха огромни (една стратегия без абстракция на картите отне около 2 TB и 14 процесорни ядро-години за изчисляване и все още беше *експлоатируема* чрез залози извън дървото), а стъпката на транслацията сама по себе си беше източник на слабост.⁸⁷ DeepStack затваря тази празнина, като изоставя цялата конструкция: никога не изгражда абстракция на цялата игра и никога не съхранява план, разсъждавайки за всяка ситуация *така както тя действително възниква* и замествайки само далечното остатъчно продължение на играта с научена оценка.

6.2.2 Архитектура

DeepStack се разделя ясно на **офлайн** фаза, в която се изгражда интуицията, и **онлайн** фаза, в която с нея се търси (вж. фигурата по-долу).⁸⁷

⁸⁸Lisý, V. & Bowling, M. (2017). “Equilibrium Approximation Quality of Current No-Limit Poker Bots.” *AAAI Workshop on Computer Poker* – локален най-добър отговор (LBR); arXiv:1612.07547.



Фигура 24: Архитектура на DeepStack: предварителното (офлайн) изграждане на „интуиция“ (горе) обучава обща дълбока мрежа за контрафактни стойности, която след това служи за оценяване на листата в онлайн цикъла на непрекъснато пререшаване (долу).

Офлайн системата генерира милиони случайни покер ситуации и ги решава с CFR решавач, за да получи целеви *контрафактни стойности* – условни „какво-ако“ печалби за всяка възможна ръка – и тези двойки (ситуация → стойност) обучават **дълбоките мрежи за контрафактни стойности (CFV мрежи)**. Онлайн DeepStack не използва план: между отделните решения той запазва само два вектора – своя *диапазон* (разпределението върху възможните ръце, които може да държи) и *контрафактните стойности* на опонента – а при всеки ход изпълнява CFR върху малко дърво за предвиждане, вкоренено в истинското текущо състояние, като използва CFV мрежата за изчисляване на стойностите на листата на границата на дълбочината. Тъй като CFR вече изчислява както диапазоните, така и контрафактните стойности, той се вписва по естествен начин в този цикъл.

Това е евристично търсене с три компонента: коректно изчисляване на локална стратегия (непрекъснато пререшаване), предвиждане с ограничена дълбочина, при което научената функция на

стойността замества останалата част от играта, и ограничен набор от действия за предвиждане, който поддържа всяко решаване бързо. Невронната мрежа се използва *само* на границата на дълбочината като оценител на листата; търсенето се извършва от CFR; и – най-важното – в цикъла не се прилага абстракция на цялата игра.

6.2.3 Ключова иновация: непрекъснато пререшаване с научени контрафактични стойности

Приносът на DeepStack е в обединяването на две идеи, като всяка от тях прави другата осъществима.

Първата е **непрекъснатото пререшаване**: възстановяване на свежа локална стратегия при всяко решение въз основа на поддържащия диапазон и вектора на контрафактичните стойности на опонента, след което тя се изхвърля, така че агентът никога не съхранява или не се ангажира с глобална стратегия. Класическото пререшаване (Burch и др., 2014) показва, че за да се възстанови стратегия за под-игра, не е необходима цялата стратегия – достатъчни са само собственият диапазон при влизане в под-играта и векторът на контрафактичните стойности на опонента. DeepStack избутва това до предела: той *никога* не съхранява стратегия за цялата игра. Всеки път, когато трябва да действа, той пререшава текущото публично състояние въз основа на тези два вектора, извършва едно действие и изхвърля стратегията.⁸⁹

Всичко това е възможно благодарение на начина, по който се поддържат двата вектора. След всяко събитие DeepStack актуализира своите два вектора по прости правила: при **собствено действие** той заменя пререшените контрафактични стойности за избраното действие и извършва байесови актуализации на своя диапазон; при **случайно събитие** той заменя стойностите на тази карта и анулира вече невъзможните ръце; а при **действието на опонента** той не прави *абсолютно нищо*. Този последен момент е тихият майсторски ход: тъй като той проследява *стойностите* на опонента, а не неговия *диапазон*, и никога не се нуждае от конкретното действие на опонента, за да поддържа тези стойности, той заобикаля стъпката на трансляция на действията, която парализира ботовете, основани на абстракция. Опонентът може да направи всеки залог с всякакъв размер; DeepStack просто пререшава от състоянието, което този залог е произвел.

Втората идея е **научената мрежа за контрафактични стойности** – „интуицията“ на DeepStack. В игра с пълна информация оценителят на листо съпоставя едно състояние към едно число; при непълна информация той трябва да съпостави *цяло публично състояние плюс диапазоните и на двамата играчи* към *вектор* от контрафактични стойности, по една за всяка ръка, защото стойностите в един възел се променят с диапазоните, които достигат до него. DeepStack научава това съпоставяне с мрежа с право разпространение от седем скрити слоя с по 500 неврона, приемайки размера на пота и двата диапазона (компресирани в 1000 клъстера от ръце) като вход и извеждайки контрафактични стойности за всяка ръка като части от пота. Специален външен слой налага

⁸⁹Burch, N., Johanson, M. & Bowling, M. (2014). “Solving Imperfect Information Games Using Decomposition.” AAAI, pp. 602–608 – пререшаване и разширената под-игра.

ограничението за нулева сума: той формира двете имплицитни стойности на играта от диапазоните и суровите изходи и изважда половината от тяхната сума, така че оценките да са взаимно съгласувани, а цялата мрежа да остане диференцируема. С тази мрежа, която доставя стойности в края на текущия рунд за залози, ограничението по дълбочина свива пререшаваната игра от 10^{160} до най-много 10^{17} точки на решение, а описаният по-долу разреден набор от действия – до около 10^7 – достатъчно малко, за да бъде решена за по-малко от пет секунди на един GPU.

Сдвояването е доказуемо коректно.⁸⁷ Ако грешката на мрежата за стойности е най-много ϵ , а пререшаването изпълнява T итерации на CFR, експлоатируемостта на получената стратегия е ограничена от

$$\text{експлоатируемост} < k_1 \epsilon + k_2 / \sqrt{T},$$

с игрово-специфични константи k_1, k_2 . Първият член представлява цената на несъвършената интуиция, а вторият – обичайната **сходимост на CFR**. Тази граница е теоретичното ядро на статията – гаранцията, че евристичното **търсене** може да бъде пренесено в условия на **непълна информация**, без **стратегията** да стане **експлоатируема** – и същият шаблон „ограничено по дълбочина решаване плюс научена стойност“, формализиран допълнително от Brown, Sandholm & Amos през следващата година, е в основата на всяка система в тази глава.

6.2.4 Уговорки, задънени улици и какво статията не описва

Чистият разказ по-горе представлява основния текст, докато инженерната реалност се съдържа в приложението, където именно акцентът върху „ограниченията и еволюцията“ оправдава своето място.

Най-важната забележка е, че **внедрената система не е тази, за която важи теоремата**. За да играе с човешка скорост, DeepStack ограничава своето предвиждане до разреден набор от залози (отказ, плащане, два или три размера на залога, ол-ин) и в статията ясно се посочва, че това „отменя свойството на коректност на Теорема 1“. Така гаранцията на внедрената система е емпирична – подкрепена от резултатите LBR по-долу – а не доказана. Второ отклонение утежнява това: доказателството за коректност предполага стойности на *най-добър отговор*, но DeepStack всъщност използва стойности от *самообучение*, които нямат теоретично обосноваване, но са били по-малко експлоатируеми в ранните тестове. Доказаният алгоритъм и печелившият алгоритъм, строго погледнато, са различни алгоритми.

Абстракцията също така се промъква в периферията. DeepStack рекламира, че не използва абстракция на карти за ограничаване на играта – но *групира* ръцете в 1000 клъстера на входа на мрежата за стойности и на ривъра изцяло се отказва от мрежата, решавайки до края на играта, като използва **абстракция на действия с клъстери** за постигане на управляемост. За да ускори пререшаването, той също така започва от **предварителна оценка** на диапазона на опонента (warm start), в консервативна комбинация през повечето време, но в агресивен вариант (когато дейст-

ва пръв), който „жертва гаранциите за пререшаване, когато оценката на диапазона на опонента е грешна“. И префлопът, далеч от евтин, изисква изброяване на всички 22 100 възможни флопове чрез флоп мрежата, смекчено единствено чрез кеширане на повтарящи се последователности от залагания. И накрая, изборът на приспособление за пререшаване (приспособлението CFR-D пред алтернативата с максимален марж) е направен, защото то „се представи по-добре в ранните тестове“, а не е изведено – и, отразявайки Глава 5, полезната дълбочина на мрежата беше ограничена от данните, а не от архитектурата, като грешката при валидиране се изравняваше след петия слой при бюджет от десет милиона проби. Нито едно от тези не подкопава резултата, но заедно те показват, че заглавието „коректно търсене без абстракции“ е стремеж, който имплементацията приближава, а не постига.

6.2.5 Изчислителни разходи и достъпност

Разходите на DeepStack са почти изцяло **офлайн и почти изцяло в решаването с CFR, чрез което се получават целевите стойности за мрежите** – не в обучението на невронна мрежа и не във времето за игра. Генерирането на целите означаваше приблизително решаване на милиони случайни под-игри: само **мрежата за търна** изразходва около **175 процесорни ядро-години** на 6 144-ядрен клъстер, като **флоп мрежата** добави приблизително половин **GPU-година** на 20 GPU; за разлика от това, самите мрежи се обучиха за около два дни всяка на един GPU и по време на игра DeepStack работи на **един стандартен GPU с под пет секунди на решение**.⁸⁷ Формата на тази сметка е истинският урок: „интуицията“ се купува веднъж, предварително, чрез грубо **намиране на равновесие**, след което внедряването е евтино – огледален образ на система, чиито разходи идват главно от **изчисленията по време на изпълнение**. По отношение на достъпността това постави *от нулата* изграждането в обсега само на добре осигурена лаборатория по онова време (**предварителните изчислителни разходи** са в мащаба на клъстер), въпреки че обучената система след това работеше на хардуер, който всеки ентусиаст притежава; десетилетие по-късно тези **предварителни изчислителни разходи** са много по-поносими на стандартни **облачни изчисления** и с по-бързи съвременни програми за решаване, а публична референтна имплементация на **Ледюк Холдем** допълнително намалява бариерата за влизане.

6.2.6 Силни страни и ограничения

Централната сила на DeepStack е **коректност с ниска експлоатируемост**: това е първият метод за търсене при непълна информация с реална гаранция, като емпирично LBR – което излага състезателните ботове като губещи хиляди mbb/g – не успява да намери начин да го победи, а самото то губи от DeepStack над 350 mbb/g.⁸⁷ Той **не изисква абстракция на цялата игра и трансляция на действия**, така че залозите на опонента извън дървото се обработват точно, без закръгляне; неговата **времеемкост на играта е скромна**; и той се обучава **без човешки данни и с минимални експертни познания**. Допълнително предимство е оценъчната синергия – функцията на стойността на DeepStack съвпада с изискванията на оценяващия метод за намаляване на дисперсията AIVAT, като намалява стандартното отклонение в човешките проучвания с 85% и прави постига-

нето на значимост възможно само в няколко хиляди раздавания.

Ограниченията определят дневния ред за останалата част от главата. DeepStack е приложим **само в игри за двама с нулева сума**; неговата **коректност** се основава на тази структура. Той запазва **рядка абстракция на залозите** при предвиждането и **абстракция на действията** на ривъра, както и наследява **грешка на приближението на мрежата за стойности** (член $k_1 \epsilon$), най-голяма на флопа, където разчита най-много на мрежата. Той плаща **високите предварителни изчислителни разходи**, отбелязани по-горе, и **пререшава от нулата** при всяко решение, като е валидиран единствено срещу хора и LBR – никога в директен сблъсък срещу най-силните ботове, основани на абстракция.

6.2.7 Наследство и съвременна значимост

Отстранете специфичните за покера детайли и основната идея на DeepStack е **търсене с ограничена дълбочина с научен модел на стойност в листата**, адаптиран към скрита информация – аналогът при непълна информация на търсенето с насочване по стойност зад AlphaGo и AlphaZero. Тази идея не остаря и не беше забравена; тя се превърна в шаблон. Brown, Sandholm & Amos формализираха **теорията на решаването с ограничена дълбочина** през следващата година⁸⁴, а парадигмата след това беше обобщена в пълни **рамки за обучение с подкрепление и търсене**: ReBeL (2020)⁹⁰ преформулира научаването на стойността на листата около публични състояния на убежденията със **самообучение в стила на AlphaZero**, а Student of Games (2023)⁹¹ – който споделя няколко автори с DeepStack – обедини **игра с пълна и непълна информация** в един алгоритъм. Същата „рецепта за **планиране в цикъл с научен модел на стойност**“ по-късно достигна отвъд **покера за двама с нулева сума**, например в играта на Diplomacy на човешко ниво от CICERO.⁹² Още по-широко, DeepStack е ранен и чист пример за принципа, който сега е централен за водещия изкуствен интелект: **изразходвайте изчисления в момента на вземане на решение чрез търсене, насочвано от научена интуиция, вместо да вграждате всичко в една гигантска предварително изчислена стратегия** – същата теза за „**изчисления по време на изпълнение**“ (test-time compute) зад днешните **модели за разсъждение**, с уточнението на самата област, че непълната информация се нуждаеше от търсене, *отчитащо убежденията*, именно защото обикновеното **търсене в дърво на Монте Карло**, което работи за Го, не работи за покер.

Няколко конкретни подсистеми остават използвани и днес: **самият шаблон „ограничена дълбочина плюс научена стойност на листо“**; **диференцируемият изходен слой, който налага условието за нулева сума** в мрежите за стойности за двама играчи; **намаляване на дисперсията в стил AIVAT**, което превръща научен модел на стойност в контролна променлива за нис-

⁹⁰Brown, N., Bakhtin, A., Lerer, A. & Gong, Q. (2020). “Combining Deep Reinforcement Learning and Search for Imperfect-Information Games.” *NeurIPS* 33, 17057–17069; arXiv:2007.13544.

⁹¹Schmid, M. et al. (2023). “Student of Games: A unified learning algorithm for both perfect and imperfect information games.” *Science Advances*, 9(46), eadg3256. doi:10.1126/sciadv.adg3256 (arXiv:2112.03178, first posted in 2021 as “Player of Games”).

⁹²Meta Fundamental AI Research Diplomacy Team (FAIR), Bakhtin, A., Brown, N., Dinan, E. et al. (2022). “Human-level play in the game of Diplomacy by combining language models with strategic reasoning.” *Science*, 378(6624), 1067–1074. doi:10.1126/science.ade9097.

кодисперсна оценка на всеки стохастичен агент; и малкият, но преносим трик да се изразяват стойностите като **части от пота** за обобщаване, инвариантно спрямо мащаба. Това, което е наистина остаряло, е ръчно проектираната техника *около* идеята – клъстеризацията по метода на k-средните с 1000 клъстера, отделните мрежи за всеки рунд, ръчно настроеното пререшаване – които по-общите научени представяния на ReBeL и Student of Games заменят. Честната оценка: **като внедрена покер система DeepStack е остарял, но далеч не е просто стъпало** – неговата централна парадигма спечели и сега е основна; а за тази конкретна дисертация неговото състояние (диапазон, контрафактични стойности на опонент) и границата на разпространение на грешката са директни семена за базирано на убеждения моделиране на опонента (**принос 1**) и анализ на безопасната експлоатация (**принос 2**).

6.3 Libratus (2017/2018)

DeepStack отговори на своето *какво-ако*, като изостави парадигмата „абстракция и план“, но все пак запази рядка абстракция на залозите в собственото си предвиждане и никога не беше изпитан в директен двубой срещу най-силните по-ранни ботове или срещу специалисти по HUNL в дълъг и строго проведен мач. Libratus (Brown & Sandholm, 2017; Carnegie Mellon)⁸⁶ – на латински *балансиран*, както в приближаване към равновесие на Наш, и *силен*, заради мощната си игра – беше създаден независимо и обявен същата година, и направи противоположния залог: запази парадигмата „абстракция и план“ и излекува единствения ѝ фатален недостатък. Неговият водещ въпрос е огледален образ на този на DeepStack: **какво ще стане, ако решим груб план на цялата игра офлайн, след това го поправим в реално време там, където абстракцията е твърде груба – с доказуема гаранция, че поправката никога няма да ни направи по-експлоатируеми?** През януари 2017 г. Libratus стана първата програма, която победи най-добрите човешки специалисти по HUNL в дълъг мач, побеждавайки четирима професионалисти със 147 mbb/g (хилядни от големия блайнд на раздаване, единицата за темп на печалба от предишния раздел; ~50 mbb/g е значително професионално предимство) в 120 000 раздавания при 99,98% значимост – и, за разлика от DeepStack, първо победи убедително в директен двубой най-добрия дотогава изкуствен интелект за покер.

Таблица 10: Libratus (2017/2018) накратко.

С един поглед	Libratus (2017/2018)
Играчи	2 (един срещу един)
Тип игра	HUNL – безлимитен тексаски холдем за двама (игра за двама с нулева сума)
План (офлайн)?	Да – абстрахирана стратегия за цялата игра, решена офлайн с MCCFR (подробна в началото, груба в края)
Невронна компонента	Няма – чисто табличен CFR / абстракция (без невронни мрежи никъде)

С един поглед	Libratus (2017/2018)
Механизъм за търсене	Вложено безопасно решаване на под-игри (решаване в реално време на късната игра с CFR+ и преизпълнение за всеки залог на противника извън дървото)
Абстракция?	Да – абстракция на карти (търн/ривър, само в плана) + асиметрична абстракция на действията; вътре в под-игрите в реално време отпада изцяло (<i>без абстракция на картите</i>)
Също и с пълна информация?	Не (само непълна информация)
Изчислителни разходи	Високи и офлайн, и онлайн: ~25 милиона процесорни ядро-часа на свръхкомпютъра Bridges; ~50 възела и десетки секунди за всяко късно решение; без GPU
Ключова иновация	План + вложено в реално време <i>безопасно</i> решаване на под-игри + самоусъвършенстване: точни, доказуемо безопасни отговори на залози извън дървото вместо трансляция на действия

6.3.1 Пропускът, който запълни

Предидният раздел очерта парадигмата, която **DeepStack** отхвърли – абстракция плюс офлайн равновесие плюс трансляция – и посочи нейния фатален недостатък. **Libratus** се насочва точно към този недостатък, без да отхвърля парадигмата. Планът се решава предварително върху *фиксирано* меню от размери на залози; по време на игра всеки залог на противника, който не е в това меню, се *транслира* – закръгля се до най-близкия размер, който ботът познава – след което ботът отговаря така, сякаш противникът е направил закръгления залог. Това закръгляне е единствената най-голяма експлоатируема слабост в покера, основан на абстракция: проверка с локален най-добър отговор показва, че водещите състезателни ботове губят хиляди mbb/g от най-лошия възможен противник, а през 2015 г. предшественикът на **Libratus** – **Claudico** – загуби първия мач *Brains vs. AI* срещу професионалисти с 91 mbb/g, отчасти защото опонентите можеха да усетят и накажат неговите граници на трансляция.⁹³

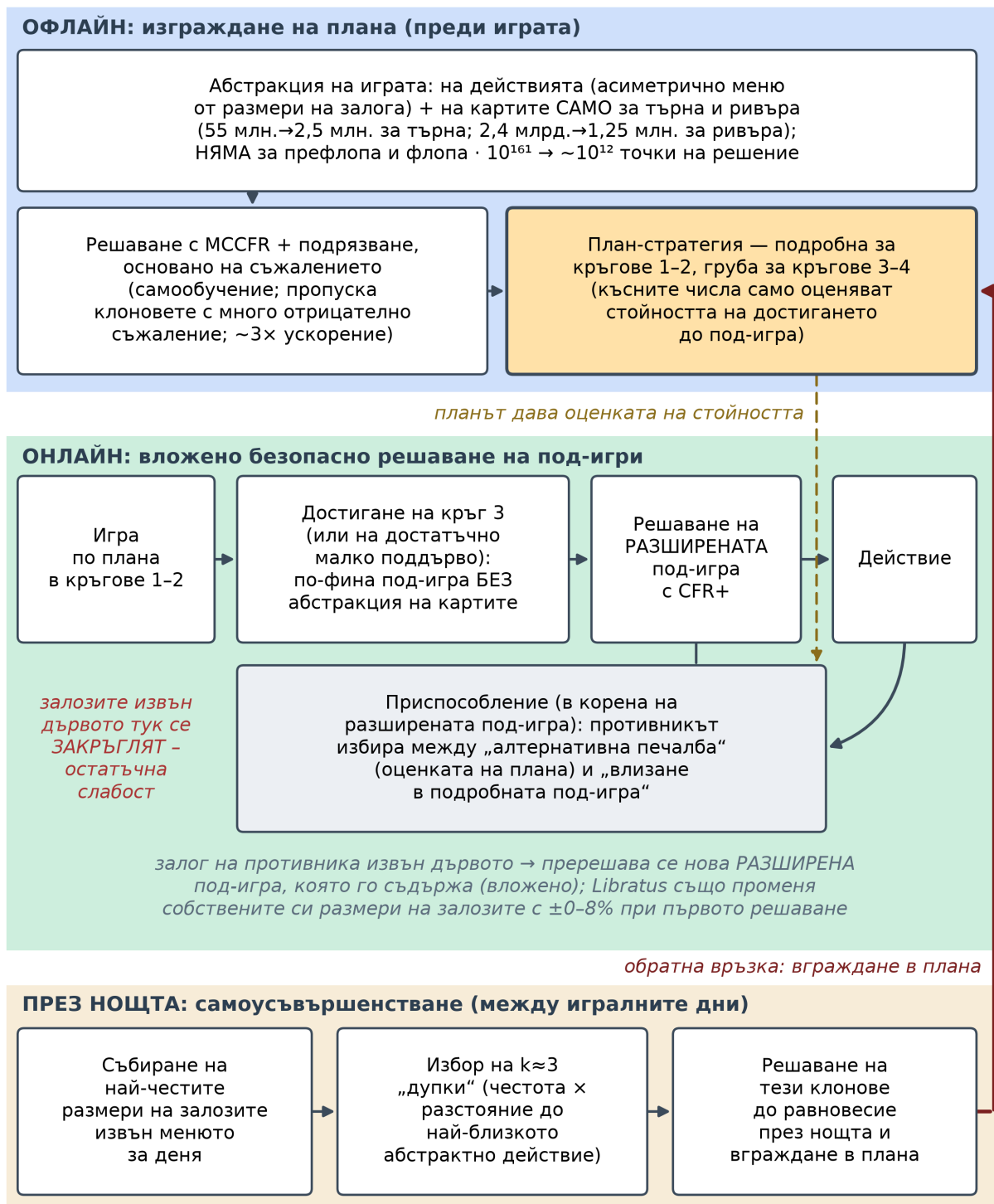
Концептуалният въпрос, на който **Libratus** отговаря, следователно е по-тесен и по-целенасочен в сравнение с този на **DeepStack**: *може ли парадигмата на абстракцията, съществуваща от десетилетия, да бъде издигната до свръхчовешко ниво чрез поправка единствено на поведението ѝ в реално време – а именно точното отговаряне на залози извън менюто вместо чрез закръгляне – и може ли тази поправка да се извърши с гаранция, че локалните корекции никога не увеличават глобалната експлоатируемост?* Отговорът е тримодулната архитектура по-долу. Докато **DeepStack** премахна напълно плана и разчиташе на научена оценъчна функция, **Libratus** запазва плана като евтин скелет и добавя решавач в реално време, който, когато противникът излезе извън дървото, изгражда

⁹³Ganzfried, S. (2016). “Reflections on the First Man vs. Machine No-Limit Texas Hold ’em Competition.” *ACM SIGecom Exchanges*, 14(2), 2–15; arXiv:1510.08578.

и решава по-фина под-игра *съдържаща действителния залог* и вплита резултата обратно в плана. Същата година тогава предложи две противоположни решения на един и същ проблем – едно невронно и без план, друго таблично и базирано на план – и **Libratus** е доказателството, че по-старата парадигма, правилно поправена, все още беше достатъчна, за да достигне свръхчовешка игра.

6.3.2 Архитектура

Libratus представлява конвейер от три модула, които функционират на три различни времеви мащаба – **офлайн** (преди мача), **онлайн** (при всяко вземане на решение) и **през нощта** (между отделните дни на игра) – и, за разлика от всички останали системи в тази глава, той не съдържа **никаква невронна мрежа** (вж. фигурата по-долу).⁸⁶



Фигура 25: Тримодулният конвейер на Libratus: предварително изчисленият план (горе), онлайн модулът за вложено безопасно решаване на под-игри, чрез който планът се изиграва (в средата), и нощният модул за самоусъвършенстване, който вгражда решените „дупки“ обратно в плана (долу, стрелката за обратна връзка).

Модул 1 – планът (офлайн). Libratus първо компресира приблизително 10^{161} точки на решение на HUNL (статията на DeepStack закръгля същия брой до 10^{160}) до около 10^{12} чрез два вида абстракция: *абстракция на действията*, която запазва само дискретно меню от размери на залозите (главно кръгли дробни и кратни на пота, взети от размерите, предпочитани от водещите конкурентни ботове, с няколко ранни размера, настроени чрез алгоритъм за оптимизация на параметри), и *абстракция на карти*, която групира стратегически сходни ръце.⁹⁴ От решаващо значение е, че **не се използва абстракция на карти в първите два рунда на залагане** – достатъчно малки, за да се позволи пълна резолюция – а се групират само търнът и ривърът, и то дори там единствено в плана. След това тази абстрактна игра се решава чрез самообучение с усъвършенстван **метод Монте Карло за минимизиране на контрафактичното съжаление (MCCFR)**, който вероятно премахва клонове с много отрицателно съжаление, което представлява приблизително трикратно ускорение и също така тихо отстранява патология на абстракциите с непълна памет, при която няколко различни ситуации споделят стратегията на един клъстер и „се борят“ за нея. Резултатът е **планът**: пълна, но неравномерна стратегия – детайлна в началото, груба в края – чиито стойности от късните рундове се използват не за игра, а единствено за *оценяване на стойността на достигане до под-игра*.

Модул 2 – вложено безопасно решаване на под-игри (онлайн). Libratus използва плана само в ранните кръгове. При достигане на третия рунд за залози – или всяка по-ранна точка, където останалата част от раздаването е достатъчно малка – той изоставя грубия план за късната игра и вместо това **изгражда нова, по-финозърнеста под-игра без абстракция на карти и я решава в реално време** с високо оптимизиран CFR+ (бърз, детерминистичен вариант на CFR). Така всяко раздаване се играе поотделно в късната игра. Определящата характеристика е това, което се случва, когато противникът направи залог, който не фигурира в менюто на плана: вместо да го закръгли, Libratus решава нова под-игра, която *съдържа точно този залог*, и повтаря това за всяко следващо действие извън менюто – *вложено решаване на под-игри*. Този модул е обект на разглеждане в следващия подраздел.

Модул 3 – модулът за самоусъвършенстване (през нощта). Тъй като решаването в реално време се пропуска в първите два рунда (решаването толкова рано само по себе си би изисквало тежка абстракция), залозите на противника извън менюто *там* все още се закръглят. Модулът за самоусъвършенстване стеснява тази остатъчна слабост между игралните дни: той събира най-често използваните от противниците размери на залозите извън менюто, избира няколко от най-вредните пропуски, изчислява правилни игровитеоретични отговори на тях през нощта и присажда тези клонове в плана. Нарочно това **не е експлоатация на противника** – той никога не се опитва да моделира и наказва грешките на хората (което би изложило Libratus на контраексплоатация). Вместо това използва залозите на противниците единствено като насока *кои от собствените пропуски на Libratus да закръпни*, а закръпванията са универсални, подобрявайки играта срещу всеки бъдещ противник.

⁹⁴Brown, N. & Sandholm, T. (2017). “Libratus: The Superhuman AI for No-Limit Poker (Demonstration).” *IJCAI-17*. <https://www.ijcai.org/proceedings/2017/0772>

Класическите градивни елементи от по-ранните глави са на видно място: **абстракция на играта** и MCCFR изграждат **плана** офлайн, CFR+ извършва **решаването на под-игра** онлайн, а „търсене“ представлява **пререшаване в реално време** за всяко решение. Забележителното отсъствие – липсата на **мрежа за стойности**, на **мрежа на стратегията** и на каквото и да е научено чрез **градиентен спуск** – е архитектурният подпис, който отличава Libratus от всяка друга система в тази глава.

6.3.3 Ключова иновация: вложено безопасно решаване на под-игри

Централният принос на Libratus е да направи **решаването на под-игра в реално време едновременно безопасно и вложено** – достатъчно силно, за да победи хората, и доказано неспособно да направи стратегията значително по-експлоатируема от плана, който усъвършенства. И двете прилагателни са важни и всяко от тях отстранява конкретен недостатък на по-ранните методи.

Трудността е в това, че под-игра с непълна информация *не може да бъде решена изолирано*. В шаха, когато се достигне ендшпил, той може да бъде решен самостоятелно, тъй като дъската е напълно известна. В покера правилната стратегия за една под-игра зависи от стратегиите в *други, недостижими* под-игри, защото именно те определят разпределението на вероятностите за скритите ръце, които противникът може да държи тук. По-ранните програми за решаване в реално време заобикаляха това, като приемаха, че противникът е следвал плана до този момент – **небезопасно** решаване на под-игра – което противникът може лесно да накаже с отклонение, понякога произвеждайки усъвършенствана стратегия значително *по-лоша* от плана.

Libratus премахва това предположение с едно малко **приспособление**, *разширената под-игра*. В корена на под-играта на противника се предоставя избор за всяка ръка, която той може да държи: или да вземе фиксирана **алтернативна печалба** – оценката на плана за това колко струва тази ръка тук – или да *влезе* в детайлната под-игра и да я изиграе. Решаването на тази разширена игра принуждава усъвършенстваната стратегия на Libratus да направи противника **не по-добре от тази оценка за всяка възможна ръка**, което е точното значение на „безопасен“: локалната корекция не може да бъде експлоатирана спрямо плана, независимо какво прави противникът. Libratus изостря това с вариант, който авторите наричат **Estimated-Maxmargin** – той максимизира *най-малкия* резерв за безопасност по всички възможни ръце на противника – и с две усъвършенствания, които го правят силен, а не просто безопасен: той използва оценки на плана за стойностите на противника вместо консервативни **горни граници** (по-малко предпазлива игра) и *придава по-малка тежест на ръцете, които противникът би могъл да държи само ако е допуснал грешка по-рано* (безсмислено е да се защитава срещу ръка, която рационален противник никога не би довел до тази ситуация), освобождавайки капацитет за защита срещу ръцете, които той реалистично може да има.

Комбинацията идва с гаранция, която е паралелна на тази на DeepStack.⁹⁵ Ако σ^* е най-малко

⁹⁵Brown, N. & Sandholm, T. (2017). “Safe and Nested Subgame Solving for Imperfect-Information Games.” *NeurIPS* 30, 689–699; arXiv:1705.02955.

експлоатируемата стратегия, която се различава от плана само в рамките на решените под-игри, а оценката на плана за стойностите на под-играта на противника има отклонение не повече от Δ , тогава експлоатируемостта на усъвършенстваната стратегия се подчинява

$$\text{експлоатируемост}(\sigma_{\text{усъв}}) \leq \text{експлоатируемост}(\sigma^*) + 2\Delta.$$

Членът 2Δ представлява цената на несъвършените оценки на стойностите и е структурно аналогичен на члена $k_1\epsilon$ в DeepStack – и двата определят колко една грешка в стойностите, подадени на решавача, може да повлияе върху експлоатируемостта в най-лошия случай. По този начин двете системи стигат до един и същи вид твърдение за коректност, но от противоположни посоки: ϵ при DeepStack е грешката на приближението на невронна мрежа при границата на дълбочината; Δ при Libratus е грешката на оценката на таблична схема при границата на под-играта.

„Вложено“ е втората половина. Вместо да разширява менюто за залози на плана (което би раздуло предварителното офлайн решаване), Libratus създава **отделен отговор на всеки залог извън менюто в реално време**: когато противникът направи залог с размер, който не е бил изброяван преди, той изгражда разширена под-игра, чиято алтернативна печалба е най-доброто в *менюто* действие, което противникът би могъл да предприеме вместо това, решава я и го прави отново за всеки следващ залог извън менюто по време на раздаването. В контролираните експерименти това превъзхожда предишния стандарт – закръгляне на залога до най-близкия размер в менюто – с повече от порядък по отношение на експлоатируемост в най-лошия случай (119 срещу 1465 mbb/g в докладвания тест с малки игри). И тъй като всяка под-игра се решава в движение, Libratus може също така да *променя собствените си размери на залозите* между раздаванията – той ги увеличава или намалява със случаен процент между 0 и 8% при първото решаване – така че хората никога не се сблъскват с фиксирана цел за анализ. Това е точният механизъм, чрез който най-слабото място на парадигмата на абстракцията, транслацията на действия, се отстранява от късната игра.⁹⁵

6.3.4 Уговорки, задънени улици и какво статията не описва

Както при DeepStack, ясната тримодулна структура прикрива къде всъщност се крие инженерната работа – и откровеността относно ограниченията – с изключение на това, че тук голяма част от нея дори не е в статията в *Science*, а в придружителната статия в IJCAI и в лекционните бележки на авторите.

Най-разкривателното ограничение е, че **само планът не е свръхчовешки – той дори не побеждава предишния бот**. Срещу Baby Tartanian8, победителя от състезанието през 2016 г., суровият план на Libratus не го побеждава (-8 ± 15 mbb/g при 95% доверителен интервал); едва когато вложеното решаване на под-игри беше включено, същата система спечели с 63 ± 28 mbb/g.⁸⁶ С други думи, предварителната стратегия е скеле и по същество цялото предимство на Libratus идва от търсене в реално време – резултат, който тихо преформулира цялата система и предвещава обрата на областта към търсене по време на изпълнение.

Няколко други важни детайла си струва да се отбележат. **Транслацията на действия не е премахната, а само ограничена:** в първите два рунда на залагане Libratus все още закръгля залозите на опонента извън менюто, а целият модул за самоусъвършенстване съществува, за да работи върху този остатъчен проблем, по три „дупки“ на нощ, без никога да го отстранява напълно. **Безопасното решаване на под-игри трябваше да бъде преработено, за да стане изобщо приложимо** – в продължение на три години се смяташе за непрактично, тъй като в директен сблъсък учебниково-безопасното решаване губеше от теоретично необоснования *небезопасен* вариант; използването от страна на Libratus на стойностни *оценки*, а не консервативни *горни граници*, заедно с намаляването на значението на ръцете, които опонентът би държал само по грешка, направи безопасността конкурентна и дори тогава Libratus умишлено използва *небезопасен* метод за решаване веднъж, при първото си влизане в третия рунд, защото там е по-евтин и емпирично достатъчно добър. Изборът на **Estimated-Maxmargin** сам по себе си жертва малко теоретична чистота в полза на силата: използването на оценки вместо горни граници може по принцип да повиши експлоатируемостта *над* тази на плана, ограничена единствено от 2Δ на теоремата. А абстракцията с непълна памет в основата означава, че няколко различни ситуации споделят една стратегия и „се борят“ за нея – известно изкривяване, което подрязването, базирано на съжаление, само частично смекчава. И накрая, **данните за изчислителните разходи почти липсват в основната статия:** водещата публикация по същество не дава никакви данни, а истинската цена (следваща) излиза наяве едва във вторичните източници, както и фактът, че **кодът никога не е бил публикуван**, оставяйки независимата проверка да се основава единствено на публикувания псевдокод.

6.3.5 Изчислителни разходи и достъпност

Където сметката на DeepStack се плаща почти изцяло офлайн, а играта му е евтина, Libratus е **скъп и в двата края**. Проектът е консумирал приблизително **25 милиона процесорни ядро-часа** на свръхкомпютъра Bridges в Pittsburgh Supercomputing Center за една година – от които около 6 милиона са отишли за изграждане и решаване на плана, около 3 милиона за решаване на под-игри в реално време по време на мача, около 3 милиона за модула за самоусъвършенстване и останалите ~13 милиона за експериментални опити и оценка.⁹⁴ Оперативно изчисленията на плана са заемали приблизително 195 възела за периоди от една до осем седмици; всяко **решаване на под-игра в реално време е използвало около 50 възела и е отнемало порядък на десетки секунди**; модулет за самоусъвършенстване през нощта е работил върху до няколкостотин възела в продължение на часове; а стратегиите и моментните снимки са консумирали около 2,6 петабайта дисково пространство.⁹⁶ **Не са използвани нито графични процесори (GPU), нито обучение на невронни мрежи** – всеки ядро-час е изразходван за CFR или абстракция.

Формата на тази сметка носи поуката. Libratus не постига евтино внедряване чрез еднократно скъпо обучение, както прави DeepStack; неговите *изчислителни разходи по време на игра* сами по себе си са в мащаба на свръхкомпютър, защото силата произтича от решаването на нова под-игра

⁹⁴Sandholm, T. (2021). “State-of-the-art for two-player no-limit Texas hold’em: Libratus.” Lecture 13 slides, 15-888 *Computational Game Solving* (Fall 2021), Carnegie Mellon University.

при почти всяко късно решение. От гледна точка на достъпността това направи Libratus, както беше внедрен, по същество невъзможно да бъде възпроизведен извън голям център за свръхкомпютри както по време на обучението, така и по време на играта – много по-висока летва от агента на DeepStack с един GPU – а липсата на публикуван код подсили това. Основните техники са независими от приложението и работят значително по-евтино на днешния хардуер и с днешните програми за решаване, но артефактът от 2017 г. беше демонстрация на това, което е възможно при наличието на изобилни изчислителни ресурси, а не на това, което е широко достъпно.

6.3.6 Силни страни и ограничения

Главното предимство на Libratus е **решителна, строго демонстрирана свръхчовешка игра**. Той е първият изкуствен интелект, който побеждава водещи специалисти по HUNL в дълъг мач – 147 mbb/g за 120 000 раздавания при 99,98% значимост, като побеждава всеки от четиримата професионалисти индивидуално – и, за разлика от DeepStack, той също така побеждава **предходния най-добър изкуствен интелект за покер в директен сблъсък** с 63 mbb/g, отделяйки своята сила от въпроса за човешкото умение. Основната му техника – **вложено безопасно решаване на под-игри** – е едновременно **доказуемо безопасна** (2Δ границата) и **емпирично с порядък по-малко експлоатируема** от транслацията на действия, която заменя, като не използва **човешки данни и никакви специфични за домейна знания**. Модулният дизайн сам по себе си е принос: евтин план, модул за решаване в реално време, който го коригира там, където е необходимо, и нощен цикъл, който закъпва собствените му слабости – всеки модул компенсира недостатък на друг – като цялата система е с **устойчивост на първо място**, отказвайки да експлоатира опонентите, за да не бъде контраексплоатиран.

Ограниченията са също толкова ясни и подготвят останалата част от главата. Libratus е **само за игра за двама с нулева сума**; неговите гаранции за безопасност се опират на тази структура. Той остава **основан на абстракция** и все още **транслира заложи извън менюто в ранните рундове**, което представлява експлоатируема слабост, която модулът за самоусъвършенстване само стеснява. Той не използва **никакво невронно обобщаване**: нищо не се прехвърля между ситуациите, планът е огромен (петабайти от съхранена стратегия), а времето за игра е в **мащаба на свръхкомпютър**, а не един GPU, както беше необходимо на DeepStack. Компромисът на Estimated-Maxmargin допуска малко, ограничено повишаване на експлоатируемостта в името на по-голяма сила; едно небезопасно решаване се промъква в конвейера и – тъй като както хората, така и изкуственият интелект се адаптираха по време на мача – дори значимостта в заглавието е, стриктно казано, „сякаш-независима“ стойност, въпреки че предимство от 147 mbb/g за 120 000 раздавания не оставя никакво реално съмнение.⁸⁶

6.3.7 Наследство и съвременна значимост

Извадете абстракционния механизъм и трайната идея на Libratus е **търсене в реално време, насложено върху груба предварително изчислена стратегия, направено безопасно при скрита информация**: решавайте локалната ситуация точно в момента на вземане на решение, в

съответствие с евтин глобален план, и реагирайте на това, което опонентът действително прави, а не на закръглено приближение на него. Тази идея не само оцеля – тя се превърна, заедно с непрекъснатото пререшаване на DeepStack, в половината от основата, върху която е изградена останалата част от главата. Вложеното безопасно решаване на под-игри на Libratus е пряк предшественик на търсенето в реално време в **Pluribus** (2019), то е обединено с пререшаването на DeepStack чрез теорията за **ограничено по дълбочина решаване**, публикувана от авторите през следващата година, и същият модел „усъвършенствайте глобална оценка на стойността с локално решаване“ се появява отново, сега с *научени* стойности, заменящи табличната схема, в **ReBeL** (2020).

Libratus е също така, в точен смисъл, **върхът на една парадигма, която областта след това изостави**. Той постигна свръхчовешка игра **без никакво дълбоко обучение** – чиста абстракция и CFR – и въпреки това последващите системи са всички невронни, защото ръчно създадената абстракция и петабайтовите планове нито се обобщават между ситуациите, нито се мащабират отвъд двама играчи. Честната оценка е, че Libratus е *надминат като архитектура, но оправдан като теза*: неговият залог, че **търсенето в реално време има по-голямо значение от по-голяма предварително изчислена стратегия**, беше напълно точен, дори когато залогът му върху табличната абстракция беше надминат. Тази първа теза оттогава се превърна в централна тема за най-напредналите системи с изкуствен интелект – наблюдението, че добавянето на търсене в момента на решението струва колкото 100 000-кратно мащабиране на предварителните офлайн изчисления, е ранен, конкретен пример за аргумента за „изчисления по време на изпълнение“ (test-time compute), който сега се прави за моделите за разсъждение,⁹⁷ а разделянето план-тогава-търсене на Libratus предвещава съвременната рецепта предварително обучение и търсене.

Няколко подсистеми остават директно използвани: **безопасно решаване на под-игри / на енд-шпила** като начин за локално усъвършенстване на глобална стратегия *без да се нарушават нейните гаранции*; **помощната конструкция на разширената под-игра** – чистата конструкция „вземете известна алтернативна стойност или влезте в под-играта“ за закрепване на локално решение към глобална оценка на стойността; **подрязване, базирано на съжаление** за мащабиране на CFR; и моделът на **самоусъвършенстването**, който използва наблюдавана игра, за да открие и *поправи собствените си слабости*, вместо да експлоатира противника. Това, което наистина е остаряло, е заобикалящата го структура – ръчно настроената **абстракция на действията** и картите, много-петабайтовата **таблична схема** и внедряването с клъстер по време на игра – и всички те отпадат с научените представяния на по-късните системи. За настоящата дисертация Libratus дава две основи: неговата граница на експлоатируемостта при безопасно решаване на под-игри е шаблон за **безопасна експлоатация при грешка в стойността (Принос 2)**, а умишленият му отказ да експлоатира – „поправете собствените си слабости, не моделирайте противника“ – е точният контрапункт, спрямо който може да бъде дефинирана *ограничена, целенасочена адаптация към противника* (Принос 1).

⁹⁷Nuñez, M. (2024, 23 Oct.). “OpenAI’s Noam Brown stuns TED AI Conference: ‘20 seconds of thinking worth 100 000× more data’.” *VentureBeat*.

6.4 Pluribus (2019)

Libratus сложи точка на въпроса за безлимитния холдем за двама, но неговите гаранции за безопасност – и всъщност самото значение на „решаването“ на играта – се основаваха на структурата на игра с нулева сума за двама играчи: там равновесието на Наш е непобедимо и решаването в реално време може да бъде доказуемо *безопасно* спрямо него. Холдемът, както хората всъщност го играят обаче, събира шестима. Pluribus (Brown & Sandholm, 2019; Carnegie Mellon и Facebook AI)⁸⁵ се изправи директно пред въпроса за игрите с много играчи: **какво се случва с рецептата план-плюс-търсене в реално време, когато махнете патерицата на игрите за двама и седнете на маса за шестима – обстановка, в която равновесието на Наш не е нито уникално, нито ефективно изчислимо, нито дори гаранция, че няма да загубите?** Отговорът му беше емпиричен и категоричен. В два формата – петима професионалисти, седнали с едно копие на Pluribus, и един професионалист срещу пет копия – той победи петнадесет елитни професионалисти (тринадесет, които се редуваха във формата с петима души, и двама – шампионите от WSOP и WPT Chris Ferguson и Darren Elias – във формата с един човек), печелейки около **48 хилядни от големия блайнд на раздаване** срещу петима души наведнъж (mbb/g, мерната единица за темп на печалба от предишните раздели – приблизително пет големи блайнда на сто раздавания, решаващ марж на маса с шестима) при 95% статистическа значимост. И го направи след обучение в продължение на **осем дни на един 64-ядрен сървър за около \$150 облачни изчисления** – с около хиляда пъти по-малко изчисления, отколкото изразходва свръхкомпютърът на Libratus.

Таблица 11: Pluribus (2019) накратко.

С един поглед	Pluribus (2019)
Играчи	6 (маса с шестима, 6-маx) – първият свръхчовешки изкуствен интелект в която и да е еталонна игра с повече от двама играчи/отбора
Тип игра	6-маx NLHE – безлимитен тексаски холдем с шестима играчи (непълна информация; с много играчи, <i>не</i> е игра за двама с нулева сума)
План (офлайн)?	Да – цялостен план за играта, решен офлайн чрез Linear MCCFR; използва се <i>директно</i> само в първия рунд за залози, след това служи като скеле
Невронна компонента	Няма – изцяло табличен CFR / абстракция (без невронни мрежи никъде, както при Libratus)
Механизъм за търсене	Търсене в реално време с <i>ограничена дълбочина</i> с $k=4$ стратегии за продължение в листата (вложени <i>небезопасни</i> решавания, пререшаване от началото на всеки рунд за залози); Linear CFR вътре в под-игрите
Абстракция?	Да – действия (1–14 размера на залозите в плана; 1–6 при търсене) + информационна абстракция (без загуби в първия рунд; със загуби в по-късните, по-фини при търсенето, отколкото в плана)

С един поглед	Pluribus (2019)
Също и с пълна информация?	Не (само непълна информация)
Изчислителни разходи	Забележително ниски: план ~12 400 ядро-часа / 8 дни / < 512 GB на един 64-ядрен сървър (~\$144); игра на 2 процесора (28 ядра), < 128 GB, без GPU, 1–33 s/решение
Ключова иновация	Свърхчовешка игра с шестима играчи чрез план + търсене с ограничена дълбочина със стратегии за продължение – спечели <i>емпирично</i> , с минимален бюджет, без гаранция за безопасност при N играчи

6.4.1 Пропускът, който запълни

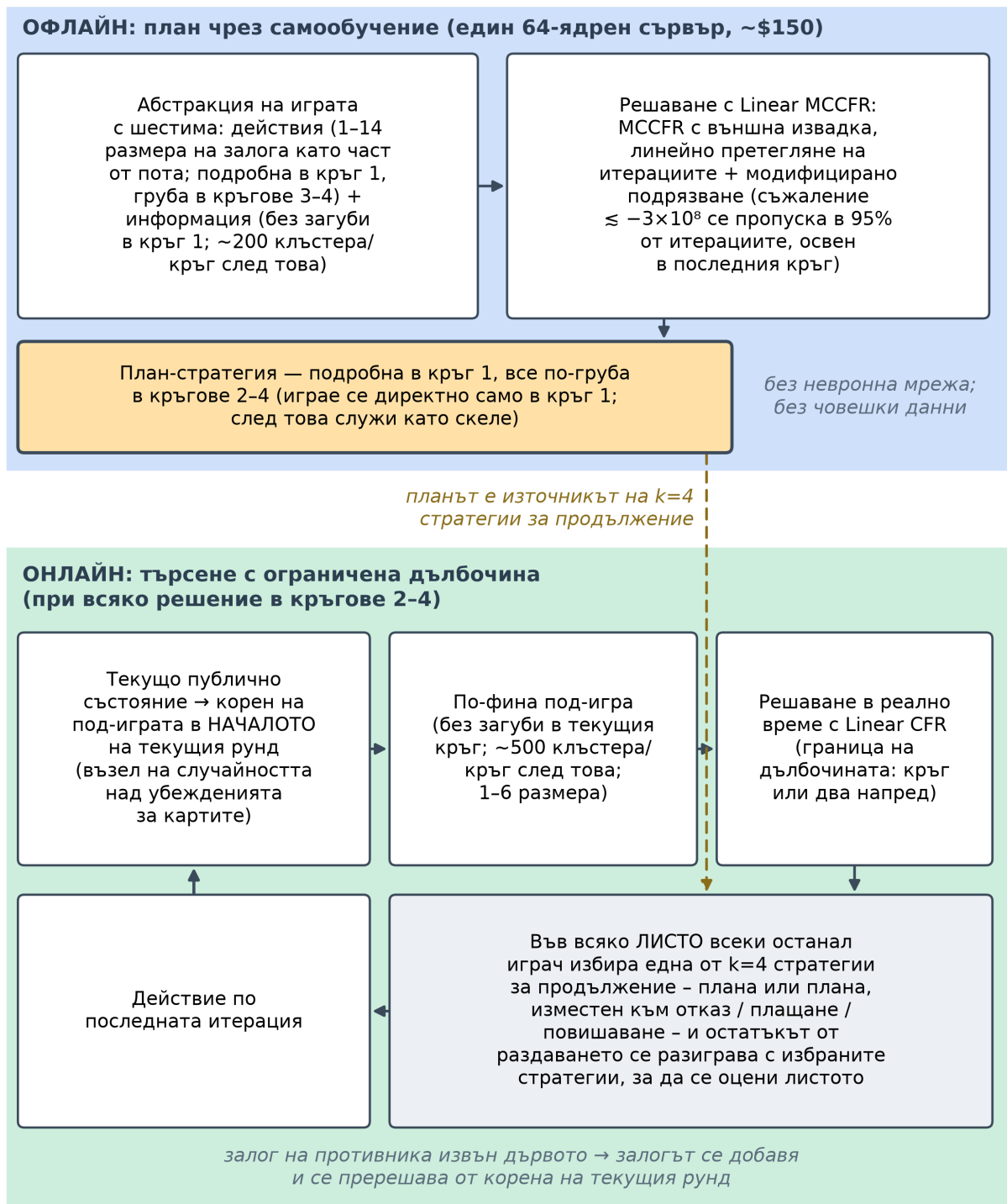
Всеки свърхчовешки игрови изкуствен интелект преди Pluribus – дама, шах, Го и двете предходни покер програми – споделяше едно скрито предположение: двама играчи, нулева сума. В тази обстановка равновесието на Наш носи свойство, което го прави очевидната цел: на всеки играч, който възприеме такова равновесие, е *гарантирано, че няма да загуби* средно (по математическо очакване), независимо какво прави противникът, и ако двама играчи независимо изчислят равновесия, техните стратегии все още се комбинират в равновесие. „Решаването“ на такава игра следователно *означава* приближаване на равновесие на Наш, а DeepStack и Libratus бяха, по същество, два начина да направят точно това. Покерът с шестима играчи обезсмисля всичко това. При повече от двама играчи намирането – дори приближаването – на равновесие на Наш е изчислително непосилен проблем в общия случай; обикновено има *много* равновесия; и, фатално, ако всеки играч независимо избере едно, получената съвместна стратегия може да не бъде равновесие изобщо, така че играчът може да *загуби*, докато играе безупречна равновесна стратегия. Brown и Sandholm илюстрират това с играта „Щанд за лимонада“, в която играчите се разполагат около пръстен: има безкрайно много равновесия и независимо избраните рядко се съчетават. Заключение е рязко – в покера с много играчи равновесието на Наш *не е нито уникално, нито гаранция за безопасност*, а дефиницията на успеха в тази област от две десетилетия просто изчезва.

Pluribus затваря тази празнина, като променя целта и след това я постига. Вместо да преследва концепция на решение, на която не може да се довери, той цели единствено *емпирично и последователно да побеждава елитни хора* и предварително приема, че неговите алгоритми не носят гаранция за сближаване към равновесие извън играта за двама с нулева сума. Тази отстъпка е цялата същност – и това е точната празнина, която принос 2 на тази дисертация се стреми да запълни. Другата половина на празнината е механична. Libratus постигна свърхчовешка игра, като решаваше всяка под-игра в късната фаза *докрай* в реално време; с шестима играчи под-игрите експлодират експоненциално и решаването докрай става непрактично, докато алтернативата на DeepStack – стойности на листата, обусловени от разпределения на убеждения – вече беше скъпа в покера за двама и се влошава с увеличаване на броя играчи. Следователно Pluribus се нуждаеше

от търсене в реално време, което да гледа само малко напред и да спира – в игра, в която, както показват следващите раздели, ранното спиране е точно това, което простото търсене не може безопасно да направи.

6.4.2 Архитектура

Както и при Libratus, Pluribus се разделя на офлайн фаза, която изгражда план, и онлайн фаза, която извършва търсене – но балансът на силите между тях е обърнат и, отново както при Libratus, **няма невронна мрежа никъде** (вж. фигурата по-долу).



Фигура 26: Двухазната архитектура на Pluribus: предварително изчисленият с Linear MCCFR план (горе) служи като източник на $k=4$ стратегии за продължение в онлайн търсенето с ограничена дълбочина (долу).

Офлайн Pluribus изчислява план за цялата игра чрез самообучение, започвайки от случайна игра и подобрявайки се чрез **Монте Карло CFR (MCCFR)** с **външна извадка**, който при всяка итера-

ция избира един играч за „обхождащ“ (traverser), симулира раздаване и насочва този играч към действията, които са се представили по-добре. Той абстрахира играта по два начина – ограничен брой размери на залозите (*абстракция на действията*) и групи от стратегически сходни ситуации с карти (*информационна абстракция*) – но само грубо, и използва този план *директно* само в първия от четирите кръга на залагане, където абстракцията на действията се запазва по-фина и не се използва групиране на карти. Навсякъде другаде – флоп, търн, ривър – Pluribus изхвърля грубия план локално и **търси в реално време** по-фина стратегия, точно както направи Libratus, с една решаваща разлика: той не решава до края на играта. Той разглежда само един или два кръга напред до граница на дълбочината и спира, което прави шест играчи изчислително достъпни.

Познатите градивни елементи са налице – абстракция и MCCFR изграждат плана, линейният CFR извършва онлайн решаването, а „търсенето“ е пререшаване при всяко решение – и двата архитектурни отличителни белега са *отсъствия*. Няма **невронна мрежа** (системата е изцяло таблична, като Libratus) и, за разлика от Libratus, няма **модул за самоусъвършенстване**: Pluribus никога не поправя своя план между сесиите, тъй като търсенето с ограничена дълбочина в три от четирите кръга вече извършва поправките. Две съставки правят ранното спиране коректно и те са предмет на задълбочения анализ по-долу: внимателен отговор на въпроса какво изобщо може да означава „стойност на листо“ при скрита информация, и търсене, което започва не от текущото решение, а от *началото* на текущия кръг на залагане.

6.4.3 Ключова иновация: търсене с ограничена дълбочина със стратегии за продължение

Евристичното търсене в шах или Го работи, като гледа на фиксирано разстояние напред и отчита стойност от хоризонта при предположението, че и двете страни ще играят добре след това. Това предположение напълно се проваля при скрита информация, а Brown и Sandholm илюстрират провала с еднократна последователна игра „камък–ножица–хартия“: ако търсещият приеме, че противникът ще играе равновесие (всеки ход една трета от времето) отвъд хоризонта, тогава всяко действие изглежда еднакво оценено на нула и търсещият може да се установи на „винаги играй камък“ – при което противникът преминава към „винаги хартия“ и истинската стойност рязко спада. Листо в игра с непълна информация следователно няма *една-единствена стойност*: неговата стойност зависи от стратегията, която търсещият ще възприеме, което е именно това, което търсенето се опитва да определи.

Централният принос на Pluribus е начинът, по който все пак се определя стойност за листата. Когато търсенето достигне границата на дълбочината, вместо да се фиксира една стратегия за продължение, **всеки играч, който все още е в раздаването, избира измежду четири различни стратегии за продължение** за остатъка от играта и може да ги смесва. Четирите са умишлено опростени: самият план и три изместени копия, които умножават вероятността за *отказ*, за *плащане* и за *повишаване* (всеки след това се ренормализира). Тъй като противникът винаги може да премине към стратегия за продължение, която най-много наказва търсещия, небалансирана стратегия – покер еквивалентът на винаги играене на камък – вече не се възнаграждава и търсещият се насочва към баланс. Тази идея първоначално беше доказана в предшественика за двама

играчи Modicum (Brown, Sandholm & Amos, 2018), който победи двама бивши шампионски бота, докато работеше на 4-ядрен лаптоп с 16 GB памет – поразителен знак, че търсенето с ограничена дълбочина със стратегии за продължение може да замести свръхкомпютър.⁸⁴

Pluribus обобщава идеята от двама играчи на шестима и добавя фин, но важен обрат. В Modicum само *противникът* избираше измежду стратегии за продължение, докато търсещият винаги следваше плана – подход, който е обоснован в **игри за двама с нулева сума**, но дава инициативата на противника и оставя търсещия колеблив и с ниска стойност. Pluribus позволява на **търсещия също да избира измежду стратегиите за продължение**, което балансира играчите и, по думите на авторите, е „по-ефективно, по-лесно и по-елегантно“. Освен това той не пререшава от текущата точка за вземане на решение, а от *началото на текущия рунд за залози*, като фиксира единствено вече предприетите действия. Това „небезопасно“ търсене – наречено така, защото, за разлика от търсенето на Libratus, не дава гаранция, която да ограничава **експлоатируемостта** – е по-евтино (на маса с шестима играчите се отказват веднага от повечето ръце, така че малко от тях изобщо се нуждаят от стратегия) и, тъй като започва непосредствено след случайно събитие с множество разклонения, на практика се оказва трудно за експлоатиране.

Това, което очевидно липсва от всичко това, е една *гаранция*. DeepStack ограничи своята експлоатируемост до $k_1\epsilon + k_2/\sqrt{T}$,⁸⁷ а Libratus – до 2Δ ,⁹⁵ Pluribus не предлага такова ограничение и пропускането е принципно, а не поради небрежност. Двигателят на CFR все още, във всяка крайна игра, свежда *средното съжаление* на всеки играч до нула,

$$\frac{R_i^T}{T} \rightarrow 0 \quad (\text{без съжаление}),$$

но изводът, който доведе двамата предшественици от липсата на съжаление до безопасност – *играта без съжаление (no-regret) се сближава към равновесие на Наш и едно равновесие на Наш не може да бъде победено* – важи само когато има двама играчи и играта е с нулева сума. При шестима играчи първото следствие отпада (самообучението не е задължително да се доближава до равновесие), а второто е безсмислено (едно равновесие не е непобедимо). Самите итерации, които *доказаха* DeepStack и Libratus безопасни, следователно носят на Pluribus само емпирична сила: свръхчовешка на практика, без нищо сертифицирано. За да направи тези итерации достатъчно евтини за изпълнение с шестима играчи, Pluribus разчита на **линеен CFR** – претегляйки приноса на итерация t чрез t , така че лошите ранни итерации да се изчистват около три пъти по-бързо – и на **модифицирано подрязване на действията с отрицателно съжаление**, което пропуска безнадеждни действия при повечето итерации (навсякъде с изключение на последния кръг); заедно с отложено разпределение на паметта, това са нещата, които намаляват сметката до един сървър.

6.4.4 Уговорки, задънени улици и какво статията не описва

Както и при двамата предшественици, публикуваната статия е изчистеният разказ, а инженерната честност се намира в допълнителните материали – още повече тук, тъй като Pluribus е шестстра-

нича статия в *Science*, чиято архитектура почти изцяло е прехвърлена към нейните допълнителни материали и към две придружаващи статии. Най-същественото признание е, че Pluribus използва **небезопасно** решаване на под-игра – той предполага, че опонентите са играли стратегията, която той изчислява за тях – което, както авторите заявяват ясно, „няма теоретични гаранции за представянето дори в игри за двама с нулева сума и има случаи, в които води до силно експлоатируеми стратегии“.⁹⁸ Съществуват безопасни алтернативи, но при директен сблъсък те се представиха по-зле, така че Pluribus взема емпиричната победа и смекчава риска само като винаги пререшава от началото на рунда за залози. Втора слабост е наследена и никога не е напълно затворена: в *първия* рунд за залози залозите на опонента, които се отклоняват твърде много от менюто на плана, все още се **закръглят** чрез транслация на действия – същата слабост, която модулет за самоусъвършенстване на Libratus съществуваше, за да преодолее – и Pluribus, тъй като няма такъв модул, просто живее с нея.

Няколко други детайла са от значение. Основните нововъведения на Pluribus **никога не са оценени поотделно (чрез аблация)**: авторите признават, че дисперсията на безлимитния покер и цената на човешките изпитания правят „твърде скъпо“ измерването на приноса на всяко едно от тях, така че докладваните ускорения на компонентите (приблизително 3× от линейния CFR, 2× от модифицираното подрязване, повече от 2× в паметта от отложеното разпределение) са оценки, а общото предимство не е разложено. Статията за предшественика за двама играчи обаче разсейва едно изкушаващо погрешно схващане: приемането на *една-единствена* стратегия за продължение (плана) в листата – очевидното нещо за опитване – губи от Baby Tartanian8 (-10 ± 8 mbb/g) и не побеждава Slumbot (-1 ± 15), докато версията с четири стратегии за продължение побеждава и двата бота ($+6 \pm 5$ и $+11 \pm 9$);⁸⁴ наборът от стратегии за продължение върши реална работа, а не е украшение. По-малки любопитни факти допълват картината: Pluribus играе стратегията от **последната** итерация на търсенето, а не обичайната усреднена по итерациите стратегия, за да избегне остатъчни лоши действия; по време на самообучението той се научи да **избягва влизането само с плащане на блайнда („лимпинг“)**, но залага „донк“ (с изпреварващ залог) **много често от хората**; и – както Libratus – неговият код **никога не е бил публикуван**, оставяйки само псевдокод за независима проверка, тъй като покерът се играе комерсиално.

6.4.5 Изчислителни разходи и достъпност

Историята за изчислителните разходи на Pluribus е тази, която повечето хора помнят, и тя наистина обръща с главата надолу своите предшественици. Планът беше обучен за **осем дни на един 64-ядрен сървър** за около **12 400 ядро-часа** и под 512 GB памет – приблизително **\$144** при спот цени в облака – а на масата Pluribus работи на **два CPU (28 ядра) и под 128 GB, без GPU на който и да е етап**, като взема от една до тридесет и три секунди за решение и играе около два пъти по-бързо от човек.⁸⁵ В сравнение с останалите контрастът е почти комичен: AlphaGo използва 1920 CPU и 280 GPU, Deep Blue – 480 специално проектирани чипа, а Libratus – около 25 милиона ядро-часа общо (от тях около 6 милиона за плана)⁹⁴ и 100 процесора по време на игра. Pluribus достигна

⁹⁸Brown, N. & Sandholm, T. (2019). Supplementary Materials for “Superhuman AI for multiplayer poker.” *Science*, 365(6456).

по-труден етап – повече играчи, по-голяма игра – за около една хилядна от изчисленията за обучение на Libratus. Докато DeepStack концентрира разходите си офлайн, а Libratus плащаше скъпо и в двата края, Pluribus е **евтин и в двата края**, и това, също толкова колкото резултатът с шестима играчи, е тезата на статията.

Това преформулира изцяло достъпността. За първи път една свръхчовешка покер система беше възпроизводима, по принцип, от един добре оборудван изследовател, а не от суперкомпютърен център – статията сама противопоставя този резултат на „всички други скорошни постижения на свръхчовешкия изкуствен интелект в игрите, които са използвали голям брой сървъри и/или ферми от графични процесори“. ⁸⁵ Рязкото поевтиняване не е магия, а резултат от алгоритмите: съчетаването на търсене с ограничена дълбочина (което авторите оценяват, че спестява поне пет порядъка спрямо решаване до края), линеен CFR и агресивно подрязване и пестене на памет. Единственото уточнение е, че самият артефакт остава затворен – без код – така че „достъпен“ описва *метода*, демонстриран в мащаб на лаптоп от Modicum, повече отколкото изтегляема програма.

6.4.6 Силни страни и ограничения

Главното предимство на Pluribus е просто в това, че е **първи**: първият изкуствен интелект, който постига свръхчовешко представяне в която и да е широко призната еталонна игра с повече от двама играчи или два отбора и в най-популярната форма на покера. ⁸⁵ Победата беше решителна и строга – **+48 mbb/g (p = 0,028)** срещу петима елитни професионалисти на масата и **+32 mbb/g (p = 0,014)** с пет копия срещу един-единствен професионалист – измерена с намалителя на дисперсията **AIVAT** (който намали дисперсията около девет пъти) в 20 000 раздавания, срещу петнадесет професионалисти, всеки от които е спечелил над един милион долара и които имаха дни да търсят слабости; темпът на печалба почти не се колебаеше. И, подобно на Libratus, Pluribus е с **устойчивост на първо място**: той играе фиксирана стратегия и никога не моделира опонентите, нито се адаптира към тях, така че не може да бъде примамен в контраексплоатируема корекция (дори не знае самоличността им, което изключва и умишлен сговор между копията му) – дисциплина, която заедно с **липсата на човешки данни и експертни познания** запазва резултата чист.

Ограниченията са именно това, от което се отказва, за да стигне дотук. Pluribus **няма абсолютно никаква гаранция за безопасност** в обстановката с шестима играчи – няма сходимост към равновесие на Наш, няма граница на експлоатируемостта – така че неговото свръхчовешко състояние е *емпиричен* факт за петнадесет силни играчи в 20 000 раздавания, а не теорема; достатъчно координирана маса или просто различна игра не носят никаква увереност. Той разчита на **небезопасно** търсене, все още **закръгля залозите извън дървото в първия рунд**, и остава **изцяло табличен и основан на абстракция**, без нищо научено, което да обобщава в различни ситуации – планът е гигантска справочна таблица, а не модел. Статията твърди само, че „съществуват мащабни и сложни среди с много играчи и непълна информация, в които внимателно конструиран алгоритъм за самообучение с търсене може да създаде свръхчовешки стратегии“; за игри, в които играчите могат да **комуникират или да се сговарят**, тя не твърди нищо. ⁸⁵ И неговият темп на печалба от 48 mbb/g на маса с шестима, макар и решителен, не е в същата „валута“ като 147-те

mbb/g на Libratus в игра един срещу един – различна игра, петима опоненти, по-висока дисперсия. Повечето от тези пропуски са поправени от по-късни системи; този, който тази дисертация откроява – **че успехът в игра с много играчи дойде без никаква гаранция за безопасност** – е изричната цел на принос 2.

6.4.7 Наследство и съвременна значимост

Две идеи надживяват спецификите на покера. Първата е техническа: **търсене с ограничена дълбочина, направено коректно при скрита информация чрез даване на всяко листо на малко меню от избираеми стратегии за продължение вместо една-единствена стойност** – чист начин да се погледне само малко напред в игра, където наивно не можеш. Втората е икономическа и областта я възприе най-дълбоко: Pluribus е каноничната демонстрация, че **по-труден проблем може да бъде решен с хиляда пъти по-малко изчисления чрез по-добри алгоритми и търсене в момента на вземане на решение**, вместо повече хардуер. Noam Brown оттогава посочи точно това – че 20 секунди търсене на раздаване са подобрили неговия покер бот толкова, колкото би го подобрило 100 000-кратно увеличаване на модела – като ранен, конкретен пример за аргумента за „**изчисления по време на изпълнение**“ (test-time compute), който сега е централен за модели за разсъждение като o1.⁹⁷ В архитектурната дъга на тази глава Pluribus е точката, в която *барьерата на броя играчи* пада; *барьерата на обобщаването*, която той оставя изправена – всичко все още е таблично и ръчно абстрахирано – е това, което **ReBeL** и **Student of Games** разрушават след това, като заменят плана с научени стойности на състоянието на убеждението.

Няколко от частите на Pluribus остават директно използваеми: конструкцията със **стратегии за продължение (листо с няколко стойности)** за стабилно търсене с ограничена дълбочина във всяка среда с непълна информация или многоагентна среда; **Линеен (дисконтиран) CFR**, вече стандартен ускорител на сходимостта и отправна точка за по-късни дисконтирани варианти на CFR; **подрязване, базирано на съжаление** за мащабиране на CFR; **AIVAT** като нискодисперсностна оценка; и обработката *добавяне на действието и пререшаване* на залозите извън дървото, наследена от Libratus. Това, което е заменено, е заобикалящата таблична конструкция – ръчно настроената абстракция и планът под формата на таблица за търсене – които научените представления на ReBeL и SoG премахват, заедно с *небезопасното* търсене, което тези методи със състояние на убеждението правят ненужно. Честната оценка отразява тази на Libratus: **заменен като внедрена архитектура, но оправдан и изострен като теза**. За тази дисертация по-специално Pluribus е ключовият камък на принос 2 – емпиричното доказателство, че методите, съчетаващи равновесие на Наш и търсене, *работят* в игри с N играчи и непълна информация, без да предлагат **абсолютно никаква гаранция за безопасност**, което е точно пропускът, който една теория на многоагентната безопасна експлоатация трябва да затвори.

6.5 ReBeL (2020)

Pluribus преодоля барьерата на броя играчи, но го направи, като остана точно това, което беше Libratus – гигантска, ръчно абстрахирана таблица за търсене без нищо научено, което да се

прехвърля от една ситуация в друга – и постигна победата си с шестима играчи, като изцяло се отказа от безопасността, разчитайки на *небезопасно* търсене без никаква гаранция за експлоатируемост. ReBeL (Brown, Bakhtin, Lerer & Gong, 2020; Facebook AI Research)⁹⁰ прави крачка назад от шестима към двама играчи и задава обратния въпрос: **какво ще стане, ако вместо ръчно да създаваме абстракция и предварително да изчисляваме план, пуснем AlphaZero – обучение с подкрепление чрез самоигра плюс търсене, както по време на обучението, така и по време на игра – в игра със скрита информация, възстановявайки гаранциите, които Pluribus отхвърли, като същевременно изцяло премахнем абстракцията?** Неговият отговор, *рекурсивно обучение на основата на убеждения* (Recursive Belief-based Learning), е по думите на авторите първият алгоритъм, който прави обучението с подкрепление и търсенето доказуемо коректни в игри с непълна информация. Трикът е да се преформулира играта като „игра с пълна информация“ върху **публични състояния на убежденията (PBS)** – вероятностни разпределения за това какво може да държи всеки играч при даденото общо знание – върху които функциите на стойността и на стратегията са добре дефинирани; след това цикъл в стил AlphaZero обучава невронна мрежа за стойности (и стратегия) върху тези състояния, като минимизирането на контрафактичното съжаление (CFR) решава под-игри с ограничена дълбочина на листата. ReBeL доказуемо клони към равновесие на Наш във всяка игра за двама с нулева сума и побеждава Донг Ким – професионалист в хедс-ъп, загубил най-малко от Libratus – със 165 mbb/g (хилядни от големия блайнд на раздаване – мерната единица от предишните раздели) в рамките на 7 500 раздавания, като използва *значително по-малко* експертни познания от всеки предишен покер изкуствен интелект – и, за разлика от затворените Libratus и Pluribus, неговата реализация (за Liar’s Dice) беше **отворена**.

Таблица 12: ReBeL (2020) накратко.

С един поглед	ReBeL (2020)
Играчи	2 (хедс-ъп) – <i>гаранциите</i> са за игра за двама с нулева сума; формализмът е за N играчи, но гаранциите и експериментите са само за двама
Тип игра	Произволни игри за двама с нулева сума и непълна информация; оценен върху HUNL покер и Liar’s Dice (и опростения вариант turn endgame hold’em). Редуцира се до алгоритъм в стил AlphaZero при игри с пълна информация
План (офлайн)?	Няма съхранен план – офлайн продуктът е <i>научена мрежа за стойности (+ стратегия) върху PBS</i> , получена чрез самообучение, а не таблица на стратегиите
Невронна компонента	Мрежа за стойности върху PBS + (по избор) мрежа за стратегия върху PBS; MLP (GeLU/LayerNorm), за покера 6 скрити слоя по 1536 неврона; вход = убеждения върху 1326-те възможни ръце на всеки играч + общите карти + пота + флаг за залог

С един поглед	ReBeL (2020)
Механизъм за търсене	CFR (CFR-D / CFR-AVG; също FP/FLOP), решаващ под-игра с ограничена дълбочина и корен в PBS, като научената мрежа за стойности дава (зависещите от итерацията) стойности в листата – както по време на обучението , така и по време на игра
Абстракция?	Няма – няма абстракция на карти/информация (със загуби или без); мрежата за стойности я замества. Запазва се само малко (≤ 9) ръчно избрано меню от размери на залозите, като залозите извън дървото се добавят в реално време
Също и с пълна информация?	Не (представен и оценен като метод за игри с непълна информация), с уточнението, че <i>дегенерира</i> до търсене в стил AlphaZero, ако частната информация бъде премахната – пълното обединяване е претенция на Student of Games
Изчислителни разходи	Обучение на GPU: пълният агент за HUNL използва ~90 DGX-1 възела $\times$ 8 V100 GPU за генериране на данни чрез самообучение (за разлика от ~\$150 само за CPU при Pluribus); CFR в една нишка на CPU; игра < 2 s на раздаване, ≤ 5 s на решение
Ключова иновация	Публични състояния на убежденията + цикъл на самообучение в стил AlphaZero, който обучава мрежа за стойности/стратегия върху PBS, а CFR работи в пространството на убежденията: по думите на авторите първото <i>коректно</i> съчетание на RL и търсене за игри с непълна информация; възстановява доказуема гаранция за равновесие на Наш при 2p0s без абстракция и без план

6.5.1 Пропускът, който запълни

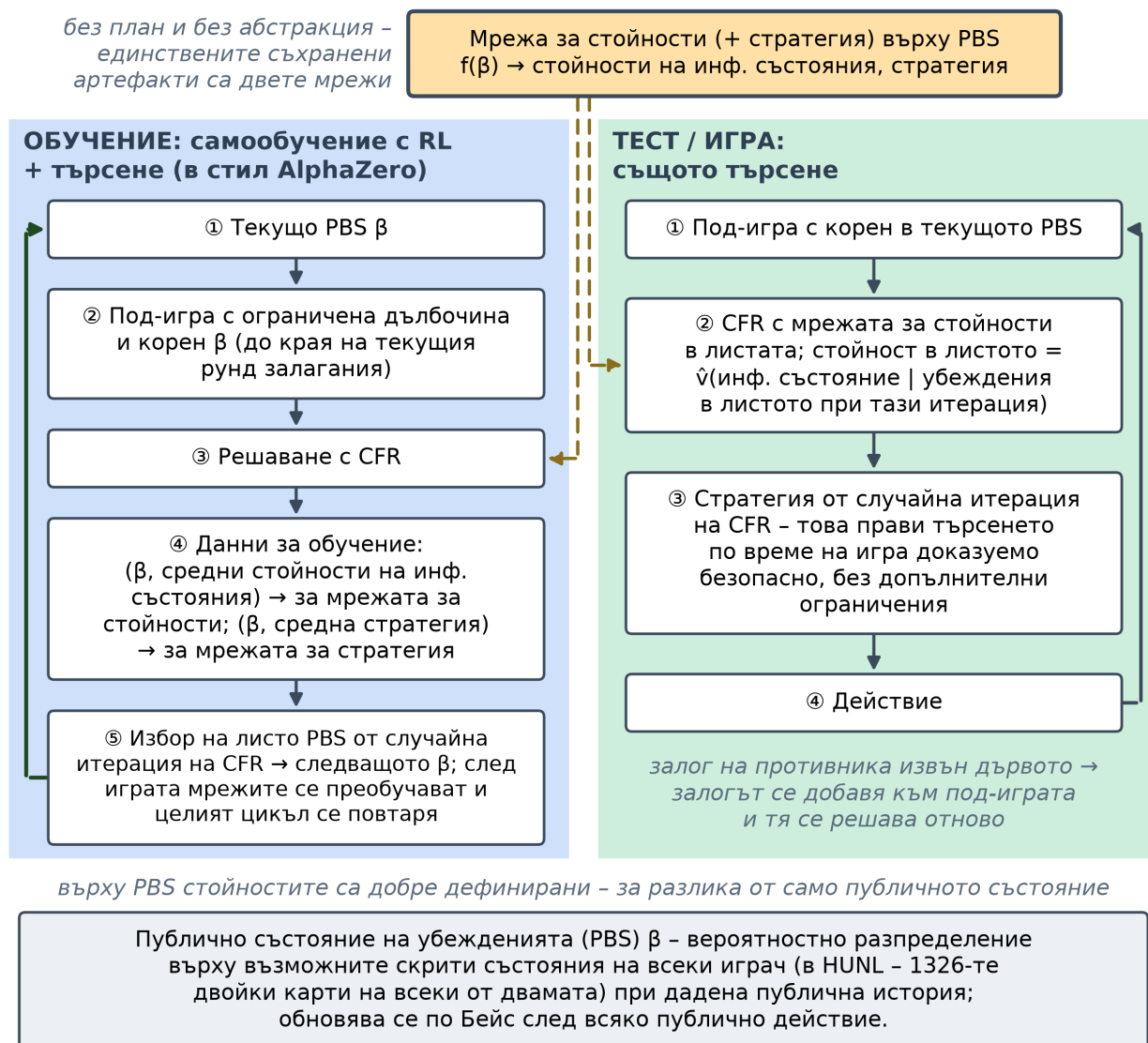
Предидните три системи стигнаха до една и съща рецепта – предварително изчислен план плюс търсене в реално време – а Pluribus я разшири до шестима играчи, но всяка от тях, в основата си, беше *табличен* обект: стратегия или план, съхранени върху ръчно проектирана абстракция, без нищо научено, което да се обобщава към ситуация, която никога не е била изрично решена. Само DeepStack имаше невронна компонента, но дори и той разчиташе на ръчно извлечени признаци, клъстеризация в 1000 клъстера на входа на мрежата и абстракция на ривъра. Междувременно най-успешната парадигма в изкуствения интелект за игри – съчетанието в AlphaZero на обучение с подкрепление чрез самоигра и търсене, което научава собствената си оценка от нулата и я използва както за обучение, така и за игра – беше *недостъпна* за игри с непълна информация. Отвореният въпрос, на който отговаря ReBeL, е: **може ли рецептата на AlphaZero да работи коректно в игри със скрита информация?** Дотогава алгоритмите, съчетаващи обучение с подкрепление и търсене, не са били теоретично коректни при непълна информация и не е било показано, че са успешни в такива игри.⁹⁰

Причината, поради която това не беше възможно преди, е фина и представлява концептуалната същност на цялата глава. AlphaZero предполага, че всяко състояние има една единствена добре дефинирана стойност: шахматна позиция струва толкова, колкото струва, независимо колко често достигате до нея. Скрытата информация унищожава това предположение. Статията го прави нагледно с модифициран камък-ножица-хартия, в който ножицата печели (или губи) двойно: равновесието е да се хвърля камък и хартия 40% от времето и ножица 20%, и при това равновесие всяко действие има очаквана стойност нула. Ако Играч 1 извърши еднопластово търсене с предвиждане, което работи в шаха – използвайки равновесната стойност в листата – всеки ход изглежда еднакво добър, така че търсещият може да се спре на „винаги камък“, след което противникът преминава към „винаги хартия“ и истинската стойност на камъка се срива от 0 до -1. **В игра с непълна информация стойността на едно действие зависи от вероятността, с която то се изиграва**, така че състояние, дефинирано само от последователността на действията, няма единствена стойност – и търсенето в стил AlphaZero е просто некоректно. Pluribus беше закръпил симптома с избираеми стратегии за продължение в листата⁸⁴; ReBeL лекува болестта, като променя това какво е едно „състояние“. Той предефинира състоянието така, че да включва разпределението на вероятностите върху скритата информация – *публично състояние на убежденията* – върху което, както показват следващите раздели, стойностите отново стават добре дефинирани и цялата машина на AlphaZero може да бъде прехвърлена. При това той също така премахва двете патерици, на които все още се опираха DeepStack и системите за абстракция: ръчно създадената абстракция и предварително изчисления план изчезват, заменени от функция на стойността, научена единствено от самообучение.

6.5.2 Архитектура

ReBeL е най-добре да се възприема като **AlphaZero за непълна информация**: цикъл на самообучение, който обучава невронни мрежи за стойности и стратегии, при който „търсенето“, използвано както по време на обучение, така и при игра, представлява решаване с CFR на под-игра с ограничена дълбочина – но всичко се извършва върху *публични състояния на убежденията*, а не върху сурови игрови състояния, като оценителят на листата е научена мрежа за стойности, а не симулация до края на играта (rollout) (вж. фигурата по-долу).⁹⁰⁹⁹

⁹⁹Bakhtin, A. (2020, 3 Dec.). “ReBeL: A general game-playing AI bot that excels at poker and more.” *Meta AI blog*.



Фигура 27: Цикълът на ReBeL в стил AlphaZero: самообучението (вляво) и играта (вдясно) използват една и съща мрежа за стойности и стратегия върху PBS; и в двата случая се решават под-игри с ограничена дълбочина и корен в публичното състояние на убежденията (общото определение е в полето долу).

Цикълът има три основни компонента. **Обучението с подкрепление чрез самоигра** е движещата сила: започвайки от коренно PBS, ReBeL построява под-игра с ограничена дълбочина, решава я, записва решените стойности и стратегия като цели на обучението, избира листо, което става следващото коренно PBS, и продължава до края на играта – след това преобучава мрежите и повтаря процеса, по същия начин както AlphaZero редува самообучение с обучение на мрежата. **Търсенето** представлява вътрешното решаване: ReBeL изпълнява CFR (конкретно CFR-D, „CFR с декомпозиция“, или неговия вариант CFR-AVG) върху ограничената по дълбочина под-игра и на всяка итерация на CFR задава стойността на всяко листо чрез заявка към **научената мрежа за**

стойности върху PBS – така стойностите в листата се променят от итерация до итерация, което е именно това, което гарантира коректността на търсенето при наличие на скрита информация. **Невронните мрежи** възплъщават научената интуиция: мрежа за стойности, която съпоставя PBS със стойностите на възможните ръце на играча, и незадължителна мрежа за стратегия, която дава на CFR добра начална стратегия (warm start) и така намалява броя на необходимите итерации.

Архитектурните особености се изразяват в *липсата* на елементи, характерни за по-ранните системи. Няма **предварително изготвен план** – единствените резултати от обучението са двете мрежи. Няма **абстракция** от какъвто и да е вид: докато DeepStack, Libratus и Pluribus групираха стратегически сходни ръце, ReBeL изчислява отделна стратегия за всяко информационно състояние, като подава на мрежата единствено суровото разпределение на убежденията върху възможните 1326 ръце и на двамата играчи, плюс общите карти, пота и един флаг „дали някой е заложил този рунд“. А **търсенето с CFR** работи в **една нишка на CPU** без никаква абстракция; основните изчислителни разходи са за самообучението на GPU, с което се обучава мрежата за стойности (вж. *Изчислителни разходи*). Класическите градивни елементи от по-ранните глави все още са видими – **CFR** (Глава 3) изпълнява търсенето, а **невронното приближение на стойността** (Глава 5) е оценителят на листата – но сега те са обединени в един цикъл „самообучение плюс търсене“, като **публичното състояние на убежденията** (разделът по-долу) служи за представянето, което прави това обединение възможно.

6.5.3 Ключова иновация: публични състояния на убежденията и научени стойности в пространството на убежденията

Приносът на ReBeL е един концептуален ход с три технически последствия. Ходът е да се спре търсенето върху **състояния** и да се започне търсене върху **убеждения за състоянията**.

Публичното състояние на убежденията (PBS) е сърцевината на всичко това. Блогът на Meta AI дава най-ясната интуиция.⁹⁹ Вземете игра с карти и я променете така, че играчите да не могат да виждат собствените си карти – само безпристрастен съдия може; на всеки ход играч обявява, за всяка карта, която *може* да държи, вероятността, с която би предприел всяко действие, и съдията избира хода за истинската карта на играча. Тъй като стратегиите на всички играчи се приемат за общоизвестни, всеки може да проследява, чрез правилото на Бейс, вероятността всеки играч да държи всяка възможна ръка. Тази модифицирана игра е **стратегически еквивалентна** на оригиналната, но тя не съдържа *частна информация*: нейното състояние – векторът от тези вероятности – е напълно наблюдаемо за всички. ReBeL нарича това състояние публично състояние на убежденията: формално, **съвместно разпределение на вероятностите** върху възможните информационни състояния на играчите при дадени общи публични наблюдения.⁹⁰ От решаващо значение е, че разглеждането на игрите с непълна информация като игри с непрекъснато състояние и пълна информация по този начин е стара идея (тя датира от работите върху децентрализирани многоагентни POMDP); постижението на ReBeL е, че за първи път я съчетава с обучение с подкрепление чрез самообучение в **състезателна среда**.

Първото следствие е, че **стойностите отново стават добре дефинирани**. Под-игра с непълна информация с корен в обикновено *публично състояние* няма единствена стойност, защото стойността ѝ зависи от скритите ръце, с които се достига до нея – което е точно причината евристичното търсене да се провали. Но под-игра с корен в *публично състояние на убежденията* има такава: в игрите за двама с нулева сума всяко публично състояние на убежденията β има единствена стойност $V_i(\beta)$ с $V_1(\beta) = -V_2(\beta)$, определена от това, че и двамата играчи играят равновесие на Наш в под-играта от β . Това е именно свойството, на което AlphaZero разчита в шаха, сега възстановено за покера – и то *формализира и обобщава контрафактичните стойности при дадени убеждения на DeepStack*, които бяха първият пример за функция на стойността върху PBS, но бяха обучени по много по-импровизиран начин.

Второто следствие е, че **цикълът на AlphaZero** може да работи с **CFR като алгоритъм за търсене**. По принцип би могло да се стартира търсене в дърво на Монте Карло на AlphaZero директно върху играта на убежденията, но представянето на убежденията е много високодимензионно непрекъснато пространство (в малкия пример с карти от блога само пространството на действията има 156 измерения) и там търсенето с дърво е безполезно. За щастие, в игрите за двама с нулева сума тези представяния на убежденията са *изпълнили* оптимизационни задачи, а CFR на практика е градиентен метод за решаването им. Така ReBeL търси с CFR вместо с търсене в дърво на Монте Карло – единствената съществена разлика от AlphaZero. Техническа тънkost, която статията доказва (Теорема 1), е, че количествата, които CFR действително изисква – стойностите за всяко информационно състояние – са *свърхградиент* на вдлъбнатата функция на стойността върху PBS, така че ReBeL обучава мрежа за стойности на информационните състояния, а не един скалар за всяко PBS.

Третото и най-важно следствие е, че цялата схема е **доказуемо коректна** и възстановява гаранцията, от която Pluribus се отказа. Най-дълбокият резултат засяга *етапа на игра* (test time). По време на самообучение ReBeL може да приеме, че стратегиите и на двамата играчи са общо знание, така че винаги знае истинското PBS; но срещу реален противник той не знае стратегията му и следователно не знае в кое PBS се намира, а при наивен подход това прави търсенето некоректно. Елегантното решение на ReBeL е да изпълнява *същия алгоритъм* по време на игра и **да действа според стратегията на произволно избрана итерация на CFR**. Статията доказва (Теорема 3), че това води до безопасно търсене (изиграната стратегия е равновесие на Наш *по математическо очакване*) без допълнителни ограничения, докато всеки предишен метод за „безопасно търсене“ добавяше ограничения толкова скъпи, че никога не бяха използвани изцяло в състезателен бот. Конкретно, с мрежа за стойности върху PBS, чиято грешка е най-много δ и T итерации на CFR на под-игра, ReBeL играе ε -равновесие на Наш – такова, което никой противник не може да експлоатира за повече от ε - с

$$\varepsilon = \delta C_1 + \frac{\delta C_2}{\sqrt{T}},$$

за специфични за играта константи C_1, C_2 . Структурата умишлено отразява тази на $k_1\epsilon + k_2/\sqrt{T}$ от DeepStack – член за грешката в стойността плюс обичайния член за сходимостта на CFR – но докато границата на DeepStack управлява експлоатируемостта на едно пререшаване, тази на ReBeL управлява *сходимостта към равновесие на Наш на цялата игра*, и двата члена вече се мащабират с грешката на научената стойност δ : с подобряването на мрежата за стойности $\delta \rightarrow 0$ и играта на ReBeL се доближава до точно равновесие на Наш. Това е точният смисъл, в който ReBeL *се връща към игрите за двама с нулева сума и възстановява теоретичните гаранции, които Pluribus се отказва да даде* – същият $1/\sqrt{T}$ гръбнак на CFR, който доказва безопасността на DeepStack и Libratus, сега обвит около стойност на състоянието на убежденията, научена чрез самообучение, вместо ръчно изградена абстракция.

6.5.4 Уговорки, задънени улици и какво статията не описва

ReBeL обръща обичайния модел на тази глава: *теорията* е в основния текст, а *инженерната честност* – по-специално точният смисъл, в който се използва „значително по-малко експертни познания“ – е преместена в приложенията.

Най-важният детайл е това, което ReBeL умишлено **отхвърля**, каталогизирано в Приложение D. Всеки предишен водещ изкуствен интелект за покер, включително DeepStack, използваше **информационна абстракция**, за да групира стратегически подобни ръце; ReBeL не използва *никаква*, нито със загуби, нито без загуби, като изчислява отделна стратегия за всяко информационно състояние от суровото разпределение на убежденията. DeepStack обучаваше своята мрежа за стойности върху *случайно генерирани* PBS, извлечени от ръчно настроен генератор; ReBeL генерира своите тренировъчни PBS изцяло от самообучение, твърдейки, че случайната извадка „би била като да се научи функция на стойността за Го чрез случайно поставяне на камъни върху дъската“ – и наистина показва (фиг. 2 в статията), че мрежа за стойности, обучена върху случайни убеждения, „се проваля в научаването на нещо ценно“. Предишните агенти предварително изчисляваха точни **таблици със стойността при отиграване докрай** и решаваха *до края на играта* на третия рунд за залози (използвайки **абстракция на ривъра**, за да го направят изчислимо); ReBeL не прави нито едно от двете, като винаги решава само до края на *настоящия* рунд и сам научава дори стойностите при отиграване докрай. Резултатът е честен, но с двойно острие: ReBeL трябва да научи *шест* „слоя“ от стойности, където DeepStack се нуждаеше от три, което *увеличава* повърхността за **разпространение на грешка** – реална цена, приета в замяна на премахването на абстракцията. И не е без експертни познания: запазва ръчно избран списък с най-много осем или девет размера на залози (леко варирани по време на обучението), единствената отстъпка в полза на изчислимостта, като залозите извън дървото се обработват чрез добавянето им към под-играта и **пререшаването** ѝ, както в Libratus и Pluribus.

Няколко други важни момента трябва да се отбележат. Чистите теореми стъпват върху **идеализации**: сходимостта в Теорема 2 предполага съвършен апроксиматор на функции, а гаранциите изискват стратегиите на играчите да са общо знание (резултатът от §6 на статията премахва това предположение за *етапа на игра*, но анализът на обучението все още идеализира мрежата).

Вариантът, който агентът реално използва за ефективност – модифициран **CFR-AVG** – според самите автори *не е доказано коректен* („дали тази модифицирана форма на CFR-AVG е теоретично коректна, остава отворен въпрос“)⁹⁰, въпреки че се представя добре в покера – позната празнина между доказания алгоритъм и внедрения. Гаранцията за безопасно търсене също зависи от избора на *случайна* итерация на CFR, която може да се окаже много лоша в ранните етапи; това се смекчава единствено чрез използването на Linear CFR, който намалява тежестта на ранните итерации. И както при предшествениците си, по същество цялата архитектура, хиперпараметрите и най-важните данни за изчислителните разходи са представени в приложенията, а не в шестте страници основен текст – статията е по замисъл теоретична.

6.5.5 Изчислителни разходи и достъпност

Профилът на разходите на ReBeL е огледален образ на този на Pluribus и контрастът е най-рязък в тази глава. Там, където интуицията на Pluribus беше план, решен чрез **CFR само с процесори на един 64-ядрен сървър за около \$150**, интуицията на ReBeL е невронна мрежа, а обучението ѝ изисква **клъстер от графични процесори**. Тясното място е генерирането на данни чрез самообучение – последователни решавания с CFR, при които на всяка итерация мрежата оценява всички листа – затова статията паралелизира генерирането на данни върху „до 128 машини с по 8 GPU всяка“, а пълният агент за HUNL в частност използва **около 90 възела DGX-1, всеки с осем GPU Nvidia V100 с 32 GB** (общо около 700 GPU), захранвайки една единствена тренировъчна машина с над 1750 епохи от по милиони примери всяка. Това са разходи за обучение на невронна мрежа, а не за CFR: самото търсене с CFR се изпълнява в една процесорна нишка. На масата обаче ReBeL е евтин и подобен на DeepStack – средно под две секунди на раздаване и никога повече от пет секунди на решение, като под-игрите на префлопа се кешират, за да бъде още по-бърз.

Историята с достъпността се разделя на две и това е най-ясният контраст със системите за покер от по-ранен период. От една страна *методът* на ReBeL е най-общият и най-малко ръчно настрояният от четирите, и – решаващо – неговата **реализация беше с отворен код** (за Liar's Dice), докато кодът както на Libratus, така и на Pluribus остана затворен (и двете статии публикуват само псевдокод).⁹⁰⁸⁶⁸⁵ Това отворено издание е истинска стъпка към възпроизводимост, която системите на CMU никога не предлагаха. От друга страна, основният *покер* резултат *не* беше публикуван – авторите го задържаха с изричното основание, че ReBeL „може да изчисли стратегия за произволни размери на стакове и произволни размери на залози за секунди“, което го прави готов инструмент за измама – а възпроизвеждането на този резултат в пълния му мащаб изискваше голям парк от V100, какъвто повечето групи не притежаваха. Така че ReBeL е „отворен“ по начин, по който неговите предшественици не бяха, но отвореният артефакт е изследователската игра, а не свръхчовешкият покер бот.

6.5.6 Силни страни и ограничения

Главното предимство на ReBeL е **общност, подкрепена с гаранция**. По думите на авторите това е първият алгоритъм, който прави парадигмата на AlphaZero „обучение с подкрепление + търсе-

не“ *доказуемо коректна* в игри с непълна информация⁹⁹; той доказуемо клони към равновесие на Наш във всяка **игра за двама с нулева сума** и подкрепя теорията с резултати: побеждава предишните шампиони Slumbot (+45 mbb/g) и BabyTartanian8 (+9 mbb/g), побеждава с голяма разлика и **локалния най-добър отговор (LBR)**⁸⁸, както и професионалиста в хедс-ъп Донг Ким (със 165 mbb/g в 7500 раздавания) – всичко това, използвайки *значително по-малко експертни познания от всеки предишен покер изкуствен интелект*, без **абстракция на картите** и с най-общото представяне от четирите. Същият алгоритъм, непроменен, клони към **приблизително равновесие на Наш** в Liar’s Dice, демонстрирайки, че това е *рамка*, а не покер програма; в игри с пълна информация той елегантно се свежда до метод, подобен на AlphaZero. И той възстановява, за невронната ера, безопасността, която Pluribus трябваше да изостави – търсене, което е **доказуемо безопасно** по време на игра без допълнителни ограничения.

Ограниченията са точни и подготвят финалната система. Гаранциите на ReBeL важат **само за игри за двама с нулева сума**: единствената стойност на PBS, изпъкналостта, която позволява CFR да се използва като търсене, и доказателствата за коректност се основават на тази структура. Формализмът на статията е записан за N играчи, но всички експерименти и гаранции са за двама – „2 играча“ в таблицата „С един поглед“ се отнася до *гаранцията*, а не до формализма. Най-конкретната му стена на мащабиране е, че **пространството на убежденията нараства с количеството скрита информация**: входът на мрежата се мащабира с броя на информационните състояния в едно публично състояние, така че игри с голяма стратегическа дълбочина, но малко общо знание – статията посочва Reconnaissance Blind Chess (шах „на сяло“ с разузнаване) – раздват представянето, а добавянето на играчи само увеличава пространството на убежденията. Алгоритъмът предполага, че **точните правила на играта са известни** (разширение в стил MuZero към неизвестна динамика е посочено като бъдеща работа). Той наследява остатъчна **грешка на приближението на стойността** δ , която ограничава колко близо до равновесие на Наш всъщност играе, а най-силният му емпиричен резултат се основава на **един човешки противник** в 7500 раздавания, а не на мач срещу няколко професионалисти.

6.5.7 Наследство и съвременна значимост

Ако оставим покера настрана, трайната идея на ReBeL е свързана с представянето: **превърнете скритата информация в състояние на убежденията и цялата мощ на обучението с подкрепление чрез самоигра плюс търсене може да се пренесе**. Публичните състояния на убежденията оттогава са в основата и на по-късните методи за коректно търсене, например Student of Games⁹¹, а ReBeL е най-ясната формулировка на „AlphaZero за непълна информация“ – търсене, вплетено както в *обучението*, така и в *играта*. Най-близкият му наследник е **Student of Games** (2023), който пренася идеята за състояние на убежденията плюс коректно търсене по-нататък в единен алгоритъм, обхващащ игри с пълна и непълна информация (авторите му определят ReBeL като „най-близкия“ до своя алгоритъм⁹¹); същата линия – обучение с подкрепление плюс търсене с научени модели – води и до **CICERO** (2022)⁹², агента на Meta, който играе Diplomacy на човешко ниво и планира с научени модели в игра за седем играчи със смесени мотиви и комуникация на

естествен език, далеч извън гаранциите на ReBeL за двама играчи. По-широко погледнато, ReBeL принадлежи към линията работа – DeepStack, Libratus, Pluribus и сега ReBeL – която днес често се представя като ранен пример за **изчисления по време на изпълнение (test-time compute)**: изразходване на изчисления за търсене в момента на вземане на решение, насочвано от научена интуиция, вместо всичко да се влага предварително в една изчислена стратегия – същата идея, която днес е централна за моделите за разсъждение (reasoning models).

Няколко подсистеми остават директно използвани. Най-важно е самото **представяне чрез PBS** – разпределение на убежденията върху скритите състояния на противниците при наличие на общо знание – и то представлява естествената основа за базирано на убеждение моделиране на противника: Принос 1 на тази дисертация разширява PBS, така че да включва убеждения относно *типа стратегия* на противника, а не само неговите карти. **Обучената чрез самоигра функция на стойността/стратегията върху състояния на убежденията**, схемата **CFR като търсене с научен оценител на листата**, трикът за „**безопасно търсене безплатно**“ чрез произволни итерации и бързият търсач на равновесие **FLOP**, представен в статията, са всички преносими. Това, което е преодоляно, е основно ръчно изграденото скеле на DeepStack, което самообучението на ReBeL без абстракция прави излишно, а след това и самият ReBeL като *обща* рамка, която Student of Games обхваща. Честната оценка е, че **ReBeL е междинна, но все още актуална стъпка, а не крайна точка**: като внедрен покер артефакт той никога не беше пуснат и сега е засенчен, но като идея – публични състояния на убежденията плюс обучение с подкрепление и търсене в стил AlphaZero – той е основополагащ и все още актуален. За тази дисертация конкретно той действа в две посоки: от една страна е отправната точка за по-богатото състояние на убежденията в Принос 1, а от друга – доказателството, че коректността, изоставена от Pluribus, *може* да бъде възстановена – макар все още не и при повече от двама играчи, което е точно границата, която Принос 2 се стреми да премине.

6.6 Student of Games / SoG (2023)

ReBeL възстанови коректността в покера като игра за двама с нулева сума и изостави както абстракцията, така и плана – но той остана метод с непълна информация, който просто *се свеждаше* до AlphaZero в игри с пълна информация, вместо да ги обедини, решаваше *фиксирана* под-игра с ограничена дълбочина и обвързваше търсенето си по време на игра с процедурата си за обучение. Student of Games (Schmid et al., 2023; Google DeepMind, с партньори от Университета на Алберта и EquiLibre Technologies в Прага) запълва последния пропуск в главата, като задава най-амбициозния въпрос от всички: **какво ще стане, ако един единствен алгоритъм, който се учи чрез самообучение, без човешки данни, може да играе шах, Го, покер един на един и Scotland Yard – разраствайки собственото си дърво за търсене според нуждите и оставяйки доказуемо коректен както за игри с пълна, така и с непълна информация?** Неговият отговор обединява двете големи линии на главата – по думите на първия автор Мартин Шмид, това е

„AlphaZero и DeepStack в един голям унифициран алгоритъм“¹⁰⁰. Двигателят е **минимизирането на контрафактичното съжаление с растящо дърво (Growing-Tree CFR, GT-CFR)**: търсене, което може да бъде прекъснато във всеки момент (anytime) и изгражда дървото на играта постепенно, насочвано от мрежа за стратегия, с научена мрежа за стойности, оценяваща листата, които все още не са разширени. Върху поддървета с пълна информация GT-CFR се държи като MCTS на AlphaZero; върху тези с непълна информация – като CFR; и една единствена теорема за коректност покрива и двете. Системата достига **силно аматьорско до професионално ниво в шах и Го, побеждава Slumbot – най-силния публично достъпен бот за HUNL – и побеждава най-силния съществуващ агент за Scotland Yard**, като същевременно е **доказано, че клони към минимаксно оптимална игра с нарастването на изчислителния ресурс и точността на мрежата**. Той споделя няколко автори от DeepStack (Bowling, Moravčík, Burch, Schmid) – линията от Алберта, която започна тази глава, тук в контраст с произхода на ReBeL от Facebook/FAIR – и първоначално се появи през 2021 г. под името *Player of Games*, преди да бъде преименуван за своята публикация в *Science Advances* през 2023 г.

Таблица 13: Student of Games (2023) накратко.

С един поглед	Student of Games (2023)
Играчи	2 (игра за двама с нулева сума) – гаранциите и оценката са за 2p0s (в Scotland Yard детективите действат като един отбор, т.е. като един играч); основната формализация е по-обща, но извън 2p0s гаранцията на равновесието на Наш е „по-малко смислена“
Тип игра	Както с пълна, така и с непълна информация – единствената унифицирана система: шах + Го (пълна) и HUNL покер + Scotland Yard (непълна)
План (офлайн)?	Не – офлайн продуктът е научена мрежа за стойности и стратегия, а не съхранена таблица на стратегиите (както в ReBeL)
Невронна компонента	Една-единствена контрафактична мрежа за стойности и стратегия (CVPN) : една мрежа извежда контрафактични <i>стойности</i> за всяко информационно състояние и априорна <i>стратегия</i> , за всички етапи на играта
Механизъм за търсене	GT-CFR – редува фаза на обновяване на съжалението в CFR (стойности на CVPN в листата) с фаза на разширяване по PUCT в стил AlphaZero, която <i>разраства</i> публичното дърво; работи вътре в непрекъснатото пререшаване, както по време на обучение , така и по време на игра
Абстракция?	Няма от вида на картите/информацията (CVPN я заменя); запазва само малко <i>рандомизирано</i> меню от залози (действия) в покера ($\approx 20\,000 \rightarrow 4\text{--}5$ действия), както ReBeL

¹⁰⁰Kilcher, Y. (2022). “Player of Games: All the games, one algorithm! (w/ author Martin Schmid).” Video interview, YouTube. <https://www.youtube.com/watch?v=U0mxx7AoNz0>

С един поглед	Student of Games (2023)
Също и с пълна информация?	Да – отличителният ред; единствената система в главата, демонстрирана върху игри с пълна информация, със <i>същия</i> алгоритъм и коректност, покриваща и двата класа
Изчислителни разходи	Обучена на TPU, умишлено съпоставена с бюджета на AlphaZero (Google TPUnv4; Го е най-скъпата); докладвано <i>относително</i> спрямо AlphaZero, без една единствена сума в долари; търсенето е $O(kT^2)$ ($O(T)$ за игри с пълна информация); пълният агент/код не беше публикуван
Ключова иновация	Growing-Tree CFR + коректно самообучение: един алгоритъм за самообучение с търсене, коректен и за двата класа игри , който разраства дървото си постепенно с CVPN в листата и доказуемо клони към равновесие на Наш

6.6.1 Пропускът, който запълни

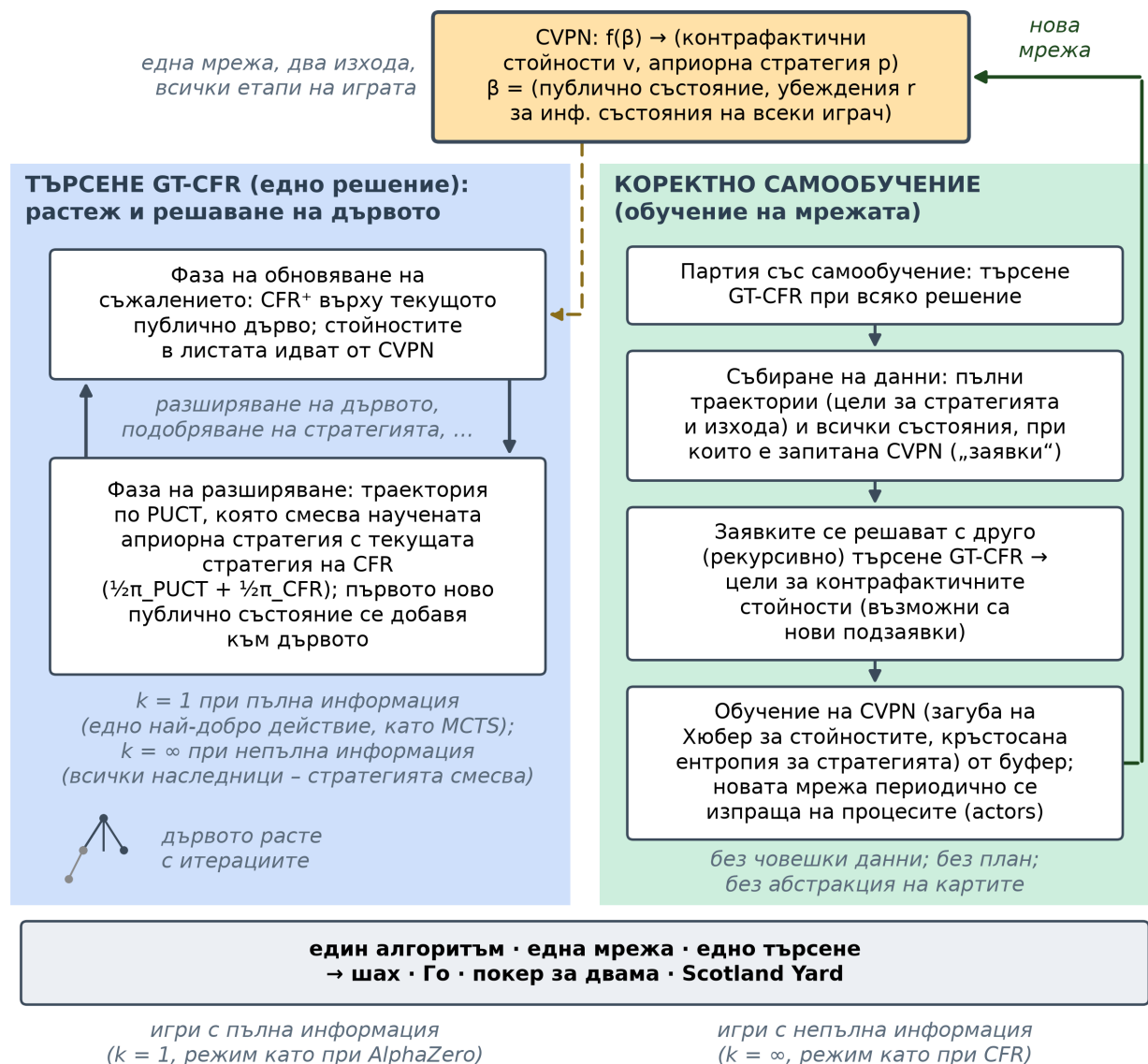
От програмата за игра на дама на Самюъл през 50-те години насам пробивите в изкуствения интелект за игри идваха по една игра наведнъж, а двете големи традиции се развиваха поотделно.⁹¹ Едната – минимакс, алфа-бета, търсене в дърво на Монте Карло и в крайна сметка AlphaZero – завладя игрите с *пълна информация*, като комбинира търсенето с научена функция на стойността, а най-общото ѝ изражение, AlphaZero, овладя шах, шogi и Го с един алгоритъм. Другата – минимизиране на контрафактичното съжаление и четирите покер системи в тази глава – завладя покера с *непълна информация* чрез игровитеоретично разсъждение. Но пътищата никога не се срещнаха. AlphaZero не можеше да играе покер: неговото търсене в дърво на Монте Карло е *некоректно* при скрита информация, по причината, която разделът за ReBeL изясни – стойността на едно състояние зависи от вероятността, с която се достига до него, така че стойностите не могат просто да бъдат върнати назад от локални поддървета. А покер системите не можеха да играят Го: те бяха изградени около структурата на залаганията на една игра. Дори ReBeL, най-общата от тези системи, беше представен и имаше гаранции само за непълна информация, *свеждайки се* до AlphaZero в случая с пълна информация, вместо наистина да обхваща и двата класа.

Отвореният въпрос, на който Student of Games отговаря, е дали *един* алгоритъм – една процедура за търсене, една архитектура на мрежата, един цикъл на самообучение – може да бъде **коректен и силен и в двата класа игри едновременно**. Трудността не е просто инженерна. Търсенето при пълна информация цели да разшири най-добрата линия дълбоко и да прочете една-единствена стойност от всяко **листо**; търсенето при непълна информация трябва да поддържа *разпределение* върху действията (за да не изтича частна информация) и да разсъждава за *убеждение* относно скритите състояния във всяко **листо**. Съгласуването на тези два подхода в един алгоритъм означава намиране на търсене, което разширява дърво по начина, по който го прави AlphaZero – насочено, асиметрично и с възможност за прекъсване във всеки момент – като същевременно изчислява иг-

ровотеоретично коректните величини, които CFR изчислява. Student of Games запълва този пропуск, като изгражда точно това търсене, GT-CFR, върху представянето чрез публични състояния на убежденията, което ReBeL беше въвел, и като обучава своите мрежи чрез *коректно самообучение*, чиито цели произтичат от самото търсене. Резултатът е кулминацията на главата: към линиите „от абстракция към невронни мрежи“ и „от предварително изчисление към търсене“ се добавя трета – обединяването на игрите с пълна и непълна информация.

6.6.2 Архитектура

Student of Games е, по своята структура, **цикълът на самообучение на AlphaZero, при който търсенето в дърво на Монте Карло е заменено от GT-CFR и при който мрежата за стойности и стратегия е дефинирана върху публични състояния на убежденията** – офлайн фаза, която обучава мрежа чрез самообучение, и онлайн фаза, която търси с нея, но при която същото търсене вече се изпълнява и в двете фази (вж. фигурата по-долу).



Фигура 28: Student of Games: цикълът на търсене GT-CFR (вляво) и цикълът на коректно самообучение, който го запазва (вдясно), използват една и съща CVPN; долната лента обобщава обединяването на игрите с пълна и с непълна информация.

Представянето е наследено от линията на ReBeL: **публичното състояние на убежденията** $\beta = (s_{\text{pub}}, r)$ съчетава публичното състояние (в покера – историята на залаганията и общите карти) с **диапазон** r – двойка разпределения върху информационните състояния, в които всеки играч може да се намира, без другите да знаят.⁹¹ Игрите с пълна информация са просто дегенеративният случай, при който всяко публично състояние има точно *едно* информационно състояние и убеждението е точкова маса, което е именно причината едно и също представяне да може да обслужва и двата класа. Върху това представяне стъпват двата основни компонента. **Контрафактичната мрежа за стойности и стратегия (CVPN)** е *една-единствена* мрежа, която при подадено състоя-

ние на убежденията извежда както вектор от контрафактични стойности (по една за всяко информационно състояние и всеки играч), така и априорна стратегия – една мрежа, която изпълнява и двете задачи за всеки етап от играта, докато DeepStack използваше отделни мрежи само за стойности, по една за всеки рунд. **Търсенето GT-CFR** е механизмът, който превръща тази мрежа в стратегия, а цикълът на **коректно самообучение** е този, който обучава мрежата въз основа на собствения изход от търсенето. Офлайн процесите за генериране на данни (actors) играят игри чрез самообучение, изпълняват търсене GT-CFR при всяко решение и генерират два вида данни за обучение – пълни игрови траектории и състоянията на убеждението, при които мрежата е била запитана по време на търсене – докато обучаващите процеси (trainers) обучават нова CVPN и периодически я изпращат обратно. Онлайн агентът изпълнява същото търсене GT-CFR (чрез непрекъснато пререшаване), за да избере всеки ход. Класическите градивни елементи са всички видими – CFR⁺ (Глава 3) е вътрешният цикъл на търсенето, приближението на стойността и стратегията (Глава 5) е CVPN, а публичните състояния на убежденията и разлагането (линията DeepStack/ReBeL) са представянето – но свързващата новост е GT-CFR, търсенето, което *разраства* своето дърво, и коректното самообучение, което поддържа всяко търсене съгласувано с всяко друго.⁹¹

6.6.3 Ключова иновация: Growing-Tree CFR и коректно самообучение

Приносът на Student of Games е търсещ алгоритъм, който за игри с непълна информация прави това, което търсенето в дърво на Монте Карло направи за игри с пълна информация – интелигентно разраства дърво към линиите, които имат значение – без да жертва коректността, която изисква скритата информация. Две идеи го правят работещ, а една единствена теорема го прави общ.

Първата идея е **разрастване на дървото чрез редуване на две фази**. MCTS на AlphaZero разширява един възел при всяка симулация и никога не се връща назад, защото в игра с пълна информация стойността на даден възел, след като бъде изчислена, вече не се променя – бъдещето не влияе върху миналото. При непълна информация това не е вярно: както отбелязва Шмид, наблюдението на *бъдещо* действие на противника променя вашето *убеждение* за частното състояние, което той е държал в миналото, така че всяка промяна на стратегията в дървото се разпространява навсякъде. Затова едно търсене не може просто да разширява и оценява; след всяко разширяване то трябва да преизчислява *цялото* дърво, за да го поддържа самосъгласувано. GT-CFR прави именно това. Всяка итерация изпълнява последователно две фази. **Фазата на обновяване на съжалението** извършва няколко итерации на Public-Tree CFR⁺ върху *текущото* дърво и където то спре – на своите гранични листа – запитва CVPN за контрафактичните стойности на под-играта отдолу, точно както решаването с ограничена дълбочина използва научена стойност на хоризонта. **Фазата на разширяване** след това разраства дървото: тя симулира една единствена траектория от корена, избирайки действия по правилото PUCT, което *смесва* априорната стратегия на мрежата с текущата стратегия на CFR, и добавя първото публично състояние, до което достига и което все още не е в дървото. Итерирането – „разшири дървото, подобри стратегията, разшири, подобри“ – дава търсене, което *може да бъде прекъснато във всеки момент* и което, подобно на MCTS, насочва изчисленията към релевантните линии, но за разлика от него на всяка стъпка търси игровитеоретично

коректна стратегия.⁹¹

Втората идея е **единственият параметър, който адаптира това едно търсене към двата класа игри**. Когато разширява, AlphaZero добавя само едното най-обещаващо действие, което е идеално, когато оптималната игра може да бъде детерминистична. Но оптималната **игра с непълна информация** обикновено е *стохастична* – трябва да се смесват действия, за да се избегне изтичане на информация – така че ангажирането само с едно действие е грешно. Затова GT-CFR разширява k -те действия с най-голяма априорна вероятност и задава $k = 1$ за игри с пълна информация (една добра линия е достатъчна; търсенето тогава на практика е това на AlphaZero) и $k = \infty$ – всички деца – за **игри с непълна информация** (така че стратегията може да смесва действия между тях). При $k = \infty$ търсенето има дори гаранция за качеството на стратегията *при краен брой итерации*, а не само асимптотична. Този един превключвател е техническото сърце на уеднаквяването: същият код се свежда до търсене, подобно на MCTS, върху шахматна позиция и до итерация, подобна на CFR, върху покер решение.

Третият елемент е **коректното самообучение**, което обучава CVPN. Подобно на AlphaZero, агентът се обучава да предсказва резултата от собственото си търсене: всяко запитване към мрежата, направено по време на търсене, дефинира под-игра, тази под-игра се пререшава чрез *друго* GT-CFR търсене и мрежата се обучава към изхода от търсенето (с по-високо качество): стойностите – чрез функцията на загуба на Хюбер (Huber loss), стратегията – чрез кръстосана ентропия, като резултатите от цялата игра осигуряват останалите целеви стойности. Думата *коректно* е ключовата: всяко търсене, използвано за генериране на данни, трябва да бъде съгласувано с мрежата и с другите търсения по траекторията (то не трябва да сглобява фрагменти от две различни равновесия), което се осигурява чрез изпълнение на търсенията върху спомагателна игра за безопасно пререшаване. Това е аналогът при непълна информация на самообучението с бутстрапинг на AlphaZero и именно това кара научените стойности действително да клонят към игровитеоретичните.

Двойката идва с гаранция, която завършва семейството от **граници на коректност** в тази глава. Теорема 1 гласи, че съжалението на средната стратегия на GT-CFR след T итерации се разделя на член, който акумулира грешката ϵ на мрежата за стойности върху *границата* на дървото (листата, които CVPN оценява) и обикновен член на съжалението на CFR върху *вътрешността* на дървото (възлите, където CFR работи); разделяйки на T , експлоатируемостта на средната стратегия е ограничена от

$$\text{експлоатируемост}(\bar{\pi}^T) \lesssim \underbrace{|\mathcal{F}| \epsilon}_{\text{грешка на мрежата (граница)}} + \underbrace{\frac{|\mathcal{N}| U \sqrt{A}}{\sqrt{T}}}_{\text{сходимост на CFR (вътрешност)}},$$

като $|\mathcal{F}|$ е размерът на границата, $|\mathcal{N}|$ – броят на информационните състояния във вътрешността, U най-голямата разлика в стойностите и A максималният брой действия. Формата вече е позната – член за грешката в стойността плюс член за сходимостта на CFR от вида $1/\sqrt{T}$, пряк структурен наследник на границите на DeepStack ($k_1 \epsilon + k_2/\sqrt{T}$) и на ReBeL ($\delta C_1 + \delta C_2/\sqrt{T}$) – но две

неща са отличителни. Първо, коефициентите са *размерите на растящата граница и вътрешност на дървото*, така че оценката е буквално твърдение за постепенния растеж на дървото: добавянето на възли с течение на времето не струва нищо по отношение на реда на сходимост. Второ, и решаващо, **това същото твърдение важи както за дърво на игра с пълна информация, така и за публично дърво с непълна информация** – така една и съща гаранция обхваща и двата класа. Съпътстващ резултат, Теорема 2, след това показва, че рекурсивното извикване на GT-CFR чрез непрекъснато пререшаване в рамките на цял епизод поддържа агента коректен, като експлоатируемостта нараства само *линейно* с дължината на играта (фактор приблизително $5D + 2$ за D стъпки на пререшаване) – свойството, което позволява на Student of Games да оцелее в хоризонта от двадесет и четири рунда на Scotland Yard. Заедно теоремите казват това, което AlphaZero можеше само да предполага: с подобряването на мрежата ($\epsilon \rightarrow 0$) и задълбочаването на търсенето ($T \rightarrow \infty$) играта клони към равновесие на Наш – вече както в покера, така и в шаха.

6.6.4 Уговорки, задънени улици и какво статията не описва

В съответствие с модела на главата, основният текст в *Science Advances* разказва чистата история на високо ниво, а инженерната откровеност е в допълнителните материали: теоремите са формулирани в основния текст „само неформално“, а архитектурите, хиперпараметрите, пълният псевдокод, изчислителните разходи и доказателствата са прехвърлени към Допълнителния текст.

Основното ограничение е открито признато: **Student of Games е по-слаб от AlphaZero в шах и Го при еднакви ресурси**, а разликата не е малка в Го – най-силната му конфигурация спечели само *2 от 400* игри срещу напълно обучен AlphaZero с 8000 симулации на ход, т.е. загуби 99,5%, макар че разгроми класическата програма Pachi с над 1100 точки Ело. Авторите наричат това „цената на общността на SoG“ и формулират причината точно: CFR просто е по-малко ефективен от търсенето в дърво на Монте Карло при игри с пълна информация, така че унифицираното търсене плаща за своята широчина с пикова сила. Шмид откровено заявява, че това е умишлено – целта е един алгоритъм „по-добър от хората“ във всичко, а не най-силната шахматна програма – но това означава, че унификацията е доказателство за *коректност и компетентност* в различните класове, а не за доминация в нито един от тях.

Две стени за мащабиране са признати като реалните ограничения. Първата е **експлозията в пространството на убежденията**, която Шмид определя като основното ограничение: CVPN трябва да *изброи информационните състояния за всяко публично състояние*, така че неговата цена се мащабира с броя на частните състояния, които играч може да заеме – управляемо за 1326-те възможни ръце в покера, но „непосилно скъпо в някои игри“⁹¹, където представянето „се разпада“. Това е същото ограничение на ReBeL, свързано с входа от PBS, наследено непроменено; статията предлага да се направи приближаване с генеративен модел, който *генерира извадка* от състояния на света вместо да ги изброява, но не изгражда такъв. Втората е **изискването за известен модел**: подобно на AlphaZero и ReBeL, Student of Games се нуждае от точен симулатор на правилата, което според думите на Шмид го поставя „зад линията на AlphaZero“ спрямо MuZero, който *научава* своя модел.

Няколко по-малки детайла допълват картината. Системата все още използва **рандомизирана абстракция на залозите** в покера (около двадесет хиляди действия, сведени до четири или пет) – единственото експертно познание, което не е премахнато; отстраняването му е посочено като бъдеща работа. Нейното търсене е **квадратично спрямо броя на итерациите** ($O(kT^2)$) извиквания на мрежата, тъй като дървото нараства с всяка стъпка) – неефективност, която авторите отбелязват, че може да бъде премахната чрез паралелизация, но не е била. Хиперпараметърът c – колко актуализации на съжалението да се изпълняват при всяко разширяване – е признато **недостатъчно проучен** („ $c = 1$ не винаги е най-добрият избор на практика“, и авторите се надяват „да проучат това по-задълбочено в бъдеще“). А чистият аргумент за сходимост на обучението е **идеализиран**, валиден само „асимптотично, когато $T \rightarrow \infty$ и при много голяма (експоненциална) памет“, така че практиката разчита на крайния капацитет на мрежата. Нито един от тези недостатъци не подкопава резултата, но заедно те определят Student of Games като първо, умишлено общо доказателство за концепция, а не като настроен и мащабируем продукт.

6.6.5 Изчислителни разходи и достъпност

Разходите на Student of Games са съсредоточени в **предварителното самообучение на TPU клъстери**, и статията ги отчита *спрямо AlphaZero, а не в абсолютни стойности* – умишлен избор, който позволява честно сравнение без претенции за хардуерен рекорд. Базовата версия на AlphaZero използва 3500 паралелни процеса за генериране на данни (actors), всеки на един Google TPuv4, в продължение на 800 000 стъпки на обучение, а Student of Games „е обучен с подобно количество TPU ресурси“⁹¹, като обучението по домейни варира от приблизително един милион стъпки (Го) до седемнадесет милиона (Scotland Yard). Според самия Шмид шахът и покерът могат да бъдат обучени „на няколко GPU“, докато Го „безспорно най-трудното“ е изисквало голям брой TPU – главно защото хардуерът е бил наличен, а не защото методът го е изисквал. Важно е да се отбележи, че **няма една единствена сума в долари** от рода на тази, с която Pluribus стана известен⁸⁵: тъй като сравнението е обвързано с *бюджет за мрежови извиквания*, а не с времето за изпълнение, честният извод е „сравним с AlphaZero и значителен“ – всъщност авторите включват намаляването на тези изчислителни разходи сред своите нерешени проблеми. По време на игра търсенето може да бъде прекъснато във всеки момент и бюджетът му се настройва, но растежът му $O(kT^2)$ (който се свежда до $O(T)$ за игри с пълна информация, където всеки възел се оценява само веднъж) означава, че силните конфигурации също не са евтини.

По отношение на достъпността присъдата е смесена и заслужава да бъде изразена точно. **Пълният агент Student of Games и неговите обучени мрежи не бяха публикувани** – няма декларация за достъпност на кода освен данните в статията и допълнението – така че по оста „главен артефакт“ той е заедно със затворените Libratus и Pluribus, а не с отворения ReBeL. Това, което е отворено, е заобикалящата екосистема: същата група DeepMind/Alberta поддържа **OpenSpiel**, който предоставя семейството алгоритми CFR и самите еталонни игри (Ледюк Холдем, Liar’s Dice, Scotland Yard), така че компонентите и средите са възпроизводими, въпреки че сглобената система не е. На практика това прави Student of Games *отворен метод върху отворена основа със затворен флаг-*

ман – разумна средна позиция, но такава, която оставя пълното възпроизвеждане на резултатите за шах/Го/покер/Scotland Yard извън обсега на типична група.

6.6.6 Силни страни и ограничения

Главното предимство на Student of Games е **общност, съчетана с коректност** – комбинация, която нито една предишна система в тази глава не предлагаше. Той е *доказуемо коректен както за игри с пълна, така и за игри с непълна информация* (Теорема 1–2) и е проверен експериментално и в двата класа, като подкрепя теорията с широчина, която никой конкурент не достига: в един-единствен дизайн достига експертно до професионално ниво в **шах и Го, побеждава Slumbot**, най-силния публично достъпен бот за HUNL (с около +7 mbb/g – единицата за темп на печалба от предишните раздели, която тази статия записва като mbb/hand – и, което е решаващо, локалният най-добър отговор (LBR), който открива слабости в по-слабите ботове, *не успява да го експлоатира* – SoG печели срещу него +434 mbb/g), и **побеждава PimBot**, най-силния съществуващ агент за Scotland Yard, изграден върху търсене в дърво на Монте Карло с ръчно настроени евристики, дори когато на PimBot са дадени десет милиона симулации за търсене спрямо няколкостотин на Student of Games. Той **не използва човешки данни, предварително изчислен план или абстракция на картите**, като научава всичко от самообучение; качеството му **расте** с изчислителния ресурс (повече търсене и повече обучение понижават експлоатируемостта, емпиричното лице на Теорема 1); и демонстрира, че *усъвършенства стратегията си по време на игра чрез търсене*, „за разлика от всеки чист алгоритъм за обучение с подкрепление“ – най-ясната формулировка в главата на идеята за търсенето като част от вземането на решение (search-as-inference).

Ограниченията са точната форма на неговата амбиция. Той е **по-слаб от специалистите** в техните собствени области – особено забележимо в Го – признатата цена за един алгоритъм за всичко.⁹¹ Неговите гаранции, подобно на тези на ReBeL, важат само за **игри за двама с нулева сума** (детективите в Scotland Yard са обединени в един отбор, за да се вместят), така че празнината в *безопасността при много играчи*, разкрита от Pluribus, остава незасегната дори тук. Неговото **представяне на убеждения не се мащабира** до игри с огромни пространства от частни състояния и той **изисква известен модел** на средата – двете стени, които самите автори отбелязват първи. Той запазва малка **абстракция на залозите**, неговото търсене е **квадратично** по отношение на итерациите и – уникално неудобно за твърдението за обобщеност – **флагманският код никога не беше публикуван**. Честният прочит е, че Student of Games доказва, че уеднаквяването е *възможно и коректно*, и плаща цената на това доказателство в пикова сила, мащаб и възпроизводимост.

6.6.7 Наследство и съвременна значимост

Student of Games е завършекът на аргумента в тази глава и обединява трите ѝ сюжетни линии едновременно. В рамките на петте системи областта премина от **ръчно изградена абстракция към научено невронно приближение** (DeepStack и ReBeL премахнаха абстракцията, от която зависеха Libratus и Pluribus), и от **решаване на всичко офлайн към обучение и търсене** (планът се сви от петабайтите на Libratus до мрежа за стойности и решаване в реално време). Student

of Games добавя третото и последно уеднаквяване – **пълна и непълна информация в един алгоритъм** – и с това преформулира цялата сюжетна линия. Концептуалният извод е, че рецептата на AlphaZero (самообучение, научена мрежа за стойности и стратегия и дърво, разраствано чрез насочено търсене) никога не е била специфична за пълна информация; тя просто се нуждаеше от *коректно* търсене върху *убеждения*, а не върху *състояния*. Заменете **търсене в дърво на Монте Карло** с GT-CFR и го изпълнете върху публични състояния на убежденията, и същата рецепта обхваща шах и покер с една единствена гаранция за сходимост. Това е централната теза на главата, изразена буквално: един алгоритъм, една мрежа, едно търсене, четири много различни игри.⁹¹

Показателно е как SoG се съотнася със съседните направления. Student of Games е *общата* рамка, към която ReBeL насочи – той разраства своето дърво, вместо да фиксира под-игра с ограничена дълбочина, отделя търсенето по време на игра от обучението (така че, подобно на AlphaZero, може просто да търси повече по време на игра, за да играе по-силно) и е валидиран в различни класове игри, а не само върху игри с непълна информация. Най-видимият му *непокрит* съсед е **MuZero**: докато MuZero премахва нуждата на AlphaZero от известен модел чрез научаване на динамиката, Student of Games все още предполага точен симулатор, така че естествената следваща стъпка – обединяване на GT-CFR с научен модел и генеративна извадка на убеждения – би премахнала наведнъж и двете му посочени ограничения. И той се намира на една ос от **CICERO**, играча на Diplomacy на Meta, който достига човешко ниво, като пренася търсене плюс обучение в игра със седем играчи, със смесени мотиви и на естествен език: ярко напомняне, че Student of Games уеднаквява *информационната структура*, но не и *броя играчи*. Няколко от неговите части вече са използвани сами по себе си – **GT-CFR** като прекъсваемо (anytime) търсене с нарастващо дърво, насочвано от мрежата за стратегия и приложимо за всеки клас игри; **CVPN** като обща мрежа за стойности и стратегия върху състояния на убежденията; **коректното самообучение** като начин за буутстрапинг на целеви стойности от взаимно съгласувани под-търсения; и **независимо от домейна непрекъснато пререшаване**, което вече не се опира на специалната структура на покера.

За тази дисертация по-специално Student of Games е едновременно таван и пътепоказател. Като най-общата **рамка за коректно търсене**, базирана на PBS, тя е естественият контекст за **Принос 1**: нейното убеждение – разпределение върху възможните информационни състояния на противника – е точно субстратът, който трябва да бъде *обогатен* с убеждения относно *типа* стратегия на противника, превръщайки убеждение, насочено към равновесие на Наш, в адаптивно. И неговото внимателно ограничаване на всички гаранции до **игри за двама с нулева сума** – статията изрично посочва, че извън тях гаранцията на равновесието на Наш е „по-малко смислена“ – преформулира, от най-напредналата гледна точка в литературата, точния пропуск, към който е насочен **Принос 2**: уеднаквяването на *информационната структура* на игрите не затвори *празнината в безопасността при много агенти*, която Pluribus разкри първи. Това е най-ясният възможен преход от тази глава към останалата част от дисертацията, а **синтезът**, който следва, обединява петте системи – **таблица на еволюционните промени** за това какво всяка добави и загуби, диаграмата на еволюцията и картата за повторна употреба на компоненти – за да направи нерешените

проблеми и приносите, които ги адресират, явни.

6.7 Синтез

С петте системи на разположение синтезът ги обединява – не за да определи победител, а за да открие три основни момента: формата на седемгодишната еволюция, общите механизми, които системите използват, и нерешените проблеми, с които тази глава предава щафетата на останалата част от дисертацията. Обобщаващите таблици на всяка система вече съдържат подробностите по отделните измерения, така че тук се прави обобщение, а не повторно извеждане на тези данни.

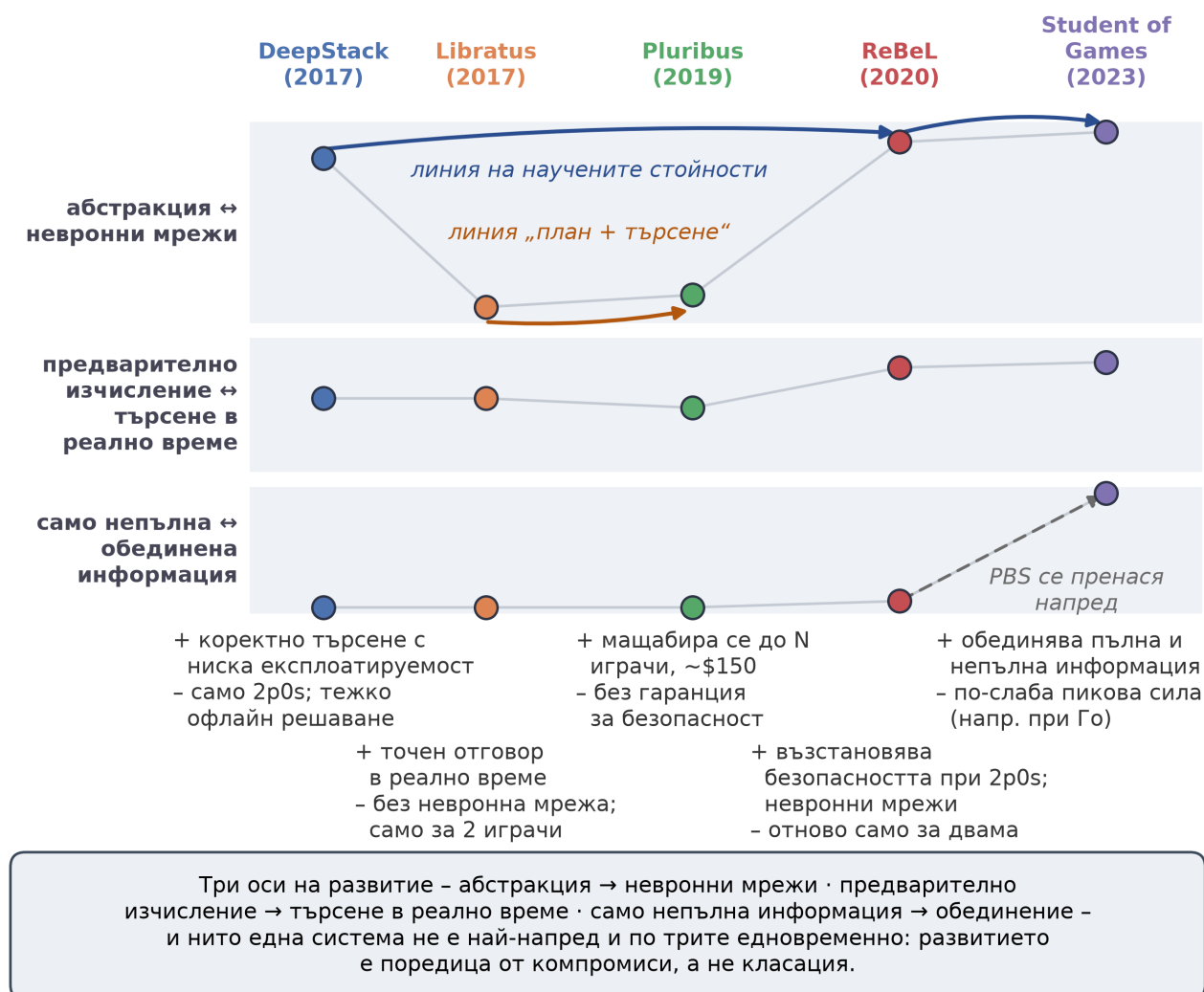
6.7.1 Развитието накратко

Прогресията е най-добре да се разглежда не като изкачване, а като разходка по трите оси от въведението, при която всяка стъпка напред по една ос беше платена с нещо друго. DeepStack и Libratus откриха този период, като атакуваха една и съща слабост – парадигмата „абстракция и план“ със загуби – от противоположни посоки: DeepStack изостави парадигмата и я замени с научени стойности и непрекъснато пререшаване, докато Libratus я запази и добави доказуемо безопасна корекция в реално време.⁸⁷⁸⁶ И двете системи останаха за двама играчи и нито една не съобщи резултат по основната метрика на другата (DeepStack не игра мачове срещу други ботове, а за Libratus не е публикуван резултат срещу LBR)⁹⁰, така че още в началото „по-добър“ вече беше многоизмерен. Pluribus след това придвижи оста *брой играчи* до шест – първият свръхчовешки резултат в еталонна игра с повече от двама играчи или отбора – но го направи, като се отказа напълно от безопасността и остана табличен – странично движение по оста на представянето и крачка назад по гаранциите. ReBeL обърна и двете тези отстъпки, връщайки се към двама играчи, за да *възстанови* гаранция за коректност, като същевременно стигна най-далеч по оста на представянето – публични състояния на убежденията със стойности и стратегия, обучени чрез самоигра, без абстракция и без план – а Student of Games завърши картината, като добави третата ос, обединявайки игра с пълна и непълна информация в един алгоритъм, за сметка на пиковата сила във всяка от тях. Обобщено в едно изречение: областта замени *ръчно изградена широчина* с *научена общност* и *предварително изчисление* с *търсене в реално време*, а нито една система не доминира по всички оси наведнъж.

Таблицата по-долу представя тази разходка като отчет за това какво е добавила всяка система и каква е била цената, което представлява полезното сравнение при еволюционен прочит; фиксирани стойности по измерения (играчи, тип игра, план, невронна компонента, търсене, абстракция, пълна информация, изчислителни разходи, ключова иновация) са в обобщаващата таблица на всеки раздел и тук не се повтарят.

Таблица 14: Петте системи по ред: какво добавя всяка и от какво се отказва в замяна.

Система (година)	Какво добавя	Какво жертва / каква е цената
DeepStack (2017)	Първо <i>коректно</i> търсене с ограничена дълбочина при скрита информация; научени контрафактични стойности вместо съхранена стратегия; изоставя парадигмата „абстракция и план“ на цялата игра	Все още запазва разредена абстракция на залозите в рамките на своето предвиждане; никога не е тестван директно срещу предишни ботове; приблизително 175 CPU-години офлайн етикетирание на стойности
Libratus (2017/18)	Първа победа както срещу ботове, така и срещу хора, чрез вложено безопасно решаване на под-игри в реално време с доказуема граница; нощен модул за самоусъвършенстване, който запълва собствените пропуски на плана	Запазва и зависи от абстракция + петабайтов план; без невронно обобщаване; изчислителни разходи в мащаба на суперкомпютър (~25 милиона ядро-часа)
Pluribus (2019)	Първа свръхчовешка игра с много играчи (шестима); известна с ниската си цена (~150 долара на един сървър)	Отказва се от <i>всички</i> гаранции за безопасност (без граници за N играчи); разчита на <i>небезопасно</i> търсене; остава изцяло табличен и абстрахиран
ReBeL (2020)	„AlphaZero за непълна информация“: публични състояния на убежденията + научени стойности/стратегия + CFR в пространството на убежденията; <i>възстановява</i> гаранцията за 2p0s; елиминира абстракцията и плана; с отворен код за Liar’s Dice	Гаранциите се връщат към двама играчи; експлозия в пространството на убежденията там, където общото знание е оскъдно; обучение върху GPU клъстер; изисква известен модел
Student of Games (2023)	Обединява пълна и непълна информация в един коректен алгоритъм (GT-CFR + една мрежа за стойности и стратегия + коректно самообучение)	По-слаб от специалистите (осезаемо при Го); гаранциите все още са 2p0s; експлозия в пространството на убежденията; изисква известен модел; основният код не е публикуван

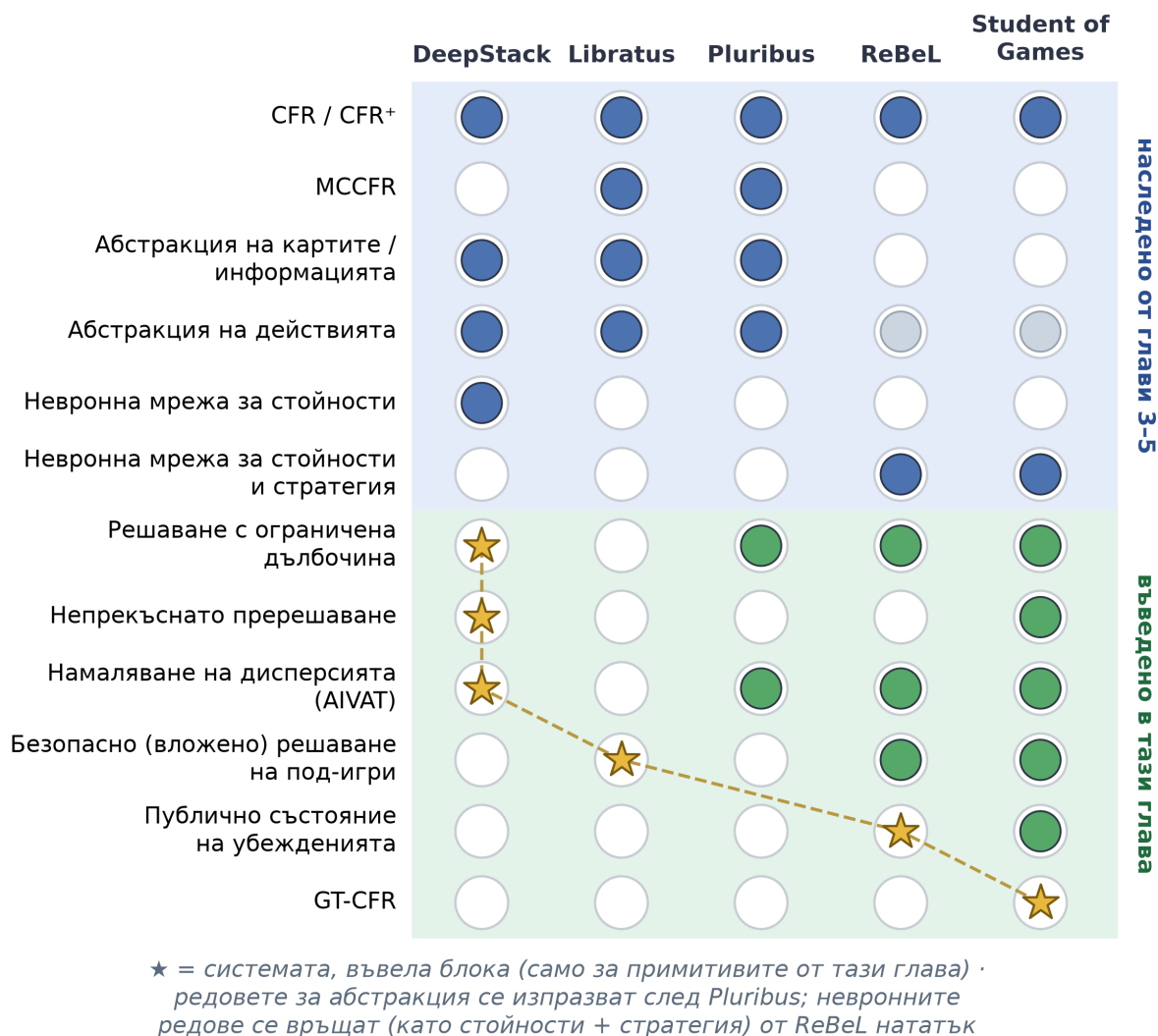


Фигура 29: Седемгодишното развитие като поредица от компромиси: петте системи по три оси (абстракция → невронни мрежи; предварително изчисление → търсене в реално време; само непълна информация → обединение), със стрелки на наследяване и бележка за спечеленото и изгубеното от всяка система.

6.7.2 Какво се пренася напред

Почти нищо в тези системи не е изцяло ново; всяка от тях представлява рекомбинация на малък набор от градивни елементи, повечето от които са въведени в глави 3–5. **CFR и CFR⁺** (глава 3) са вътрешният решавач на всяка система; **Монте Карло CFR** (глава 3) обучава плановите на Libratus и Pluribus; **абстракцията на картите и на действията** (глава 4) е в основата на Libratus и Pluribus и оцелява само като остатъчно меню за залагания в ReBeL и Student of Games; а **невронното приближение на стойността** – а след това **на стойността и стратегията** (глава 5) е именно това, което позволява на DeepStack, ReBeL и Student of Games да премахнат абстракцията, от която зависят останалите. Върху тези глават наслагва свои собствени примитиви, всеки въве-

ден от една система и използван повторно от нейните наследници: **решаване с ограничена дълбочина, непрекъснато пререшаване, безопасно вложено решаване на под-игри, публично състояние на убежденията и Growing-Tree CFR**. Ако се проследят *първите появи*, еволюцията в главата се преразказва чрез компонентите: блоковете се натрупват, тези от ерата на абстракцията избледняват, а научените поемат контрол.



Фигура 30: Карта на повторно използваните компоненти: градивните елементи от глави 3–5 и примитивите, въведени в тази глава (редове), срещу петте системи в хронологичен ред (колони); звездите отбелязват системата, въведа съответния примитив, а пунктирната „стълба“ свързва първите им появи.

6.7.3 Защо това е важно за нашето изследване

Тази глава представлява съвременното равнище, върху което дисертацията се изгражда и от което се отклонява, а нейният най-важен принос към тази работа е рамкиращ. Всяка система, разгледана тук, е „сляпа“ за **противника по замисъл**: всяка изчислява стратегия, устойчива в най-

лошия случай (приближение на равновесие), и не я адаптира към конкретния противник. Статията за Pluribus го казва изрично: системата „играе фиксирана стратегия, която не се адаптира към наблюдаваните тенденции на противниците“, не знае кои са противниците ѝ, а отклонението към експлоатираща стратегия се отхвърля, защото прави самата система уязвима за експлоатация.⁸⁵ Прочетена като обзор, следователно, главата е каталог на пет системи от световна класа, които споделят точно един пропуск: нито една не се адаптира към конкретния, погрешим противник пред себе си. Именно този общ пропуск е нишата, в която работи цялата дисертация, а останалата част от този подраздел разглежда какво предлага главата за всеки от трите планирани приноса.

За Принос 1 (Рамка за поведенческа адаптация). Най-прекият принос на главата е **публично-то състояние на убежденията (PBS)**, въведено от ReBeL и наследено от Student of Games – разпределение върху скритите състояния, които противникът може да заема при наличие на общо знание, което Принос 1 разширява, за да носи убеждения относно *типа стратегия* на противника, а не само неговите карти. Два по-малко очевидни момента изострят това. Първо, PBS е въведено, за да направи *търсенето доказуемо коректно*, а коректността е ортогонална на моделирането на противника – така че изграждането на адаптация върху PBS наследява целия механизъм за пререшаване и неговите гаранции за сходимост *непроменени*, позволявайки на адаптивното убеждение да бъде актуализирано вътре в доказуемо коректно търсене, вместо да бъде добавено отвън; адаптацията получава възможността да използва повторно не само структура от данни, но и доказателствата, прикрепени към нея. Второ, всяка система в главата споделя **двустепенна архитектура** – евтина глобална стратегия (план или научена мрежа за стойности), усъвършенствана чрез локално решаване в реално време – и собственият дизайн на дисертацията, „започнете от безопасна стратегия и се подобрявайте срещу слаби противници“, е същата двустепенна форма, при която целта на второто ниво е променена от *точност* към *ограничена експлоатация*. Следователно главата предоставя както субстрата за адаптация, така и системния скелет, в който тя се вписва.

За Принос 2 (Многоагентна безопасна експлоатация). Ключовият елемент е Pluribus: свръхчовешки покер за шестима играчи, постигнат *без каквато и да е гаранция за безопасност*, което е точно пропускът, който една теория за многоагентна безопасна експлоатация трябва да запълни – точка, която главата заявява директно. По-фин прочит превръща собствената уговорка на системите в оправдание за приноса. Статията за Pluribus посочва, че при повече от двама играчи равновесията на Наш не са единствени, намирането им е изчислително трудно, а независимо избрани равновесни стратегии може да не образуват равновесие – т.е. равновесната игра не гарантира, че играчът няма да загуби;⁸⁵ Student of Games ограничава гаранциите си до игри за двама с нулева сума, защото извън тях гаранцията на равновесието е „по-малко смислена“.⁹¹ Ако при много играчи равновесието изначално не осигурява безопасност, тогава отказът да се експлоатира видимо слаб противник означава отказ от гаранция, която никога не е била притежавана – което преформулира умишлената експлоатация от безразсъдно отклонение от безопасността в *рационална цел*, след като безопасността е недостижима само чрез равновесие. Това е най-ясната мотивация, която литературата предлага за многоагентната експлоатация, която Принос 2 преследва.

За Принос 3 (Методология за оценяване). Главата стандартизира **експлоатируемостта** – най-лошата загуба спрямо **най-добре реагиращ противник** – като универсален измерител, прилаган еднакво към всичките пет системи, и тази однородност сама по себе си представлява шаблон за **рамка за оценяване между системи**. Тя също така предоставя на дисертацията два инструмента, преназначени за нова употреба. **Локалният най-добър отговор (LBR)**, използван в цялата глава за *сертифициране* на **неексплоатируемост**, се превръща в *тест за безопасност*, когато бъде насочен към **адаптивен агент** – не става ли агентът **контраексплоатируем**, когато преследва слаб противник? – което е точно състезателната оценка от типа на „обучаващата атака“, за която настоява глава 8. А **AIVAT, методът за намаляване на дисперсията чрез научени стойности** зад докладваните резултати на DeepStack и Pluribus, е инструментът, който прави малкото, но *растящо* предимство на адаптацията статистически видимо въпреки огромната дисперсия в игрите с карти, където **суровите печалби** биха подвели.

По-нататъшни опорни точки. Следващите бележки са кратки екстраполации, а не твърдения на самите системи; всяка е идея в един ред, която следващите глави могат да развият.

- **Многократно използвана форма на доказателство (Принос 2).** Гаранциите в главата имат сходна форма – член за грешката на приближението на стойността и (без Libratus) член за сходимост от вида $1/\sqrt{T}$ (DeepStack: $k_1\epsilon + k_2/\sqrt{T}$; Libratus: 2Δ ; ReBeL: $\delta C_1 + \delta C_2/\sqrt{T}$; SoG: $|\mathcal{F}|\epsilon + |\mathcal{N}|U\sqrt{A}/\sqrt{T}$) – което дава на Принос 2 готов шаблон за ограничаване на експлоатируемостта, *породена* от умишлено отклонение.
- **Експлоатация с ограничена дълбочина (Принос 1/Принос 2).** Многозначните листа на Pluribus (k избираеми стратегии за продължение)⁸⁴ структурно представляват меню от поведения на противника на хоризонта; ако това меню се тълкува като *типове* противници, се получава ефективно изчислима експлоатация отвъд дълбочината на търсенето – точно проблемът, с който се занимават Miles и др. (2025), цитирани в глава 8.
- **Карта за изграждане спрямо цитиране (методология).** Подразделите „Изчислителни разходи и достъпност“ разделят областта на възпроизводимото в мащаба на една докторантура (CFR на Pluribus върху един сървър, еднонишковият CPU решаващ на ReBeL, отворената среда OpenSpiel, малките игри) и на онова, което може само да се цитира (суперкомпютърът на Libratus, TPU клъстерът на SoG) – неявна обосновка да се надгражда евтината и отворена линия, вместо да се възпроизвежда някоя от водещите системи.
- **Пренасочен механизъм (рамкиране).** Самоусъвършенстването през нощта на Libratus е корекция на стратегията в мача, насочена към *собствените* му слабости; дисертацията прилага същия цикъл „наблюдение → корекция“, насочен към *противника*, така че приносът е пренасочване на доказан свръхчовешки механизъм, а не неговото изобретяване.
- **Връзка със съвременния дебат (рамкиране).** И петте системи могат да се разглеждат като ранни примери за „изчисления по време на изпълнение“ (test-time compute) – изразходване на търсене в момента на вземане на решение, вместо всичко да бъде вградено в предварително изчислена стратегия – така че представянето на адаптацията в реално време като изчисления за *адаптация* по време на изпълнение свързва тази работа със съвременния разказ

за моделите на разсъждение.

- **Съгласуваност като безопасност (Принос 2).** „Коректното самообучение“ на Student of Games изисква всяко локално търсене да е съгласувано с мрежата за стойности и с предишните търсения по същата траектория.⁹¹ Това напомня предлаганото в Принос 2 закотвяне на експлоатиращата стратегия към безопасния план чрез KL-регуларизация. Такова закотвяне е собствено предложение на дисертацията: в riKL котвата е стратегия, имитираща човешка игра, и регуларизацията не дава гаранция за най-лошия случай.
- **Абстракция, прехвърлена към типowo пространство (Принос 3).** Урокът от глава 4, че абстракцията със загуби създава експлоатируеми шевове, се пренася от пространството на състоянията към пространството на *типовете* на противника: твърде грубият типов модел е експлоатируем по същия начин, както твърде грубата абстракция на картите, което предсказва вид провал, който Принос 3 трябва да може да улови.

6.7.4 Нерешени проблеми и връзка с глави 7–12

Четири нерешени проблема произтичат пряко от синтеза и всеки от тях е отправна точка за някоя от следващите глави. Първият е самата „**слепота**“ **за противника**: всички най-съвременни системи изчисляват фиксирана **стратегия, оптимална в най-лошия случай**, и я играят безусловно, оставяйки празнината между статично равновесие и динамичен, **съобразен с противника начин на игра** широко отворена – предмет на глава 7 (извеждане на поведението на противника от **следи от действия**) и частта от **Принос 1**, посветена на извеждането. Вторият е **безопасността при много играчи**: Pluribus показва, че методите, съчетаващи самообучение и търсене, *печелят* в игри с N играчи и **непълна информация**, макар да нямат известни теоретични гаранции извън игрите за двама с нулева сума⁸⁵, а Student of Games потвърди, че дори най-общата **рамка за коректно търсене** все още ограничава своите гаранции до двама играчи – така че ефективно изчислима теория на *безопасната експлоатация отвъд игрите за двама с нулева сума* (глава 8, **Принос 2**) остава нерешен проблем. Третият е **адаптация отвъд границата на дълбочината**: системите за решаване в реално време в главата гледат само малко напред и коректното пренасяне на **модела на противника** отвъд този хоризонт – вместо той да се изоставя в листата – е нерешен проблем; глава 8 само го очертава чрез работата на Miles и др. (2025). Четвъртият е **бюджетът за изчисления в реално време**: разходите на системите се различават с няколко порядъка и **адаптивен агент**, който трябва да пререшава и да преоценява противник онлайн, наследява най-строгата версия на това ограничение.

Тези проблеми са в основата на глави 7 и 8 – ядрото на дисертацията – и се развиват в следващите глави: многоагентно обучение, популации и образуване на коалиции (глави 9–11) и модели на последователности и агенти с големи езикови модели (глава 12); междудомейнната рамка за оценяване остава за бъдеща работа. Петте системи в тази глава колективно представляват платформата, върху която стъпва настоящата дисертация, както и контрапункта, спрямо който тя се определя: тя наследява техните представяния на състояния на убежденията, тяхното търсене в реално време и тяхната дисциплина по отношение на експлоатируемостта и се стреми да добави ед-

на способност, която всички те по замисъл пропускат – способността да забележи, че противникът не играе оптимално и да предприеме действия по въпроса, без самата тя да стане експлоатируема в замяна.

7 Глава 7 – Моделиране на противника в игри с непълна информация

Това е цялостна глава за моделиране на противника: формулировката на проблема, математическият му апарат, семейството от методи и набор от контролирани експерименти, проведени върху две малки покер игри. Главата е написана така, че да бъде разбираема и без предварително познаване на програмния код на проекта. Всички представени експериментални резултати са получени от възпроизводими изпълнения на двете тестови среди (Кун покер и Ледюк Холдем) и, когато е възможно, са поставени в *точни* аналитични граници, а не в симулирани.

Къде се вписва това в дисертацията. Игровитеоретичният агент от предходните глави изчислява *равновесие на Наш* – стратегия, която не може да бъде победена в дългосрочен план. Настоящата глава изгражда противоположната способност: **сензор**, който наблюдава как *конкретен* противник действително играе и превръща тези наблюдения в оценка на неговата стратегия, така че агентът да може да се отклони от равновесието с цел да накаже грешките на този противник. Този сензор представлява първата половина от рамката за поведенческа адаптация на дисертацията (Принос №1). Втората половина – как да се експлоатира *безопасно*, без да се създаде уязвимост в собствената игра – е формализирана в Глава 8.

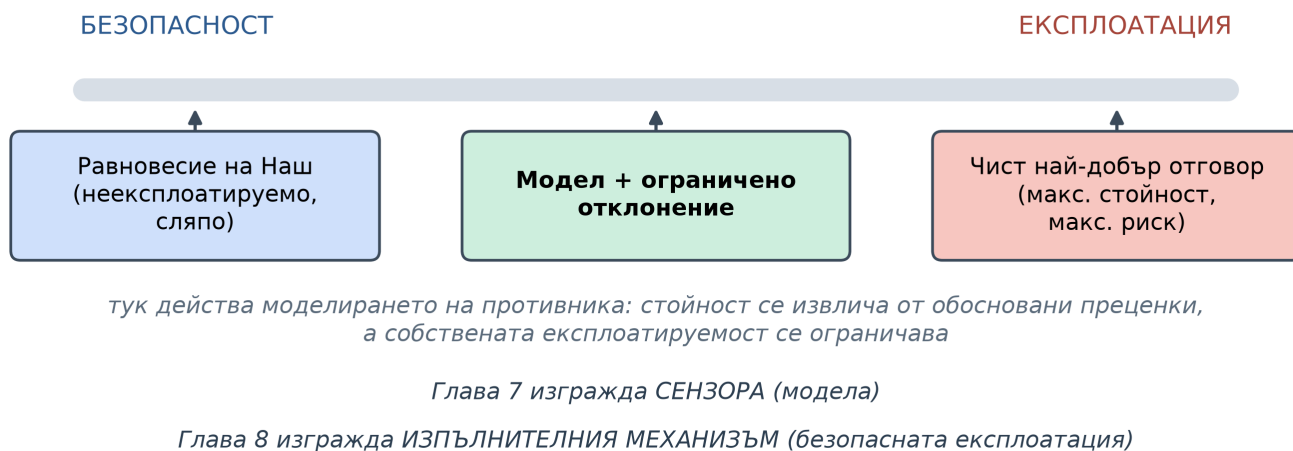
7.1 Защо е необходимо моделиране на противника

Равновесна стратегия на Наш е изградена така, че в дългосрочен план никога да не води до загуба, *независимо срещу кого се играе*. В игра за двама с нулева сума тази гаранция е реална и ценна: една равновесна стратегия има фиксирана *стойност* и нито един противник – колкото и да е умен – не може да я свали под нея. Но същото свойство представлява и ограничение. Една равновесна стратегия играе **идентично** както срещу световен шампион, така и срещу някой, който се отказва всеки път при направен залог. Тя никога не се адаптира, защото е създадена именно така, че да не се нуждае от адаптация.

Този отказ да се адаптира има цена и тази цена е измерима. **Моделирането на противника** е актът на наблюдение на действията на конкретен противник, формиране на убеждение относно неговата стратегия и **отклонение от равновесието на Наш, за да се използва забелязаната закономерност** – повече блъфове срещу някой, който се отказва твърде често, и залози за стойност с по-слаби ръце срещу някой, който плаща твърде често.

Стойността се вижда най-ясно в примера с „камък-ножица-хартия“. Вашият противник тайно хвърля камък 70% от времето. Безопасната стратегия е да избирате трите хода с равни вероятности и така завинаги да излизате на нула. Но ако *забележите* пристрастието, хвърляте повече хартия и започвате да печелите. Този пример съдържа цялата глава в умален вид по две причини: пристрастието трябва да се **изведе** от шумен поток от хвърляния (няколко хвърляния не стигат), а ако **заложите прекалено много** на хартията, ставате предвидими и противникът ви смазва

с ножица. Извеждането на пристрастието е моделиране на противника; знанието докъде да се наклоните е компромисът между експлоатация и безопасност, който преминава през всичко по-долу.



Фигура 31: Скалата безопасност–експлоатация: чистата равновесна игра е неексплоатируема, но „сляпа“; чистият най-добър отговор извлича най-голяма стойност, но носи най-голям риск. Моделирането на противника действа между двете крайности. Глава 7 изгражда сензора (модела), а Глава 8 изгражда изпълнителния механизъм (безопасната експлоатация).

7.1.1 Колко е заложено – измерено в Кун покер

Кун покер е най-малката нетривиална покер игра: тесте от три карти (Вале, Дама, Поп), по една карта на всеки, един рунд за залози. Тя е достатъчно малка, за да бъде решена *точно*, което ни позволява да изчислим стойността на моделирането, вместо да я предполагаме. За всеки от няколко постоянни противникови стила ние изчислихме три точни величини:

- **Очаквана стойност при равновесие на Наш** – това, което равновесната стратегия носи срещу този противник (на раздаване);
- **Стойност на най-добрия отговор** – това, което стратегия, която *познава противника перфектно*, носи: точният таван за експлоатация;
- **разликата** между тях – сумата, която равновесната игра оставя неизползвана.

Таблица 15: Потенциал за експлоатация в Кун: очаквана стойност при равновесие на Наш, при най-добър отговор и разликата между тях, по тип опонент.

Противник (Кун)	Очаквана стойност		Разлика
	при равновесие на Наш	Стойност на най-добрия отговор	
Пасивен играч (никога не се отказва)	+0,119	+0,333	0,215
Камък (ангажира се само с Поп)	-0,060	+0,167	0,227
Маниак (винаги залага/плаща)	+0,121	+0,333	0,213
„Наш“ (равновесна игра)	-0,055	-0,052	0,004

Срещу трите експлоатируеми стила разликата е огромна – **около 0,21-0,23 на раздаване** (от 0,11 до 0,28, ако се вземат предвид и двете позиции на масата), в игра, чиято цялостна равновесна стойност е около една двадесета от жетон. Срещу противник, който играе равновесие на Наш, разликата е по същество **нула**, точно както изисква теорията: не можете да експлоатирате равновесие, защото най-добрият отговор не може да надмине стойността на играта. (Малките отрицателни стойности при равновесие на Наш са известният недостатък на първия играч в Кун покер от $-1/18 \approx -0,056$ - това е характеристика на самата игра, а не грешка; равновесието тук е приближение, получено с 20 000 итерации на CFR, затова срещу отделен противник стойността може да падне с няколко хилядни под $-1/18$.)

Два урока вече се открояват. Първо, **моделирането си струва**: стойността е значителна и реална. Второ, **експлоатацията зависи от вида на слабостта** – точният най-добър отговор на „камък“ *блъфира повече* (той залага най-слабата ръка в отворен пот, защото „камъкът“ се отказва от всичко освен Поп). Останалата част от тази глава е посветена на това как да реализираме тази стойност, без да я губим.¹⁰¹

7.2 Байесовото ядро – убеждение, наблюдения и реакция

Под всеки метод в тази глава стои един цикъл, който е добре да бъде обяснен с прости думи преди въвеждането на каквито и да било означения. Представете си, че сте детектив. Започвате с предположение за начина, по който играе противникът (**априорно разпределение**). Всяко негово действие е следа: то прави някои обяснения по-вероятни, а други – по-малко вероятни. Вие

¹⁰¹Southey, F. et al. (2005). “Bayes’ Bluff: Opponent Modelling in Poker.” *UAI*.

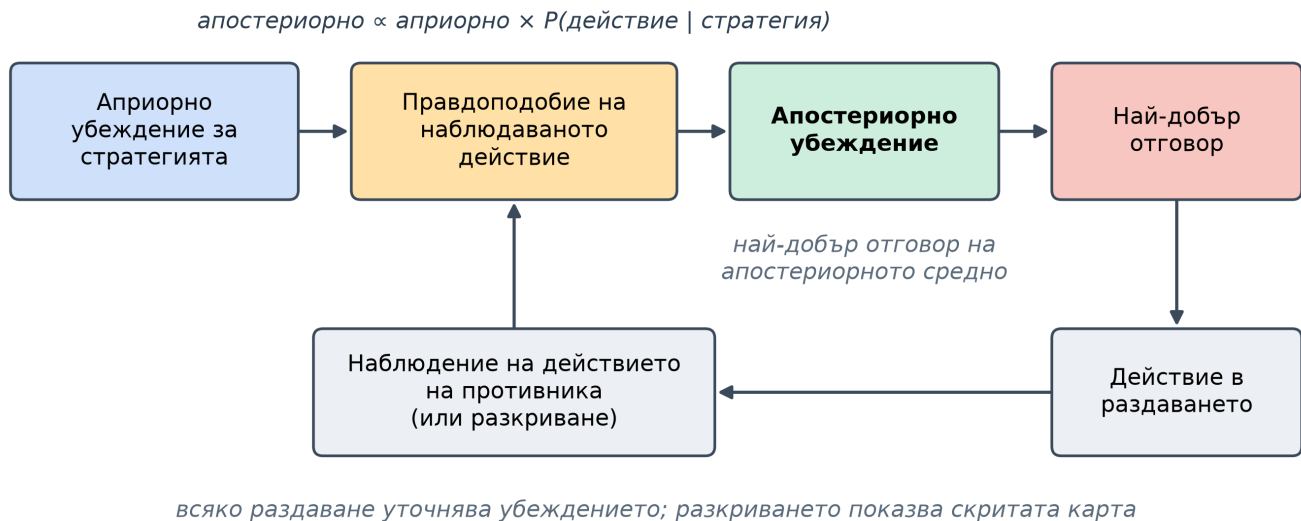
включвате тази следа в подозренията си (**апостериорно разпределение**) и повтаряте процеса. Моделирането на противника *представява* именно този цикъл:

убеждение за стратегията → наблюдавано действие → обновяване на убеждението → най-добър отговор → повторение.

Формално, обновяването се извършва по правилото на Бейс. Ако σ обозначава кандидат-стратегия на противника и a е действието, което току-що беше наблюдавано,

$$P(\sigma | a) \propto P(\sigma) \cdot P(a | \sigma),$$

с думи: **апостериорното разпределение** $\propto$ **априорното разпределение** $\times$ **колко добре дадена стратегия обяснява току-що извършеното действие**. Действие, което кандидатът смята за невъзможно, намалява вероятността му към нула; действие, което предвижда добре, я увеличава. След множество раздавания убеждението се съсредоточава върху най-подходящото обяснение.



Фигура 32: Цикълът на моделиране на противника: априорното убеждение се умножава по правдоподобие на всяко наблюдавано действие и дава апостериорното разпределение, спрямо което се изчислява най-добрият отговор; резултатът от раздаването захранва следващото наблюдение.

7.2.1 „Зоопарк“ от типове

Най-простият начин да направим това конкретно е да разсъждаваме върху малък набор от предварително дефинирани противникови **типове**, вместо върху цялото безкрайно множество стратегии. В рамките на главата използваме повтарящ се набор от типове, всеки от които изпълнява двойна роля – като противник, срещу когото да играем, и като хипотеза, която детекторът обмисля:

Таблица 16: „Зоопаркът“ от типове опоненти: поведението на всеки постоянен стил.

Тип	Поведение
Пасивен играч	Никога не залага и никога не се отказва – пропуска хода без залог (чек), когато може, и плаща всеки залог.
Камък (стегнат-пасивен)	Вкарва жетони само с най-добрата ръка; с всичко останало се отказва. Най-експлоатируемият тип.
Маниак (разпуснат-агресивен)	Винаги залага или плаща, независимо от ръката си.
„Наш“	Балансирана равновесна игра; използва смесени стратегии; неексплоатируем.

7.2.2 Актуализацията, изчислена на ръка

Механиката е най-лесна за разбиране, когато сами завъртите механизма веднъж. Да предположим, че трима кандидати залагат определена ръка с вероятност 0,8, 0,5 и 0,1 съответно, като започват от равномерно априорно разпределение $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$. Наблюдават **залог**:

$$\text{апостериорно} \propto \left(\frac{1}{3} \cdot 0,8, \frac{1}{3} \cdot 0,5, \frac{1}{3} \cdot 0,1\right) = (0,267, 0,167, 0,033) \xrightarrow{\text{нормализиране}} (0,571, 0,357, 0,071).$$

Наблюдаваме **втори** залог и отново умножаваме по $(0,8, 0,5, 0,1)$:

$$(0,711, 0,278, 0,011).$$

Убеждението се натрупва върху типа играчи, които правят чести залози, и би го направило *още по-бързо*, колкото повече кандидатите се разминават в мненията си. Тази интуиция „колко много се разминават“ се появява отново в раздел 7.6 като обяснение защо някои опоненти се идентифицират след четири раздавания, а други – след двеста.

7.2.3 Защо Дирихле и защо е нужна само средната стойност

Два факта от литературата оформят всеки практически модел. Първо, когато оценявате разпределение на действия чрез броене, естественото априорно разпределение е **разпределението на Дирихле**, защото то е *спрегнатото* априорно разпределение за мултиномиалното разпределение: обновяването на разпределение на Дирихле с наблюдаваните броеве дава отново разпределение

на Дирихле, в което броевете просто се **добавят** към псевдоброевете на априорното разпределение. Няма нужда от интеграл или симулация – апостериорната средна стойност на действие i е

$$\mathbb{E}[p_i] = \frac{\alpha_i + n_i}{\sum_j (\alpha_j + n_j)},$$

където α_i са априорните псевдоброеве (които едновременно изпълняват ролята на *изглаждане*, като предотвратяват стойността нула за което и да е действие), а n_i са наблюдаваните броеве. Тази затворена форма съответства на „непрекъснатия“ модел, разгледан в раздел 7.4.

Второ – и това е най-често използваният резултат в областта – за да се избере оптималният отговор, не е нужно цялото апостериорно разпределение върху стратегиите; достатъчна е само неговата **средна стойност**. Очакваната печалба срещу *разпределение* от стратегии на опонента е равна на печалбата срещу единствената осреднена стратегия. Това свежда изчислително непосилен интеграл върху пространството на стратегиите до задачата „най-добър отговор на един осреднен опонент“ и именно това прави байесовата експлоатация приложима на практика. (Раздел 7.5 се връща към това: отговорът на средната стойност е оптимален по отношение на печалбата, но, както ще се окаже, оценката може да не *клони* към истинската стратегия на опонента – тънкост, която работата от 2025 г. разрешава.)¹⁰²

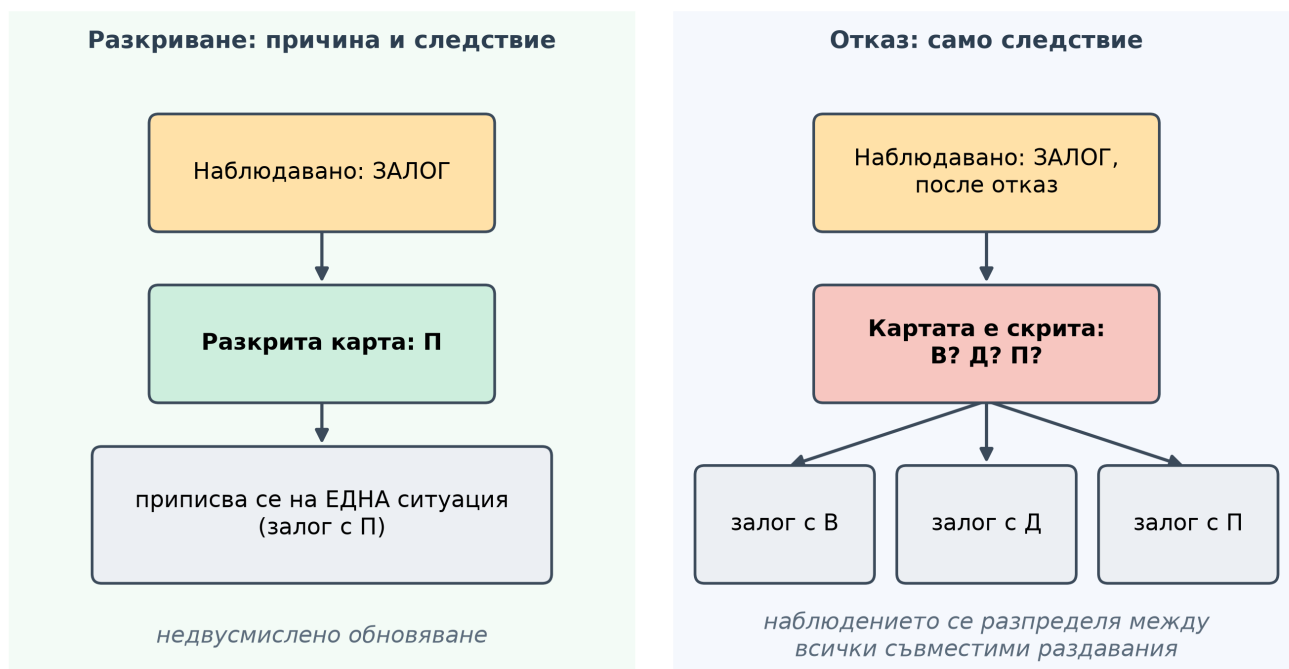
7.3 Виждане през ключалка – проблемът с частичната наблюдаемост

Има причина моделирането на противника в покера да е по-трудно от броенето на честоти в „камък-ножица-хартия“ и това заслужава отделен раздел, тъй като определя характеристиките на всеки следващ модел.

В покера наблюдават **действия** – залог, плащане, отказ – но **не** и частната ръка, която ги е породила. Един и същ „залог“ може да произтича както от силна ръка, така и от блъф. Единственото изключение е **разкриването**, когато картите се показват в края на раздаването, ако то не е приключило с отказ. По този начин всяко раздаване попада в една от двете категории според това какво разкрива:

- **Разкриване – причина и следствие.** Виждате както действията на противника, *така и* частната ръка зад тях. Можете да свържете поведението с точната ситуация, която го е породила. Това представлява златния стандарт за наблюдение.
- **Отказ – само следствие.** Раздаването завършва без разкриване на картите. Видяхте *какво* са направили, но не и *какво са държали*, така че не можете да сте сигурни коя ситуация е причината за това.

¹⁰²Ganzfried, S. & Sun, Q. (2018). “Bayesian Opponent Exploitation in Imperfect-Information Games.” *IEEE Conference on Computational Intelligence and Games (CIG)*, DOI 10.1109/CIG.2018.8490452; препринт arXiv:1603.03491 (2016). (Теорема 2.1: печалбата срещу средната стойност на апостериорното разпределение е равна на печалбата срещу цялото разпределение.)



Фигура 33: Частична наблюдаемост: при разкриване картата на противника става видима и действието се свързва с една-единствена ситуация; при отказ картата остава скрита и моделът трябва да разпредели наблюдението между всички карти, които противникът би могъл да държи.

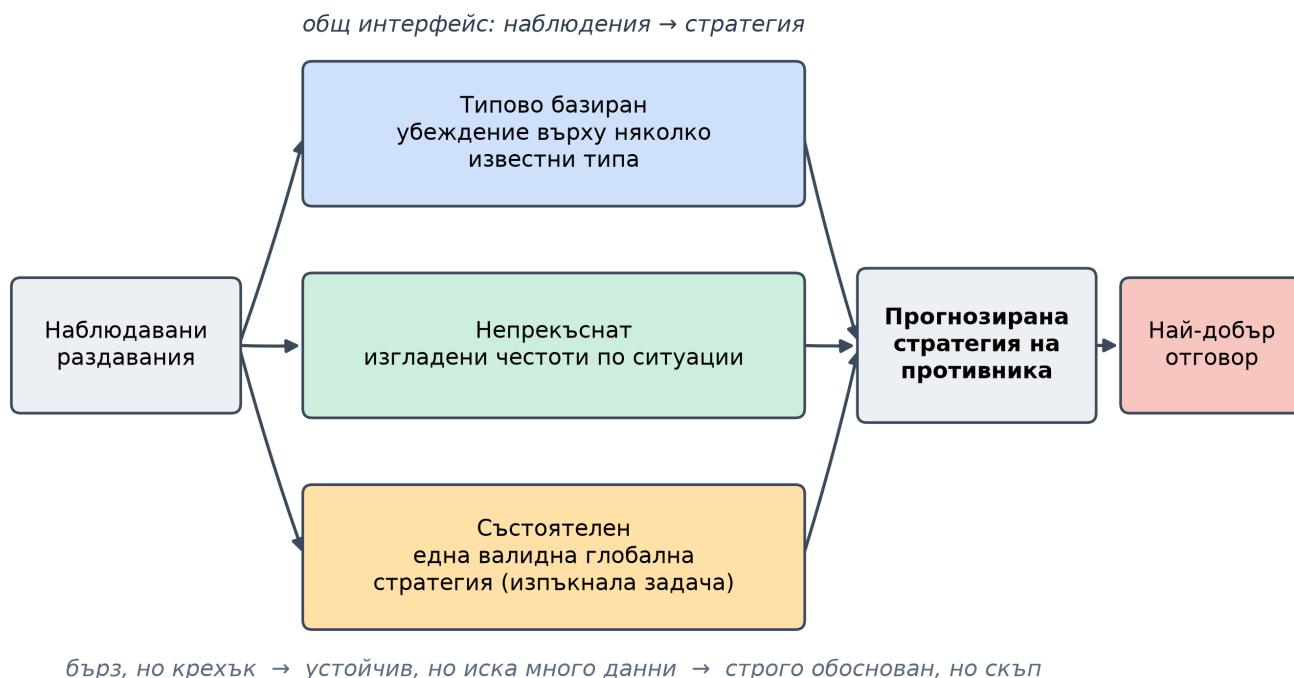
Следствието е истинско теоретично ограничение, а не неудобство при имплементацията: **без да наблюдавате частната информация на противника, можете да научите колко често той избира всяко действие, но не и с кои карти го прави.** Много различни стратегии пораждат едни и същи честоти на действията и повече раздавания не могат да ги различат. Разкриването е именно информацията, която премахва тази двусмисленост.

Как се справя моделът с отказ? Като разсъждава върху **всички карти, които противникът би могъл да държи.** Конкретно, той изброява раздаванията, съгласувани с това, което действително е било видяно, преиграва решенията на противника при всяка хипотетична тайна карта и разпределя наблюдението между тях – като претегля всяка хипотеза според нейната правдоподобност. Разкриването свежда този набор до една-единствена възможност; отказът го оставя разпределен. Този ход „маргинализиране върху скритата карта“ е техническото ядро на всеки модел в следващите два раздела и – неслучайно – формалният обект, около който е изградена теорията на състоятелността от раздел 7.5.

7.4 Три модела с единен интерфейс

Няма един-единствен модел на противника. Има семейство, което прави компромис между скорост на сходимост, устойчивост към изненади, интерпретируемост и изчислителни разходи. Този

проект имплементира и сравнява три точки от този спектър, умишлено изградени зад **общ интерфейс** – всеки приема поток от наблюдавани раздавания и генерира прогнозирана стратегия на противника – така че *същият* последващ най-добър отговор може да бъде приложен към всичките три и сравнението да е при равни условия.



Фигура 34: Три модела с общ интерфейс: дискретно убеждение върху типове, свободно преброяване по ситуации и глобално съгласувана оценка в последователна форма; и трите връщат прогноза за стратегията на противника, която постъпва в една и съща стъпка за най-добър отговор.

- **Типово базиран.** Поддържа убеждение (разпределение на вероятностите) върху фиксирано меню от типове противници, актуализира го чрез правилото на Бейс (раздел 7.2) и докладва апостериорно претеглената средна стратегия. Той се сближава *изключително* бързо **когато противникът е в менюто** и изходът му е интерпретируем („80% камък, 20% манияк“). Неговата слабост също е ясна: ако реалният противник не прилича на нито един от типовете, моделът няма честен начин да го каже (раздел 7.6).
- **Непрекъснат.** Премахва менюто. Оценява вероятностите за действията на противника *директно във всяка ситуация* чрез броене на това, което те са направили там, изгладено с априорно разпределение на Дирихле (раздел 7.2). Той може да представи **всяка** стратегия, включително смеси и нови стилове, така че е устойчивият избор срещу непознати противници. Цената е данни: той научава всяка ситуация независимо, без споделяне на структура, така че се нуждае от много повече наблюдения – и, както показва раздел 7.7, тази нужда от данни се оказва сериозен проблем в по-голямата игра.
- **Състоятелен.** Оценява една-единствена *глобално съгласувана* стратегия – такава, която със сигурност е валидна стратегия в цялото дърво на играта – вместо всяка ситуация поотделно.

Това е най-строго обоснованият и най-новият модел; той е достатъчно важен и централен за дисертацията, за да получи собствен раздел (раздел 7.5).

Следващата таблица обобщава сравнението и служи за ориентир в останалата част от главата:

Таблица 17: Трите модела на противника, сравнени по представяне, сходимост, устойчивост, интерпретируемост и цена.

Модел	Представява	Сближаване	Устойчив на непознати противници?	Интерпретируемост	Изчислителна цена
Типово базиран	няколко известни типа	най-бързо	не	висока (именувани типове)	евтина
Непрекъснат	всяка стратегия за конкретна ситуация	по-бавно (жаден за данни)	да	средна (за конкретна ситуация)	евтина
Състоятелен	една валидна глобална стратегия	доказуемо към истината	да	средна	скъпа (оптимизация)

Очевидният въпрос – *кой от тях трябва да използва дисертацията?* – няма единствен отговор и това само по себе си е откритие. Естествената цел е хибриден подход: да се разчита на структурни априорни знания (типове), когато данните са оскъдни, и постепенно да се приближава към състоятелната оценка с натрупването на наблюдения. Тази глава изгражда и измерва крайните точки, така че хибридният модел да има стабилна основа, върху която да стъпи.¹⁰³

7.5 Проблемът на състоятелността и решението в последователна форма

Този раздел е най-напредналата теоретична част на главата и е частта, която дисертацията най-пряко доразвива. Той поставя въпрос, който по-ранните модели подценяват: *ако даден модел добре описва наблюдаваното поведение, означава ли това непременно, че сме възстановили истинската стратегия на опонента?* Изненадващият отговор е **не** – дори при наличие на безкрайни данни – и съществува принципно решение.¹⁰⁴

¹⁰³Bard, N., Johanson, M., Burch, N. & Bowling, M. (2013). "Online Implicit Agent Modelling." AAMAS, 255–262 – ос „явни срещу неявни“, която рамкира тази таксономия.

¹⁰⁴Ganzfried, S. (2025). "Consistent Opponent Modeling in Imperfect-Information Games." *arXiv:2508.17671*.

7.5.1 Недостатъкът: напасването към средната стойност не е равносилно на откриването на истината

Класическата байесова рецепта отговаря на **средната стойност** на апостериорното разпределение на стратегията (раздел 7.2). Един метод за моделиране на противника ще наречем **състоятелен**, ако неговата оценка се сближава със същинската стратегия на опонента σ^* при натрупване на наблюдения срещу постоянен противник. Бихме очаквали класическата рецепта да бъде състоятелна, но тя не е.

Най-ясният контрапример е камък-ножица-хартия. Да предположим, че моделиращият разсъждава върху множество от *извадени* кандидатски стратегии и винаги реагира на някаква тяхна средна стойност. Ако истинската стратегия $\sigma^* = (0,8, 0,1, 0,1)$ лежи **извън** областта, до която тези извадки могат да се комбинират, моделът – тъй като винаги е претеглена средна стойност на извадките – просто **не може да я достигне**, независимо колко много данни постъпват. Това е интуитивно. По-дълбокият резултат е, че методът може да се провали **дори когато истината лежи вътре** в достижимата област: ако истинската стратегия се намира в центъра на три извадки, чиято средна стойност съвпада с нея, апостериорното тегло на единствената най-добре пасваща извадка нараства неограничено спрямо останалите, така че асимптотично убеждението **колабира върху една извадка**, вместо да се установи на истинската смес.¹⁰⁴ Колкото по-плътнo моделът напасва данните, толкова повече се сближава с *грешната* стратегия.



Фигура 35: Защо напасването към средната стойност може да пропусне истината: моделът може да породи само точки в оцветената обвивка на своите извадки. Истинска стратегия извън обвивката е недостижима, а дори такава вътре в нея бива изоставена, защото апостериорното разпределение се свива към един-единствен връх.

Това е важно, защото обезсилва интуитивната представа за „сигурност чрез натрупване на повече данни“. Състоятелността не е автоматична; тя трябва да бъде изградена.

7.5.2 Решението: задача на изпъкналата оптимизация в последователна форма

Подходът преформулира оценката така, че „валидна стратегия“ и „съответства на наблюденията“ да се превърнат в ограничения на една-единствена оптимизационна задача с добри свойства. Ключовата промяна на променливите е **последователната форма**: вместо вероятности за действие във всяка ситуация, стратегията на опонента се представя чрез **реализационни тегла** y_r върху последователности от действия. Допустимите стратегии тогава са точно тези, които удовлетворяват набор от **линейни** ограничения $Fy = f, y \geq 0$ - полиномиални по размер на дървото на играта, вместо експоненциални.

Частичната наблюдаемост (раздел 7.3) се въвежда чрез **функция на наблюдаемост**: за всяко наблюдавано раздаване множеството от траектории, съгласувани с това, което действително е било видяно (отказът скрива картата на опонента и допуска множество траектории; разкриването я показва и допуска само една). Правдоподобие на едно наблюдение представлява сума от реализационните тегла върху това съгласувано множество. Поставянето на **априорно разпределение на Дирихле** върху теглата и максимизирането на **логаритъма на апостериорната вероятност** води до програмата

$$\max_y \sum_r (\alpha_r - 1) \log y_r + \sum_t \log \left(\sum_{r \in o(\ell_t)} q_r y_r \right) \quad \text{при } Fy = f, y \geq 0,$$

където $o(\ell_t)$ е множеството от съгласувани траектории за наблюдение t и q_r са (нормализирани) вероятности на случайността. Решаващото свойство е, че когато $\alpha_r \geq 1$ тази целева функция е **вдлъбната** – първият член е вдлъбнат, вторият е логаритъм на афинна функция, а ограниченията са афинни – така че това е **изпъкнала** задача без **локални оптимуми**. Всеки решавач, който намери локален оптимум, е намерил глобалния. Достатъчен е стандартен **градиентен метод с проекция**: извършва се стъпка на градиента, след което резултатът се проектира обратно върху множеството от ограничения. В псевдокод цикълът е елементарен, което е и целта -

```
initialize y feasible (F y = f, y >= 0)
repeat until converged:
    g <- gradient of the (negative) log-posterior at y
    z <- y - eta * g                                # gradient step
    y <- project z onto { y : F y = f, y >= 0 }      # convex projection
```

Този метод връща **модата** на апостериорното разпределение (неговата единствена най-вероятна точка), а не средната стойност. Това е ключовият компромис: модата е **състоятелна** – при стандартни условия (истинската стратегия лежи във вътрешността на множеството от допустими стратегии и има положителна априорна плътност, различни стратегии водят до различни наблюдения

и всяка ситуация на противника се посещава безкрайно често) оценката доказано се сближава към σ^{*104} – докато оптималната по отношение на печалбата средна стойност по правило е изчислително непосилна за точно пресмятане. Следователно основната ос на дизайна, върху която стъпва целият процес, е **средна стойност срещу мода**: оптимална по отношение на печалбата, но изчислително непосилна, спрямо ефективно изчислима и състоятелна.

7.5.3 Какво установихме, откровено казано

Ние внедрихме този състоятелен оценител и проверихме неговата работа при възстановяване на стратегия в играта Кун, където той се представя според очакванията: възстановената стратегия е много близка до истинската (разстоянието по обща вариация е приблизително **0,004 до 0,021**), като съвпада или превъзхожда резултатите от възстановяването на непрекъснатия модел в същата игра. Това потвърждава както правилната работа на механизма, така и валидността на теорията върху малката тестова среда.

Ние **не** го стартирахме в рамките на онлайн цикъла на експлоатация, нито върху по-голямата игра. Причината е практическа: оценката представлява решението на оптимизация, която се **пререшава с натрупването на наблюдения**, като цената ѝ нараства с историята – от части от секундата в началото до много секунди за всяко повторно решаване след изиграването на десетки хиляди раздавания. За мач в реално време, раздаване по раздаване, това е непрактично в наивната му форма. Вместо да го разглеждаме като пропуск, ние приемаме това за *емпиричен отговор* на въпрос, който главата поставя изрично – *достатъчно бързо ли е решаването на изпъкналата задача при всяка актуализация за игра в реално време?* – а именно **не, без инкрементални методи** (всяко решаване да започва от предишното решение, а членовете за отделните раздавания да се кешират), което е точно видът приближение, с който се занимава Глава 8. Разбирането и резултатът от възстановяването са това, от което дисертацията се нуждае от този модел сега; онлайн инженерството остава за бъдеща работа.¹⁰⁴

7.6 Когато моделът е уверен, но греши

Раздел 7.5 беше за фин провал при безкрайни данни. Този раздел разглежда груб провал при крайни данни и именно той най-пряко мотивира *безопасната* експлоатация. Това е различен провал от липсата на състоятелност: тук проблемът е **неправилно зададено меню** – противник, който типово базираният модел просто не може да представи – и как се държи моделът, когато бъде притиснат в ъгъла.

Изградохме скрит противник, който представлява 50/50 смес от действията на „камък“ и „маниак“ – съзнателно **ниито един** от четирите кандидата. Очаквано е апостериорното разпределение да разпредели убеждението си между двата най-близки типа. Това обаче не се случва. Апостериорното разпределение, изчислено като произведение на вероятностите, се концентрира върху *единственото най-добро обяснение*, поради което убеждението прескача от един тип към друг и в

крайна сметка твърдо се ангажира с типа „Наш“ – единственият кандидат, който приписва реална вероятност както на залог, така и на чек със средна ръка и по този начин никога не е категорично опроверган. В резултат моделът става **уверен** (апостериорното разпределение е близо до 1,0 върху един тип) и същевременно **грешен** (този тип не отговаря на действителния противник).

Това разкрива един капан, който си струва да бъде изказан ясно: апостериорното разпределение е **относителна** величина. „Деветдесет и пет процента хипотеза на Наш“ означава „най-добро приближение *сред тези четири кандидата*“, а не „добро приближение в абсолютен смисъл“. Измерването на *абсолютното* съответствие на победителя разкрива проблема – същата хипотеза на Наш, която се представя добре срещу противник, който наистина играе равновесие на Наш, се представя значително по-зле срещу тази смес, като същевременно отчита еднаква увереност на хартия. Малкото стереотипно меню няма честен начин да каже „нито едно от горните“.

7.6.1 Колко дълго може да остане грешен? Проверка на устойчивостта

Тъй като уверено-грешните убеждения са опасни, подложихме феномена на стрес-тест върху **300 изпълнения (с различни начални числа) от по 500 раздавания**, като разграничихме две неща, които една наивна метрика обединява: *бавна сходимост* и *отпадане след сходимостта*.

Таблица 18: Надеждност на детектора при 300 начални числа по 500 раздавания.

Измерване (300 начални числа × 500 раздавания)	Резултат
Правилен дългосрочен победител до раздаване 500	300 / 300 (100%)
Някога отстъпва, след като трайно поеме лидерството	0 / 300 (никога)
Грешен тип все още води след раздаване 100	40 / 300 (~13%)
Грешен тип все още води след раздаване 200	14 / 300 (~5%)
Раздаване, на което истината окончателно се установява	медиана 23 – 90-ти персентил 125 – най-лош случай 461

Добрата новина е, че убеждението в крайна сметка винаги се установява върху най-близката представима стратегия и след като се заключи, никога не отстъпва. Тревожната новина са средните редове: в немалка част от случаите моделът поддържаше **уверено грешно убеждение за повече от сто раздавания** – достатъчно дълго, за да може един най-добре реагиращ агент, който експлоатира твърдо това убеждение, да се е настроивал да победи илюзия. Това е емпиричното проявление на напрежението между експлоатацията и безопасността: на модела може да се вярва в *крайна сметка*, но „в крайна сметка“ понякога е след 200 раздавания, а действието въз основа на междинното убеждение, сякаш то е сигурно, е начинът, по който самият агент се наранява.

Честната поправка не е по-голямо меню, а да спрем да питаме „кой единствен тип?“ и да се опрема на модели, които могат да представят смеси (непрекъснатите и състоятелните модели от раздели 7.4–7.5), и да **мащабираме степента на експлоатация според това доколко е обоснована преценката за противника**. Този принцип е нишката, която преминава през следващите два раздела и към Глава 8.

7.7 От модела до печалбата – най-добрият отговор, таванът и самонанесената уязвимост

Моделът е полезен единствено ако действията, основани на него, носят печалба. За да измерим това по ясен начин, подаваме оценката на всеки модел в **същия** точен най-добър отговор и играем пълни мачове и в двете игри, като поставяме всеки резултат между два аналитични критерия, изчислени чрез ръчно проверена математика, а не чрез симулация:

- **Очаквана стойност при равновесие на Наш** – това, което безопасната равновесна игра носи срещу този опонент (дори при безполезна преценка за противника резултатът не може да е по-лош от тази стойност);
- **таван** – точната стойност на най-добрия отговор, максималното, което може да бъде извлечено при пълно познаване на опонента.

Добрият експлоататор се намира между двете и се стреми към тавана. Първо, проверка за коректност, че апаратът е надежден: решаващът възпроизвежда точната равновесна стойност на Кун $-1/18$ (остатъчна експлоатируемост $\approx 0,002$, по същество неексплоатируем), а точен най-добър отговор превъзхожда равномерната игра с предвидените разлики (+0,500 при Кун, +2,087 при Ледюк).

Кун – и двата модела достигат тавана.

Таблица 19: Кун: таванът на експлоатация спрямо реално постигнатото от всеки модел.

Опонент	таван	типово базиран	непрекъснат
Камък	0,167	0,168	0,168
Маниак	0,333	0,337	0,330
„Наш“	-0,055	-0,055	-0,053

(Чист опонент, който винаги се отказва при залог, е още по-експлоатируем – таван от 0,975, достигнат до 0,966 – но той не е сред повтарящите се типове, дефинирани по-горе, и поради това е пропуснат тук с цел еднообразие.)

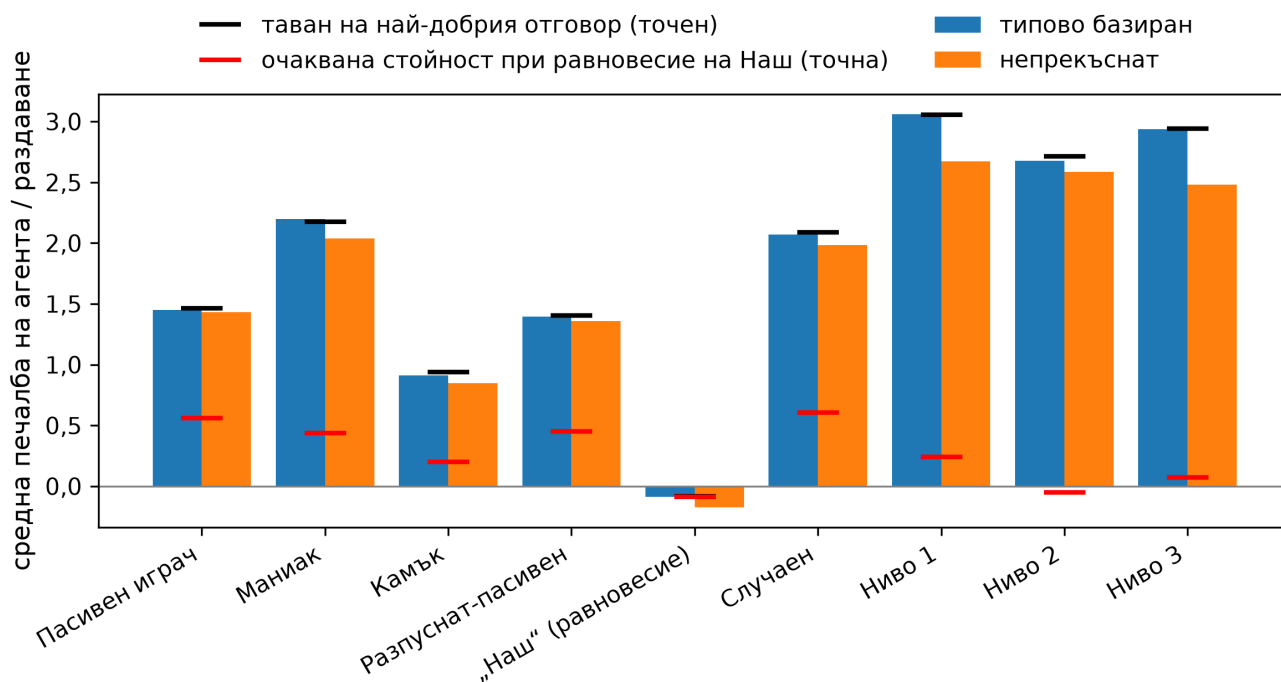
Ледюк – започва да се проявява значението на класа на модела. Ледюк Холдем добавя втори

рунд за залози и една споделена обща карта, което води до приблизително два порядъка повече ситуации в сравнение с Кун – все още точно решима, но вече достатъчно голяма, за да различи моделите.

Таблица 20: Ледюк: таванът на експлоатация спрямо реално постигнатото от всеки модел.

Опонент	таван	типово базиран	непрекъснат
Ниво 1	3,056	3,061	2,672
Маниак	2,177	2,199	2,038
Пасивен играч	1,464	1,451	1,434
Камък	0,937	0,912	0,848
„Наш“	-0,083	-0,085	-0,175

(„Ниво 1“ е противник, който играе най-добър отговор срещу случаен играч, а „Ниво k“ – най-добър отговор срещу „Ниво k-1“; в Ледюк менюто на типово базирания модел съдържа девет типа, показани на следващата фигура.)



Фигура 36: Експлоатация срещу всеки противник в Ледюк: типово базираният модел (който описва точно тези противници) следва плътно точния таван на най-добрия отговор, а непрекъснатият модел остава близо, но под тавана при най-трудните за нападение типове и пада под безопасната базова стойност срещу „Наш“.

Два резултата подчертават извода, като вторият е от съществено значение:

1. **Не можете да експлоатирате равновесие.** Срещу противника „Наш“ всеки модел реализира приблизително (отрицателната) стойност на играта и не повече. Таванът *по същество представлява* стойността на играта. Това потвърждава, че наблюдаваната експлоатация другаде е реална, а не артефакт на средата за тестване.
2. **Уверен, но недообучен модел ви прави уязвими.** В Ледюк непрекъснатият модел губи срещу „Наш“ – $-0,175$ при таван от $-0,083$. С несъвършени данни за множеството ситуации в Ледюк той реагира най-добре на *грешна* оценка на противник, който изобщо не подлежи на експлоатация, и по този начин създава **уязвимост** в собствената си игра. Недостигът на наблюдения, който срещу експлоатируеми опоненти му струва малко, се оказва активно вреден срещу неексплоатируем опонент.

Това е напрежението между експлоатацията и безопасността, изразено в едно число. Когато класът на модела пасва на противника, най-добрият отговор извлича пълната теоретична стойност; когато не съвпада, агентът както оставя пари на масата и – по-опасно – връща част от тях обратно. Тази втора цена е именно това, което предпазният механизъм от Глава 8 (ограничаване на отклонението от равновесието на Наш според увереността на самия модел) съществува, за да предотврати.

Разликите тук не се дължат на начална случайност. При пет различни начални числа стандартната грешка на печалбата на раздаване е незначителна в сравнение с наблюдаваните ефекти: типово базираният модел остава в рамките на 3% от тавана за всеки тип и в двете игри (и в рамките на две стандартни грешки за почти всички типове), а недостигът на непрекъснатия модел в Ледюк и неговата уязвимост срещу „Наш“ са многократно по-големи от стандартната грешка.¹⁰⁵

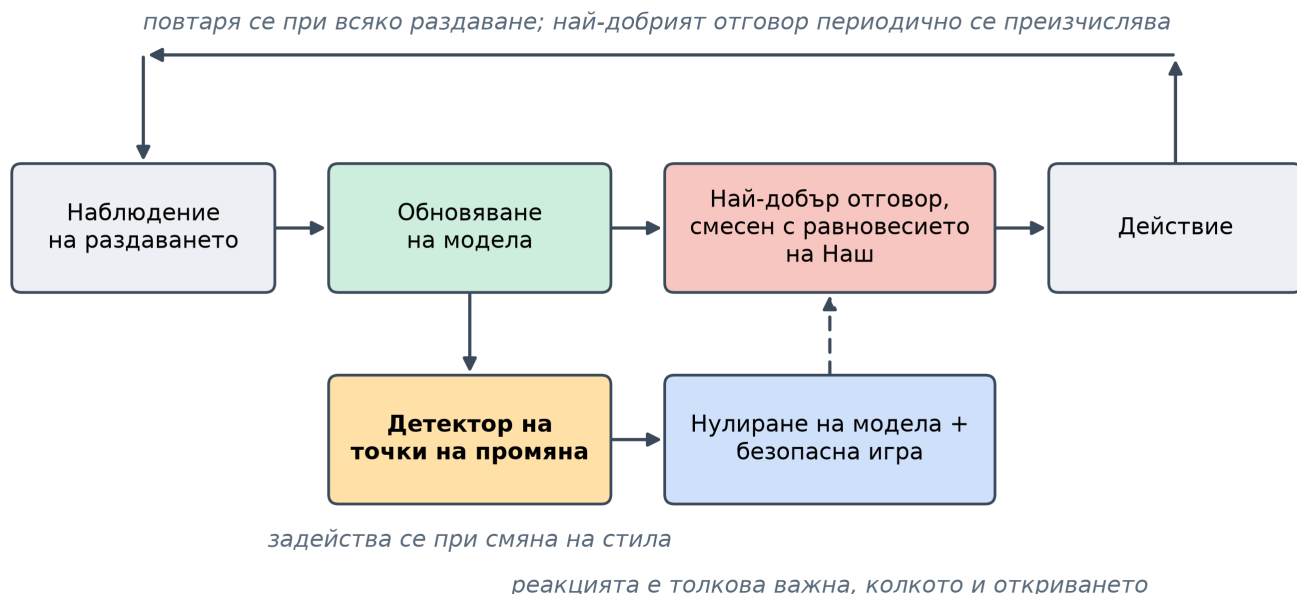
7.8 Адаптиране към промяна – нестационарни опоненти

Досега всичко се основаваше на постоянен противник. Реалните опоненти обаче се променят и адаптират, а модел, който е учил търпеливо десет хиляди раздавания, става безполезен в момента, в който неговият обект промени стила си – той вече *уверено* описва играч, който вече не съществува. Това е отворен проблем, към който е насочена дисертацията, и главата включва първи контролиран експеримент по него.

Пълният адаптивен агент изпълнява цикъла от раздел 7.2 от край до край: **наблюдава** раздавания, **актуализира** модела, периодично преизгражда собствената си стратегия като най-добър отговор на текущата оценка (по избор смесена с равновесието на Наш за безопасност) и **действа**. За да се справи с промяната, той добавя детектор: лек статистически монитор върху агресията на опонен-

¹⁰⁵Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM Transactions on Economics and Computation* 3(2), DOI 10.1145/2716322 – статията, която характеризира кога е възможен принципът „експлоатирай, но никога не губи спрямо базовата линия“; безопасната половина от скалата и основата за Глава 8.

та (байесово онлайн откриване на точки на промяна¹⁰⁶), който следи за промяна в режима и при задействане **нулира** модела и преминава към безопасна игра, докато отново се учи.



Фигура 37: Цикълът на адаптивна експлоатация: наблюдение, обновяване на модела, най-добър отговор (смесен с равновесието на Наш за безопасност) и действие; детекторът на точки на промяна може да нулира модела, когато противникът смени стила си.

Сменихме стила на опонента в средата на мач от 20 000 раздавания и сравнихме **статичен** модел (който никога не забравя) с такъв, който прилага **забравяне на точка на промяна**. Резултатът е **сценарийно-зависим**, и именно това представлява откритието:

Таблица 21: Статичен модел срещу забравяне при точка на промяна, след смяна на стила по средата на мача.

Игра	Смяна на стила (в средата)	статичен (след смяната)	точка на промяна (след смяната)
Кун	камък → маниак	-0,106 ± 0,007	+0,211 ± 0,008
Ледюк	камък → маниак	+1,834 ± 0,054	+0,552 ± 0,017

- **В Кун забравянето печели.** Стратегията, научена срещу „камък“, е блъф-ориентирана; пусната срещу маниак, който плаща всеки залог, тя *активно губи* (печалбата на статичния модел става отрицателна). Откриването на смяната и повторното обучение връщат стабилна печалба.

¹⁰⁶Adams, R. P. & MacKay, D. J. C. (2007). “Bayesian Online Changepoint Detection.” *arXiv:0710.3742*.

- **В Ледюк забравянето губи.** Там маниакът губи над два жетона на раздаване, така че непрекъснато адаптиращ се модел го експлоатира успешно и без нулиране – докато детекторът, твърде нетърпелив, задейства десетки **фалшиви аларми** по време на стабилните периоди (около шестдесет нулирания срещу една истинска промяна), всяко от които изхвърля трудно спечелени данни и преминава към безопасна игра.

Урокът не е „откриването на промени е добро“ или „лошо“ – той е, че **реакцията при открита промяна има толкова голямо значение, колкото и самото откриване.** Детектор, който задейства твърде често, съчетан с пълно връщане в безопасен режим, може да струва повече от остарелия модел, когато новият опонент е толкова експлоатируем, че остарелият модел губи малко. По-евтини и по-меки реакции – частично забравяне вместо твърдо нулиране, по-малко чувствителен детектор – са ясна следваща стъпка и те се свързват пряко с експлоатацията, мащабирана според увереността, от Глава 8. (Стойностите са средни от пет начални числа $\pm$ стандартната грешка; посоката на ефекта е една и съща при всяко от тях.)

7.9 Връзки и бъдещи насоки

Какво установява тази глава. Добрият модел на противника е *необходим, но не и достатъчен* за печеливша и безопасна адаптация. Когато класът на модела съответства на противника, най-добрият отговор достига точния таван на извлечимата стойност – реализира се пълната теоретична стойност на моделирането. Но при частична наблюдаемост и ограничени данни, недообученият модел както оставя пари на масата, така и срещу противник, който не може да бъде експлоатиран, реагира най-добре на химера и *губи*. Нестационарността добавя втори аспект: забравянето на остарял модел е полезно само когато остарялостта е активно вредна, а фалшивите аларми на небрежен детектор могат да струват повече, отколкото спестяват. Един принцип се повтаря през всичко това: **степената на експлоатация трябва да съответства на това доколко е обоснована преценката за противника.**

Обратни връзки. Тази глава е огледален образ на работата по равновесие, която я предшества. Докато една равновесна стратегия *игнорира* идентичността на противника по замисъл, моделът на противника е механизмът, който я *използва* – двата са крайните точки на спектъра безопасност–експлоатация. Механизмът за извадка, използван в предишните глави за обхождане на дърво на играта според *текущата* стратегия, се появява отново тук, за да разсъждава върху действията на противника според *изведения* модел. А идеята за групиране на много конкретни ситуации в няколко представителни категории – абстракция – е именно това, което типово базираният модел прави в пространство на стратегиите.

Напред към Глава 8 и дисертацията. Тази глава изгради **сензора**, а Глава 8 изгражда **изпълнителния механизъм** – механизма, който превръща модела в *безопасна* експлоатация, като ограничава отклонението от равновесието според това доколко е обоснована преценката за противника. Самонанесената уязвимост на непрекъснатия модел срещу „Наш“ (раздел 7.7) е емпиричният ар-

гумент за този механизъм; проверката на увереното, но грешно убеждение (раздел 7.6) определя неговия бюджет; теорията на състоятелността (раздел 7.5) е принципният гръбнак, върху който рамката се разширява.¹⁰⁷

¹⁰⁷Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Гл. 3 (игри в нормална форма); гл. 4 (изчисляване на решения на игри в нормална форма, §4.1 – линейно програмиране за игри с нулева сума); гл. 5 (игри в разгърната форма; §5.2 – игри с непълна информация и последователната форма, механизмът под всяка линейна програма в Глава 8); гл. 7 “Learning and Teaching” – рамката за учене в повтарящи се игри, включително напрежението, че действията едновременно *експлоатират* и *обучават* опонента. Свободно достъпна: <http://www.masfoundations.org/download.html>

8 Глава 8 – Безопасна експлоатация в игри с непълна информация

Тази глава въвежда от самото начало *безопасната* експлоатация на противника: формулировката на проблема, математическият апарат, семейството от методи и набор от контролирани експерименти, проведени върху две малки покер игри. Написана е така, че да може да бъде прочетена самостоятелно; не се изисква предварително познаване на програмния код на проекта. Всички експериментални числа, докладвани тук, са **измерени** при възпроизводими изпълнения на двете тестови среди (Кун покер и Ледюк Холдем) и където е възможно, са поставени в *точни* аналитични граници, а не в симулирани. Когато резултатът противоречи на онова, което теорията ме е карала да очаквам, казвам го изрично и обяснявам разминаването – тъкмо тези разминавания са най-поучителните части на главата.

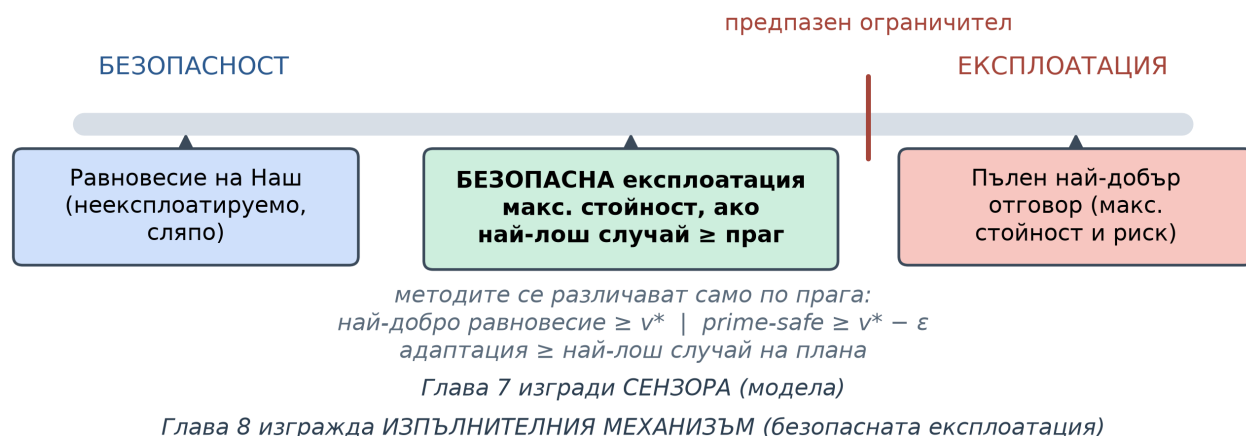
Къде се вписва това в дисертацията. Глава 7 изгради **сензор**: модел, който наблюдава как даден противник играе и оценява неговата стратегия. Тя също така разкри опасността от наивното действие въз основа на този сензор – най-добрият отговор на уверен, но недообучен модел може да отвори пробив в собствената ви игра (в Ледюк той всъщност *загуби* срещу неексплоатируемия противник „Наш“). Глава 8 изгражда **изпълнителен механизъм**: механизма, който превръща модел в печалба, **без да става експлоатируем**. Този изпълнителен механизъм е втората половина на рамката за поведенческа адаптация на дисертацията (принос №1), а определянето *точно къде неговите гаранции зависят от предположението за игра с нулева сума за двама играчи* е отправната точка за многоагентното разширение (принос №2).

8.1 Защо безопасната експлоатация – другата половина на скалата

Стратегия в **равновесие на Наш** е изградена така, че никога да не губи в дългосрочен план. В игра с нулева сума за двама играчи тази гаранция е точна: стратегията има фиксирана *стойност* v^* и никой противник, колкото и умен да е, не може да ви свали под нея. Моделирането на противника (Глава 7) ви дава възможността да се *отклоните* от тази безопасна стратегия, за да накажете грешките на конкретен противник. Очевидният начин да реализирате печалба от модел е да изчислите **най-добър отговор** към него и да играете така. Проблемът е, че най-добрият отговор обикновено сам по себе си е **силно експлоатируем**: за да накаже „винаги се отказваш при залог“, той напълно спира да блъфира с определени карти, а по-умен противник – или същият противник, след като спре да се преструва – минава право през отворения пробив.

Представете си **скала**. Ако я завъртите докрай към *безопасност*, ще играете равновесие на Наш: непобедими, но без да наказвате слаб противник. Ако пък я завъртите докрай към *експлоатация*, ще играете чист най-добър отговор на текущата си преценка за противника: максимална печалба, **ако** преценката е вярна и противникът остане на място, но сериозен риск, ако грешите, извадката ви е малка или противникът нарочно ви е подвел (показвал ви е слаб стил, за да ви примамва към голямо отклонение). **Безопасната експлоатация** поставя *ограничител* върху тази скала: можете

да се наклоните към експлоатацията колкото желаете, но никога не бива да минавате отвъд точката, в която най-неблагоприятният противник може да ви свали под вашата безопасна базова линия.



Фигура 38: Скалата безопасност–експлоатация с ограничител. Чистото равновесие на Наш е не-експлоатируемо, но „сляпо“; пълният най-добър отговор извлича най-голяма стойност, но носи най-голям риск. Безопасната експлоатация действа между двете крайности, като стойността в най-лошия случай не може да падне под определен праг. Глава 7 изгради сензора (модела), а Глава 8 изгражда изпълнителния механизъм (безопасната експлоатация). Методите се различават единствено по това къде е поставен прагът.

Причината това да не е параноя е едно число от Глава 7 и отново от тази глава: пълният най-добър отговор на стегнатия стил „камък“ в Кун носи **+0,167 на раздаване**, но стойността му в най-лошия случай е **-0,5** – противник, който на свой ред играе най-добър отговор срещу него, може да му взема по половин жетон на раздаване. Стойността в най-лошия случай при равновесие на Наш, напротив, е самата стойност на играта. Безопасната експлоатация е дисциплината да се вземе от този потенциал толкова, колкото позволяват грешките на противника, без да се поема рискът от -0.5.

Какво означава „безопасен“ в неформален план. Вашата очаквана стойност за целия мач никога не бива да пада под избран праг, **независимо какво прави противникът**. Това е твърдение за най-лошия случай, а не за конкретно раздаване – все пак ще губите раздавания; гаранцията се отнася до дългосрочната средна печалба срещу най-неблагоприятния противник. Целият въпрос на дизайна в тази глава е *какъв праг* да бъде избран и как да се изчисли стратегия, която го спазва, като същевременно извлича възможно най-много над него.¹⁰⁸

¹⁰⁸Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM Transactions on Economics and Computation* 3(2), статия 8. DOI 10.1145/2716322 – статията, която показва, че в повтаряща се игра за двама с нулева сума отклонението от равновесието може да е безопасно точно когато противникът прави „подаръци“, и дава алгоритми, които рискуват само вече спечеленото; безопасната половина от скалата и основата за Глава 8.

8.2 Експлоатацията като оптимизация с ограничения

Най-полезната идея в тази глава е, че **всеки метод за безопасна експлоатация е една и съща оптимизационна задача**, в която се сменя само една част. С други думи:

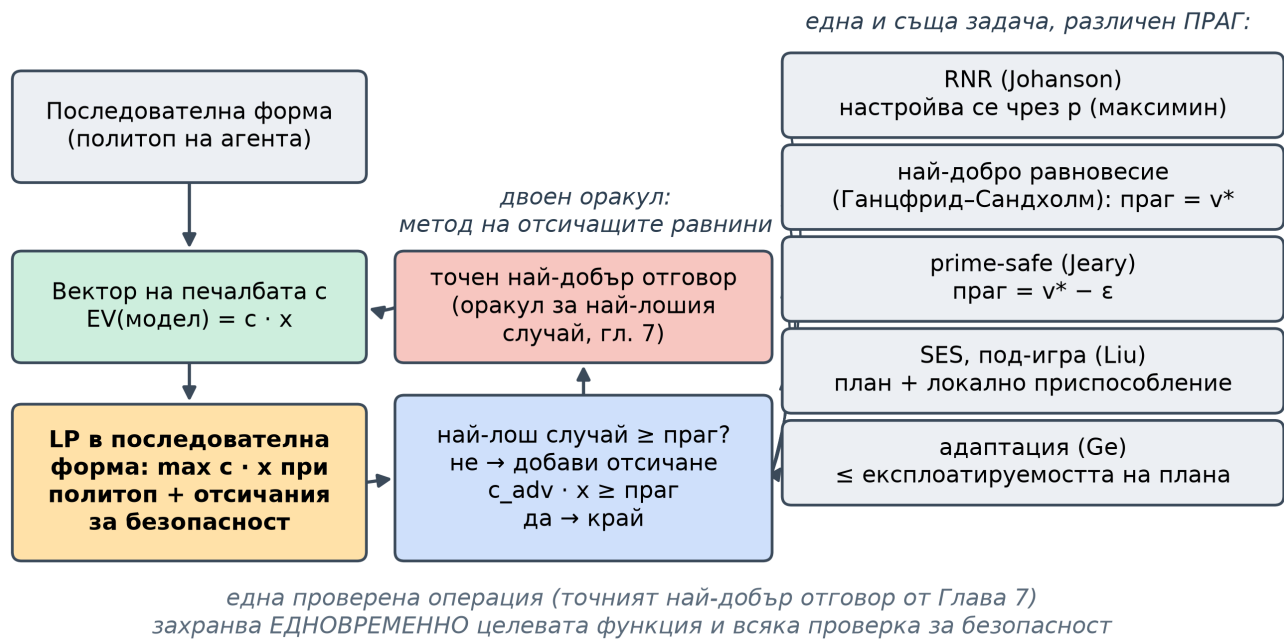
максимизиране на очакваната стойност на собствения агент спрямо модела на противника **при ограничение**: стойността му в най-лошия случай да не пада под прага на безопасност.

За да се превърне това в *изчислима* задача, представете стратегията на агента в **последователна форма**: вместо вероятности за действие според ситуацията, използвайте **план за реализация** x , който присвоява тегло на всяка от последователностите от действия на агента (поредици от собствените му избори от корена до текущата точка), подчинени на линейните ограничения на **дървовидния политоп** (treeplex): празната последователност има тегло 1; при всяко информационно множество теглата на децата се сумират до теглото на родителя; всички тегла са неотрицателни. Причината да се плати този „данък“ за смяната на променливите е решаваща: срещу **фиксирана** стратегия на противника очакваната стойност на агента е **линейна** по x ,

$$EV(x) = \sum_{z \in Z} [\pi_c(z) \cdot \pi_{-h}(z) \cdot u_h(z)] x_{\text{seq}(z)} = c \cdot x,$$

където Z са крайните състояния, $\pi_c(z)$ е вероятността на случайните ходове, $\pi_{-h}(z)$ - вероятността на ходовете на противника, а $u_h(z)$ - печалбата на агента. Това е така, защото собственото произведение на вероятностите за действие на агента по всеки път е x при последователността на този път. Така „максимизиране на очакваната стойност спрямо модела“ е **линейна целева функция** $c \cdot x$, дървовидният политоп се задава с **линейни ограничения**, а целият метод е **задача на линейното програмиране (LP)**. Пълният най-добър отговор е просто $\max_x c \cdot x$ върху дървовидния политоп – което ни позволява да проверим LP-задачата спрямо кода за точен най-добър отговор от Глава 7 (те съвпадат до 10^{-6}).

Прагът на безопасност е ограничение върху **стойността в най-лошия случай** – „за всеки противник σ' , $EV(x, \sigma') \geq \text{праг}$ “. Това вътрешно „за всеки противник“ само по себе си е задача за най-добър отговор (минимакс), поради което не го записваме като една гигантска билинейна задача; решаваме го чрез **генериране на ограничения** (метод на двойния оракул / метод на отсичащите равнини): решаваме LP-задачата, прочитаме стратегията на агента, извикваме **точния най-добър отговор като оракул за най-лошия случай** и ако стойността в най-лошия случай е под прага, добавяме единственото линейно отсичащо ограничение, което той поражда, и решаваме отново. Процедурата е крайна (съществува краен брой чисти най-добри отговори), прозрачна и лесна за отстраняване на грешки.



Фигура 39: Една LP-задача, пет прага за безопасност. Дървовидният политоп в последователна форма дава линейна целева функция (оачакваната стойност срещу модела); цикълът за генериране на ограничения използва точния най-добър отговор като оракул за най-лошия случай и добавя отсичащо ограничение, докато прагът не бъде спазен. RNR, най-доброто равновесие (базовият вариант на Ганцфрид и Сандхолм), prime-safe, SES и адаптацията са ЕДНА И СЪЩА задача с различен праг, а една вече проверена операция (най-добрият отговор от Глава 7) захранва както целевата функция, така и всяка проверка за безопасност.

Концептуалната полза от този подход е икономия: **пет метода, един двигател**. Те се различават единствено по прага, който е предмет на следващия раздел. И тъй като както целевата функция (векторът на печалбата), така и оракулът за безопасност използват повторно *проверения* код за най-добър отговор от Глава 7, цялата Глава 8 се основава на една вече проверена елементарна операция, а не на нова поредица от математически изчисления.¹⁰⁹

¹⁰⁹Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Гл. 3 (игри в нормална форма); гл. 4 (изчисляване на решения на игри в нормална форма; §4.1 – линейно програмиране за игри с нулева сума); гл. 5 (игри в разгърната форма; §5.2 – игри с непълна информация и последователната форма, механизмът под всяка LP-задача в Глава 8); гл. 7 “Learning and Teaching” – рамката за учене в повтарящи се игри, включително напрежението, че действията едновременно *експлоатират* и *обучават* противника. Свободно достъпна: <http://www.masfoundations.org/download.html>

8.3 Какво означава „безопасен“ – три определения с нарастваща степен на реализъм

„Безопасността“ звучи като абсолютна характеристика, но всъщност не е еднозначна. От значение са три различни определения, а изследователската линия по същество представлява *отслабване* на изискването, за да стане то приложимо в практиката.

(а) Безопасност по Ганцфрид и Сандхолм (2015): никога да не се печели по-малко от стойността на играта. Определението им се отнася до повтарящата се игра: средно поне v^* на раздаване, каквото и да прави противникът. Тук то се налага на стратегията за всяко отделно раздаване, с точно равновесие на Наш σ^* като базова линия:

$$EV(x, \sigma') \geq v^* \quad \text{за всеки противник } \sigma'.$$

Това е най-силното и най-чистото понятие. То се опира на **теоремата за минимакса**: в игра с нулева сума за двама играчи равновесната стратегия на Наш *гарантира* стойността v^* срещу всеки противник. На прага за всяко отделно раздаване обаче отговарят само равновесните стратегии, затова методът избира онази равновесна стратегия, която печели най-много срещу модела (вариантът „най-добро равновесие“ при Ганцфрид и Сандхолм; по-нататък методът се нарича **най-доброто равновесие**, в кода ganzfried). Пълните им алгоритми отиват по-далеч: отклоняват се от равновесието, като рискуват само „подаръците“ (грешките на противника), вече спечелени в предходните раздавания. Уловката е в предпоставката: нужно е точно равновесие на Наш, а то е практически неизчислимо във всяка голяма игра.

Какво биха добавили пълните алгоритми. Техният алгоритъм RWYWE (Risk What You’ve Won in Expectation – „рискувай средно спечеленото дотук“) решава при всяко раздаване същата LP-задача, но с праг, понижен до $v^* - k_t$, където k_t е средната стойност на натрупаните дотук подаръци, така че стратегията излиза извън равновесието точно дотолкова, доколкото грешките на противника вече са го платили; BEFFE играе най-доброто равновесие, докато натрупаните подаръци не покрият пълна експлоатация до края на мача. И двата алгоритъма са доказуемо безопасни в повтарящата се игра и могат да експлоатират повече от най-доброто равновесие; реализацията и изпитването им остават за бъдеща работа (RWYWE е естественият базов метод за двама играчи за Принос №2).

(б) Безопасност prime-safe (Jeary & Turrini 2023): корекция за несъвършена базова линия. Всяка реална базова линия е **ϵ -равновесие** (поради абстракция и ограничени изчислителни ресурси – предмет на Глава 4), така че самата тя е експлоатируема в известна степен ϵ . Поради това закрепването към точната v^* е неоправдано; методът prime-safe понижава прага точно с експлоатируемостта на самата базова линия,

$$\text{праг} = v^* - \epsilon, \quad \epsilon = \text{експлоатируемост(базова линия)} \geq 0,$$

т.е. „никога не печели по-малко от стойността в най-лошия случай на стратегията, която така или иначе щеше да играеш“. ε трябва да бъде **измерена**, а не приета наготово – точка, която експериментите приемат буквално.

(в) Безопасност на адаптацията (Ge и др. 2024): да не бъдеш по-експлоатируем от своя план (blueprint). Най-практичното понятие. Една експлоатираща стратегия е *безопасна за адаптация*, ако и само ако

$$\begin{aligned} \text{експлоатируемост}(x) &\leq \text{експлоатируемост}(\text{план}) \\ \Leftrightarrow \text{най-лош случай}(x) &\geq \text{най-лош случай}(\text{план}). \end{aligned}$$

Тъй като планът вече е ε -експлоатируем, това е по-слабо от абсолютния праг на Ганцфрид и Сандхолм (по-слабо точно с ε) и следователно *постижимо* там, където строгата безопасност не е. Опасността му е огледален образ: ако планът е *ужасен*, „не по-лошо от плана“ се удовлетворява тривиално от почти всичко – така че безопасността на адаптацията има смисъл само при разумна базова линия (отворено изискване, което отбелязвам, вместо да реша).

Таблица 22: Три понятия за безопасност: прагът, който всяко гарантира, какво изисква и къде е слабо.

Понятие за безопасност	Неформален праг	Изисква	Слабост
Ганцфрид и Сандхолм (2015)	$\geq v^*$	<i>точно</i> равновесие на Наш като базова линия	точното равновесие на Наш е практически неизчислимо в големи игри
Prime-safe (2023)	$\geq v^* - \varepsilon$	измерената експлоатируемост на базовата линия	трябва честно да измерите ε
Адаптация (2024)	$\leq$ експлоатируемостта на плана	произволен план	безсъдържателно, ако планът е лош

8.3.1 Къде се крие предположението за игра с нулева сума за двама играчи – точката, от която тръгва дисертацията

Всеки от горните прагове се основава на един факт: **в игра с нулева сума за двама играчи равновесната стратегия на Наш гарантира стойността v^* срещу всеки противник** ($\min_{\sigma'} \text{EV}(\sigma^*, \sigma') = v^*$). Именно това прави „отклонявай се към модела, но никога под прага“ последователно и приложимо ограничение, което произтича директно от теоремата за минимакса. В игра с $N > 2$ играчи това се срива: равновесната стратегия на Наш *не* гарантира

фиксирана стойност срещу произволни противници (печалбата на отделния противник вече не е равна на вашата със знак минус – в игра с нулева сума това важи само за сумата от печалбите на всички противници, – затова противниците могат да действат координирано срещу вас, а различните равновесия ви носят различни печалби), така че няма единна v^* , към която да се закотви. Равновесията в игри с повече от двама играчи не са „нито единствени, нито неексплоатируеми“.¹¹⁰ Понятия за безопасност при N играчи съществуват, но са или много консервативни (максиминната стойност срещу координирана коалиция¹¹¹), или цел, която по принцип не може да бъде гарантирана (равният дял – справедливата част C/n - доказано не може да бъде осигурен срещу противници с различни фиксирани стратегии¹¹⁰). **Това е отвореният проблем за Принос №2** – експлоатация на слаби противници в малки игри с N играчи при ограничена загуба спрямо базова линия, включително срещу противници, действащи в сговор. Тази глава не го решава; тя прави провала *прецизен*, което е точно това, от което се нуждае една глава на дисертация.¹¹²¹¹³¹¹⁴

8.4 Линеино програмиране в последователна форма и генериране на ограничения

Този раздел описва „как е изградено“. Единствената нова елементарна операция, която главата добавя, е HeroTreeplex, който (i) изброява последователностите на агента и ограниченията на дървовидния политоп (използвайки повторно кода в последователна форма от Глава 7), (ii) изгражда вектора на печалбата c за фиксиран противник чрез едно обхождане на дървото и (iii) решава LP-задачата с ограничение за безопасност с решавача HiGHS на SciPy. Всичко останало е цикълът на генериране на ограничения, който е достатъчно кратък, за да бъде представен изцяло:

```
c_model <- payoff vector of EV vs the opponent model      # linear objective
cuts    <- {}                                             # discovered safety cuts
repeat:
  x <- argmax c_model . x s.t. treeplex(x) and all cuts    # one LP solve
  pol <- behavioral strategy read out of x
  wc <- worst_case_value(pol)    # = -(opp. exact BR value): the oracle
  if wc >= floor - tol: return pol    # safe: done
  adv <- opponent's exact best response to pol
  add cut c(adv) . x >= floor - slack # valid lower bound on worst case
```

Всяко генерирано ограничение представлява вектора на печалбата спрямо *конкретен* най-добър

¹¹⁰Ge, J., Wang, Y., Li, W. & Jin, C. (2025). “Securing Equal Share: A Principled Approach for Learning Multiplayer Symmetric Games.” *ICML*; arXiv:2406.04201.

¹¹¹Celli, A. & Gatti, N. (2018). “Computational Results for Extensive-Form Adversarial Team Games.” *AAAI* 32(1). DOI 10.1609/aaai.v32i1.11462.

¹¹²Johanson, M., Zinkevich, M. & Bowling, M. (2007). “Computing Robust Counter-Strategies.” *NIPS 2007*, 721–728 – оригиналният отговор по Наш (RNR), настройваемият предшественик на методите по-rope.

¹¹³Jeary, L. & Turrini, P. (2023). “Safe Opponent Exploitation For Epsilon Equilibrium Strategies.” *arXiv:2307.12338*.

¹¹⁴Ge, Z., Xu, Z., Ding, T., Meng, L., An, B., Li, W. & Gao, Y. (2024). “Safe and Robust Subgame Exploitation in Imperfect Information Games.” *ICML*, PMLR 235, 15255–15270.

отговор на противника; то е валидна линейна долна граница на истинския най-лош случай, поради което добавянето на ограничения монотонно затяга релаксираното ограничение за безопасност към действителното. Извеждането на поведенческа стратегия от план за реализация x се извършва чрез съотношението $\beta(I, a) = x_{Ia} / x_{\text{seq}(I)}$ (равномерно в случаите, когато теглото на родителя е приблизително 0).

Единственото, което се различава между петте метода, е стойността на `floor` и, за метода на под-игра (§ 8.7), набор от *фиксиращи тегла*, които задържат играта на агента извън избраната под-игра такава, каквато е в плана. Най-доброто равновесие използва `floor = v*`; `prime-safe` използва `floor = v* - ε`; адаптацията използва `floor = worst_case(blueprint)`; RNR преформулира целевата функция като максимин по параметъра p (§ 8.5).

Една практическа особеност, която си струва да се отбележи (тя доведе до реално отстраняване на грешки при изпълненията): стойността, която *приблизителното* равновесие на Наш постига срещу самото себе си, може да бъде съвсем малко **над** истинската постижима максиминна стойност на играта, така че изискването за точно `ws ≥ floor` може в даден момент да направи LP-задачата недопустима (без допустимо решение) и, при неблагоприятна поредица от отсичащи ограничения, да върне *небезопасна* стратегия. Решението е малък резерв за допустимост в ограниченията (`floor - slack`, като `slack` $\approx 5 \cdot 10^{-4}$, поддържан независим от допустимата грешка при сходимост), така че истинската максиминна стратегия винаги да остава допустима. Това е видът числов детайл, който никога не се появява в научните статии, но решава дали методът всъщност ще върне безопасна стратегия.¹¹⁵

8.5 Ограничен отговор по Наш и скокообразното превключване

Един от най-ранните строго обосновани методи, **ограниченият отговор по Наш** (Johanson 2007; близък до ϵ -безопасните стратегии на McCracken и Bowling от 2004 г.), е концептуалният предшественик на всички останали и въвежда *настройваемия параметър*. Неговата идея: да се изчисли равновесието на агента срещу p -ограничен противник – такъв, който е принуден да играе фиксирания модел с вероятност p и е свободен да играе противнически с вероятност $1 - p$. При изменение на p от 0 до 1 се проследява път от равновесието на Наш ($p = 0$) до пълния най-добър отговор ($p = 1$). Тъй като свободният компонент дава най-добър отговор на агента, RNR е **максимин**:

$$\text{RNR}(p) = \arg \max_x \left[p \cdot \text{EV}(x, \text{модел}) + (1 - p) \cdot \min_{\sigma'} \text{EV}(x, \sigma') \right],$$

който решаваме със същия метод на отсичащите равнини плюс допълнителна променлива за най-

¹¹⁵Идеята за генериране на ограничения / двоен оракул идва от McMahan, H. B., Gordon, G. J. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML* – общата рецепта за „оптимизиране срещу най-лош случай, който се открива в хода на решаването“.

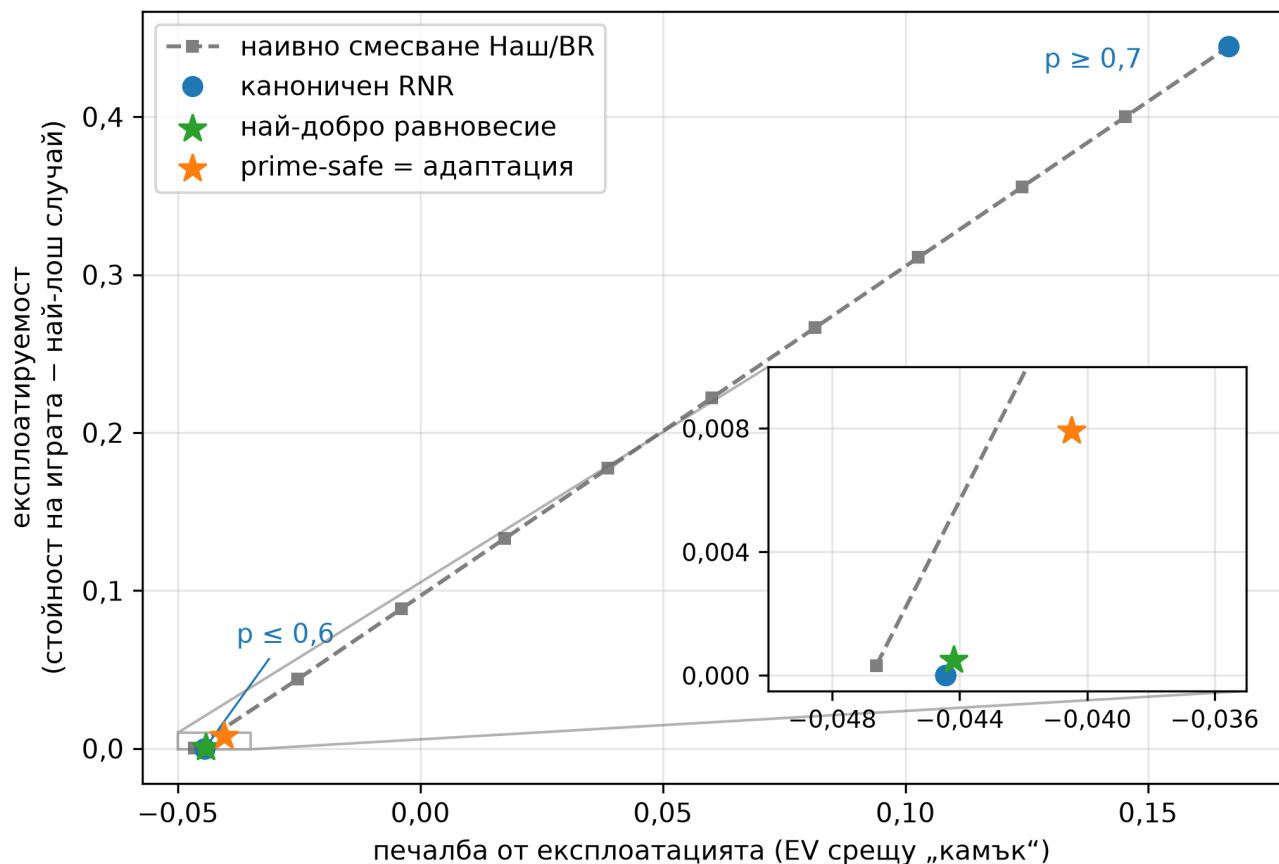
лошия случай.

Уговорка, която си струва да се запомни. В първоначалните бележки на проекта RNR беше описан като *наивно смесване*, $(1 - p) \cdot \text{Наш} + p \cdot \text{BR}$. Това е полезен интуитивен инструмент, но **не представлява** алгоритъма на Йохансон; ние реализирахме *и двата* и ги обозначихме, тъй като в малка игра поведението им е коренно различно. Наивното смесване проследява плавна крива (сивата линия на следващата фигура): печалбата и експлоатируемостта се увеличават почти пропорционално заедно – от равновесието на Наш до пълния най-добър отговор.

Измерената изненада – каноничният RNR превключва скокообразно. Предвидих, че при изменение на p каноничният RNR ще очертае гладка, монотонна граница, която доминира смесването навсякъде. Не е така. В Кун срещу „камък“ каноничният RNR връща **една и съща безопасна стратегия** за всички $p \in [0, 0,6]$ (EV $-0,044$, експлоатируемост ≈ 0) и след това **преминава направо към пълния най-добър отговор** при $p \approx 0,7$ (EV $+0,167$, експлоатируемост $0,444$). Между изпробваните стойности на p (през $0,1$) няма междинни точки.

Таблица 23: Ограничен отговор по Наш по стойността на параметъра на смесване: каноничната форма срещу наивно смесване.

	каноничен RNR –			наивно
	каноничен RNR –	експлоатируе-	наивно	смесване – экс-
p	EV	мост	смесване – EV	плоатируемост
0,0	-0,044	~0,000	-0,047	0,000
0,3	-0,044	~0,000	+0,017	0,133
0,6	-0,044	~0,000	+0,081	0,266
0,7	+0,167	0,444	+0,103	0,311
1,0	+0,167	0,444	+0,167	0,444



Фигура 40: Границата между експлоатация и безопасност в Кун с работните точки на LP-методите. Каноничният RNR дава само две групи точки (безопасния ъгъл при $p \leq 0,6$ и ъгъла на пълния най-добър отговор при $p \geq 0,7$); сивата линия е доминираното наивно смесване; звездите (най-доброто равновесие и съвпадащите prime-safe и адаптация) са в ефективния безопасен ъгъл, увеличен във вложената графика.

Защо (проверих, преди да се доверя на обяснението). Целевата функция на RNR е **линейна** по x върху **политоп**, така че нейният оптимум е **върх** и сменя върховете само когато p премине през критично съотношение. Политопът на стратегиите в Кун е малък – малко върхове – така че преходът е единичен скок. Гладката крива на Йохансон е явление от *големи игри* (много върхове) или идва от варианта *със смещение към данните* (data-biased), който задава отделно p за всяко информационно множество. Наивното смесване изглежда гладко само защото линейно интерполира две фиксирани стратегии – и то е **доминирано в безопасния ъгъл** (при експлоатируемост ≈ 0 LP-методите достигат EV $-0,044$ спрямо $-0,047$ на смесването). Скокообразно обаче е само съответствието между p и стратегията, а не самата граница: смесването на двете RNR стратегии (Йохансон и др. дефинират такова смесване) достига всяка точка от отсечката между двата ъгъла, защото очакваната стойност е линейна по смесването, а стойността в най-лошия случай е вдлъбната. При двата намерени върха превключването става при $p \approx 0,68$, а евентуален междинен връх би бил оптимален само за p между $0,6$ и $0,7$ – стойности, които серията не е включвала. В Кун срещу „камък“ тази

отсечка подобрява наивното смесване само с около 0,002, така че тук изборът *къде* да се отклониш почти не носи предимство пред равномерното смесване; силно вдлъбнати граници Йохансон и др. отчитат в голяма абстрахирана игра на холдем. Малката игра показва друго: p не е плавна скала – стратегията превключва от безопасния връх към BR-върха при праг, който *се измества според това колко експлоатируем е противникът*. Тази последна точка се повтаря в § 8.6: срещу много експлоатируем противник дори $p = 0,5$ вече е след скока.¹¹⁶

8.6 Най-доброто равновесие в Кун – безопасно, но само малко по-печелившо (основният резултат)

Централният експеримент решава всеки метод срещу *точен* модел на всеки тип противник и го оценява по **печалба** (EV срещу този противник) и **безопасност** (стойност в най-лошия случай), като прагът на Наш е $v^* = -0,056$ (известната стойност на Кун за първия играч, $\approx -1/18$). Всички числа са точни (пълно дърво) и са от изпълнението в пълен мащаб (*scale*).

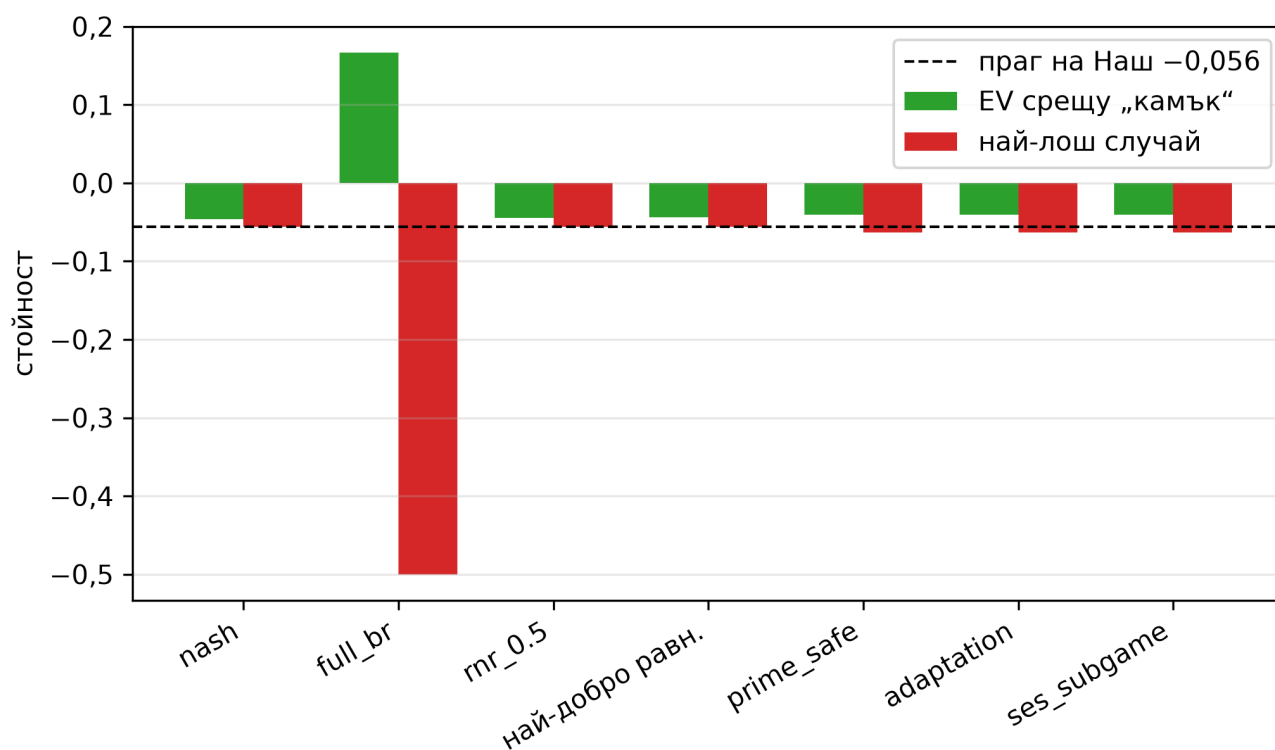
Таблица 24: Кун: очаквана стойност и стойност в най-лошия случай за всеки метод срещу всеки противник („най-добро равн.“ е базовият вариант „най-добро равновесие“ на Ганцфрид и Сандхолм, в кода ganzfried).

Противник	метрика	nash	full_br	rn_r_0.5	най-добро равн.	prime_safe	adaptation
Камък (стегнат-пасивен)	EV	-0,047	+0,167	-0,044	-0,044	-0,040	-0,040
	най-лош случай	-0,056	-0,500	-0,056	-0,056	-0,063	-0,063
Маниак (разпуснат-агресивен)	EV	+0,118	+0,333	+0,278	+0,131	+0,151	+0,151
	най-лош случай	-0,056	-0,167	-0,111	-0,056	-0,063	-0,063
Винаги залага (AlwaysBet)	EV	+0,113	+0,333	+0,333	+0,115	+0,131	+0,131
	най-лош случай	-0,056	-0,167	-0,167	-0,056	-0,063	-0,063

¹¹⁶Johanson, M. & Bowling, M. (2009). “Data Biased Robust Counter Strategies.” *AISTATS*, PMLR 5, 264–271 – прави параметъра p зависим от това колко данни подкрепят всяка част от стратегията – и именно това изглажда границата на практика.

Противник	метрика	nash	full_br	rnrr_0.5	най-добро равн.	prime_safe	adaptation
Винаги пасува (AlwaysPass; най-експлоатируем)	EV	+0,146	+0,975	+0,975	+0,222	+0,266	+0,266
	най-лош случай	-0,056	-0,500	-0,333	-0,056	-0,063	-0,063
„Наш“ (контролен)	EV	-0,056	-0,055	-0,056	-0,056	-0,055	-0,055
	най-лош случай	-0,056	-0,333	-0,056	-0,056	-0,063	-0,063

(ses_subgame не е показан: в Кун под-играта е цялата игра и методът съвпада с адаптацията. Шестият тип в данните, Thresholdish, дава същата картина: най-доброто равновесие +0,019 срещу +0,015 за Наш.)



Фигура 41: Методите срещу „камък“ в Кун: в зелено е очакваната стойност срещу противника, в червено – стойността в най-лошия случай, а пунктирът е прагът на Наш. Само при пълния най-добър отговор стойността в най-лошия случай (червено) пада до $-0,5$, далеч под прага; всички строго обосновани методи остават близо до прага и печелят колкото равновесието или малко повече.

Три основни резултата обобщават тази глава.

1. **Пълният най-добър отговор е поучителният пример.** Той винаги печели най-много срещу фиксиран модел – до $+0,975$ срещу AlwaysPass – но стойността му в най-лошия случай се срива до $-0,5$. Това е единственият метод, който е небезопасен срещу *всеки* противник ($\text{rng_}0.5$ е небезопасен срещу три от петте), защото противникът винаги може да отговори с най-добър отговор на уязвимостта, която той създава.
2. **Най-доброто равновесие е безопасно и все пак печели – но съвсем малко.** Срещу *всеки* противник стойността му в най-лошия случай остава на прага на Наш (безопасно в рамките на 10^{-3}) и печели повече от стратегията на Наш срещу всеки експлоатируем тип – най-ярко $+0,222$ срещу $+0,146$ за Наш срещу AlwaysPass и $+0,131$ срещу $+0,118$ при манияка. Именно този резултат е целта на главата: *може да се експлоатира, като доказуемо никога не се пада под равновесната стойност* – но печалбата е малка: срещу четирите експлоатируеми типа най-доброто равновесие печели с $0,002$ – $0,076$ на раздаване повече от Наш, т.е. 1–9% от онава, което печели пълният най-добър отговор. Печалбата е малка, защото прагът v^* за всяко отделно раздаване допуска само равновесни стратегии (§ 8.3).

3. Един глобален p не е настройка за безопасност. rnr_0.5 е безопасен срещу „камък“, но вече е скочил до пълния най-добър отговор срещу силно експлоатируемите AlwaysPass / AlwaysBet (същата стойност като full_br , стойност в най-лошия случай $-0,33 / -0,17$ – небезопасен). Това е прагът на превключване от § 8.5, който се измества според експлоатируемостта на противника – и конкретната причина най-доброто равновесие, което ограничава стойността, а не параметър, да е по-добрата основа.

Prime-safe и адаптацията изразходват измерен ε -бюджет. Те понижават прага от v^* до $v^* - \varepsilon$, а при изпълнението експлоатируемостта на базовата линия (CFR, спрян преждевременно) е *измерена*: $\varepsilon = 0,0074$; стойността им в най-лошия случай излиза $-0,063 = v^* - 0,008$ срещу всеки противник, което съответства на коригирания с ε праг. Изразходвайки този бюджет, те печелят малко повече от най-доброто равновесие ($+0,266$ спрямо $+0,222$ срещу AlwaysPass). Те изглеждат „небезопасни“ в таблицата само защото флагът ги сравнява с v^* ; спрямо *собствения* си праг те са безопасни по конструкция. (Prime-safe и адаптацията съвпадат по конструкция: ε се определя като v^* минус стойността на базовата линия в най-лошия случай, така че $v^* - \varepsilon$ е точно тази стойност, а реализацията използва една и съща стратегия (CFR, спрян преждевременно) и като базова линия за prime-safe, и като план за адаптацията. Праговете на Jeary и Turrini и на Ge и др. съвпадат винаги, когато двете са една и съща стратегия; разликата е в приложението – алгоритми за цялата игра срещу повторно решаване на под-игри.)¹⁰⁸

Бележка за точността на измерването (сблъсках се с този проблем на практика).

При бързата проверка (*smoke*, 30 000 итерации на CFR) дори стратегията на Наш е маркирана като „небезопасна“ – стойността ѝ в най-лошия случай е $-0,0568$, т.е. $0,0013$ под точната стойност v^* , малко над допустимото отклонение 10^{-3} . Това е чиста грешка на приближението, а не провал в безопасността: в пълен мащаб (200 000 итерации) недостигът спада до 3×10^{-4} и Наш правилно е маркиран като безопасен. Урокът за тестовата среда е, че допустимото отклонение на флага за безопасност трябва да надвишава собствената грешка на приближението на базовата линия, в противен случай базовата линия няма да издържи собствената си проверка.

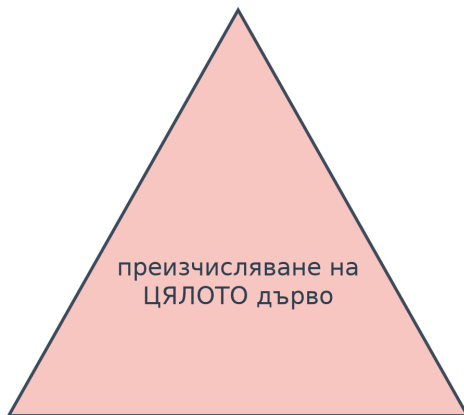
8.7 Безопасност в реално време – решаване на под-игри (SES)

Гаранцията на най-доброто равновесие е *глобална*: ограничението за безопасност се прилага върху **цялото** дърво на играта. Това е приемливо при Кун, но в реална игра не можете да решавате наново цялото дърво всеки път, когато моделът се актуализира. Идеята за **търсене на безопасна експлоатация** (Liu и др. 2022) прави безопасността **локално** свойство на една единствена **под-игра**: играйте навсякъде плана (равновесието на Наш), но в избрана под-игра оптимизирайте отново играта на агента, за да експлоатира модела – с **приспособление**, което ограничава отгоре колко локалното отклонение може да увеличи експлоатируемостта (границата расте с нивото на експлоатация и с грешката на модела на противника). По-строгата гаранция, използвана тук –

никога да не си по-експлоатируем от плана, – е безопасността на адаптацията на Ge и др., която техният метод OX-Search постига при повторното решаване на под-игри.

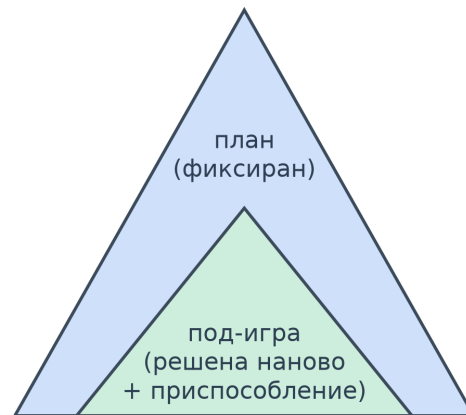
Реализацията в тази глава налага тази по-строга гаранция директно върху дървовидния политоп: **фиксирайте** всяка последователност на агента *извън* под-играта на теглото, което тя има в плана (така че агентът играе плана навсякъде освен в под-играта), и задайте прага на безопасност равен на стойността в най-лошия случай на самия план. Тъй като „играйте плана и вътре в под-играта“ винаги е допустимо и достига точно този праг, решението на LP-задачата може да бъде само поне толкова добро – локалната експлоатация е безопасна по конструкция. Единственото изискване е под-играта да бъде **затворена надолу** (веднъж влезли в нея, оставете в нея), което важи за естествени избори като „втори рунд в Ледюк“ или „след като се падне поп“.

ГЛОБАЛНА безопасност
(най-добро равновесие / prime-safe / адаптация)



Ледюк: спрени при лимит 40 итерации,
БЕЗ сходимост; най-лош случай
от $-0,64$ до $-1,33$ (силно небезопасни)

ЛОКАЛНА безопасност
(SES: под-игра + приспособление)



Ледюк: сходимост за 194–350 итерации (лимит 400);
най-лош случай $\approx -0,13$ (почти безопасен);
от $+0,25$ до $+0,68$ срещу слабите типове

Фигура 42: Глобална срещу локална безопасност. Глобалните методи решават наново цялото дърво; в Ледюк техният цикъл за генериране на ограничения не достига сходимост в рамките на лимита от 40 итерации и остави силно небезопасни стратегии. Методът с под-игра фиксира играта извън избраната под-игра според плана и решава наново само нея с приспособление – много по-малка задача, която при лимит 400 итерации достига сходимост и остана почти безопасна.

Значението на „локално“ не е само въпрос на естетика: § 8.8 показва, че в Ледюк решаването на

под-игра достига сходимост в рамките на своя бюджет, а глобалното – не.¹¹⁷¹¹⁸¹¹⁹

8.8 В мащаб – Кун работи; Ледюк се разпада глобално, но се задържа локално

Ледюк Холдем добавя втори рунд на залагане и една обща карта – приблизително два порядъка повече ситуации от Кун, все още точно решими, но достатъчно големи, за да натоварят методите. Изпълнението за Ледюк използваше конфигурация с ограничен брой итерации (лимит от 40 итерации за цикъла на генериране на ограничения, допустимо отклонение 10^{-2} и под-игра, зададена като „поп на масата“), като за всяка клетка е записано дали решаването е **достигнало сходимост**, или лимита. Стойност на играта $v^* = -0,086$.

Таблица 25: Ледюк: методът с под-игри срещу глобалните решавачи („ит.“ – итерации; „лимит“ – спрял при лимита от 40 итерации).

Противник	метрика	nash	full_br	ses_subgame	най-добро равн.	prime_safe / adaptation
Камък	EV	+0,201	+0,937	+0,247	+0,624	+0,635
	най-лош	-0,089	-1,633	-0,130	-0,838	-0,744
	случай					
	сходим?	✓	✓	✓ (194 ит.)	✗ лимит	✗ лимит
Маниак	EV	+0,438	+2,177	+0,682	+1,806	+1,842
	най-лош	-0,089	-1,100	-0,130	-0,638	-0,657
	случай					
	сходим?	✓	✓	✓ (211 ит.)	✗ лимит	✗ лимит
Пасивен играч (CallingStation)	EV	+0,559	+1,464	+0,663	+1,342	+1,363
	най-лош	-0,089	-1,000	-0,130	-0,915	-0,686
	случай					
	сходим?	✓	✓	✓ (350 ит.)	✗ лимит	✗ лимит

¹¹⁷Liu, M., Wu, C., Liu, Q., Jing, Y., Yang, J., Tang, P. & Zhang, C. (2022). “Safe Opponent-Exploitation Subgame Refinement.” *NeurIPS* – търсене на безопасна експлоатация (SES) с приспособление; добавената експлоатируемост е ограничена отгоре (теорема 4.1).

¹¹⁸Ge, Z. et al. (2024), *op. cit.* – OX-Search: повторно решаване на под-игри с приспособление, което гарантира, че експлоатируемостта не надвишава тази на плана (безопасност на адаптацията, теорема 4.3); може да се прилага вложено при всяко ново информационно множество и е насочено срещу противник, който първо „обучава“ модела, а после го експлоатира; изпитано в Ледюк и Flop Hold’em, за двама играчи.

¹¹⁹Milec, D., Kovařík, V. & Lisý, V. (2025). “Adapting Beyond the Depth Limit: Counter Strategies in Large Imperfect Information Games.” *AAMAS 2025* (разширено резюме), 2675–2677; arXiv:2501.10464 – използване на информация от модел на противника *отвъд* хоризонта на търсене.

Противник	метрика	nash	full_br	ses_subgame	най-добро равн.	prime_safe / adaptation
Разпуснат- пасивен (LoosePassive)	EV	+0,449	+1,405	+0,559	+1,306	+1,309
	най-лош случай	-0,089	-4,200	-0,133	-1,239	-1,334
	сходим?	✓	✓	✗ (400 ит.)	✗ лимит	✗ лимит

Основната констатация е отрицателна и емпирична: при лимит от 40 итерации глобалното решаване не достига сходимост дори в Ледюк. Предвидих, че най-доброто равновесие ще бъде безопасно в Ледюк, както в Кун. Вместо това **глобалните** решавачи (най-доброто равновесие `ganzfried`, `prime_safe`, `adaptation` и `rng_0.5`) всички **достигнаха лимита от 40 итерации, без да достигнат сходимост**, оставяйки стойности в най-лошия случай от **-0,64 до -1,33** – силно небезопасни. Причината, която потвърдих, вместо да предполагам: методът на отсичащите равнини добавя по едно отсичащо ограничение на итерация и при по-голямото дърво на Ледюк множеството от релевантни чисти най-добри отговори е голямо, така че 40 отсичащи ограничения изобщо не фиксират истинския най-лош случай; главната LP-задача продължава да връща оптимистични, но небезопасни стратегии. Още *преди* изпълнението това беше отбелязано като най-вероятното слабо място – и методът се провали точно там.

Положителната половина е методът на под-играта. SES **достигна сходимост** при трима от четиримата експлоатируеми противници (194–350 итерации), тъй като решава наново само малката под-игра „поп на масата“, като останалата част от дървото е фиксирана – значително по-малка LP-задача с далеч по-малко множество от отговори на противника. Той извлича реална стойност **(+0,25 до +0,68)** срещу слабите типове – повече от стратегията на Наш) при стойност в най-лошия случай $\approx -0,13$, което е с порядък по-близо до безопасността в сравнение с глобалните методи. Това е разграничението *глобална срещу локална безопасност* от § 8.7, което се проявява като измерим факт в игра толкова малка като Ледюк – конкретният аргумент за методите за под-игри в реално време и против наивно глобално решаване. Сравнението обаче не е при равни условия: глобалните решавачи са спрени след 40 итерации (около 2,5 s на клетка), а SES е имал лимит 400 и е използвал 194–400 итерации (30–79 s на клетка). Резултатът показва, че глобалният цикъл е бавен и небезопасен *в рамките на този бюджет*, а не че не може да достигне безопасност при по-голям.

За SES обаче трябва да се отбележи: неговата остатъчна експлоатируемост ($\approx 0,043$) все още надвишава допустимото отклонение от 0,01, така че и той е маркиран като небезопасен, а при LoosePassive изпълни пълните 400 итерации, без да достигне сходимост. Това нарушение обаче се мери спрямо v^* . Собственият праг на SES е стойността в най-лошия случай на плана – тук това е спряното преждевременно CFR (-0,120), а не равновесието на Наш (-0,089) – и трите клетки

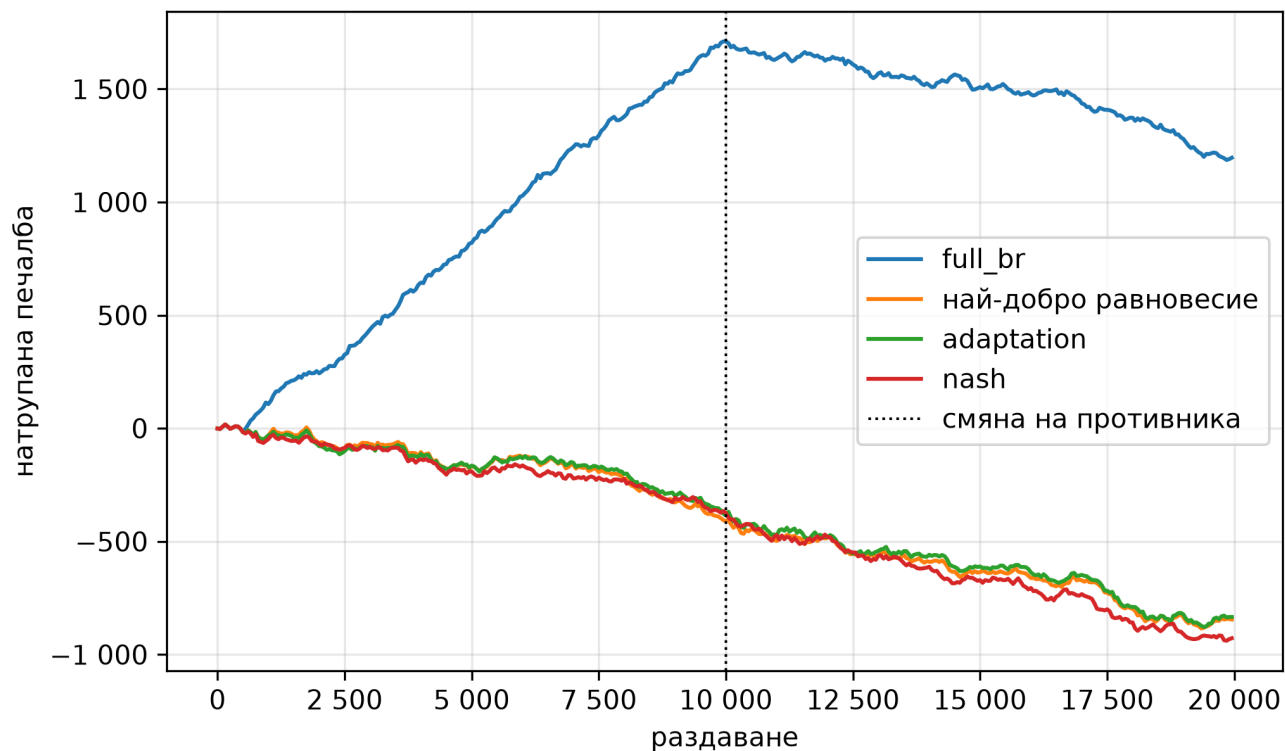
със сходимост са с 0,010 под този праг, т.е. точно на границата на допустимото отклонение 0.01. Следователно SES е спазил своя праг в рамките на допустимото отклонение, а остатъкът се дължи главно на експлоатируемостта на самия план; дали методът е *доказуемо* безопасен, ще покаже повторно изпълнение с по-малко допустимо отклонение.

8.8.1 Обучаващата атака – и защо осъществената печалба е погрешен критерий

Последният експеримент е проверка при измама – изпитанието, което цялата система за безопасност трябва да издържи. Измамлив противник играе слабия стил „камък“ като примамка в продължение на 10 000 раздавания, след което „разкрива“ силната си игра и още 10 000 раздавания играе равновесие на Наш; на всеки 500 раздавания моделът от Глава 7 подава обновената си оценка на всеки решавач (5 начални числа).

Таблица 26: Обучаващата атака: реализирана печалба и нарушения на безопасността по метод.

метод	средно на раздаване (общо)	средно на раздаване (след смяната)	нарушения на безопасността / начално число
full_br	+0,051	-0,061	40, 40, 40, 40, 40
най-добро равн. (ganzfried)	-0,048	-0,055	0, 0, 0, 0, 0
adaptation	-0,046	-0,055	40, 40, 40, 40, 40
nash	-0,051	-0,061	0, 0, 0, 0, 0



Фигура 43: Обучаваща атака в Кун, натрупана печалба (изпълнение с начално число 0). Пълният най-добър отговор (синьо) се изкачва до около +1700 по време на примамката и след смяната само бавно слиза – завършва далеч напред, защото „разкриващият“ противник, който играе равновесие на Наш, си връща само около стойността на играта на раздаване. Безопасните методи не се хващат на примамката и през цялото време плащат „данъка“ на първия играч.

Наивният разказ – „обучаващата атака наказва алчния експлоататор“ – **не се потвърди по отношение на осъществената печалба**, а разбирането защо е поучително. Пълният най-добър отговор завършва със *значителна печалба* (+0,051 на раздаване общо): той реализира неочаквана печалба по време на фазата с примамката и, тъй като „разкриващият“ противник е само „Наш“, този противник успява да си върне едва малко повече от стойността на играта на раздаване, така че тази неочаквана печалба не бива върната в рамките на 10 000 раздавания. Сигналът, който *наистина* отличава безопасните от небезопасните методи, е **точната стойност в най-лошия случай / броят нарушения на безопасността**: пълният най-добър отговор наруши прага на Наш в **40 от 40** повторни настройки, а най-доброто равновесие – в **0 от 40**. Най-лошият случай за една стратегия се реализира от противник, който *играе най-добър отговор срещу нея* – а стационарният противник, който след „разкриването“ играе равновесие на Наш, просто не е такъв противник. Коригираният извод, който сега смятам за правилния урок при проектирането, е: **измервайте безопасността чрез най-лошия случай, а не чрез осъществената печалба срещу безобиден противник**; за да накажете алчния експлоататор *по отношение на печалбата*, разкриващият противник трябва да бъде адаптивен контраексплоататор, а не фиксирано равновесие на Наш. (Това, че adaptation

показва 40 нарушения, докато *nash* показва 0, също не е грешка: адаптацията умишлено се насочва към праг *под v^** , така че по замисъл задейства брояча, който се отчита спрямо v^* .)

8.9 Връзки и бъдещи насоки

Какво установява тази глава. Безопасната експлоатация е една идея – *максимизиране на стойността спрямо модела при спазване на праг на безопасност* – и в напълно решима игра тя работи точно както предвижда теорията: най-доброто равновесие е безопасно срещу всеки противник и срещу всеки експлоатируем противник печели повече от стратегията на Наш, макар и съвсем малко, докато наивният най-добър отговор носи повече печалба, но е катастрофално експлоатируем. Prime-safe и адаптацията разширяват гаранцията към несъвършените базови линии, присъщи на всяка реална система, като изразходват *измерен ϵ -бюджет* под стойността на играта. Но най-ценният резултат от тази глава е отрицателен и емпиричен: дори в игра с малък размер като Ледюк глобалното решаване на задачата за безопасна експлоатация не достигна сходимост при лимит от 40 итерации, докато локалният метод на под-играта (с лимит 400) достигна – разликата между глобалната и локалната безопасност от теория към практика, измерена, а не просто заявена, макар и при неравни бюджети.

Обратни връзки. Тази глава е точното допълнение на предходната работа върху равновесието и моделирането на противника. Сензорът от Глава 7 създава модела на противника, който тук се превръща в *целева функция*; точният най-добър отговор от Глава 7 служи като *оракул за най-лошия случай*; а представянето в последователна форма от работата върху CFR и върху състоятелността (Глава 7) е *променливата*, която LP-задачата оптимизира. Емпиричната мотивация за прага на безопасност в тази глава идва от самонанесената уязвимост на непрекъснатия модел срещу „Наш“ в Ледюк (Глава 7, § 7.7) – и наистина тук най-доброто равновесие никога не губи срещу „Наш“ повече от стойността на играта.

Напред към дисертацията. Резултатите в Кун показват, че изпълнителният механизъм работи според теорията (при точен модел на противника и в игра, която може да се реши точно); липсата на сходимост в Ледюк е конкретният аргумент за двата мащабируеми пътя, между които дисертацията трябва да избере – **точна двойнствена LP-задача, решавана еднократно**, за ограничението за най-лошия случай, или избор на **локална безопасност на ниво под-игра** (SES, OX-Search) като механизъм в реално време. Първото допълнение е собственият алгоритъм на Ганцфрид и Сандхолм, който рискува натрупаните подаръци (RWYWE, § 8.3) – по-силният базов метод за двама играчи за Принос №2; той изисква само съществуващата LP-задача с праг, който се мени според натрупаните подаръци, а реализирането му остава за бъдеща работа. И в двата случая прагът на безопасност е това, което превръща крехкия сензор от Глава 7 в приложим адаптивен агент.

9 Глава 9 – Многоагентно обучение с подкрепление: координация, конкуренция и комуникация

Тази глава въвежда от самото начало многоагентното обучение с подкрепление (MARL): проблемът, който го отличава от всичко преди него, математиката, която го рамкира, семейството от методи, които се използват за решаването му, и набор от контролирани експерименти, проведени в малки, точно решими игри. Написана е така, че да може да бъде прочетена независимо; не се предполага предварително запознаване с кода на проекта. **Всички експериментални числа, представени тук, са измерени** при възпроизводими изпълнения на тестовите среди и, където това е възможно, са поставени в *точни* аналитични граници (равновесия на Наш, точни стойности на най-добрия отговор), а не в граници, получени от други симулации. Когато дадено изпълнение противоречи на очакванията, произтичащи от теорията, запазвам първоначалното очакване и го съпоставям с получения резултат – тези разминавания са най-поучителните части от главата.

Къде се вписва това в дисертацията. Глави 2–8 работят почти изцяло в рамките на **игри за двама с нулева сума** (Pluribus в Глава 6 е изключението, което се отказва от гаранциите): има стойност v^* , равновесната стратегия на Наш я осигурява срещу *всеки* противник и CFR доказуемо се сближава към нея. Глава 9 е повратната точка към **многоагентния** свят, където тези три удобства отслабват или изчезват. Тя предлага три връзки с дисертацията. **LOLA** (Foerster и др., 2018) – диференциране през *стъпката на обучение* на противника – може да се разглежда като *динамично* моделиране на противника, допълнение към статичната преценка за противника от Глава 7, насочено към противник, който се променя (Принос №1). **PSRO** – игра, която се играе върху *популация от стратегии* – е кандидат-рамка за безопасна експлоатация там, където няма теорема за минимакса (Принос №2), и обща многоагентна *методология за оценяване* (Принос №3). Мястото, където се нарушава всяка гаранция за игри с двама играчи – липсващата минимаксна опора при $N > 2$ – е назовано тук и оставено като отворен проблем, към който е насочена дисертацията.

9.1 Защо многоагентното обучение с подкрепление е различен проблем

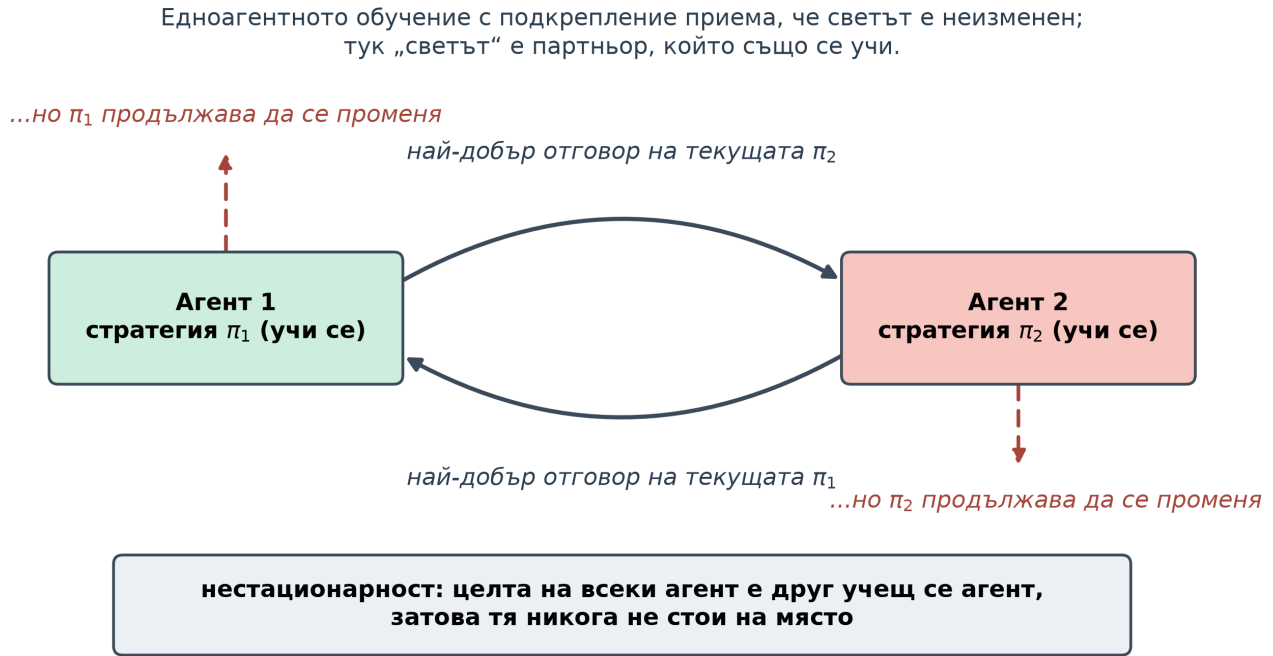
В едноагентното обучение с подкрепление (Глава 1) един агент се учи чрез проба и грешка в **непроменяща се** среда. Формално той се изправя пред стационарен марковски процес на вземане на решения: законът за преход $P(s' \mid s, a)$ и наградата $R(s, a)$ не се изменят по време на обучението, което позволява да съществува фиксирана оптимална стойностна функция Q^* , към която агентът да се сближава. В Глави 2–8 ние преминахме към едно по-високо ниво и изчислихме **равновесия** – стратегии, които са оптимални срещу *съвършено рационален* противник, който вече е завършил своите разсъждения.

Многоагентното обучение с подкрепление се намира между тези два подхода и е по-трудно и от двете. Няколко агента **се учат едновременно в споделена среда**, така че от гледната точка на всеки един агент „средата“ – която вече *включва и другите агенти* – **непрекъснато се изменя**,

докато всички се обновяват. Това е **нестационарност** и тя нарушава предположението, на което се крепи едноагентното обучение с подкрепление. От гледна точка на агент i ефективният преход и награда зависят от стратегиите π_{-i} на другите агенти, а те се променят при всяко обновяване, така че целта Q^* , която агент i преследва, самата тя е в движение. Това също така поражда два проблема, които просто не съществуват за агент, който действа сам: **координация** (как агентите, които си сътрудничат, се научават да действат заедно, без някой да им казва как?) и **разпределяне на заслугата** (когато отборът успее, чии действия са имали значение?).

Образ, който си струва да запомните: **учене на танц с партньор, който също се учи да танцува**. Ако стъпките на вашия партньор бяха фиксирани, бихте могли да научите наизуст последователност от стъпки, която им подхожда – това е едноагентното обучение с подкрепление. Ако вашият партньор беше безупречен професионалист, бихте могли да изучите известния му стил и да се подготвите с перфектен отговор – това е изчисляване на равновесие. Но когато *и двамата* се подобряват в реално време, всяка промяна, която правите, променя това, което другият трябва да направи, и обратното; вие си стъпвате един на друг по краката, прекоригирате и осцилирате – докато или не намерите общ ритъм (координирано равновесие), или не започнете да се въртите в кръг безкрайно (както в „съвпадение на монети“, Matching Pennies, раздел 9.4). Цялата област е изкуството да *подредите картите така, че синхронизацията да се случва надеждно*.¹²⁰

¹²⁰Zhang, K., Yang, Z. & Başar, T. (2021). “Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms.” *Handbook of RL and Control* – §1–2 за нестационарността и таксономията, използвана тук.



Фигура 44: Нестационарността като двама партньори, които се учат да танцуват едновременно. Всеки агент оптимизира спрямо текущата стратегия на другия (плътните стрелки), но тази стратегия също се променя, защото другият агент на свой ред оптимизира спрямо него (пунктирните стрелки). Целта, която всеки преследва, никога не стои на място, затова наивните обучаващи се агенти циклират или прекалено се коригират, вместо да се сближат с равновесие. Всеки метод в тази глава е различен начин целта да се задържи неподвижна достатъчно дълго, за да може агентът да се научи спрямо нея.

9.2 Марковски игри – мостът между глави 2–8 и MARL

Глави 2–8 разсъждаваха върху **игри в екстензивна форма (EFGs)**: дърво на играта, **информационни множества**, които групират истории, които играчът не може да различи, и контрафактични стойности, подавани към алгоритъм за минимизиране на съжалението (CFR). Глава 9 разсъждава върху **марковски (стохастични) игри**, стандартната формализация на MARL. Двете се свързват естествено и изясняването на тази връзка е това, което не позволява преминаването да изглежда като започване отначало.

Марковската игра е наредената n -торка

$$(\mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, \{R_i\}_{i=1}^N, \{\Omega_i\}, O, \gamma),$$

с множество състояния $s \in \mathcal{S}$, по едно множество действия $\mathcal{A}_i$ за всеки агент, закон за преход $P(s' \mid s, a_1, \dots, a_N)$, управляван от **съвместното** действие, наградите на отделните агенти

$R_i(s, a_1, \dots, a_N)$ и (при частично наблюдавания случай) наблюдения $o_i = O_i(s)$. Всеки агент има стратегия $\pi_i(a_i | o_i)$; заедно те формират **съвместна стратегия** $\pi = (\pi_1, \dots, \pi_N)$. Специалните случаи са именно предишните глави:

- $N = 1$ възстановява обикновено **MDP** (Глава 1).
- $N = 2$ и едно състояние дава **матрична игра** (с нулева сума при $R_1 = -R_2$; тестовата среда на тази глава, раздел 9.4).
- Последователна игра с непълна информация и нулева сума дава **EFG** (глави 2–8): марковска игра, при която „**състоянието**“ е *история*, а **частичната наблюдаемост** се изразява чрез **информационно множество**.

Това, което преминава през моста, е *речникът*. Стратегията все още съпоставя това, което агентът знае, с разпределение върху действията, така че поведенческата стратегия на EFG в информационно множество е точно стратегията в марковската игра $\pi_i(a_i | o_i)$; информационното множество се превръща в наблюдението o_i ; а контрафактичната стойност на история става оценка на централизиран оценител в състояние. Това, което **не** преминава през моста, са *гаранциите*. Контрафактичното разлагане на CFR изисква дърво на играта с пълна памет, каквото общите марковски игри (с цикли и едновременни ходове) не предоставят, а при $N > 2$ играчи CFR губи гаранцията за сходимост към равновесие на Наш дори в дърво с пълна памет (Abou Risk и Szafron, 2010),¹²¹ така че „средна стратегия $\rightarrow$ Наш“ вече не важи; MARL се връща към обучение чрез градиент/стойност, чиято сходимост не е гарантирана (оттук и цикличността в раздел 9.4). Най-съществено, изчезва **опората на минимаксната стойност**: в игра за двама с нулева сума равновесната стратегия на Наш гарантира v^* срещу всеки противник – фактът, на който се крепи безопасната експлоатация в Глава 8 – и за $N > 2$ няма такава единична стойност. Именно тази липсваща опора е празнината, която наследява Принос №2.¹²²

9.3 Семейството от методи

Всеки метод в тази глава представлява различно структурно решение на въпроса „другите агенти се учат, докато аз се уча“. Тук са от значение пет основни **семейства** (MADDPG, MAPPO и QMIX са три варианта на CTDE).

¹²¹Abou Risk, N. & Szafron, D. (2010). “Using Counterfactual Regret Minimization to Create Competitive Multiplayer Poker Agents.” AAMAS, 159–166 – гаранцията за сходимост на CFR обхваща игрите за двама с нулева сума и пълна памет; при повече играчи тя се губи.

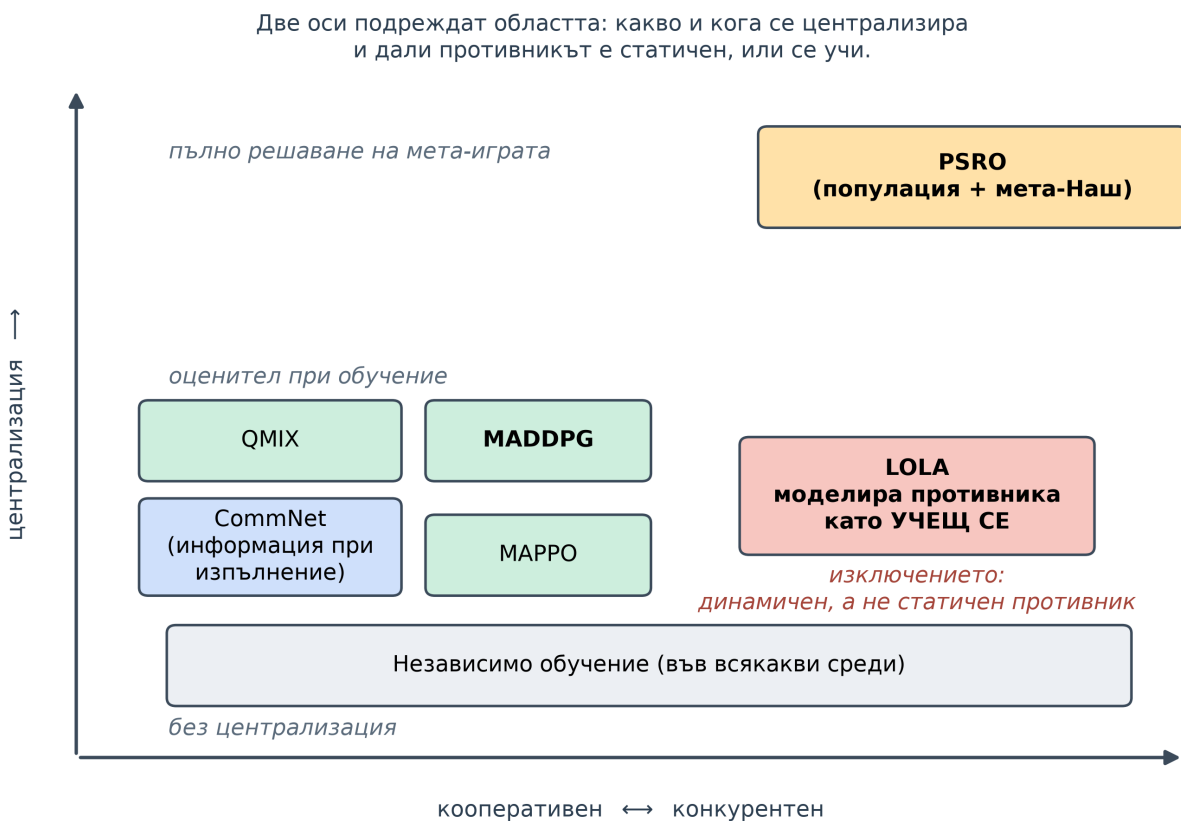
¹²²Littman, M. L. (1994). “Markov Games as a Framework for Multi-Agent Reinforcement Learning.” ICML – статията, която въведе тази рамка и алгоритъма minimax-Q.

Таблица 27: Семействата методи в MARL: какво прави всеки, кога да се използва и основната му слабост.

Подход	Какво прави	Кога да го използвате	Основна слабост
Независимо обучение (IL)	всеки агент използва собствен алгоритъм за едноагентно обучение и третира останалите като част от средата	бърза базова линия; почти стационарни среди	нестационарност → цикличност, провал на координацията
MADDPG (CTDE)	оценителят на всеки агент вижда наблюденията и действията на всички агенти по време на обучението; всеки актьор вижда само собственото си наблюдение при изпълнение	смесени кооперативно-конкурентни задачи; каноничният CTDE шаблон	входът на оценителя нараства с N ; актьорското обновяване е сложно за настройка
MAPPO (CTDE)	обикновен PPO с централизирана функция на стойността $V(s)$ на глобалното състояние и споделени параметри	кооперативен MARL, когато искате проста и надеждна базова линия	висока цена на данните при обучение по текущата стратегия (on-policy); „прост“, но чувствителен към настройката
QMIX (CTDE)	факторизира съвместната стойност монотонно на стойности по агенти	кооперативни задачи с обща награда	монотонността ограничава класа на представимите функции на стойността
PSRO	поддържа популация от стратегии; намира мета-Наш сместа върху нея; обучава най-добър отговор на тази смес; добавя го; цикълът се повтаря	конкурентни/общии игри; когато искате игровитеоретична цел на сходимост	пълен най-добър отговор във всеки кръг; приблизителният оракул отслабва гаранцията

Подход	Какво прави	Кога да го използвате	Основна слабост
LOLA	оптимизира, като приема, че противникът ще направи една стъпка на обучение ; диференцира през неговото обновяване	диференцируеми игри с двама играчи, в които наивното обучение се проваля (IPD)	предполага, че знаете и можете да диференцирате обновяването на противника
CommNet	агентите излъчват диференцируемо съобщение ; всеки получава средното от останалите и го подава на входа на своята стратегия	кооперативни задачи с частична наблюдаемост	усредняването губи информацията <i>кой</i> какво е казал

Две оси пресичат таблицата (фигурата по-долу). Първата е **какво е централизирано и кога?** IL не централизира нищо; CTDE централизира *оценителя/стойността само по време на обучението* (изпълнението остава децентрализирано); PSRO централизира цялостно *решаване на мета-играта* между кръговете на обучение; CommNet централизира *информацията по време на изпълнение* чрез канал, който агентите сами научават. Втората е **моделира ли се противникът като статичен, или като учещ се агент?** Всичко освен LOLA третира стратегията на противника като фиксирана, докато агентът ѝ отговаря; **LOLA** предвижда *следващото обновяване* на противника – динамично, а не статично.



Фигура 45: Семейството методи по две оси. Хоризонтално: доколко целевата среда е кооперативна или конкурентна (IL и PSRO обхващат и конкурентните среди; MADDPG, MAPPO, QMIX и CommNet са насочени към сътрудничество; LOLA е мостът към смесените мотиви). Вертикално: какво и кога се централизира (нищо при IL; оценителят по време на обучение при CTDE; пълното решаване на мета-играта при PSRO; информацията по време на изпълнение при CommNet). LOLA е изключението, което моделира противника като учещ се агент, а не като фиксирана стратегия.

Исторически развитието протича така: научи се да говориш (CommNet, 2016) → централизирай оценителя (MADDPG, 2017) → внеси теория на игрите в популацията (PSRO, 2017) → факторизирай стойността (QMIX, 2018)¹²³ → предвиди обучението на противника (LOLA, 2018) → и след това установи, че простият подход често е най-успешен (MAPPO, 2022).¹²⁴

¹²³Rashid, T., Samvelyan, M., Schroeder de Witt, C., Farquhar, G., Foerster, J. & Whiteson, S. (2018). "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning." *ICML*. arXiv:1803.11485.

¹²⁴Albrecht, S. V., Christianos, F. & Schäfer, L. (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches* (MIT Press), гл. 5 и 9 – съвременно учебно изложение на тези предизвикателства и методи.

9.4 Независимо обучение и начините, по които то се проваля

Най-ясният начин да се разбере *защо* останалата част от областта изобщо съществува, е чрез изпълнението на контролен експеримент, който води до провал. При независимото обучение всеки агент се поставя в собствен градиентен цикъл и се приема, че останалите агенти са част от средата. Върху четири канонични 2×2 матрични игри – всяка от които представлява различен качествен случай – това води до четири различни изхода.

Игрите и техните аналитични равновесия на Наш (обективна истина): **дилемата на затворника** (доминираща стратегия: предателството строго превъзхожда сътрудничеството, така че единственото равновесие е взаимно предателство); **„съвпадение на монети“** (Matching Pennies; игра с нулева сума; единственото равновесие е напълно смесеното $(\frac{1}{2}, \frac{1}{2})$ със стойност 0); **лов на елен** (две чисти равновесия на Наш – доминиращото по печалба (Елен, Елен) и рисково доминиращото (Заек, Заек) – плюс едно смесено); **битка на половете** (две чисти равновесия на Наш, които не са съгласни кое да изберат, плюс едно смесено).

Два независими градиентни агента (с точни градиенти, така че динамиката е чиста и без шум) дават следните измерени резултати:

Таблица 28: Независими обучаващи се агенти в четири матрични игри, сравнени с аналитичното равновесие на Наш.

Игра	Измерен резултат	NashConv	Съвпада ли с аналитичното равновесие на Наш?
дилемата на затворника	$x \rightarrow [0,001, 0.999]$ (предателство)	0,0013	да – равновесието при доминиращи стратегии
лов на елен	всички начални числа $\rightarrow$ (Заек, Заек)	0,0013	да – <i>рисково доминиращото</i> чисто равновесие на Наш
битка на половете	всички начални числа $\rightarrow$ едно чисто равновесие на Наш	0,0013	да – чисто равновесие на Наш
„съвпадение на монети“	не се сближава (последната итерация се отклонява към границата)	1,4–1,8	не – никога не се установява

В три от четирите игри агентите се сближават с истинско равновесие на Наш, а интересните детайли са в „как“. В **лов на елен** агентите надеждно избират **рисково доминиращото** равновесие

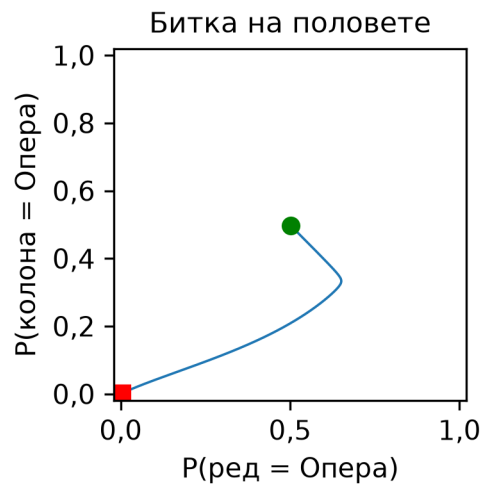
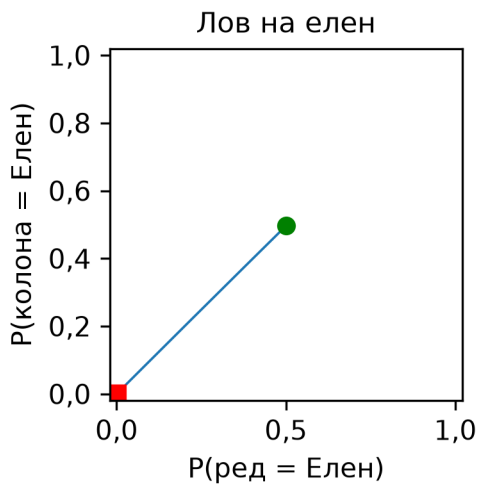
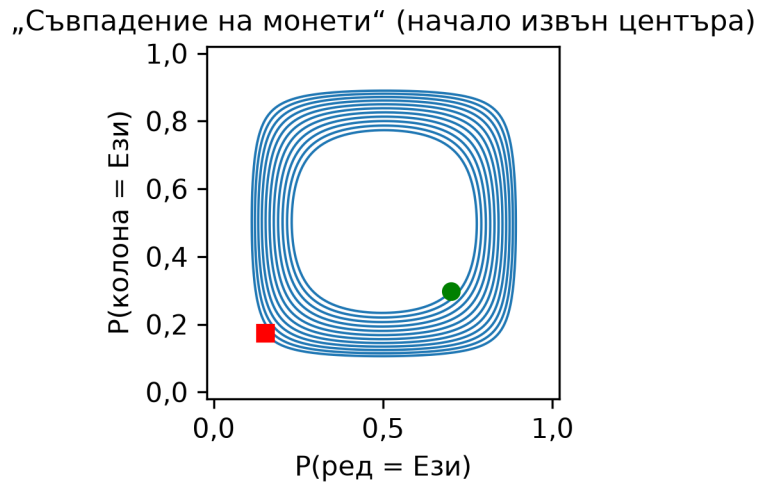
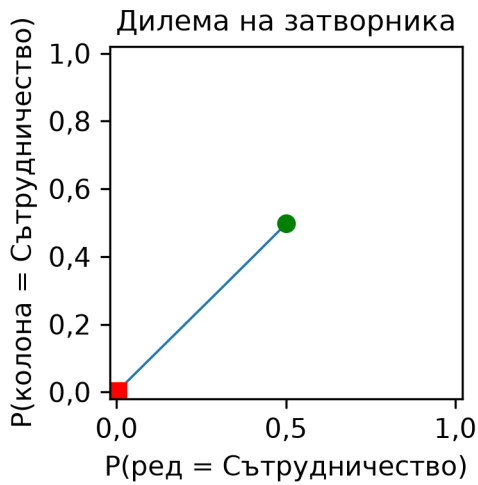
(Заек), вместо доминиращото по печалба (Елен), въпреки че „Елен“ носи по-голяма печалба – едностранният ход към „Елен“ се наказва, така че градиентната динамика се плъзга към безопасния ъгъл. В **битка на половите** всяко начално число води до *едно и също* чисто равновесие; агентите решават проблема на координацията, но не по начин, разнообразен според началните числа. Това са проблемите с избора на равновесие и координацията, представени в конкретен вид.

Играта „съвпадение на монети“ е показателна: тя **никога не се сближава с равновесие**, което е именно същината на проблема.¹²⁵

Съпоставка (запазена прогноза → какво всъщност се случи). Прогнозирах, че в „съвпадение на монети“ траекторията ще опише чиста орбита с приблизително постоянен радиус около $(\frac{1}{2}, \frac{1}{2})$. И двата вида обучаващи се агенти не успяха да **се сближат с равновесие** – но нито един не описа чиста орбита. Агентът с градиентен метод с проекция (IGA), използван в изследователските скриптове, *спираловидно се отдалечава*: измереното разстояние до равновесието на Наш нараства през времевите прозорци от 0,30 до 0,48. Агентът със softmax параметризация, използван в имплементацията, *се отклонява към ъглите* (последни профили като $x = [0,96, 0.04]$, $y = [0,03, 0.97]$, $\text{NashConv} \approx 1,8$). **Изводът остава непроменен и дори е по-ясен**: при наивно едновременно учене *последната итерация* не се сближава с равновесие в игра само със смесено равновесие – това, което се сближава, е *средното по времето* (раздел 9.6). Моделът на „енергозапазващата орбита“ е верен за непрекъснатата динамика (безкрайно малка стъпка): тогава траекторията обикаля около равновесието по затворена крива (Singh и др., 2000). При фиксирана стъпка 0,1 обаче всяка стъпка излиза леко навън от кривата и траекторията бавно се отдалечава по спирала към границата. И в двата случая последната итерация не се сближава, а това е аргумент в полза на механизмите за осредняване, разгледани по-нататък.

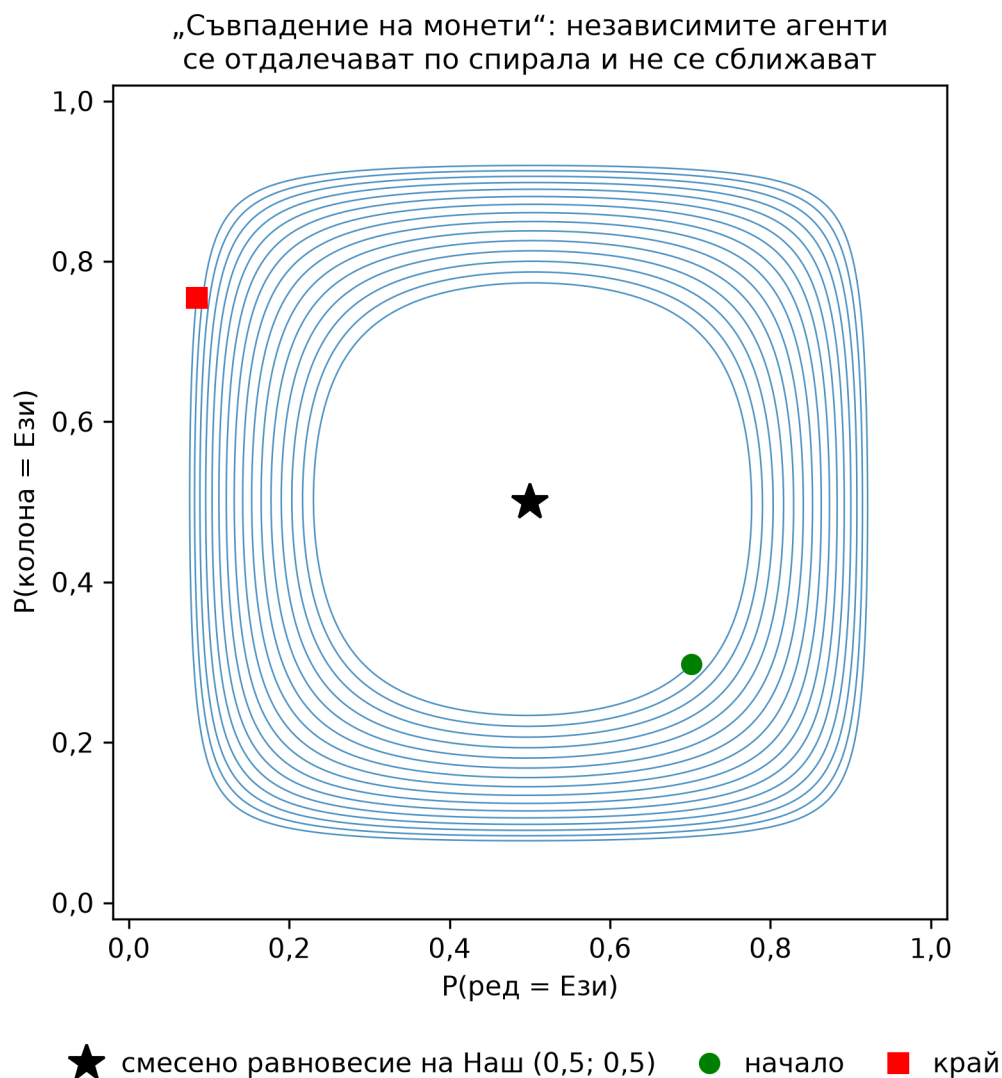
¹²⁵Singh, S., Kearns, M. & Mansour, Y. (2000). “Nash Convergence of Gradient Dynamics in General-Sum Games.” *UAI* – анализ на това защо градиентното изкачване циклира, вместо да се сближи с равновесие, в игри като „съвпадение на монети“.

Независими обучаващи се агенти в игри 2×2
(траектории в пространството на стратегиите)



● начало ■ край

Фигура 46: Независими обучаващи се агенти в четирите матрични игри: в дилемата на затворника те стигат до взаимно предателство, в „лов на елен“ и „битка на половете“ се установяват в чисто равновесие, а в „съвпадение на монети“ не се сближават (при начало извън центъра траекторията се отдалечава от смесеното равновесие на Наш, вместо да се установи в него). Контрастът показва нагледно защо са нужни механизми за координация.

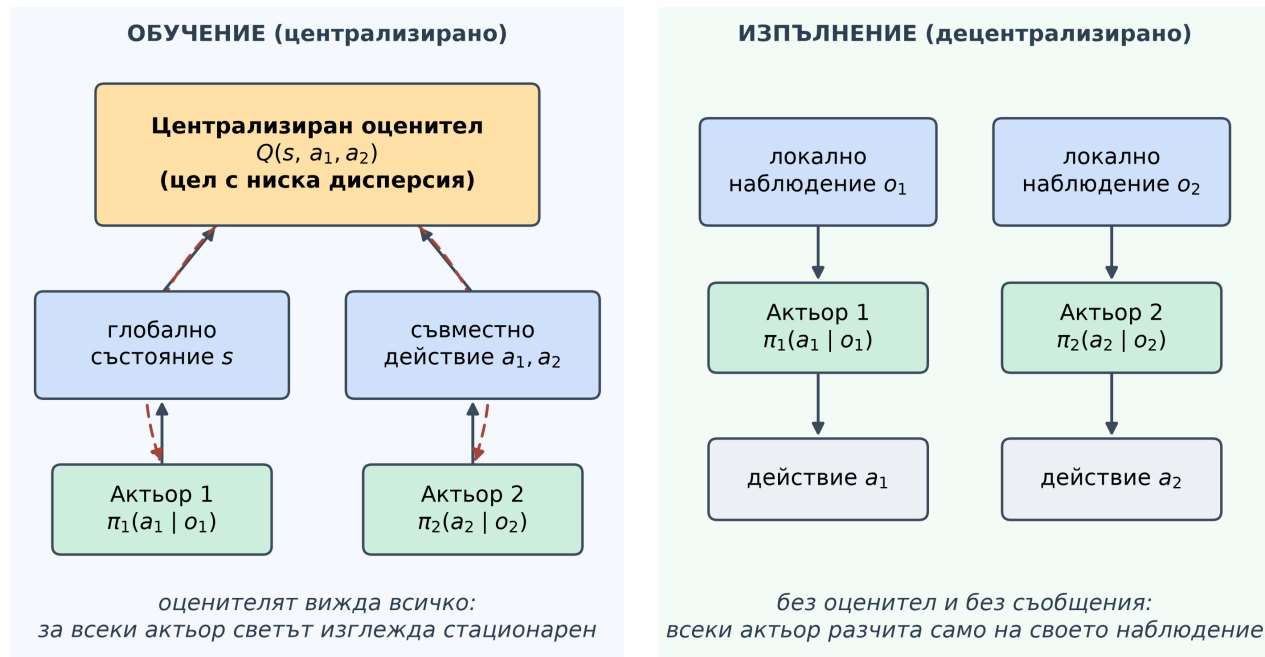


Фигура 47: Нестационарността отблизо: при „съвпадение на монети“ разстоянието до смесеното равновесие на Нaш не намалява, а расте през последователните времеви прозорци (от 0,30 до 0,48); траекторията се отдалечава по спирала към границата. Последната итерация не се сближава, колкото и дълго да продължи обучението.

9.5 Централизирано обучение, децентрализирано изпълнение (CTDE)

Първата структурна корекция запазва изпълнението реалистично – всеки агент все още действа въз основа на собственото си наблюдение – като същевременно дава на *обучаващия се агент* привилегирован достъп до информация по време на обучение. Архетипът е **MADDPG** (Lowe и др., 2017): актьорът $\pi_i(a_i \mid o_i)$ на всеки агент вижда само собственото си наблюдение, но централизираният оценител $Q_i(s, a_1, \dots, a_N)$ вижда глобалното състояние и действието на *всеки* агент. Асиметрията е същността на подхода. Оценителят за всеки агент, който наблюдава само o_i , се

сблъсква с нестационарен, частично наблюдаван свят, така че идентични входове водят до различни възвръщаемости и неговата целева стойност е шумна; централизираният оценител, който отчита цялата информация, вижда целева стойност, която е (почти) детерминистична, затова дава обучаващ сигнал със значително по-ниска дисперсия. **MARPO** (Yu и др., 2022) е минималната версия: обикновен PPO с една централизирана функция на стойността $V(s)$ на глобалното състояние вместо отделна $V(o_i)$ за всеки агент, и е силна базова линия, която често не отстъпва на по-сложни методи – предупреждение срещу прекалено усложняване.



Фигура 48: Архитектура CTDE. По време на обучението централизиран оценител (или функция на стойността) вижда глобалното състояние и съвместното действие и дава целеви стойности с ниска дисперсия; по време на изпълнението всеки актьор действа само въз основа на собственото си локално наблюдение, без достъп до оценителя и без обмен на съобщения. Асиметрията между обучение и изпълнение прави света стационарен в очите на обучаващия се агент, без да се „мами“ по време на реалната игра.

Има ли централизираният оценител действително по-ниска дисперсия? Измерено в едностъпкова кооперативна „референциална“ задача (говорителят вижда целта, слушателят – не, а двамата трябва да я назоват), след обучение до сходимост:

Таблица 29: Централизиран срещу независим оценител:
краен остатък на грешката на функцията на стойността.

Оценител	Краен остатък (грешка на функцията на стойността)
централизиран $Q(s, a_1, \dots, a_N)$	$3,2 \times 10^{-11}$
независим $Q_i(o_i)$	0,077

Централизираният оценител свежда остатъка си практически до нула – той вижда целта и съвместното действие, така че наградата е детерминистична функция на неговите входове – докато независимият оценител не може да види целта и остава блокиран в прогнозирането на базовата честота. Това потвърждава, че централизираният оценител напасва целевата си стойност много по-точно. Това само по себе си не означава по-ниска дисперсия на градиента на стратегията: при сходили оценители централизираният оценител може дори да я увеличи (Lyu и др., 2021).¹²⁶

Но по-точният оценител не е същото като решаването на координацията и именно тук една прогноза се провали.

Съпоставка (запазена прогноза → какво всъщност се случи). Прогнозирах, че в играта „изкачване“ на Claus и Boutilier¹²⁷ – кооперативна матрична игра без състояние, чийто оптимум (11) е фланкиран от —30 санкции за некоординация, с „безопасен“ атрактор при 5 – централизираният оценител ще избяга от капана и ще достигне оптимума, побеждавайки независимите обучаващи се агенти. Това не се случи. Измерени награди при алчна стратегия: **независими обучаващи се агенти 7, MADDPG 5, MAPPO 7** (оптимум 11, безопасна стойност 5). Нито един метод не достигна оптимума, а MADDPG всъщност *изостана* както спрямо IL, така и спрямо MAPPO. (Използваният вариант на MADDPG за дискретни действия заменя детерминистичния градиент на стратегията с контрафактична базова линия в стила на COMA (Foerster и др., 2018),¹²⁸ затова резултатът се отнася до този вариант, а не до MADDPG в оригиналния му вид.) Честното тълкуване: централизираният оценител напасва целевата си стойност много по-точно (потвърдено по-горе), но това **не е достатъчно**, за да преодолее относителното свръхобобщаване¹²⁹ и трудното изследване, което санкциите от —30 налагат – агентите няма да опитат рискованото съвместно действие достатъчно дълго, за да открият 11. Това, че MADDPG остава под IL, насочва вниманието към дискретното обновяване

¹²⁶Lyu, X., Xiao, Y., Daley, B. & Amato, C. (2021). “Contrasting Centralized and Decentralized Critics in Multi-Agent Reinforcement Learning.” AAMAS, 844–852. arXiv:2102.04402 – при сходили оценители по текущата стратегия централизираният оценител дава на децентрализираните актьори обновявания с по-висока дисперсия. Версия в списание: Lyu, X., Baisero, A., Xiao, Y., Daley, B. & Amato, C. (2023). JAIR 77, 295–354. DOI 10.1613/jair.1.14386.

¹²⁷Claus, C. & Boutilier, C. (1998). “The Dynamics of Reinforcement Learning in Cooperative Multiagent Systems.” AAAI, 746–752.

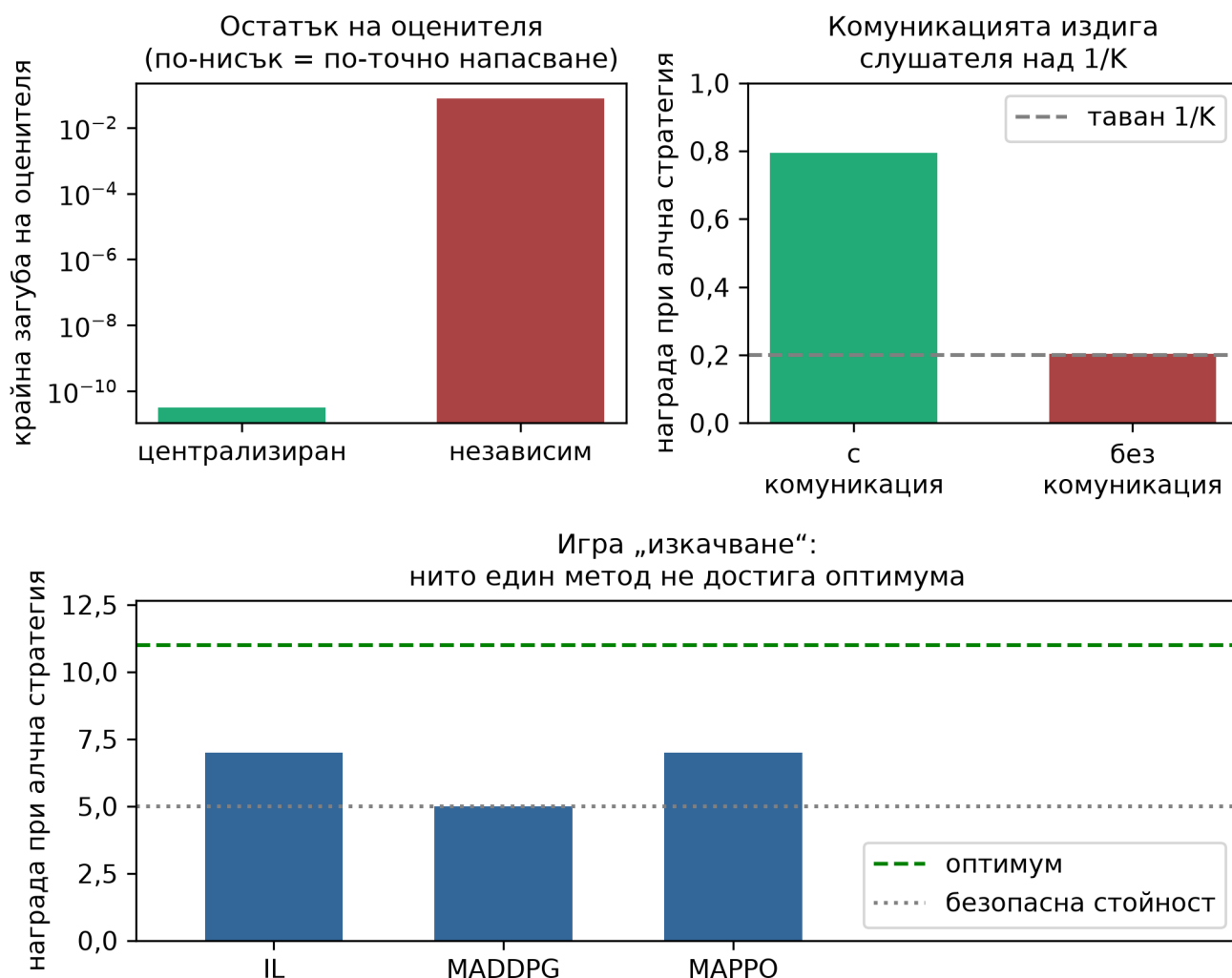
¹²⁸Foerster, J., Farquhar, G., Afouras, T., Nardelli, N. & Whiteson, S. (2018). “Counterfactual Multi-Agent Policy Gradients.” AAAI 32(1). DOI 10.1609/aaai.v32i1.11794 (COMA).

¹²⁹Matignon, L., Laurent, G. J. & Le Fort-Piat, N. (2012). “Independent reinforcement learners in cooperative Markov games: a survey regarding coordination problems.” Knowledge Engineering Review 27(1), 1–31. DOI 10.1017/S0269888912000057.

на актьора с контрафактична базова линия – то трябва да бъде проверено първо. Урокът оцелява в по-сдържан вид: **CTDE осигурява по-добър оценител, но не и автоматична координация.**

Методологическа бележка, която си струва да се запомни: и двата ефекта на CTDE по-горе бяха **невидими при бързата конфигурация за проверка (smoke)** – там загубите на оценителя бяха почти еднакви, а ползата от комуникацията (раздел 9.7) беше нула. Те се проявиха едва след обучение до сходимост в по-голямата конфигурация в пълен мащаб (scale). Бързата проверка показва, че кодът работи; за да се наблюдават явленията, е необходимо обучение до сходимост.¹³⁰

¹³⁰Lowe, R. et al. (2017). “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments.” *NeurIPS* (MADDPG); и Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A. & Wu, Y. (2022). “The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games.” *NeurIPS Datasets and Benchmarks Track*. arXiv:2103.01955 (MAPPO).



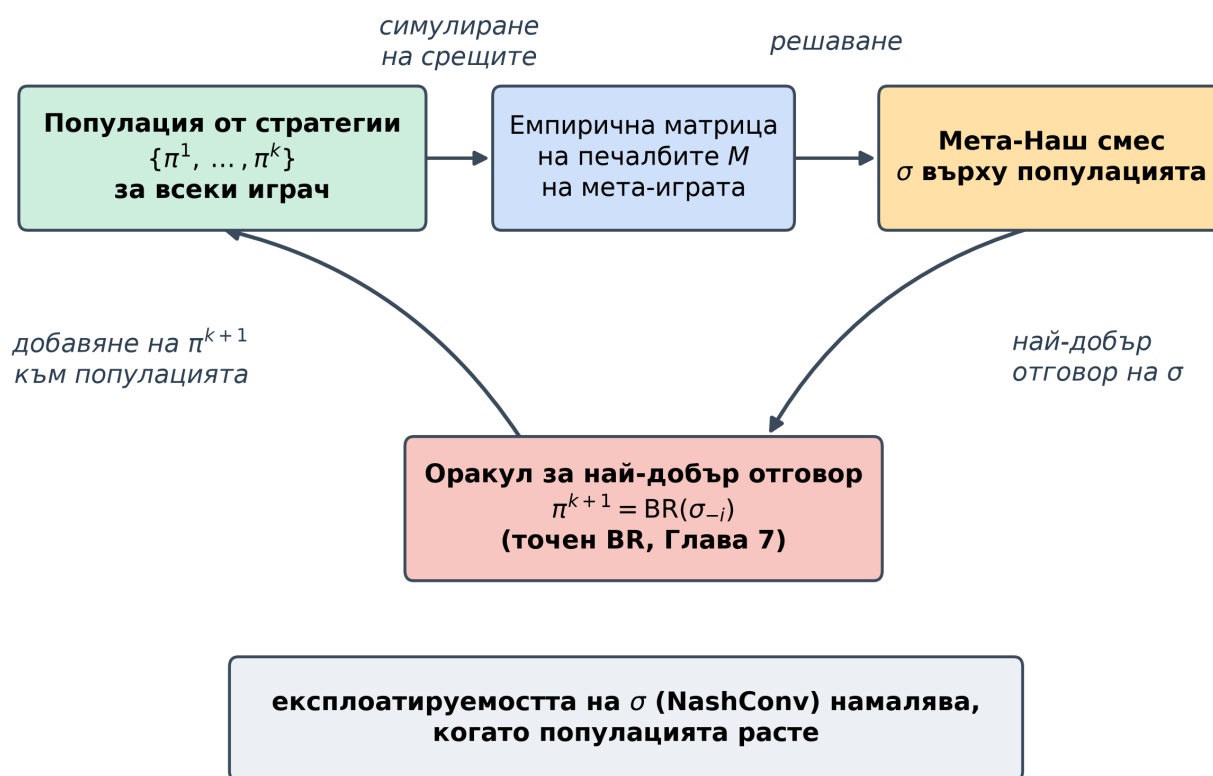
Фигура 49: Резултати за кооперативното CTDE и за комуникацията (конфигурация в пълен мащаб). Горе вляво: остатъкът на централизирания оценител е с порядъци по-малък от този на независимия (логаритмична скала). Горе вдясно: комуникацията издига слушателя далеч над тавана при случайно отгатване $1/K$ (вж. раздел 9.7). Долу: в играта „изкачване“ нито един метод не достига оптимума (11); MADDPG (5) изостава от IL и MAPPO (7) – CTDE намалява дисперсията на оценителя, но само по себе си не решава координацията, когато изследването е трудно.

9.6 PSRO – теория на игрите върху популация от стратегии

Втората структурна корекция е водещата нишка на главата и прекият наследник на **итерирания най-добър отговор** от Глава 2. **PSRO** (Policy-Space Response Oracles; Lanctot и др., 2017) разглежда цели стратегии като атомите на една по-висша игра. То поддържа **популация** от стратегии за всеки играч, изгражда емпиричната матрица на печалбите на **мета-играта** между популациите, решава **мета-Наш равновесие** върху нея, обучава **най-добър отговор** (оракул) на мета-Наш сместа на противника и добавя този отговор към популацията; цикълът се повтаря. То обединява

самообучението, фиктивната игра и метода на **двойния оракул** в една рамка и, което е от решаващо значение за този проект, неговата метрика за прогрес е *експлоатируемост* – същата NashConv, използвана в Глави 2–8.

PSRO = итерираният най-добър отговор от Глава 2, пренесен от действия към цели стратегии.



Фигура 50: Цикълът на двойния оракул в PSRO. От текущите популации се построява матрицата на печалбите на мета-играта и се намира нейната мета-Наш смес; оракулът за най-добър отговор изчислява отговор на сместа на противника и новата стратегия се добавя към популацията. Очаква се експлоатируемостта на мета-Наш сместа да намалява с нарастването на популацията. В този проект оракулът е точният най-добър отговор от Глава 7, затова сходимостта на PSRO се измерва със същата мярка за експлоатируемост като в главите по теория на игрите.

Два факта правят имплементацията точна, а не приблизителна. Първо, оракулът използва повторно **точния най-добър отговор** от Глава 7 за игрите Кун и Ледюк. Второ, мета-Наш сместа на противника върху *поведенчески стратегии* е, съгласно теоремата на Кун¹³¹ (тъй като тези игри са с пълна памет), реализационно еквивалентна на **една единствена поведенческа стратегия**; свеждането на сместа по този начин позволява модулът за точен най-добър отговор да се приложи директно.

¹³¹Kuhn, H. W. (1953). "Extensive Games and the Problem of Information." *Contributions to the Theory of Games II*, 193–216. DOI 10.1515/9781400881970-012.

Защо популация, а не просто последната стратегия от самообучението? Защото самообучението се сближава в *средното*, а не в последната итерация. Измерено върху **Кун** при фиктивна игра със самообучение: експлоатируемостта на **средната** итерация спадна от 0,24 до 0,031 за 200 итерации, докато експлоатируемостта на **последната** итерация продължи да осцилира между 0,33 и 0,83. Доверяването на последната стратегия би означавало да се доверим на стойност, която никога не се установява; мета-сместа на PSRO е популационната версия на осредняването, върху което се основават CFR и фиктивната игра.

Сходимост на PSRO, измерена (експлоатируемост = NashConv на мета-Наш сместа в цялата игра):

Таблица 30: Траектория на експлоатируемостта при PSRO, по семейство игри.

Игра	Траектория на експлоатируемостта	Извод
Кун покер	$0,917 \rightarrow \sim 2 \times 10^{-16}$ до кръг 6	сближава се до машинна нула
матрична игра („съвпадение на монети“)	$2,0 \rightarrow 0$ до кръг 2	сближава се
камък-ножица-хартия (изследване)	$2,0 \rightarrow 0,017$; популация $\rightarrow \{R,P,S\}$, смес $\rightarrow$ равномерна	сближава се
Ледюк Холдем	$4,75 \rightarrow 2,16$ за 20 кръга	намалява, но е далеч над целта
Goofspiel ($K = 3$, бърза проверка)	$1,33 \rightarrow 0$	сближава се
Goofspiel ($K = 4$)	колебае се $1,24 \leftrightarrow 2,0$ (8 кръга)	не се установява

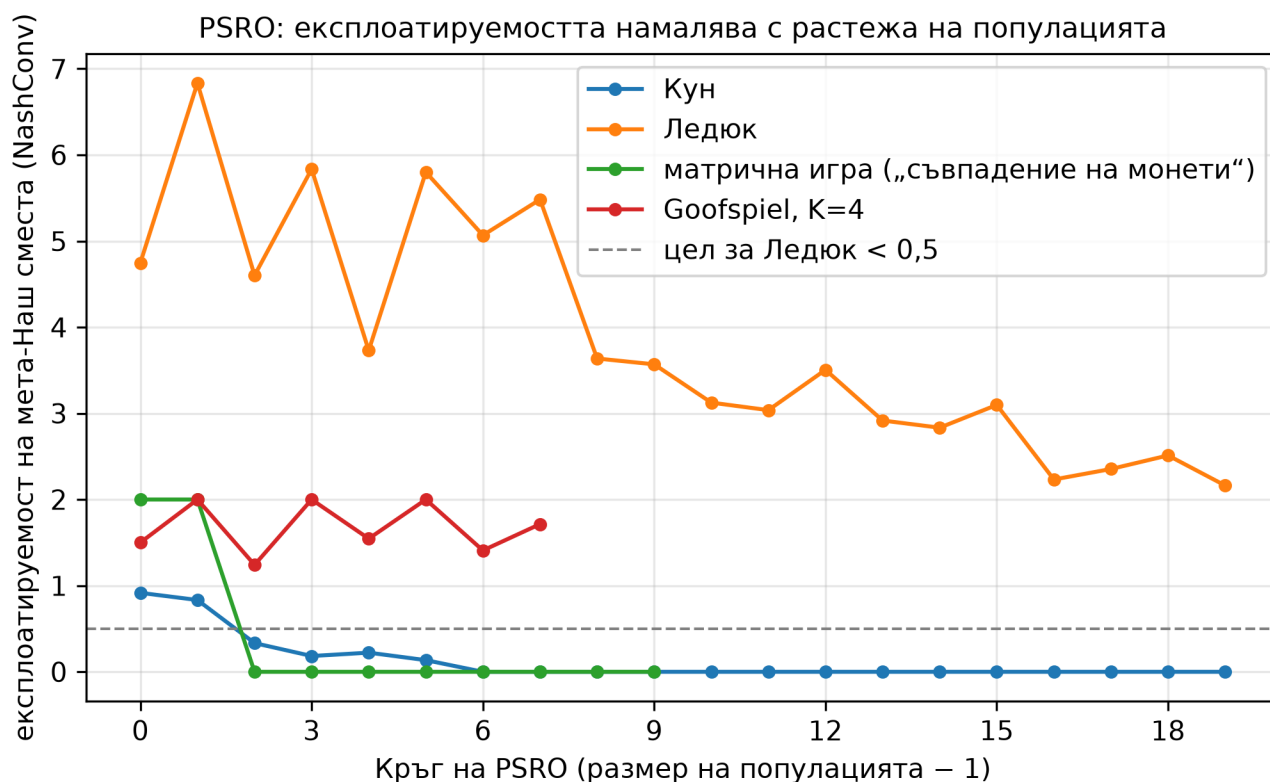
Резултатите за **Кун** / матрица / RPS са учебникови: при малките игри популацията бързо обхваща необходимите стратегии и експлоатируемостта рязко спада. Два резултата не се развиха според очакванията.¹³²

Съпоставка 1 (Ледюк). Прогнозирах, че PSRO ще намали експлоатируемостта на Ледюк под 0,5 в рамките на 20 итерации. Измерено, тя спадна от 4,75 до 2,16 - след силни колебания през първите осем кръга (до 6,83) следва ясно намаление, но стойността остава далеч от 0,5. Това е **истинска бавна сходимост**, а не грешка: Кун достигна машинна нула за 6 кръга, но игровото дърво на Ледюк е много по-голямо и популация от 20 *чисти* най-добри отговора просто е твърде малка, за да приближи добре неговото

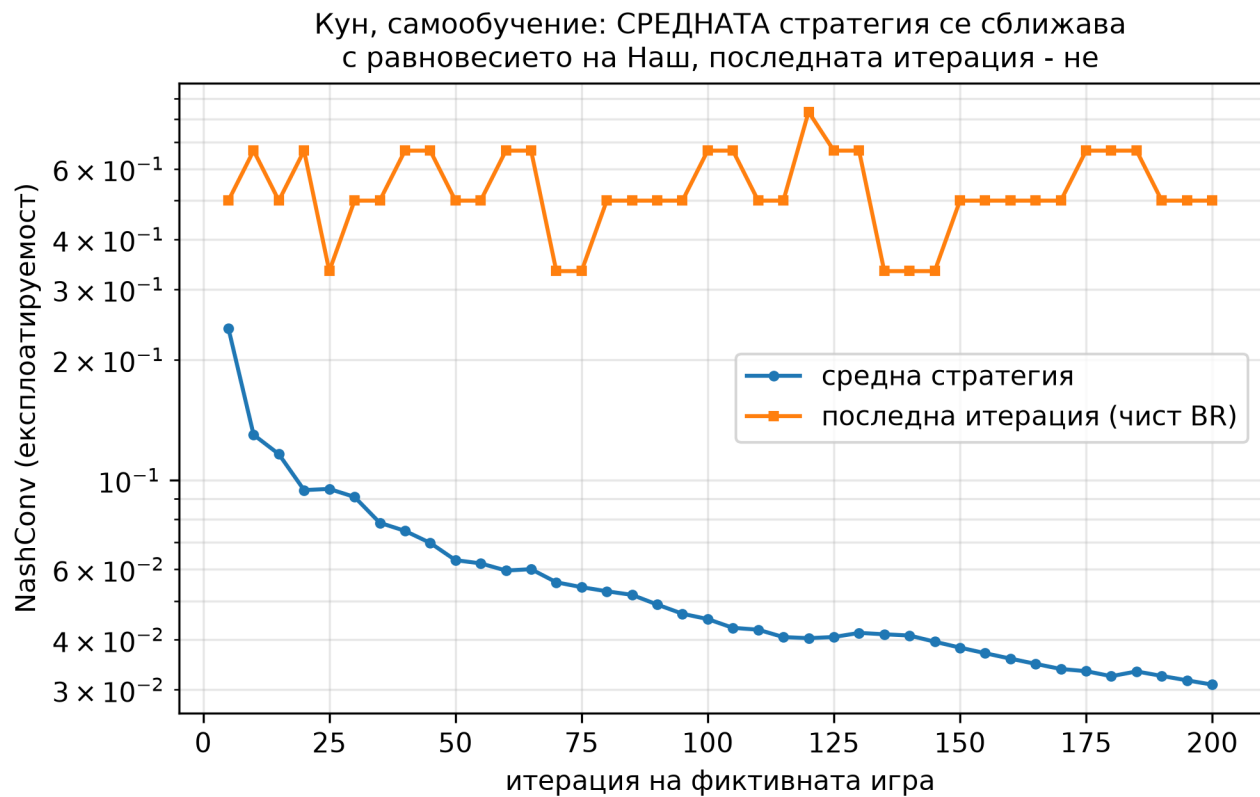
¹³²Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO). Свързани: McMahan, H. B., Gordon, G. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML* (методът на двойния оракул, който PSRO обобщава); Tuyls, K. et al. (2020). “Bounds and dynamics for empirical game theoretic analysis.” *Autonomous Agents and Multi-Agent Systems* 34(1), art. 7. DOI 10.1007/s10458-019-09432-y (EGTA).

смесено равновесие на Наш. Целта „ $< 0,5$ за 20“ беше оптимистична. Изводът – експлоатируемостта намалява с нарастването на популацията – остава валиден; *скоростта* е стената на мащабиране и тя се вписва в откритието от Глава 8 за глобално спрямо локално мащабиране.

Съпоставка 2 (Goofspiel $K = 4$). Прогнозирах ненарастваща експлоатируемост. При $K = 3$ (бърза проверка) тя се сближи до 0 напълно; при $K = 4$ тя осцилира между $\sim 1,2$ и $2,0$ и не се установява. Това е единственият резултат, който все още не мога да обясня напълно и съгласно работния процес аз го **документирам, а не го коригирам**. Две конкретни подозрения за последваща сесия: драйверът на PSRO за Goofspiel никога не премахва дублиращите се стратегии за най-добър отговор (така че мета-играта може да спре поради повторения) и популация от чисти стратегии вероятно е твърде слаба, за да представи смесения мета-Наш на по-голямата игра. Маркирано като отворен елемент в кода, а не като валидиран резултат.



Фигура 51: Експлоатируемост при PSRO спрямо размера на популацията в различни игри (конфигурация в пълен мащаб). При Кун и при матричната игра тя спада до (почти) нула за няколко кръга; при Ледюк силно се колебае през първите осем кръга, след което намалява, но след 20 кръга остава далеч над целта 0,5 (стената на мащабиране за популация от чисти стратегии); при Goofspiel с $K = 4$ се колебае, без да се установи (отбелязана аномалия). Точният оракул за най-добър отговор гарантира сходимост по принцип; размерът на играта определя скоростта.



Фигура 52: Самообучение в Кун: експлоатируемостта (NashConv) на средната стратегия постепенно спада към нула, докато тази на последната итерация продължава да се колебае. Затова самообучението и PSRO разчитат на осредняване по популацията, а не на последната стратегия.

9.7 Научена комуникация (CommNet)

CTDE централизира информацията по време на *обучение*; комуникацията я централизира по време на *изпълнение* чрез канал, който агентите **научават**, а не такъв, който дизайнерът изгражда. В **CommNet** (Sukhbaatar и др., 2016) всеки агент кодира своето наблюдение в скрит вектор, излъчва **съобщение**, и след това получава **средната стойност** на съобщенията на останалите агенти като допълнителен вход за своята стратегия. Цялото се обучава от край до край, така че протоколът възниква: в референциалната задача говорителят (който единствен вижда целта) трябва да се научи да я кодира, а слушателят – да я декодира.

Тестът е показателен, тъй като без канал слушателят е ограничен единствено до чисто отгатване, $1/K$. Измерено (конфигурация в пълен мащаб, $K = 5$, при което таванът при случайно отгатване е 0,2):

Таблица 31: Научена комуникация: отборна награда при включен и изключен канал.

Канал	Отборна награда (алчна стратегия)
с комуникация	0,795
без комуникация	0,204

С канала слушателят се издига далеч над тавана от $1/K$; без него той остава точно на тавана. Комуникацията върши реална работа – и обърнете внимание, че изводът от съпоставката в раздел 9.5 важи и тук: при конфигурацията за бърза проверка и двете числа бяха 0,24 (каналът все още не се беше научил да пренася информация), а ползата се появи едва след обучението в пълен мащаб.¹³³

9.8 LOLA – моделиране на противника като учещ се агент

Всеки метод досега разглежда стратегията на противника като фиксирана, докато агентът реагира. **LOLA** (обучение с осъзнаване за ученето на противника; Foerster и др., 2018) е изключението и е този, който е най-пряко свързан с дисертацията. Вместо да се оптимизира спрямо *настоящата* стратегия на противника, всеки агент се оптимизира спрямо стратегията, която противникът ще има *след една стъпка на обучение*, като диференцира *през* тази стъпка на обучение. Допълнителният член представлява смесена втора производна – как актуализацията на стратегията на противника зависи от *моите* параметри – и именно той превръща егоистичните агенти в кооперативни.

Класическата демонстрация е **повтарящата се дилема на затворника** с памет 1, при която стратегията на всеки агент се състои от пет вероятности за сътрудничество, а очакваната дисконтирана възвръщаемост има затворена форма чрез стационарната марковска верига върху двойките резултати. Наивните градиентни агенти, всеки от които максимизира собствената си възвръщаемост спрямо текущата стратегия на другия, стигат до взаимно предателство, докато агентите с **LOLA**, всеки от които отчита предстоящото обновяване на другия, достигат взаимно сътрудничество.

Измерено (дисконтирана възвръщаемост на стъпка; пълно сътрудничество ≈ 3 , взаимно предателство ≈ 1):

¹³³Sukhbaatar, S., Szlam, A. & Fergus, R. (2016). “Learning Multiagent Communication with Backpropagation.” *NeurIPS* (CommNet).

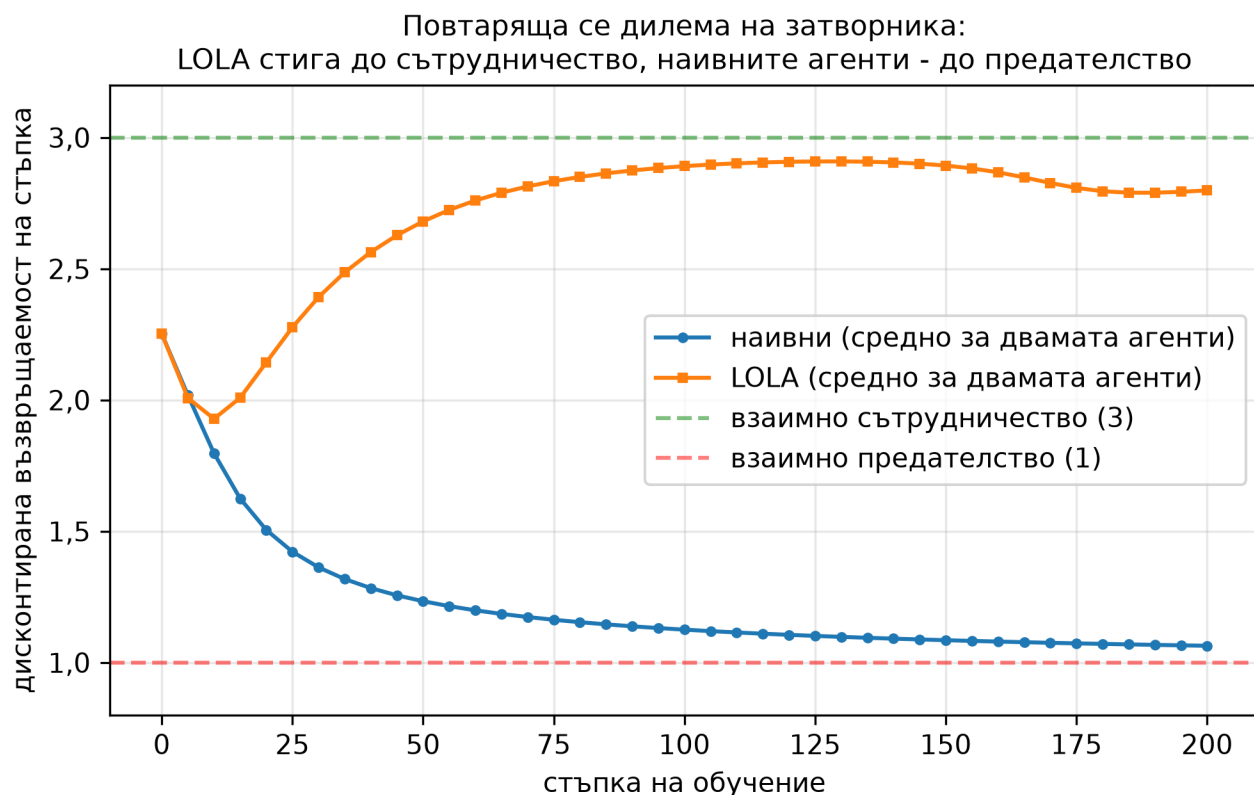
Таблица 32: LOLA срещу наивни обучаващи се агенти в повтарящата се дилема на затворника.

Обучаващи се	Възвръщаемост
наивни срещу наивни	1,04
LOLA срещу LOLA	2,82

Посоката е точно резултатът на LOLA – сътрудничеството възниква там, където наивното обучение води до предателство. (Моята прогноза за ≈ 3 беше малко завишена; измерената 2,82 е близка до пълно сътрудничество, а в изследователското изпълнение с по-голяма **скорост на обучение при предвиждане** двамата агенти достигнаха 2,67 и 2,93 (средно 2,80).) Вградена **проверка за коректност** потвърждава механизма: задаването на **скоростта на обучение при предвиждане** на нула кара **градиента** на LOLA да се редуцира точно до **наивния градиент**, така че сътрудничеството произтича именно от члена за **предвиждане** от втори ред.

Концептуално това представлява **динамично** моделиране на противника: Глава 7 извежда *текущата* стратегия на противника; LOLA предвижда неговата *траектория на обучение*. Съчетаването на двете – статична преценка, която задава началото на динамичното предвиждане – е кандидат за Принос №1, а не нещо вече решено.¹³⁴

¹³⁴Foerster, J., Chen, R. Y., Al-Shedivat, M., Whiteson, S., Abbeel, P. & Mordatch, I. (2018). “Learning with Opponent-Learning Awareness.” AAMAS, 122–130. arXiv:1709.04326 (LOLA).



Фигура 53: LOLA срещу наивни обучаващи се агенти в повтарящата се дилема на затворника: при наивните агенти възвръщаемостта на стъпка спада към взаимно предателство (~ 1), а при агентите с LOLA нараства към взаимно сътрудничество ($\sim 2,8$, средно за двамата агенти). Именно предвиждането на следващата стъпка на обучение на противника превръща динамиката от предателство в сътрудничество.

9.9 Честни бележки, ограничения и връзка със следващите глави

Какво се задържа. Потвърдените резултати са гръбнакът: независимото обучение се сближава в игрите с доминираща стратегия и в координационните игри и *се проваля* в „съвпадение на монети“; PSRO свежда експлоатируемостта до (почти) нула при Кун, матричната игра и RPS с точен оракул; самообучението се сближава в средното, но не и в последната итерация; остатъкът на централизирания оценител е с порядъци по-малък; научената комуникация издига слушателя над тавана при случайно отгатване; и LOLA превръща предателството в повтарящата се дилема на затворника в сътрудничество. Взети заедно, те проследяват предвидената дъга – от „защо наивното обучение се разпада“ до „структурните поправки, които го възстановяват“.

Какво не успя и защо това е от значение. Четири честни уговори остават в сила и за следващите глави. (1) При фиксирана стъпка траекторията в „съвпадение на монети“ се отклонява към границата, вместо да обикаля – по-остра версия на урока за несходимост. (2) PSRO при Ледюк се сближава *бавно* (експлоатируемост $\sim 2,16$ след 20 кръга, а не $< 0,5$) – стена на мащабиране на по-

пулацията с чисти стратегии. (3) Goofspiel $K = 4$ се колебае, вместо да се сближава – маркирана, необяснена аномалия в кода (документирана, но не поправена). (4) При играта „изкачване“ нито един метод не достигна оптимума и MADDPG се представи по-зле от независимите обучаващи се агенти – централизираният оценител намалява остатъка си, но сам по себе си не решава координатията, когато изследването е трудно. Двата елемента на кода (Goofspiel $K = 4$; контрафактична базова линия на MADDPG) трябва да бъдат разгледани, преди тези части да бъдат използвани отново. И един методологичен момент: ефектите при невронните мрежи бяха невидими при бързата проверка и се появиха едва в пълен мащаб, така че твърденията почиват именно на числата от конфигурацията в пълен мащаб.

Доверие. Всяка цел на равновесие е *точна* (аналитично равновесие на Наш за матричните игри; точен най-добър отговор и NashConv от Глава 7 за Кун/Ледюк/Goofspiel), така че игровитеоретичните резултати се ограничават от обективната истина, а не от други симулации. Невронните резултати представляват качествени неравенства (централизиран $<$ независим; с комуникация $>$ без комуникация), а не точни стойности, и по конструкция са чувствителни към началното число и версията. Графиките на експериментите по-горе са генерирани от JSON файловете с резултати, записани в хранилището (вж. `../figures/README.md`).

Връзки с предходните и следващите глави. Назад: PSRO представлява итерираният най-добър отговор от Глава 2, издигнат до ниво популация, и използва изцяло модула за точен най-добър отговор от Глава 7; стената на мащабиране при Ледюк отеква откритието глобално срещу локално от Глава 8. Напред: трите връзки с дисертацията вече са конкретни – LOLA като динамично моделиране на противника (Принос №1), мета-играта на PSRO като методология за оценяване (Принос №3) и, преди всичко, **липсващата минимаксна опора при $N > 2$** (Принос №2), единственото място, където престава да важи всяка гаранция за игри с двама играчи от глави 2–8.

9.10 Основни изводи за синтеза на дисертацията

- **Нестационарността е основният проблем и тя е структурна, а не въпрос на изчислителен бюджет** – в „съвпадение на монети“ последната итерация не се сближава, колкото и дълго да продължи обучението.
- **CTDE централизира оценителя по време на обучението и децентрализира актьора при изпълнението**; измерено е, че централизираният оценител напасва целевата си стойност почти без остатък ($3,2 \times 10^{-11}$ спрямо 0,077) – но по-точният оценител не реши играта „изкачване“: той не е достатъчен за координатия.
- **PSRO е мостът между теорията на игрите и MARL**: измерено е, че довежда Кун до експлоатируемост, равна на машинната нула, а RPS – до равномерна смес; Ледюк намалява, но достига стена на мащабиране – същата стена, която мотивира мащабируемите методи в дисертацията.
- **Самообучението и PSRO успяват чрез средното и популацията, а не чрез последната**

итерация (средната NashConv на Кун $0,24 \rightarrow 0.031$, докато последната итерация се колебае) – причината да съществува популация.

- **LOLA разглежда противника като учещ се агент** и превръща предателството в IPD в сътрудничество ($1,04 \rightarrow 2.82$) – динамичното допълнение към статичния модел на противника от Глава 7 (Принос №1).
- **Липсата на минимаксна стойност при $N > 2$ е отворената врата към дисертацията** (Принос №2): речникът на глави 2–8 преминава в MARL, но опората на безопасността – не.

10 Глава 10 – Обучение на базата на популации и еволюционна теория на игрите

Тази глава изгражда от основите темата за обучение и оценка на **популации** от агенти: еволюционната математика, която определя дали дадена популация изобщо може да се установи в равновесие, транзитивната/циклична структура, която решава дали процесът на самообучение ще се сближи или ще се завърти, и малка **лига** в стила на AlphaStar, изградена върху решим вариант на покер. Написана е така, че да може да се чете независимо от останалите глави. **Всички експериментални резултати, представени тук, са получени** от възпроизводими изпълнения и, когато е възможно, са съпоставени с *точни* еталонни стойности (аналитичните ESS и равновесия на Наш за матричните игри; точната експлоатируемост спрямо най-добър отговор от Глава 7 за Ледюк). Когато дадено изпълнение противоречи на очакванията, произтичащи от теорията, запазвам първоначалното теоретично предвиждане и го съпоставям с получените резултати – именно тези разминавания са най-показателните части от главата.

Място в дисертацията. Глави 2–8 решават *една* фиксирана игра; Глава 9 добавя *втори* обучаващ се агент. Глава 10 преминава към *цяла популация*, която се обучава срещу себе си. Тя има три връзки с приносите на дисертацията. **Основните експлоататори** на практика са автоматизирани модели на противника – статичната преценка за противника от Глава 7, пренесена на ниво популация (Принос №1). **Лигата на AlphaStar** използва експлоататори, за да направи основните агенти устойчиви – механизъм, близък до безопасната експлоатация на ниво популация, но без формална гаранция: авторите обосновават подхода с фиктивната игра срещу себе си, чиято смес се сближава към равновесие на Наш при игри за двама с нулева сума, а устойчивостта на лигата показват само емпирично¹³⁵. Точно тук се намира Принос №2. А EGTA / **мета-Наш** е методологията за оценяване на популацията (Принос №3).

10.1 Защо популации – и защо структурата определя всичко

Самообучението има известен **режим на отказ**: играч, който тренира единствено срещу самия себе си, може да се върти в кръг. Ако играта е **камък-ножица-хартия**, „да станеш по-добър“ няма смисъл – камък побеждава ножица, губи от хартия и подобряването срещу един противник просто те прави по-лош срещу друг. Най-важната идея в тази глава е, че игрите имат два вида структура, смесени заедно: **транзитивна част** (истинска **стълбица на уменията** – съществува *по-добър играч*) и **циклична част** (**кръг от контрастратегии** – има само *какво побеждава какво*). Дали една **самообучаваща се популация** ще се сближи до нещо добро или ще се върти вечно, зависи от това *коя част е доминираща*, а не от **скоростта на обучение**.

Една представа, която да запомните: **доджо**. **Основните ученици** са това, което се опитвате да

¹³⁵Vinyals, O. et al. (2019). “Grandmaster level in StarCraft II using multi-agent reinforcement learning.” *Nature* 575(7782), 350–354. DOI 10.1038/s41586-019-1724-z (AlphaStar; лигата и PFSP).

създадете. Поддържате зала, пълна със **спаринг партньори**, чиято единствена задача е да откриват и наказват конкретна слабост в настоящите ученици (експлоататори). И поддържате **музей на минали шампиони** – замразени „снимки“ – така че никой да не печели, като забрави как да победи стар стил. Обучението тогава представлява цикъл: учениците тренират помежду си, по-силните се копират и леко се изменят (**обучение на базата на популации**), а периодично копие на всички се замразява в музея. Лигата е това доджо, превърнато в математическа форма.¹³⁵

10.2 Семейството от методи

Всеки от разгледаните методи представлява различен начин за превръщане на **популацията** в тренировъчен сигнал или средство за оценка.

Таблица 33: Семействата методи, базирани на популации: какво прави всеки, кога да се използва и основната му слабост.

Подход	Какво прави	Кога да се използва	Основна слабост
Самообучение	обучава последната стратегия срещу (копие на) самата себе си	транзитивни игри; бърза базова линия	зацикля и забравя в цикличните игри; последната итерация не се сближава
PSRO	популация + мета-Наш + оракул за най-добър отговор; добавя най-добрия отговор, повтаря	състезателни игри; търси се игровитеоретична цел	пълен най-добър отговор на всеки кръг; <i>сместа</i> не е решението (вж. §10.7)
PBT	обучава много агенти; копира по-приспособените (експлоатация) + променя хиперпараметри (изследване)	всяка популация; онлайн търсене на хиперпараметри	може да загуби разнообразието си или да регресира (вж. §10.5)

Подход	Какво прави	Кога да се използва	Основна слабост
Лига в стила на AlphaStar	PBT с три типа агенти (основни агенти / основни експлоататори / експлоататори на лигата) + замразяване + PFSP	нетранзитивни игри, където наивното самообучение за цикля	много агенти, големи изчислителни разходи; евристичен метод без гаранция за безопасност

Два инструмента за оценка са общи за всички тях. **Репликаторната динамика** (§10.3) е непрекъснатата идеализация на подбора – тя показва накъде се движи една популация и къде може да се установи. **Декомпозицията „пумпал“** (§10.4) измерва доколко една игра (или популация) е транзитивна или циклична. Заедно те представляват диагностика, която предсказва, *преди* да бъдат направени изчислителните разходи, дали лигата от §10.5 ще се установи или ще се завърти.

Исторически линията е следната: обучение на базата на популации като онлайн търсене на хиперпараметри (PBT, 2017)¹³⁶ → теория на игрите върху популация от стратегии (PSRO, 2017)¹³⁷ → лигата с три типа агенти, достигнала ниво „гросмайстор“ в StarCraft II – над 99,8% от класираните човешки играчи (AlphaStar, 2019)¹³⁵ → геометрията, която обяснява *защо* е необходима популация (транзитивно-циклична декомпозиция, 2018-2019; „пумпал“, 2020)¹³⁸. Такива популации се оценяват с емпиричен теоретико-игрови анализ (EGTA)^{139 140}.

10.3 Репликаторна динамика – къде една популация може да се установи

Репликаторното уравнение е най-простият модел на подбора: делът на дадена стратегия нараства пропорционално на това колко тя превъзхожда средната за популацията.

$$\dot{x}_i = x_i [f_i(x) - \bar{f}(x)], \quad \bar{f}(x) = \sum_j x_j f_j(x).$$

Всяко равновесие на Наш е точка на покой, но не и обратното – например всяка чиста стратегия

¹³⁶Jaderberg, M., Dalibard, V., Osindero, S., Czarnecki, W. M. et al. (2017). “Population Based Training of Neural Networks.” *arXiv:1711.09846*.

¹³⁷Lancot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO). Вж. също: McMahan, H. B., Gordon, G. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML*, 536–543 (методът на двойния оракул, който PSRO обобщава).

¹³⁸Czarnecki, W. M., Gidel, G., Tracey, B., Tuyls, K., Omidshafiei, S., Balduzzi, D. & Jaderberg, M. (2020). “Real World Games Look Like Spinning Tops.” *NeurIPS*; arXiv:2004.09468 – геометрията „пумпал“ и защо обучението изисква популации.

¹³⁹Wellman, M. P. (2006). “Methods for Empirical Game-Theoretic Analysis.” *AAAI*, 1552–1555.

¹⁴⁰Tuyls, K., Pérolat, J., Lancot, M., Hughes, E., Everett, R., Leibo, J. Z., Szepesvári, C. & Graepel, T. (2020). “Bounds and dynamics for empirical game theoretic analysis.” *Autonomous Agents and Multi-Agent Systems* 34(1), 7. DOI 10.1007/s10458-019-09432-y.

също е точка на покой. Устойчивите точки на покой обаче винаги са равновесия на Наш, а всяка **еволюционно стабилна стратегия (ESS)** – чиста или смесена стратегия, която, възприета от цялата популация, не може да бъде изместена от малък дял мутанти – е асимптотично устойчива точка на покой¹⁴¹. Това е непрекъснатата идеализация на онова, което PBT лигата прави дискретно: копира по-успешните агенти (подбор) и внася малки случайни промени в тях (мутация).



пунктир = „дискретен аналог на“: лигата е репликаторна динамика с обучаващи се агенти вместо фиксирани дялове на стратегиите

Фигура 54: Еволюционната теория на игрите като непрекъсната идеализация на PBT лигата: подборът в репликаторната динамика съответства на копирането на по-успешните агенти, мутацията (дрейфът) – на промяната на скоростта на обучение и на ентропията, а устойчивите точки на покой (ESS) – на мета-Наш сместа, към която се стреми лигата. Лигата е репликаторна динамика, в която вместо фиксирани дялове на стратегиите участват обучаващи се агенти.

При четири канонични симетрични игри (динамиката е детерминистична, затова резултатът е

¹⁴¹Hofbauer, J. & Sigmund, K. (2003). “Evolutionary game dynamics.” *Bulletin of the American Mathematical Society* 40(4), 479–519 – т.нар. „фолклорна теорема“ за репликаторната динамика (§2.3), асимптотичната устойчивост на ESS (§2.6) и затворените орбити при „камък-ножица-хартия“ (теорема 2).

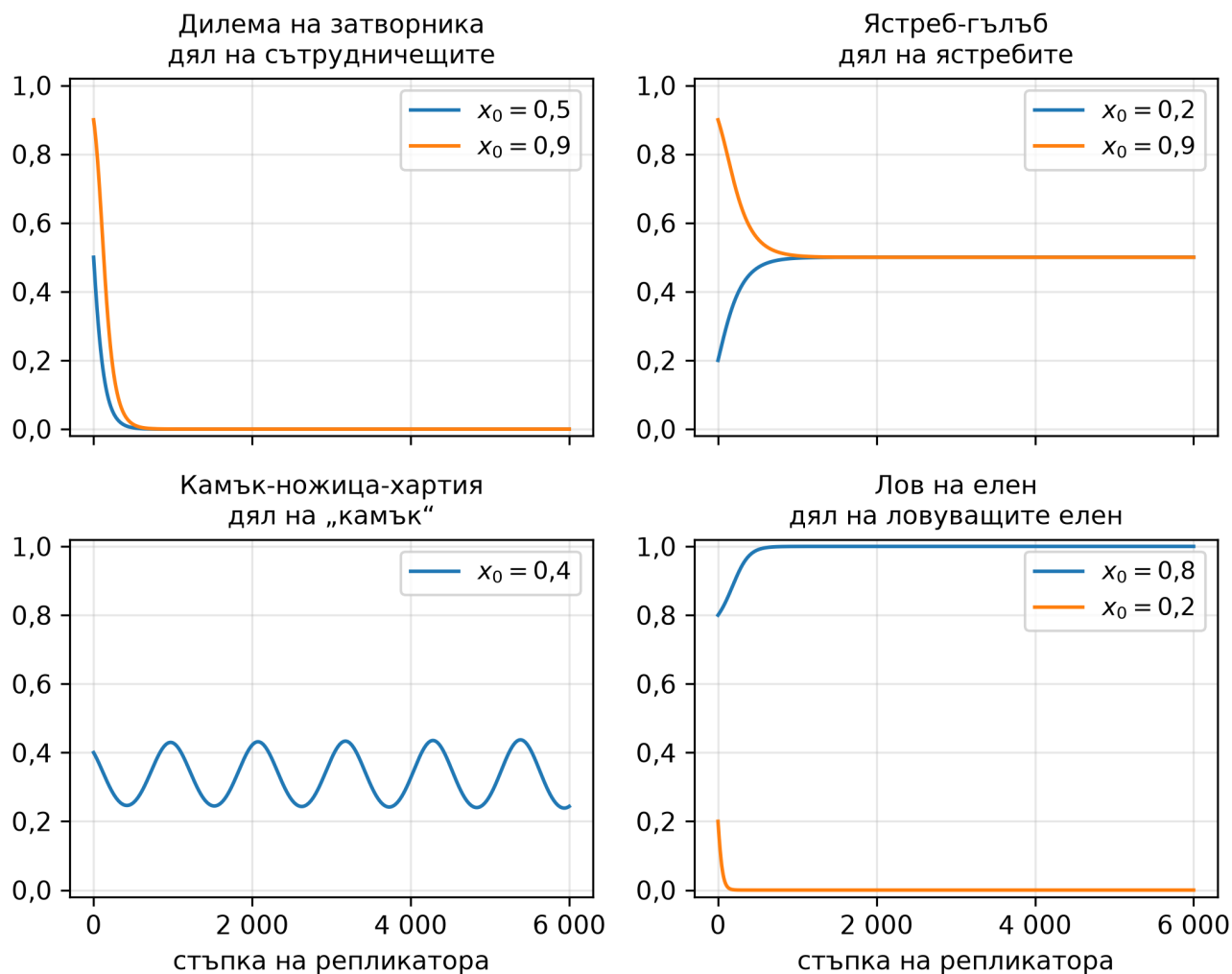
еднозначен) измерените резултати съвпадат напълно с теорията:

Таблица 34: Репликаторна динамика върху четири симетрични игри, спрямо аналитичния еталон.

Игра	Аналитичен еталон	Измерено крайно състояние	Сходим?
Дилемата на затворника	предателството доминира; ESS (0, 1)	[0,0, 1.0]	да
Ястреб-гълъб	вътрешна ESS $p(\text{ястреб}) = V/C = 0,5$	[0,5, 0.5] (орбитален радиус 0,0)	да
Камък-ножица-хартия	равновесие на Наш = равномерна стратегия; няма ESS	обикаля около центъра на разстояние 0,095	не
Лов на елен	две чисти ESS (според басейна на привличане)	[0,8, 0.2] $\rightarrow$ [1, 0]; [0,2, 0.8] $\rightarrow$ [0, 1]	да

Дилемата на затворника се свежда до доминиращата стратегия; ястреб-гълъб *достига* вътрешната 0,5 ESS; ловът на елен избира различна чиста ESS в зависимост от басейна на привличане, в който попада началната точка. **Камък-ножица-хартия никога не се сближава** – вътрешната ѝ неподвижна точка е *център*, така че популацията обикаля около нея завинаги (в непрекъснато време орбитите са затворени; явният метод на Ойлер ги превръща в бавно разширяваща се спирала). Този последен ред е цялата мотивация за останалата част от главата: цикличната игра няма стабилна популация.¹⁴²

¹⁴²Hofbauer, J. & Sigmund, K. (1998). *Evolutionary Games and Population Dynamics* (Cambridge) – репликаторна динамика, ESS и центъра при „камък-ножица-хартия“; вж. също обзора на Bloembergen, Tuyls, Hennes & Kaisers (JAIR 53, 659–697, 2015), който свързва репликаторната динамика с многоагентното обучение.

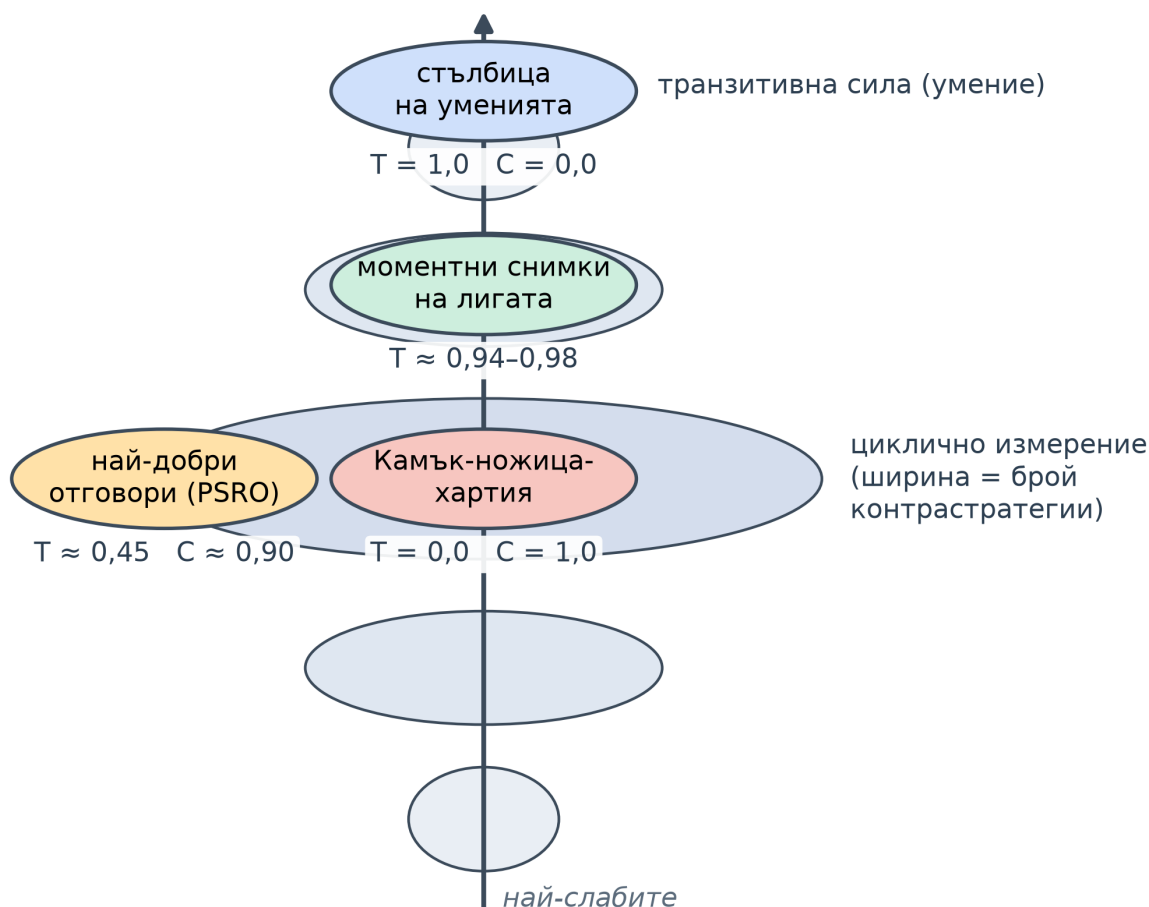


Фигура 55: Траектории на репликаторната динамика (дял на първата стратегия във времето): в дилемата на затворника популацията стига до всеобщо предателство, в „ястреб-гълъб“ – до вътрешната ESS с дял 0,5 на ястребите, в „камък-ножица-хартия“ обикаля около центъра, без да се сближава, а в „лов на елен“ стига до „всички ловуват елен“ или „всички ловуват заек“ в зависимост от началната точка. Тази несходяща орбита показва защо популациите в циклични игри се нуждаят от специални механизми, за да не се въртят в кръг.

10.4 „Пумпалът“ – транзитивна и циклична структура

Ако цикличните игри са проблемът, необходимо е да *измерим* степента им на цикличност. **Декомпозицията „пумпал“** разделя матрицата на печалбите на една игра на **транзитивна** компонента (стълбица на уменията, изразена чрез рейтинг на всяка стратегия) и **циклична** компонента

(остатъкът – частта „камък-ножица-хартия“) ¹⁴³. Името идва от формата, която Czarnecki и др. откриват при реални игри: те са най-широки (най-циклични) при стратегиите със средна сила и се стесняват към крайностите на уменията ¹³⁸.



една и съща игра, различни популации: популацията от най-добри отговори (PSRO) е в широката циклична част, а популацията от моментни снимки (лигата) се изкачва по транзитивната ос.

Фигура 56: „Пумпалът“ (схема): вертикалната ос е транзитивната сила (умението – съществува по-добър играч), а ширината е цикличното измерение (броят на контрастратегии, структурата „камък-ножица-хартия“). „Камък-ножица-хартия“ е в най-широката циклична част (транзитивно съотношение 0,0), чистата стълбица на уменията е върху транзитивната ос (1,0), мета-играта на най-добрите отговори в PSRO за Ледюк е в широката циклична част (транзитивно съотношение $\sim 0,45$), а мета-играта на моментните снимки на лигата се изкачва по транзитивната ос ($\sim 0,94-0,98$). Т – транзитивно, С – циклично съотношение.

Една тънкоост заслужава внимание: първоначалният план предлагаше декомпозиция чрез **SVD** с

¹⁴³Balduzzi, D., Tuyls, K., Pérolat, J. & Graepel, T. (2018). “Re-evaluating Evaluation.” *NeurIPS*; arXiv:1806.02643 – разлагане на Ходж на антисиметричната матрица на печалбите на транзитивна и циклична компонента; усредняване по Наш.

ранг 1, която неправилно дава транзитивно съотношение ≈ 0.707 за „камък-ножица-хартия“ – сингулярните стойности на антисиметрична матрица винаги са по двойки, затова приближението с ранг 1 обхваща най-много половината от квадрата на нормата ѝ при всяка игра. Реализацията използва вместо това комбинаторното разлагане на Ходж (по разлики в рейтингите)¹⁴³, което правилно дава транзитивно съотношение 0,0 за „камък-ножица-хартия“. Измерено:

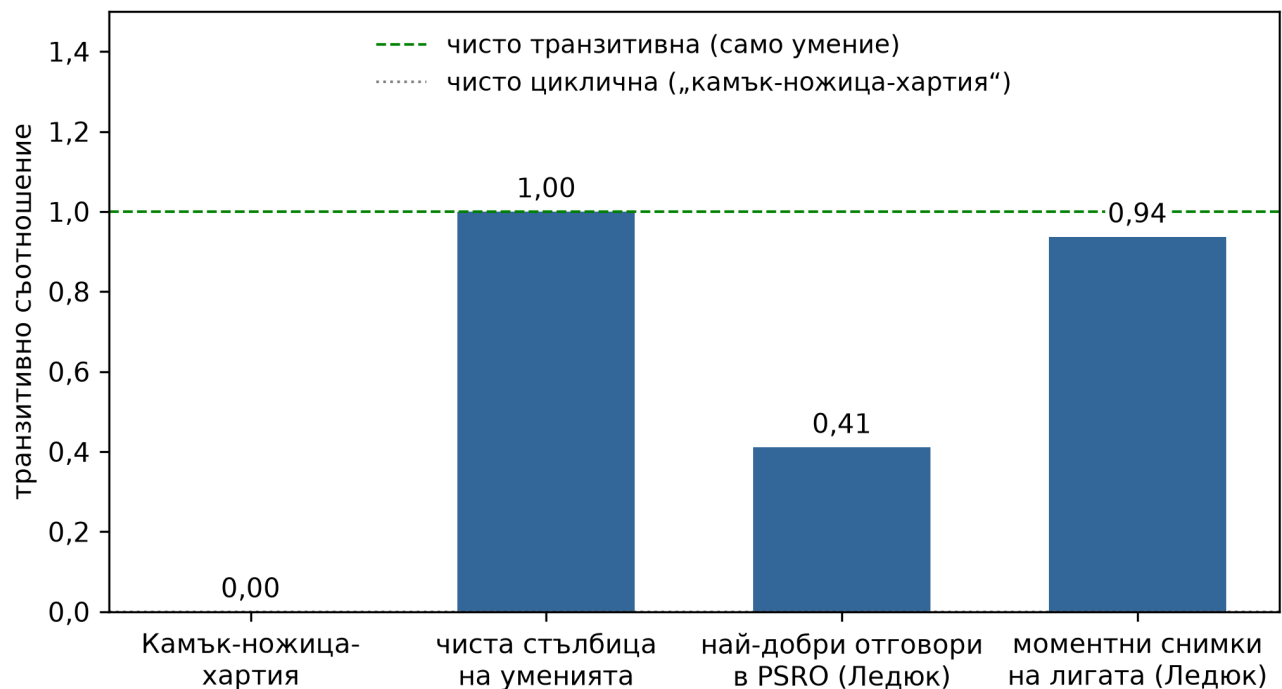
Таблица 35: Транзитивни и циклични компоненти на четири популации чрез комбинаторна декомпозиция на Ходж.

Популация	Транзитивна компонента (Ходж)	Циклична компонента	Структура
камък-ножица-хартия	0,0	1,0	чисто циклична
стълбица на уменията	1,0	0,0	чисто транзитивна
мета-игра на най-добрите отговори в PSRO (Ледюк)	0,41–0,46	0,89–0,91	предимно циклична (27 цикъла от три стратегии)
мета-игра на моментните снимки на лигата	0,94–0,98	–	предимно транзитивна

Двете *реални* популации са основният извод – и те си противоречат в рамките на една и съща игра.¹⁴⁴

Съпоставка (запазена прогноза → какво всъщност се случи). Бях приел, че покерът е стълбица на уменията, и очаквах мета-играта на Ледюк да бъде предимно транзитивна. Измерванията показват, че тя зависи изцяло от това *коя популация се разлага*. Популация от **най-добри отговори** (PSRO) е предимно **циклична** (транзитивно съотношение ≈ 0.45 , 27 цикъла от три стратегии): най-добрият отговор побеждава текущата смес, по-нов най-добър отговор побеждава *него*, и така нататък – „пумпалът“ на Czarnecki и др. в действие. Популация от **моментни снимки на траекториите на обучение** (лигата) е предимно **транзитивна** (≈ 0.94 –0,98), защото по-късните моментни снимки обикновено са по-силни от по-ранните, образувайки стълбица. Нито едното не е грешка; съотношението транзитивно/циклично е свойство на *популацията*, а изборът как да изградите популацията е избор дали ще видите кръг или стълбица.

¹⁴⁴Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML* – разлагане на функционалните игри на транзитивна и циклична част; $\text{PSRO}_{\tau N}$.

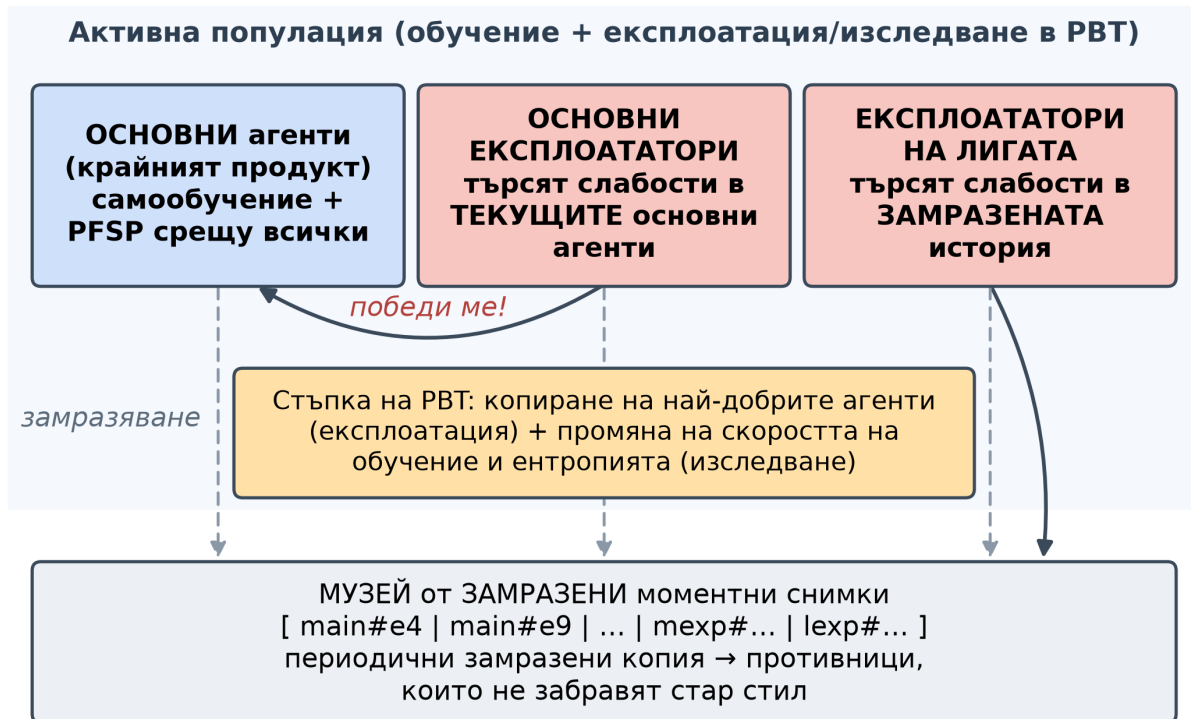


Фигура 57: Транзитивно съотношение на четири популации: „камък-ножица-хартя“ (0,0, чисто циклична), чиста стълбица на уменията (1,0), мета-играта на най-добрите отговори в PSRO за Ледюк (~0,41-0,46, предимно циклична) и мета-играта на моментните снимки на лигата (~0,94-0,98, предимно транзитивна). Една и съща игра има противоположна структура в зависимост от това как е изградена популацията.

10.5 PBT лига в стила на AlphaStar

С помощта на диагнозата изграждаме доджото. Лигата обучава **невронни** PPO агенти на **Ледюк Холдем** с три типа агенти: **основни** агенти (продуктът; те играят срещу всички чрез PFSP самообучение), **основни експлоататори** (търсят слабости в *текущите* основни агенти) и **експлоататори на лигата** (търсят слабости навсякъде в *замразената история*). Обучението на базата на популации копира най-добрите агенти (експлоатация) и променя тяхната **скорост на обучение** / **ентропия** (изследване); периодически се добавя **замразена моментна снимка** на всеки агент в музея; вдвояването по **PFSP** (приоритизирана фиктивна игра срещу себе си, Prioritized Fictitious Self-Play) избира всеки противник с вероятност, която нараства според това колко е трудно да бъде победен. От решаващо значение е всяка **невронна мрежа** да бъде извлечена до **таблична** стратегия, така че **точният** най-добър отговор от Глава 7 да измерва нейната експлоатируемост – същият NashConv, използван още от Глава 2 (NashConv е сумата от печалбите на двата най-добри отговора; в конвенцията на OpenSpiel експлоатируемостта е половината от нея).

Сдвояване по PFSP: всеки противник се избира с вероятност, която расте с трудността да бъде победен (обучението се съсредоточава там, където агентът губи).



Фигура 58: Лигата: основни агенти (крайният продукт), основни експлоататори (търсят слабости в текущите основни агенти) и експлоататори на лигата (търсят слабости в цялата замразена история), с периодично замразяване на копия в „музей“ от моментни снимки и сдвояване по PFSP, което насочва обучението към противниците, срещу които агентът губи. PBT копира най-добрите агенти и променя хиперпараметрите им.

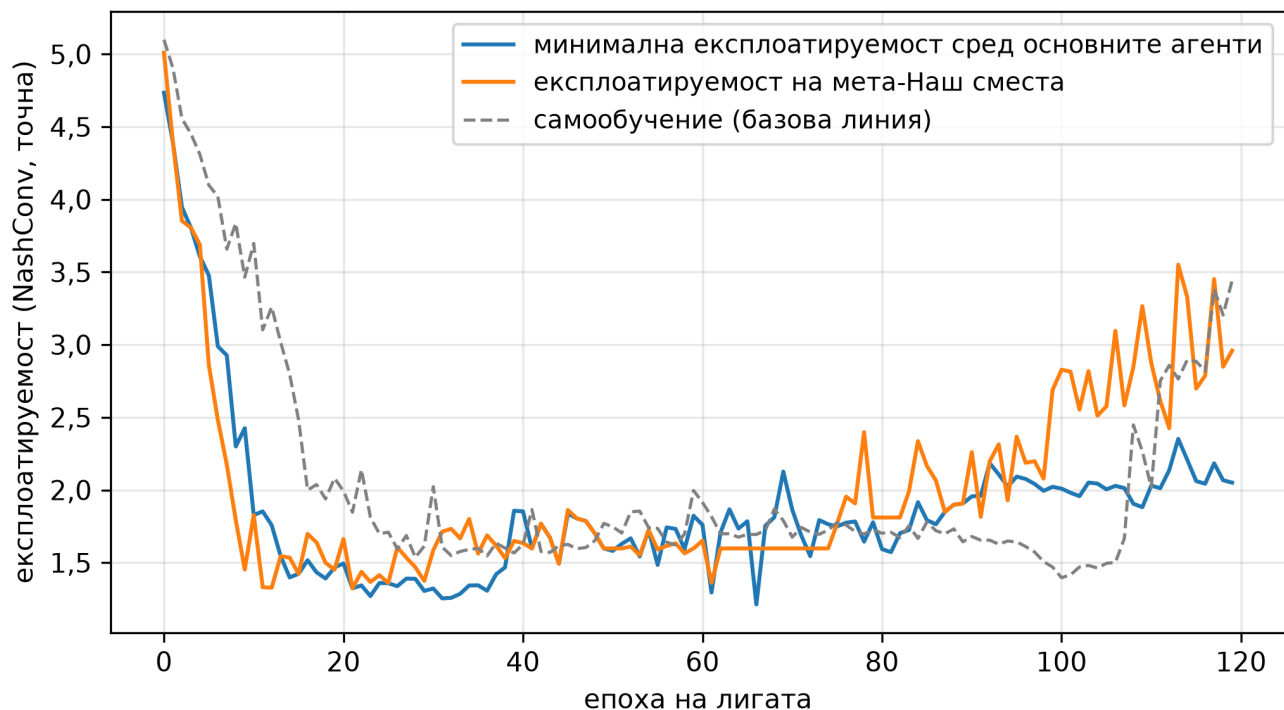
Подобрява ли се лигата? Резултатите от двете конфигурации (бърза проверка: 7 агента, 15 епохи; пълен мащаб: 8 агента, 120 епохи, 48 замразени моментни снимки) са следните:

Таблица 36: Показатели от обучението на лигата при двата мащаба.

Показател	Бърза проверка	Пълен мащаб
минимална експлоатируемост сред основните агенти	4,67 → 3,04 (най-ниската стойност е в края)	4,73 → мин $\approx 1.21 \rightarrow 2,05$
експлоатируемост на мета-Наш сместа	4,73 → 3,04	5,01 → мин ≈ 1.32 (плато 1,60) $\rightarrow 2,96$

(Стойността 2,96 е за мета-Наш сместа на популацията в началото на епоха 119, преди последната стъпка на обучение; стойността 3,418 в §10.7 е за крайната популация от 56 агента.)

Съпоставка (запазена прогноза → какво всъщност се случи). Очаквах *монотонно* намаляване на експлоатируемостта. При бързата проверка (15 епохи) очакването се потвърждава: спадът е плавен и най-ниската стойност е в края. При пълния мащаб (120 епохи) картината е друга: експлоатируемостта спада рязко през първите около 15 епохи и е най-ниска около епохи 20-40 (мета-Наш сместа достига ≈ 1.32 в епоха 21, а най-добрата моментна снимка е от епоха 29; единичният минимум сред основните агенти, ≈ 1.21 , е в епоха 66). След плато (≈ 1.60 за мета-Наш сместа) тя **нараства отново** до $\approx 2.05 / \approx 2.96$ в епоха 119. Най-добрите агенти са *замразените моментни снимки* от първата половина на обучението; *активните* основни агенти се влошават към края – стратегиите им непрекъснато се сменят и част от наученото се забравя. Самообучението, което няма експлоататори, също се влошава след епоха 100, така че натискът на експлоататорите не е единственото обяснение. Това се вижда само при достатъчно дълго обучение: лигата не се подобрява монотонно. Възможното решение (непроверено тук) е запазване на най-добрата моментна снимка или регуляризация на популацията. Методологично това повтаря извода от Глава 9: **пълният мащаб разкрива това, което бързата проверка скрива.**



Фигура 59: Експлоатируемост (NashConv) на лигата за 120 епохи (пълнен мащаб): минималната експлоатируемост сред основните агенти и тази на мета-Наш сместа спадат рязко през първите около 15 епохи, най-ниски са около епохи 20-40 и отново нарастват в края на обучението. Замразените моментни снимки запазват силните агенти от първата половина на обучението, а активните агенти се влошават към края. Пунктирната сива линия е самообучението (базова линия), което също се влошава след епоха 100.

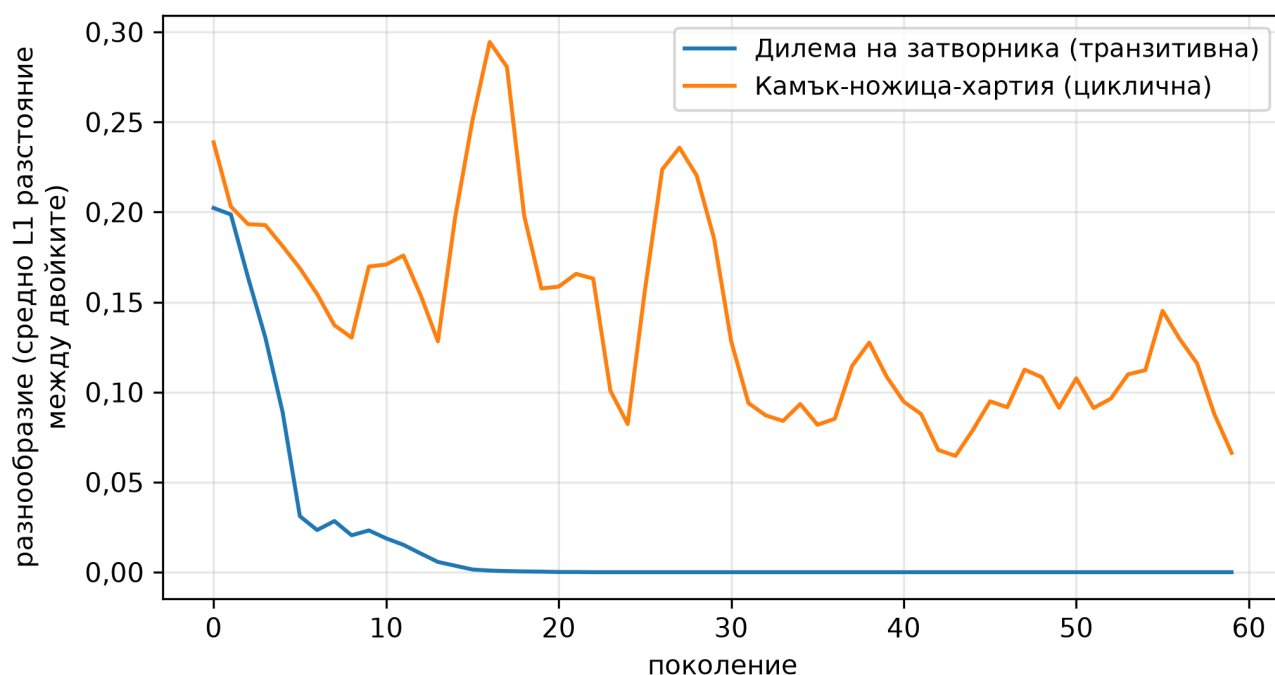
10.6 Разнообразие – наистина ли популацията е разнообразна?

Лигата е толкова добра, колкото и *разнообразието* от стратегии, които съдържа. Измерваме три показателя: **ефективен размер на популацията** (коефициентът на участие на мета-Наш теглата), **поведенческа клъстеризация** (наистина ли стратегиите са различни?) и **покриване на експлоатация**. Резултатите показват, че популацията е само слабо разнообразна: коефициентът на участие нараства от 1,0 (бърза проверка) до 1,9 (пълнен мащаб), но поведенческата клъстеризация свежда всичко до **един клъстер** и при двата мащаба – дори при пълнен мащаб, където максималното поведенческо разстояние между двойки (0,48) надвишава прага за клъстеризация (0,30), методът на единичното свързване обединява агентите във верига. Разнообразието тук е *на ниво тегла* (мета-Наш сместа разпределя теглото си между няколко агента), а не *на ниво поведение* (агентите играят почти еднакво).

Механизмът се проявява най-ясно в опростена среда – мини-RBT върху матрични игри:

Таблица 37: Разнообразие при мини-PBT през поколенията – върху транзитивна и върху циклична игра.

Игра	Разнообразие през поколенията
Дилемата на затворника (транзитивна)	спада до 0 (всички → предателство)
Камък-ножица-хартия (циклична)	непрекъснато се променя (0,07–0,29), никога не се установява



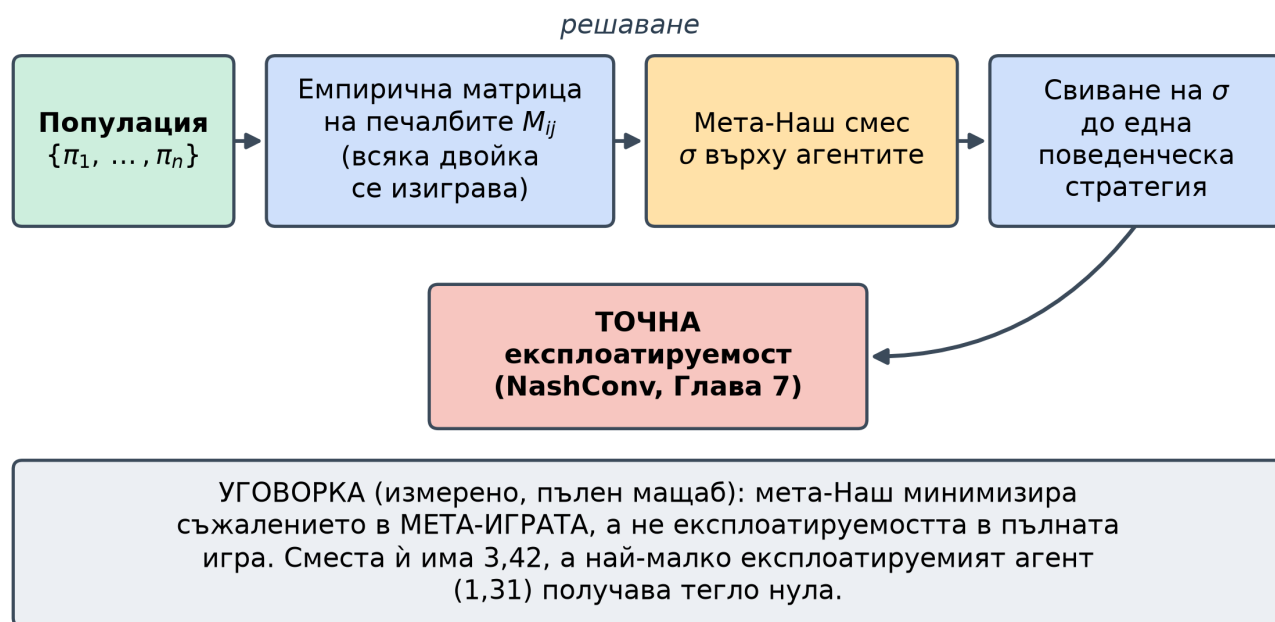
Фигура 60: Разнообразие при мини-PBT: в транзитивната дилема на затворника популацията се свежда до една-единствена стратегия (разнообразие $\rightarrow 0$), а в цикличната игра „камък-ножица-хартия“ стратегиите непрекъснато се сменят (0,07–0,29), докато популацията следва кръга от контрастратегии. Дали разнообразието ще се запази, зависи от структурата на играта.

Транзитивните игри *унищожават* разнообразието (има една най-добра стратегия и всички се сближават към нея), а цикличните игри го *налагат* (най-добра стратегия няма, затова популацията непрекъснато се променя). Лигата в Ледюк е близо до транзитивния край – затова разнообразието ѝ е ниско и затова ползите от разнообразието, описани за AlphaStar, където по време на обучението са създадени близо 900 различни агента, не се проявяват в този мащаб.¹³⁵

10.7 EGTA – оценка на популацията и неочаквано откритие

Последният инструмент е **емпиричният теоретико-игрови анализ (EGTA)**: цели агенти (стратегии за цялата игра) се разглеждат като „чисти стратегии“ на игра от по-високо ниво, всяка двойка агенти се изиграва, за да се попълни емпиричната матрица на печалбите, намира се **мета-Наш** сместа на тази матрица и тя се оценява. Така експлоатируемостта се пренася на ниво популация: търси се равновесие на Наш на игра, чиито „атоми“ са цели стратегии.

EGTA = равновесие на Наш на игра, чиито „стратегии“ са цели агенти – експлоатируемостта, пренесена на ниво популация.



Фигура 61: Процесът на EGTA: всяка двойка агенти се изиграва, за да се построи емпирична матрица на печалбите, намира се мета-Наш сместа, тя се свежда до една поведенческа стратегия и се измерва ТОЧНАТА ѝ експлоатируемост в пълната игра. Измерената уговорка: мета-Наш минимизира съжалението в мета-играта, а не експлоатируемостта в пълната игра, затова сместа може да е по-експлоатируема от най-добрия член на популацията, който дори може да не участва в нея.

Очакването (от първоначалния план) беше, че **мета-Наш** сместа на **лигата** ще бъде *по-малко* експлоатируема от всеки отделен член. Измерено:

Таблица 38: Експлоатируемост на мета-Наш сместа спрямо най-добрия отделен агент, по конфигурация.

Конфигурация	експлоатируемост на мета-Наш сместа	най-добър отделен агент	мета-Наш $\leq$ най-добрия?
Бърза проверка	2,665	2,665	да (цялото тегло е върху най-добрия агент)
Пълен мащаб	3,418	1,305	не

Съпоставка (запазена прогноза → какво всъщност се случи). Очаквах **мета-Наш сместа** да бъде поне толкова **неексплоатируема**, колкото най-добрият член на популацията. Бързата проверка го потвърди тривиално: **мета-Наш** даде *цялото* тегло на единствения най-добър агент, така че сместа = най-добрия = 2,665. При пълния мащаб **мета-Наш** разпределя теглото (0,645 върху един агент и по-малки тегла върху още два), а сместа, сведена до една поведенческа стратегия, има експлоатируемост 3,418 - **по-висока** от тази на най-добрата отделна моментна снимка (1,305). Това не е грешка в смесването: същият код дава смес = най-добрия при бързата проверка, а сместа е по-малко експлоатируема от всеки от трите си компонента (3,558, 3,932 и 4,262). Причината е в целта: **мета-Наш** минимизира **съжалението в мета-играта** – тоест доколко добре се представя *срещу популацията* – а това е *различна цел* от минимизирането на **експлоатируемостта в пълната игра**. Затова най-малко експлоатируемите агенти получиха тегло нула. Изводът е обратен на очакването: като резултат от лигата трябва да се използва избран член, **устойчив срещу най-добър отговор**, а не **мета-Наш сместа**.

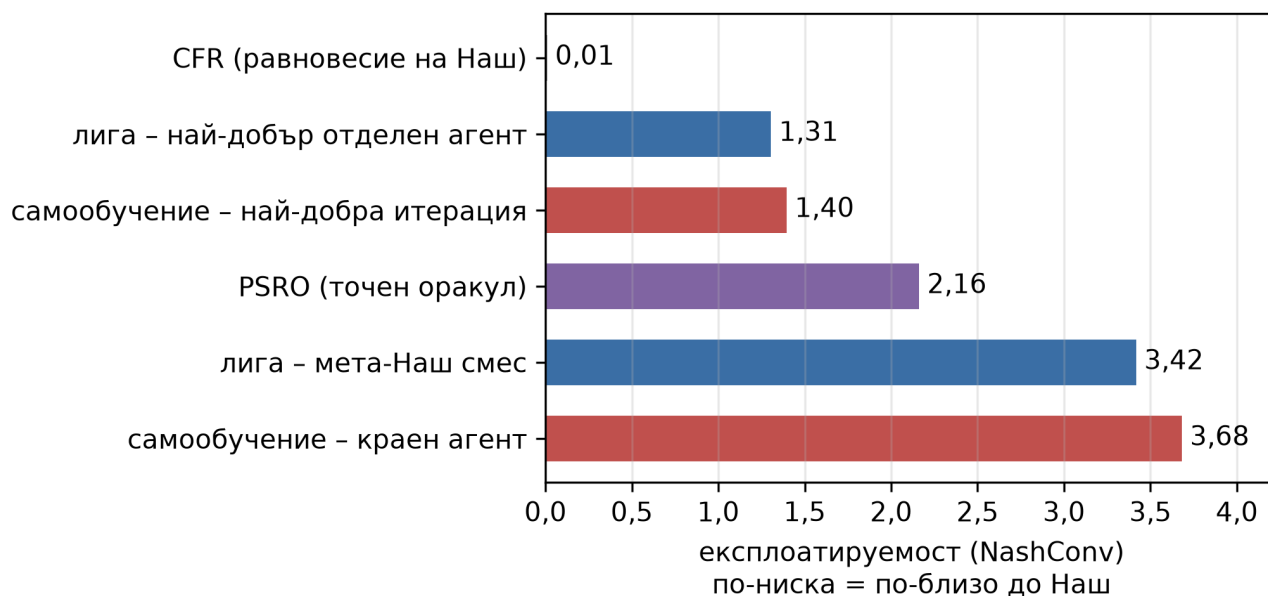
Как се представя **лигата** спрямо останалите методи по показателя за **експлоатируемост** в **Ледюк**?

Таблица 39: Експлоатируемост в Ледюк (NashConv) по метод; при самообучението са дадени и крайният агент, и най-добрата итерация.

Метод (пълен мащаб)	Експлоатируемост (NashConv)
CFR, равновесие на Наш (долна граница)	0,0099
Лига – най-добър отделен агент (за цялото обучение)	1,305
Самообучение – най-добра итерация (епоха 100)	1,396
PSRO (точен оракул за най-добър отговор)	2,163

Метод (пълнен мащаб)	Експлоатируемост (NashConv)
Лига – мета-Наш смес	3,418
Самообучение – краен агент	3,683

Най-добрият отделен агент на лигата (1,305, най-добрата моментна снимка за цялото обучение) е най-добрият резултат сред методите, основани на обучение, но предимството му трябва да се чете внимателно: PSRO достига 2,163, а при самообучението най-добрата итерация е 1,396 (епоха 100) и едва накрая се влошава до 3,683. Всички тези методи остават далеч над долната граница, зададена от CFR. Мета-Наш сместа на лигата (3,418) е по-добра от крайния агент на самообучението, но по-лоша от PSRO и от най-добрия отделен агент (вж. съпоставката по-горе). Затова като резултат от лигата трябва да се използва избран отделен агент, а не сместа.¹³⁷



Фигура 62: Експлоатируемост (NashConv) в Ледюк по методи (пълнен мащаб): долна граница CFR (равновесие на Наш) ~0,01; най-добрият отделен агент на лигата (1,31, най-добрият за цялото обучение) е сравним с най-добрата итерация на самообучението (1,40) и е под PSRO (2,16), под мета-Наш сместа на лигата (3,42) и под крайния агент от самообучението (3,68).

10.8 Честни бележки, ограничения и предаване на щафетата

Какво се потвърди. Основните резултати се потвърдиха: репликаторната динамика възпроизвежда всеки аналитичен резултат за матричните игри (ESS или равновесие на Наш), включително затворената орбита при „камък-ножица-хартия“, която не се сближава¹⁴¹; разлагането на Ходж ясно отделя умението от цикличността и правилно определя чистите случаи; лигата постига бързо

ранно подобрене и дава отделен агент с експлоатируемост 1,305 - по-добър от PSRO (2,163) и сравним с най-добрата итерация на самообучението (1,396); а EGTA дава работеща мярка за експлоатируемост на ниво популация. Заедно те проследяват пътя от „в кои игри една популация изобщо може да се установи?“ до „какво се случва, когато една популация сама се обучава?“.

Какво не се получи и защо това има значение. Три уговорки остават валидни и за следващите глави. (1) При пълен мащаб експлоатируемостта на лигата **нараства отново в края на обучението** ($4,73 \rightarrow \approx 1.21 \rightarrow \approx 2.05$) – лигата не се подобрява монотонно, а най-добрите агенти са замразени моментни снимки. (2) При пълен мащаб **мета-Наш сместа е по-експлоатируема от най-добрия член на популацията** (3,42 срещу 1,31), защото не му дава никакво тегло – изборът на мета-Наш не гарантира устойчивост, затова като резултат трябва да се използва избран отделен агент, а не сместа. (3) Мета-играта на Ледюк е **циклична като популация от най-добри отговори**, но **транзитивна като популация от моментни снимки** – структурата е свойство на начина, по който е изградена популацията. И един методологичен извод: късната регресия и слабостта на сместа не се виждат при бързата проверка – пълният мащаб разкрива това, което бързата проверка скрива.

Доверие. Всяка еволюционна цел е *аналитична* (ESS или равновесие на Наш на матричните игри), а всяка стойност на експлоатируемостта в лигата е *точният* NashConv от Глава 7, изчислен върху таблично извлечените стратегии – така игровитеоретичните резултати са проверени спрямо точни стойности. Невронните резултати идват от едно изпълнение на PBT за всяка конфигурация: надеждни са *посоките* (ранно подобрене; най-добрата моментна снимка < PSRO < крайния агент на самообучението; късна регресия; мета-Наш сместа > най-добрия член на популацията), а не точните стойности до третия десетичен знак. Графиките с експериментални резултати са построени от съхранените в проекта JSON файлове с резултати, а концептуалните диаграми – от скриптовете `make_*_figure.py`.

Връзки с предходните и следващите глави. С предходните: лигата е асинхронен вариант на PSRO от Глава 9 с невронни оракули и изцяло използва точния най-добър отговор от Глава 7; „мета-Наш на популация“ е равновесието на Наш от Глава 2, пренесено едно ниво по-горе. Със следващите: основните експлоататори на практика са автоматизирани модели на противника (Принос №1); EGTA/мета-Наш е методологията за оценяване (Принос №3); а липсващата гаранция на лигата – тя може да се влоши и сместа ѝ може да е експлоатируема – е популационната форма на **липсващата опора за безопасност при $N > 2$ играчи** (Принос №2). Транзитивната/цикличната диагностика предсказва, че коалиционните игри всеки срещу всеки от Глава 11 ще бъдат силно циклични, така че наивното PBT там ще зацikli.

10.9 Основни изводи за синтеза на дисертацията

- **Структурата определя сходимостта.** Транзитивното/цикличното съотношение („пумпал“) е *диагностика преди обучението*: транзитивните игри се установяват (и потискат разнообраз-

разието), цикличните игри се въртят (и налагат разнообразие). Измерено транзитивно съотношение: „камък-ножица-хартия“ 0,0, стълбица на уменията 1,0, PSRO в Ледюк ≈ 0.45 (циклична), моментни снимки на лигата $\approx 0.94-0.98$ (транзитивна).

- **Лигата произвежда силни отделни агенти, но не носи гаранция.** Най-добрата моментна снимка (1,305) е по-добра от PSRO (2,163) и сравнима с най-добрата итерация на самообучението (1,396) – и двете при едно изпълнение; въпреки това експлоатируемостта *нараства отново* към края ($\rightarrow 2.05$), а мета-Наш *сместа* (3,42) е по-лоша от най-добрия член на популацията (1,31).
- **Мета-Наш оптимизира съжалението в мета-играта, а не експлоатируемостта в пълната игра** – двете не съвпадат, затова при оценката на популацията експлоатируемостта на сместа трябва да се измерва пряко, а като резултат да се използва избран отделен агент.
- **Точната оценка е опората.** Извличането на невронните стратегии в табличен вид и оценяването им с точния най-добър отговор от Глава 7 прави всяко твърдение за популацията проверимо спрямо точни стойности – със същия NashConv като в Глава 2.
- **Липсата на гаранция за безопасност на ниво популация е отворен проблем (Принос №2):** механизмът с експлоататори в лигата на AlphaStar е аналог на ниво популация на безопасната експлоатация от Глава 8, но е евристичен: устойчивостта му е показана само емпирично¹³⁵, а в малката лига от тази глава той не осигури нито монотонно подобрене, нито неексплоатируема смес.

11 Глава 11 – Формиране и засичане на коалиции

Това е цялостна глава за нещо, което възниква едва когато една игра има **трети играч**: временни **съюзи**, които се формират, експлоатират и предават. Тя използва **So Long Sucker (SLS)** – играта за четирима на Хауснер, Наш, Шапли и Шубик, публикувана през 1964 г., в която изходът зависи почти изцяло от договарянето между играчите¹⁴⁵, – и допълва единствения публикуван подход с обучение с подкрепление за нея¹⁴⁶, чиито агенти не отчитат **коалициите**, с детектор на коалиции и награда по Шапли. Чрез нея главата осмисля прехода от $N = 2$ към $N \geq 3$, където **равновесието на Наш и експлоатируемостта стават изчислително непосилни и губят смисъла си**, така че на въпроса „проработи ли?“ вече не може да се отговори с едно-единствено число. Написана е така, че да може да се чете самостоятелно. **Всички експериментални числа, докладвани тук, са измерени** във възпроизводими изпълнения и, когато е възможно, са поставени в рамките на *точни* еталонни стойности (учебниковите стойности на Шапли и ядрото за опростени кооперативни игри; точен минимаксен решавач за крайната фаза на SLS с двама играчи). Когато някое изпълнение противоречи на очакванията ми – включително истинска грешка в двигателя на играта – запазвам първоначалното очакване и го съпоставям с действителния резултат; тези разминавания са най-поучителната част от главата.

Къде се вписва това в дисертацията. Глави 2–10 разчитаха на игра с двама играчи с *точен* оракул за най-добър отговор. Глава 11 се отказва от него и навлиза в нерешените проблеми на областта. Тук главата се свързва с трите приноса на дисертацията. **Детекторът на коалиции** пренася моделирането на противника от въпроса „какъв тип играч е това?“ към въпроса „кой с кого е в съюз?“ (Принос №1). **Безопасната базова линия губи опората си в равновесието на Наш** – без минимаксна стойност и при празно ядро няма стабилно разпределение, на което да се опре безопасността; регуларизация към поведенческа опорна стратегия по подобие на piKL е кандидат, но не дава гаранция – празнина, която главата очертава, но не запълва (Принос №2). **Мета-играта на EGTA и приносът по Шапли** заместват експлоатируемостта, която при повече от двама играчи вече не показва какво е гарантирано на агента, още по-малко срещу коалиция (Принос №3).

11.1 Защо появата на трети играч е повратна точка

С двама играчи една игра с нулева сума има *стойност*: съществува оптимална по минимакс стратегия и „колко далеч сте от оптималната?“ (експлоатируемост) е едно-единствено смислено число, на което се опираше всяка глава от Глава 2 насам. Добавянето на трети играч променя основата. Сега двама играчи могат да **се обединят** срещу третия и интересният въпрос вече не е „какво е равновесието?“, а „с кого се съюзява кой, за колко дълго и кой предава пръв?“. Равновесието на Наш в игра всеки срещу всеки с четирима играчи е едновременно **изчислително непосилно и**

¹⁴⁵Hausner, M., Nash, J., Shapley, L. & Shubik, M. (1964). “So Long Sucker – A Four-Person Game.” B M. Shubik (Ed.), *Game Theory and Related Approaches to Social Behavior*, 359–361. New York: Wiley.

¹⁴⁶Sharan, M. & Adak, C. (2024). “Reinforcing Competitive Multi-Agents for Playing ‘So Long Sucker’.” *arXiv:2411.11057*.

лишено от гаранции: намирането му е поне толкова трудно, колкото при общите игри с двама играчи¹⁴⁷, а ако всеки играч избере равновесие независимо от другите, получената комбинация може изобщо да не е равновесие¹⁴⁸. Освен това равновесието на Наш защитава само от едностранни отклонения, а не от съвместните отклонения на коалициите, които всъщност определят изхода от играта¹⁴⁹.

Полезна аналогия: **вечеря** с четирима гости и една къща. Всеки се съюзява по двойки, за да свърши работата, но най-умният ход е тихо да предадете партньора си един ход *преди* той да ви предаде. So Long Sucker прави това буквално: държите цветни **жетони**, поставяте ги в купчините на други играчи (ръкостискане) или вземате техните купчини (ножът), и бивате елиминирани, когато останете без **жетони**. В двигателя на тази глава съюзът е **кодиран изцяло в ходовете**: преговорите, които в истинската игра са свободни и необвързващи, не се моделират, затова детекторът може да прочете коалицията директно от потока от ходове.

Последствието за методологията е най-важната идея в тази глава: **при $N \geq 3$ точната оценка отстъпва място на емпиричната**. Вместо експлоатируемост се използват делът на победите, коалиционният резултат от детектора и цикличното съотношение на емпиричната мета-игра, закотвени в единствената подигра, която е точно решима: крайната фаза с двама играчи.

Прочетете повече: Hausner, Nash, Shapley & Shubik (1964) – правилата на играта; De Carufel & Jerade (2024) – характеристика на крайната фаза с двама играчи¹⁵⁰. Точната опорна точка в тази глава е собственото минимаксно решение на двигателя; то още не е сверено с тази характеристика.

11.2 Детекторът на коалиции – извличане на съюзи от ходовете

Първият инструмент обобщава моделирането на противника. Вместо да извежда скрит *тип* или *ръка* (Глава 7), детекторът извежда скрита *социална структура*. Той следи дневника на ходовете и натрупва две матрици: **помощ** (играч i поставя жетон в купчината на играч j) и **вреда** (играч i взема купчината на играч j). Разликата между тях е **нетна подкрепа**, а двойката с най-висока взаимна нетна подкрепа представлява най-силната коалиция.

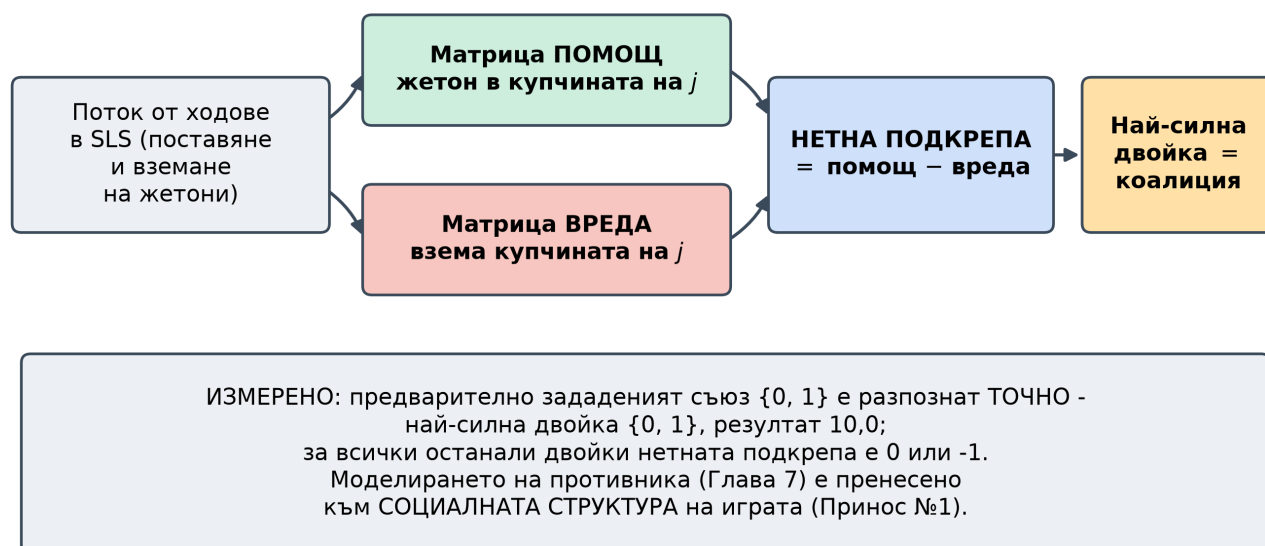
¹⁴⁷Daskalakis, C., Goldberg, P. W. & Papadimitriou, C. H. (2009). “The Complexity of Computing a Nash Equilibrium.” *SIAM Journal on Computing*, 39(1), 195–259.

¹⁴⁸Brown, N. & Sandholm, T. (2019). “Superhuman AI for multiplayer poker.” *Science*, 365(6456), 885–890.

¹⁴⁹Bernheim, B. D., Peleg, B. & Whinston, M. D. (1987). “Coalition-Proof Nash Equilibria I. Concepts.” *Journal of Economic Theory* 42(1), 1–12. DOI 10.1016/0022-0531(87)90099-8.

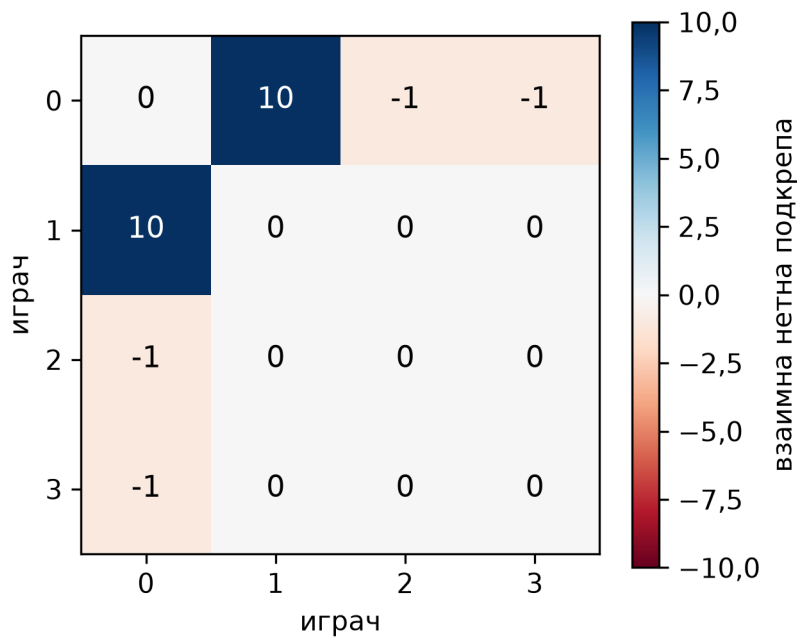
¹⁵⁰De Carufel, J.-L. & Jerade, M. R. (2024). “So Long Sucker: Endgame Analysis.” *arXiv:2403.17302*.

В този двигател съюзът се вижда само в ходовете - преговорите не се моделират.



Фигура 63: Детекторът на коалиции: в потока от ходове на SLS поставянето на жетон в купчината на друг играч се отчита като ПОМОЩ, а вземането на купчина – като ВРЕДА. От натрупаните матрици на помощта и вредата се получава нетната подкрепа, в която взаимният съюз се вижда като силна двупосочна връзка. Моделирането на противника от Глава 7 се пренася от въпроса „каква ръка има?“ към въпроса „кой с кого е в съюз?“ (Принос №1).

Работи ли? Програмираме двама играчи да си помагат систематично и да вредят на останалите, без да даваме на детектора никаква информация, и го караме да посочи коалицията. Той разпознава **точно** предварително зададената коалиция $\{0, 1\}$, с отчетливо по-висок резултат (10,0 за съюзническата двойка, 0 или -1 навсякъде другаде). Коалицията се разпознава напълно само по начина, по който се поставят жетоните. Това е проверка срещу една програмирана, открито действаща двойка; честотата на фалшивите тревоги при игра без съюзи и разпознаването на прикрити съучастници не са проверявани.



Фигура 64: Матрица на коалиционния резултат, изведена само от поставянето на жетоните: предварително зададеният съюз между играчи 0 и 1 се вижда като силна взаимна подкрепа (10), а за останалите двойки стойностите са неутрални (0) или враждебни (-1). Детекторът разпознава коалиция, за която не е получил никаква информация.

Прочетете повече: реализацията на моделирането на противника от Глава 7 – принципът „наблюдение на действията → обновяване на убежденията“, който детекторът пресъздава като матрици на помощта и вредата.

11.3 Принос по Шапли в състезателна игра

Ако искаме един **агент** да се *научи* да формира коалиции, е необходим сигнал за **награда**, който да показва „ти допринесе за една коалиция“. Класическият инструмент е **стойността на Шапли**: единственото разпределение на стойността на коалицията между членовете ѝ, което удовлетворява аксиомите на Шапли (ефективност, симетрия, нулев играч, адитивност); то осреднява пределния принос на всеки член по всички възможни редове на присъединяване. Две учебникови игри определят какво означават „справедливо“ и „стабилно“, като и двете се възпроизвеждат точно:

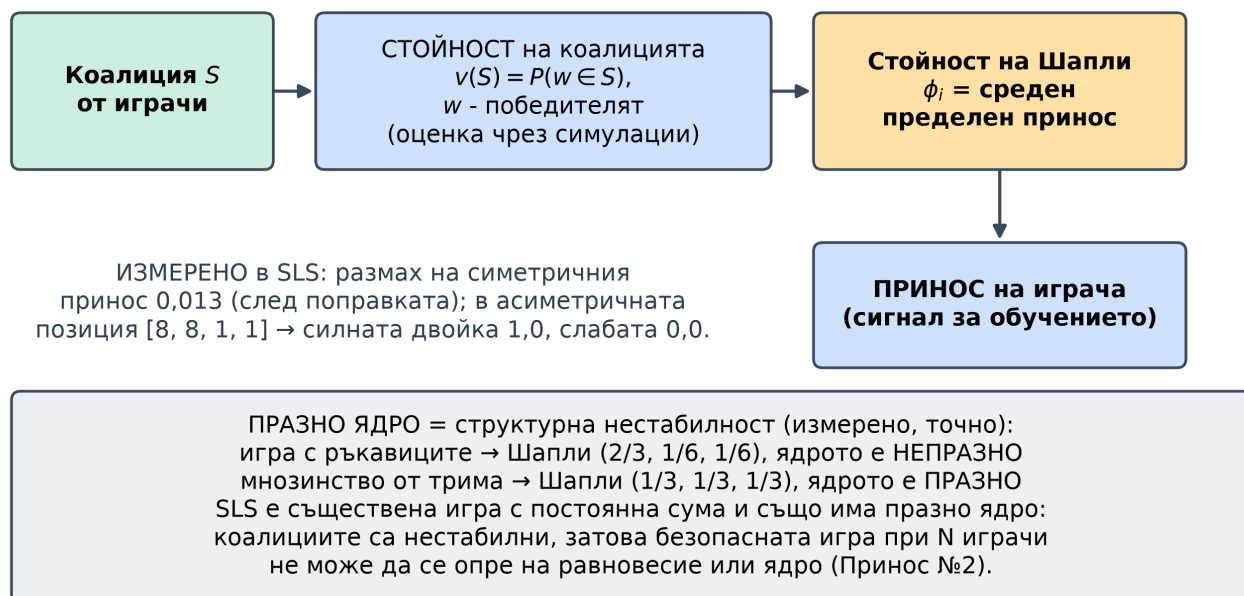
Таблица 40: Стойност на Шапли и стабилност на ядрото при два опростени кооперативни модела.

Опростена кооперативна игра	Стойност на Шапли	Ядро (стабилност)
Игра с ръкавиците	$(2/3, 1/6, 1/6)$	непразно (съществува стабилно разпределение)
Мнозинство от трима играчи	$(1/3, 1/3, 1/3)$	празно (няма стабилно разпределение)

Празното ядро на играта на мнозинството е ключовата идея на главата: това е игра, при която всяко разпределение може да бъде оспорено от някаква коалиция, така че сътрудничеството е *по своята същност нестабилно*. SLS е игра с постоянна сума (печели само един), а всяка съществена игра с постоянна сума има празно ядро¹⁵¹; следователно и коалиционната игра на SLS, в която стойността на коалицията е гарантираната ѝ вероятност за победа, няма стабилно разпределение (играта е съществена, защото никой играч не може сам да си гарантира победа). Заедно с липсата на минимаксна стойност при повече от двама играчи това обяснява защо „безопасната“ игра при N играчи не може да се опре на стабилно равновесие (Принос №2).

¹⁵¹Ferguson, T. S. *Game Theory* (лекционни записки, UCLA), Part IV “Games in Coalitional Form”, § 2.3, Theorem 1.

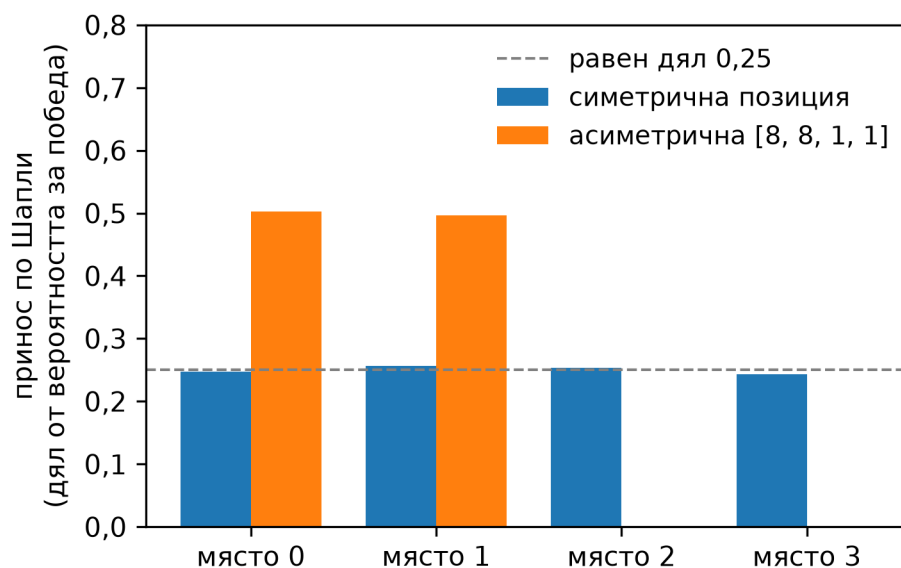
Стойността на Шапли е единственото разпределение, което удовлетворява аксиомите на Шапли; тук стойността на коалицията е вероятността ѝ за победа.



Фигура 65: Принос по Шапли в изцяло състезателна игра: „стойността“ на коалицията се предефинира като ВЕРОЯТНОСТТА неин член да спечели (оценена чрез симулации), а приносът на всеки играч е стойността на Шапли на тази функция. Празното ядро на играта на мнозинството (няма стабилно разпределение) илюстрира защо коалициите в SLS рано или късно се разпадат.

SLS е състезателна, а не кооперативна игра – няма обща печалба, която да се поделва, – затова **пре-дефинираме стойността на коалицията** като *вероятността неин член да спечели*, оценена чрез симулации по метода Монте Карло. Приносът на всеки играч е стойността на Шапли на тази функция на вероятността за победа. Резултати върху позиции в SLS: наистина симетрична позиция дава почти равни приноси (размах 0,013), а асиметричната позиция [8, 8, 1, 1] приписва *целия* принос на силната двойка (стойност на коалицията 1,0). Симетричният резултат е получен **след** отстраняване на грешка – вж. съпоставката по-долу.¹⁵²

¹⁵²Shapley, L. S. (1953). “A value for n-person games.” В *Contributions to the Theory of Games II* (Annals of Mathematics Studies 28), 307–318. Princeton University Press; Chalkiadakis, G., Elkind, E. & Wooldridge, M. (2011). *Computational Aspects of Cooperative Game Theory*. Morgan & Claypool (ядро, стойност на Шапли, нуклеолус); Wang, J., Zhang, Y., Kim, T.-K. & Gu, Y. (2020). “Shapley Q-Value: A Local Reward Approach to Solve Global Reward Games.” *AAAI* 34(5), 7285–7292 (разпределяне на приноса в MARL чрез стойността на Шапли).



Фигура 66: Принос по Шапли в две позиции на SLS: почти равен за четирите места в симетричната позиция (размах 0,013 след поправката) и изцяло съсредоточен върху силната двойка в асиметричната позиция [8, 8, 1, 1]. Приносът следва реалното предимство едва след като правилото при равенство в двигателя стане безпристрастно.

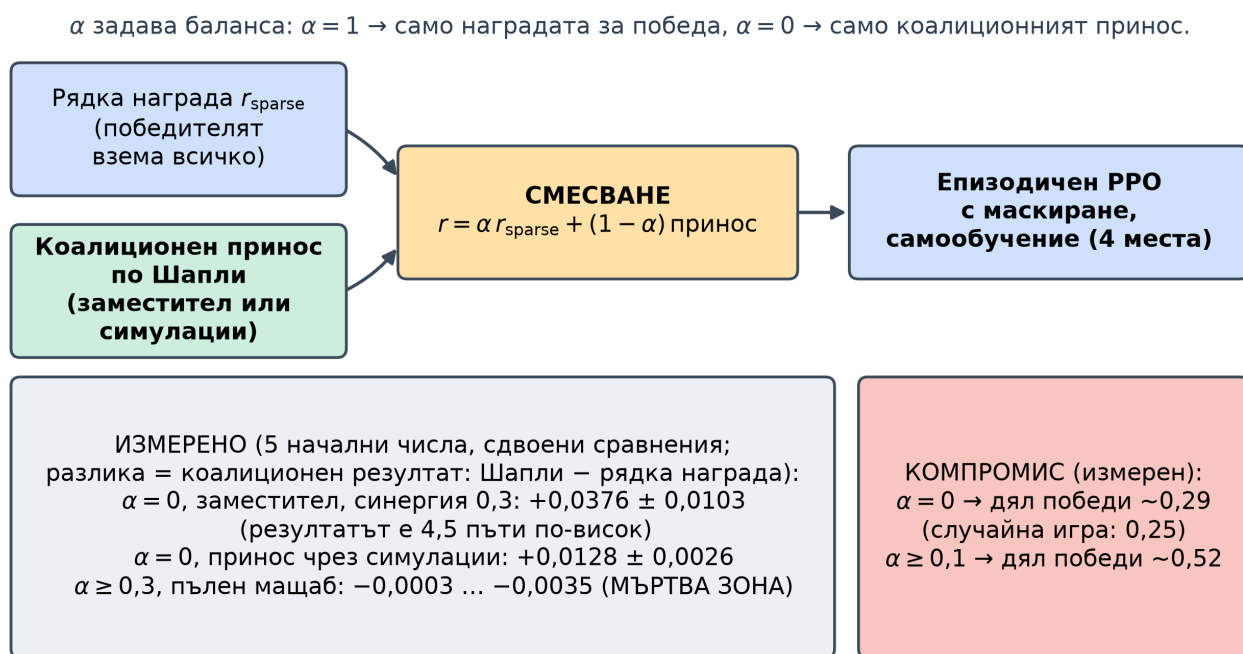
Съпоставка (запазена прогноза → действителен резултат). Прогнозирах, че симетрична позиция ще даде симетрично разпределение на приноса (размах $< 0,15$). Първото изпълнение даде 0,54 (неуспешна проверка), като играч 0 печелеше около два пъти повече от справедливия си дял в три независими скрипта. Тъй като заподозрях първо двигателя, а не прогнозата, открих механизма: **~99,5% от случайните игри в този двигател завършват с пат** (всички играчи, останали в играта, са без жетони в ръка – следствие от опростените правила, които пропускат играч с празна ръка), затова победителят се определя от правилото при равенство по брой жетони, а то тихомълком даваше предимство на място 0, защото избираше играча с най-малък индекс. **Безпристрастното случайно разрешаване на равенството** отстрани проблема: размахът на симетричния принос спадна от 0,54 на 0,013, а победите вече се разпределят равномерно. Това разкри също, че впечатляващият дял победи $\sim 0,87$ на собствения агент е бил *същият* артефакт (агентът винаги е заемал място 0); реалната стойност е $\sim 0,41$. Урокът: в игра, която почти винаги завършва с почти равен резултат, правилото при равенство е най-важният ред в кода на двигателя, а симетричната *позиция* дава симетричен *изход* едва когато това правило е безпристрастно.

11.4 MAPPO с отчитане на коалициите – и кога всъщност расте коалиционният резултат

Следва обучението. Всяко място в SLS се заема от отделен агент PPO с маскиране на недопустимите ходове, който се обучава по цели епизоди чрез самообучение. Наградата е **смес**:

$$r = \alpha r_{\text{sparse}} + (1 - \alpha) (\text{коалиционен принос по Шапли}),$$

където r_{sparse} е рядката награда „победителят взема всичко“, а вторият член е центрираният принос по Шапли от раздел 11.3 – изчислен или евтино, от оценките на оценителя (заместител), или по-скъпо, чрез симулации (в кода counterfactual). Теглото α определя баланса между „просто победи“ ($\alpha = 1$) и „формирай коалиции“ ($\alpha = 0$).



Фигура 67: Смесване на наградите в MAPPO с отчитане на коалициите: рядката награда „победителят взема всичко“ се смесва с коалиционния принос по Шапли с тегло α ($\alpha = 1$ – само рядка награда, $\alpha = 0$ – само принос). Измерено: α е определящият параметър – коалиционният резултат нараства значимо само при ниско α , а при $\alpha \geq 0,3$ сигналът е потиснат. Евтиният заместител, изчислен от оценките на оценителя, дава по-силен ефект от по-скъпата оценка чрез симулации.

Формират ли агентите, обучени с коалиционен принос, повече коалиции от агентите с рядка награда? Честният отговор изискваше **сдвоени сравнения с 5 начални числа** за всички комбинации на α , вида на приноса и синергията, тъй като една-единствена конфигурация ме подведе сериозно (вж. по-долу). Сравненията са на две нива: бърза проверка (5 жетона, 400 игри за обучение) и пълен мащаб (7 жетона, 1500 игри). Резултати (сдвоена разлика = коалиционен

резултат на агентите с принос по Шапли минус този на агентите с рядка награда; ** = разлика, по-голяма от две стандартни грешки, което при 5 начални числа отговаря приблизително на $p < 0,12$):

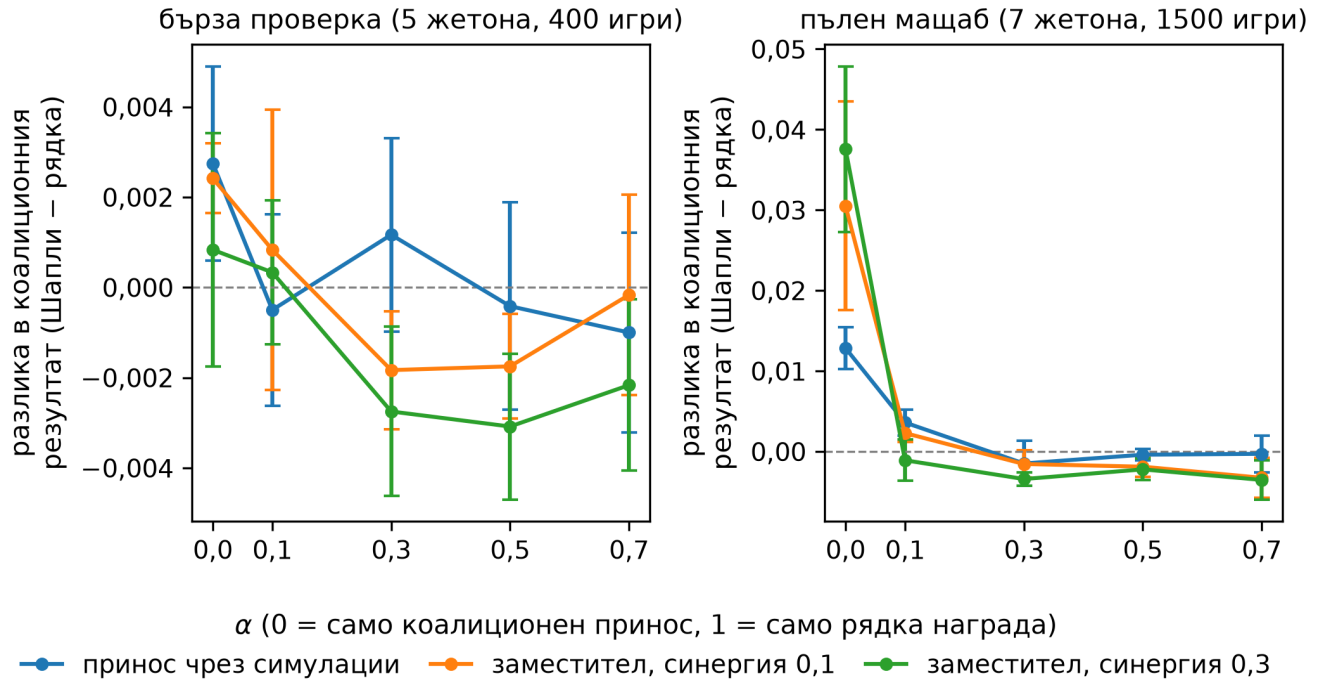
Таблица 41: Сдвоена разлика в коалиционния резултат между агентите с принос по Шапли и базовата линия с рядка награда, по режим.

Режим	Сдвоена разлика (Шапли – рядка награда)
пълнен мащаб, заместител, $\alpha = 0$, синергия 0,3	$+0,0376 \pm 0,0103$ (резултатът на агентите с принос е ~4,5 пъти този на базовата линия)
пълнен мащаб, чрез симулации, $\alpha = 0$	$+0,0128 \pm 0,0026$
пълнен мащаб, $\alpha \geq 0,3$	$-0,0003 \dots - 0,0035$ (навсякъде отрицателна; при бързата проверка: $-0,003 \dots + 0,001$, без значими)
бърза проверка, заместител, $\alpha = 0$, синергия 0,1	$+0,0024 \pm 0,0008$ (много малка)

Съпоставка (запазена прогноза → действителен резултат). Експериментите ми с една-единствена конфигурация използваха стойността по подразбиране $\alpha = 0,3$ и показаха, че коалиционният сигнал изчезва при пълния мащаб – което първоначално тълкувах като „приносът от заместителя е твърде слаб при по-дълго обучение“. Серията експерименти напълно обърна този извод: **α е доминиращият параметър, а 0,3 е мъртва зона.** Коалиционният резултат нараства статистически значимо *само при ниски стойности на α* (при $\alpha \approx 0$ агентите с принос по Шапли надминават базовата линия с $+0,038$), докато *всяка* клетка с $\alpha \geq 0,3$ при пълния мащаб е отрицателна – членът с рядка награда „всичко или нищо“ заглушава коалиционния сигнал. Две допълнителни изненади: ефектът е **по-голям при пълния мащаб** (7 жетона и 1500 игри за обучение срещу 5 и 400, така че размерът на играта и продължителността на обучението не са разделени; обратно на първоначалното ми тълкуване „положителен ефект при бързата проверка, нулев при пълния мащаб“, което се дължеше на това, че и в двете беше използвано $\alpha = 0,3$), и **евтиният заместител дава по-силен ефект от скъпата оценка чрез симулации.** Ефектът е устойчив при ниско α и просто не се вижда в режима, в който преобладава членът с рядка награда; дали той отразява формиране на коалиции, още не е проверено (вж. уговорката по-долу). Решението е „да се даде голяма тежест на коалиционния принос“, а не „да се изчисли по-точен принос“.

Уговорка за механизма. В `shapley.py` стойността от симулациите е $v(S) = \sum_{i \in S} P(i \text{ печели})$ – адитивна игра, в която стойността на Шапли на всеки играч е просто собствената му вероятност за победа (а в обучението тя се изчислява веднъж за партида, от началното състояние). Центрираният принос на заместителя е пропорционален на собствената оценка на оценителя за агента минус

средната за масата; синергията само го мащабира. Нито един от двата сигнала не показва кой на кого помага, а коалиционният резултат е средната *абсолютна* взаимна нетна подкрепа и расте и при взаимна враждебност. Ефектът при ниско α следователно показва, че замяната на наградата за победа с плътен сигнал променя взаимодействията между агентите; дали така възникват коалиции, изисква контролен експеримент с еднакво плътна награда без коалиционна информация.



Фигура 68: Сдвоена разлика в коалиционния резултат за всички комбинации $\alpha \times$ вид на приноса $\times$ синергия (5 начални числа, $\pm$ една стандартна грешка): голяма и значима само при ниско α (максимум +0,038 при заместителя и $\alpha = 0$); при пълен мащаб е отрицателна при всяко $\alpha \geq 0.3$. Първоначалният нулев резултат идва от измерване в „мъртвата зона“ $\alpha = 0.3$.

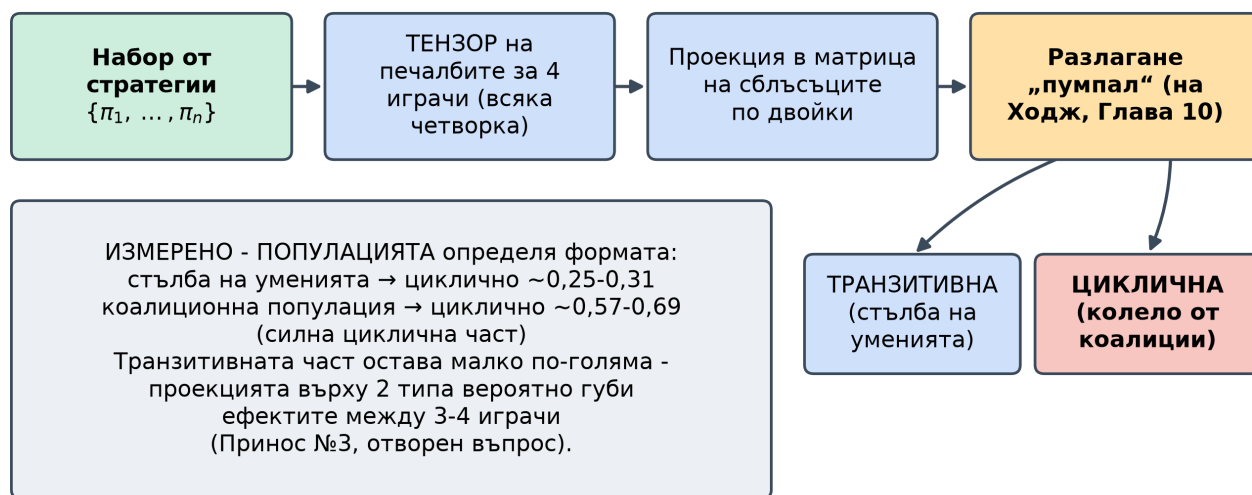
Няма безплатен обяд: при изцяло коалиционен принос ($\alpha = 0$) делът на победите спада до $\sim 0,29$ (близо до нивото на случайната игра 0,25), докато $\alpha \geq 0,1$ го запазва на $\sim 0,52$. Основната цел е формирането на коалиции, а победата е второстепенна – точно както беше заложено в плана на главата, но вече количествено.¹⁵³

¹⁵³Реализацията на MAPPO с централизиран оценител е от Глава 9. piKL (регуларизация на търсенето към стратегия, наподобяваща човешката игра) е въведен от Jacob, A. P. и др. (2022). “Modeling Strong and Human-Like Gameplay with KL-Regularized Search.” *ICML*, arXiv:2112.07544, и е приложен в Diplomacy от Bakhtin, A. и др. (2023). “Mastering the Game of No-Press Diplomacy via Human-Regularized Reinforcement Learning and Planning.” *ICLR*, arXiv:2210.05492. Опорната стратегия там е имитация на човешка игра и не дава гаранция за безопасност; използването ѝ като опора за безопасна експлоатация при N играчи е предложение на дисертацията.

11.5 EGTA и въртящият се пумпал – SLS колело ли е, или стълба?

Крайният инструмент оценява *популацията*. **Емпиричният игровитеоретичен анализ (EGTA)** разглежда целите стратегии като атомите на мета-игра, изиграва всяка двойка, за да попълни емпирична матрица на печалбата, и анализира нейната структура. За да се използват повторно решаващът за мета-равновесие на Наш от Глава 9 и разлагането „пумпал“ (на Ходж) от Глава 10 – и двата инструмента за игри с двама играчи – тензорът на печалбите за четирима играчи се **проецира** в матрица на сблъсъците по двойки, която след това се разлага на **транзитивна** (стълба на уменията) и **циклична** (камък-ножица-хартия) компонента.

EGTA = емпирична мета-игра, чиито „стратегии“ са цели обучени агенти; експлоатируемостта не дава гаранция срещу коалиция.



Фигура 69: EGTA и разлагането „пумпал“ за SLS: всяка четворка стратегии се изиграва многократно, за да се попълни тензорът на печалбите за четирима играчи; той се проектира в матрица на сблъсъците по двойки, която се разлага на транзитивна (стълба на уменията) и циклична (контрастратегии между коалициите) компонента. Уговорка: проекцията върху двойки вероятно губи коалиционните ефекти между трима и четирима играчи, затова цикличната компонента може да е подценена.

Глава 10 предвиди, че коалиционните игри всеки срещу всеки ще бъдат силно циклични. Измерванията показват, че това зависи изцяло от **популацията, която се разлага** – същият урок, който даде и Глава 10:¹⁵⁴

¹⁵⁴Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML*, PMLR 97, 434–443 – разлагане на транзитивна и циклична компонента; Czarnecki, W. M. и др. (2020). “Real World Games Look Like Spinning Tops.” *NeurIPS*, arXiv:2004.09468 – геометрията „пумпал“.

Таблица 42: Циклично съотношение и структура на две популации в SLS.

Популация	Циклично съотношение	Структура
Популация тип „стълба на уменията“	0,25-0,31	преобладаващо транзитивна (стълба)
Коалиционна популация (стратегии „съюзник/предател“)	$\sim 0,57-0,69$	силна циклична компонента; транзитивната е малко по-голяма

Съпоставка (запазена прогноза → действителен резултат). Очаквах голяма циклична компонента и първоначално видях почти съвършена стълба на уменията (циклично съотношение $\sim 0,07$). Причината беше отчасти грешката с място 0 (раздел 11.3) и отчасти **съставът на популацията**: базовата популация по подразбиране е стълба на уменията. Популация с коалиции (фиксиран съюзник + стратегии „предател“) повишава цикличното съотношение до $\sim 0,57-0,69$ - голяма нетранзитивна компонента, която силно потвърждава *посоката* на прогнозата, но остава **под прага на строго доминиране** (квадратът на цикличното съотношение е малко под 0,5). Основната вероятна причина е, че **проекцията върху двойки типове губи коалиционните ефекти между трима и четирима играчи**; разлагането директно на тензора остава отворен въпрос. Нито едно от двете тълкувания не е грешка: съставът на популацията определя дали SLS изглежда като колело, или като стълба.

11.6 Бележки, ограничения и връзка със следващите глави

Какво се потвърди. Точните опорни точки са надеждни: двигателят на SLS съвпада с минимаксното решение на крайната фаза с двама играчи без **нищо едно** несъответствие; детекторът разпознава **точно** предварително зададената коалиция ($\{0, 1\}$, резултат 10,0); кодът за стойността на Шапли възпроизвежда опростените кооперативни игри – *включително тяхното ядро* – с точност до четвъртия знак след десетичната запетая. Основният резултат от обучението се потвърждава в сдвоени сравнения с 5 начални числа: при ниско α коалиционният резултат е значимо по-висок (+0,038), макар че, както е реализиран, приносът по Шапли не носи информация за коалициите (раздел 11.4).

Какво не се получи и защо това е важно. Две поправки остават в сила и за следващите глави. (1) „Симетричната“ игра не беше симетрична – **грешка в правилото при равенство** по брой жетони даде на място 0 около два пъти по-голям дял от справедливия и завиши дела на победите на собствения агент от реалните $\sim 0,41$ до фалшивите $\sim 0,87$; грешката е поправена, но показва, че рискът се крие в *правилата* на двигателя, а не в решавача. (2) „Коалициите не възникват при

пълния мащаб“ се оказа следствие от **неподходящо тегло на смесване** ($\alpha = 0,3$ е мъртва зона), а не провал на метода. Методологичен паралел с глави 9-10: **една-единствена конфигурация скрива онова, което разкрива серия експерименти с няколко начални числа.**

Доверие. Всяка точна проверка (минимаксът на крайната фаза, опростените кооперативни игри) е детерминирана и възпроизводима; твърденията за обучението се основават на сдвоени сравнения с 5 начални числа и интервали на грешката, така че *посоките* са надеждните твърдения, а не величините до третия знак след десетичната запетая. Основната уговорка остава **верността на двигателя спрямо правилата**: двигателят е сертифициран спрямо собствения си правилник, а не спрямо формализацията на De Carufel & Jerade – и грешката в правилото при равенство показва, че тази сверка не е козметична. (Записаният файл `scale_results.json` е резултат отпреди поправката и се цитира само като доказателство за грешката; меродавните данни за пълния мащаб идват от серията експерименти.)

Връзки с другите глави. Назад: детекторът е моделът на противника от Глава 7, пренесен към социалната структура; мета-равновесието на Наш и разлагането „пумпал“ от глави 9–10 се прилагат повторно върху проецираната мета-игра; без минимаксна стойност и при празно ядро определението от Глава 8 „безопасно = ограничено отклонение от равновесието на Наш“ губи опората си, а регуляризацията към поведенческа опорна стратегия (по подобие на `riKL`) остава кандидат без гаранция. Напред: Глава 12 разглежда езиковите модели като стратегически агенти, но не и договарянето; безопасността при N играчи (Принос №2) и оценката чрез EGTA върху тензора на печалбите като многоагентно обобщение на експлоатируемостта (Принос №3) остават задачи на дисертацията.

11.7 Основни изводи за синтеза на дисертацията

- **При $N \geq 3$ точната оценка отпада.** В игра всеки срещу всеки с четирима играчи равновесието на Наш е изчислително непосилно и не дава гаранции; оценката става емпирична (дял на победите, коалиционен резултат и циклично съотношение от EGTA), закотвена в единствената точно решима подигра (минимаксното решение на крайната фаза с двама играчи, 0 несъответствия).
- **Коалиционният резултат расте, когато наградата за победа се замени с плътен сигнал, но това още не доказва научени коалиции.** При $\alpha \approx 0$ коалиционният резултат на агентите с принос по Шапли надвишава този на агентите с рядка награда с $+0,038$ (5 начални числа, пълен мащаб), а при $\alpha \geq 0,3$ ефектът изчезва. Както е реализиран, приносът не показва кой на кого помага, а коалиционният резултат отчита и враждебните взаимодействия; нужна е контролна награда без коалиционна информация.
- **Голямата тежест на приноса намалява дела на победите.** $\alpha = 0$ понижава дела на победите до $\sim 0,29$, близо до нивото на случайната игра ($0,25$); при $\alpha \geq 0,1$ той се възстановява до $\sim 0,52$ - цената на плътния сигнал вече е измерена.

- **Празното ядро означава структурна нестабилност.** Като игра с постоянна сума SLS има празно ядро, както играта на мнозинството: няма стабилно разпределение и коалициите *ще* се разпадат. Затова „безопасната“ игра при N играчи не може да се закотви в равновесие на Наш или в ядрото; регуляризацията към поведенческа опорна стратегия (по подобие на $piKL$) е кандидат, а не гаранция (Принос №2).
- **Съставът на разлаганата популация определя дали се вижда стълба, или колело** (уро-
кът от Глава 10 се потвърждава): популацията „стълба на уменията“ е преобладаващо тран-
зитивна (циклично съотношение $\sim 0,3$), а коалиционната има силна циклична компонента
($\sim 0,57-0,69$), макар транзитивната да остава малко по-голяма.
- **Верността към правилата на двигателя е по-важна от кода на решавача.** Две от оста-
налите слабости (пристрастието към място 0 и цикличното съотношение под прага на доми-
ниране) се дължат на опростяванията на двигателя, а не на математиката на Шапли и EGTA,
която издържа точно учебниковите проверки; третата – приносът без коалиционна инфор-
мация (раздел 11.4) – е в начина, по който е дефиниран приносът.

12 Глава 12 – Модели на последователности и агенти с голям езиков модел в стратегически среди

12.1 Защо да се разглежда обучението с подкрепление като прогнозиране на последователност

Класическото обучение с подкрепление оптимизира, като оценява функцията на стойността, разпространява наградите назад (белманово обновяване) и подобрява стратегията. Decision Transformer (DT, „трансформър за вземане на решения“; Chen и др., 2021) предлага друг подход: разглежда целия проблем като **моделиране на последователности чрез обучение с учител**. На модел от типа GPT се подава траектория от тройки

$$(\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots)$$

където $\hat{R}_t = \sum_{t' \geq t} r_{t'}$ е *остатъчната възвръщаемост* (return-to-go), а моделът се обучава да предсказва a_t въз основа на всичко, случващо се до s_t . Не се използва оператор на Белман и не се извършва стъпка за подобряване на стратегията. При използването на модела той просто се **обулавя**: задава се желана висока остатъчна възвръщаемост и моделът генерира действията, които в данните са предшестващи такива възвръщаемости.

За дисертацията подходът е привлекателен, защото работи изцяло **офлайн** – върху фиксиран набор от записани данни, какъвто е случаят с историите на реални игри, при които самообучението не е възможно.¹⁵⁵

12.2 Капанът между късмета и умението

Преформулирането прикрива едно предположение: че остатъчната възвръщаемост е нещо, което агентът *е заслужил*. В стохастична среда това не е вярно. Paster и др. (2022) показват, че стратегиите, обусловени от възвръщаемостта, систематично преследват късмета, а най-простият пример го прави очевидно.

Разгледайте едностъпкова задача с два възможни избора (двурък бандит). Действие А изплаща 0,5 детерминистично, а действие В изплаща 1,0 с вероятност 0,4 и 0 в останалите случаи, така че очакваната му стойност е $\mathbb{E}[B] = 0,4 < 0,5$. Действие А е оптимално по очаквана стойност. Единственият начин обаче една траектория да постигне възвръщаемост от 1,0 е чрез избор на действие В и получаване на късмет. Ако обусловим по „възвръщаемост = 1,0“, ще получим единствено траектории с действие В:

$$P(a = B \mid \hat{R} = 1,0) = 1,00 \quad (\text{измерено в собствена симулация})$$

¹⁵⁵Chen et al., *Decision Transformer: Reinforcement Learning via Sequence Modeling*, NeurIPS 2021 (arXiv:2106.01345).

Обучаваният модел, обусловен по възвръщаемостта, уверено избира действието с по-ниска очаквана стойност. Това не е грешка в модела, а самата същност на обуславянето по изхода, когато изходът отчасти зависи от случайността.¹⁵⁶

12.3 ARDT – обуславяне по гарантираната възвръщаемост

Решението е да се обуславя не по възвръщаемостта, която *се е случила*, а по възвръщаемостта, която агентът може да си гарантира срещу противник в най-лошия случай – минимаксната остатъчна възвръщаемост. ARDT (Tang и др., 2024) я оценява чрез **регресия с експектили**, която минимизира асиметричната квадратична загуба

$$L_{\text{ER}}^{\alpha}(u) = \mathbb{E}_u[|\alpha - \mathbf{1}(u > 0)| \cdot u^2]$$

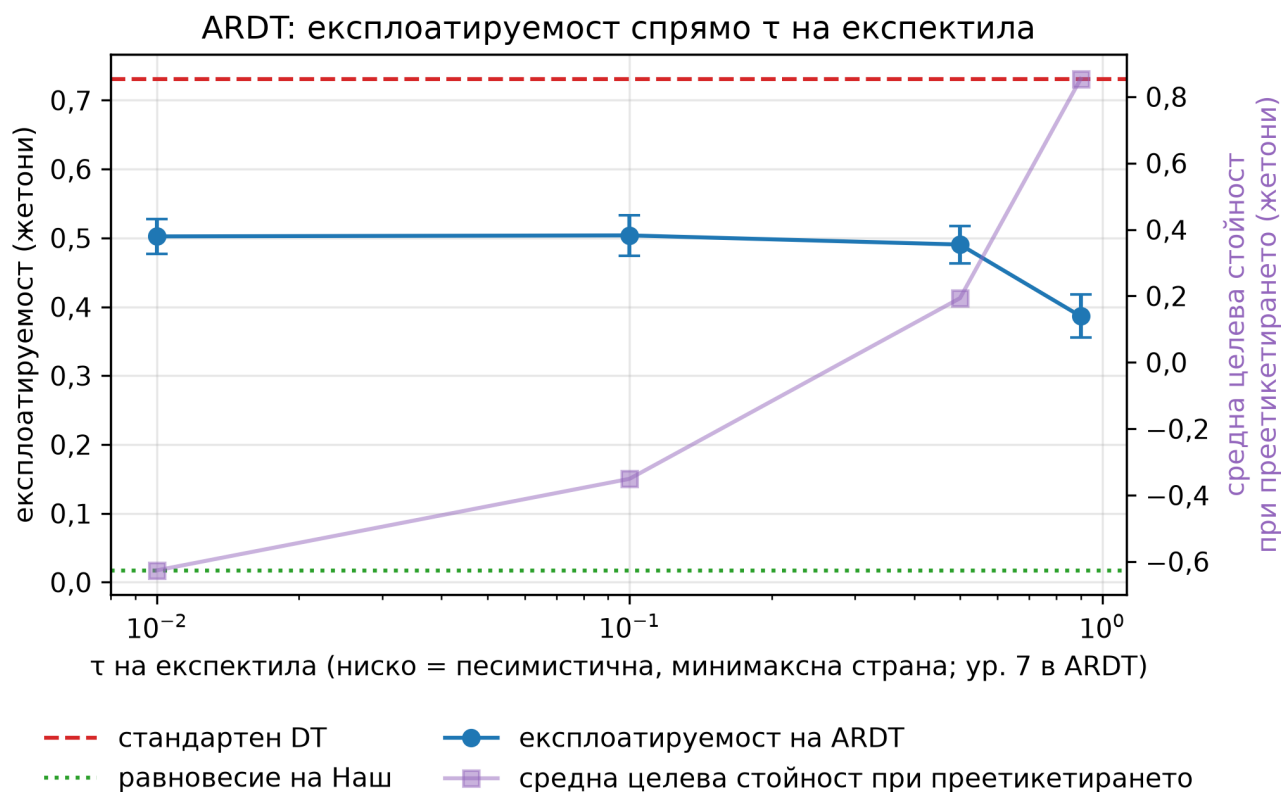
чиито граници дават необходимите оператори: $\alpha \rightarrow 0$ възстановява **минимума**, а $\alpha \rightarrow 1$ - **максимума**. Траекториите се преетикетират с оценената минимаксна възвръщаемост и върху тях се обучава стандартен DT.

Два детайла от публикувания метод са съществени; и двата са проверени в самата статия, а не само в резюмето ѝ. Първо, песимистичната страна съответства на **ниско** α (в първоначалния план на главата $\tau = 0,9$ погрешно е описано като песимистично); самата статия използва $\alpha = 0,01$. Второ – и по-важно – целевата стойност при преетикетирането е

$$\tilde{R}_t = \tilde{Q}_{\nu}(s_t, a_t)$$

стойност на двойката **състояние-действие**, получена от два *свързани* оценителя, които се напасват редуващо се със загуби α и $1 - \alpha$. Стойност на състояние $V(s)$ не може да различи „това състояние е лошо“ от „това действие в това състояние е лошо“, а точно на това разграничение се основава методът.

¹⁵⁶Paster, McIlraith & Ba, *You Can't Count on Luck: Why Decision Transformers and RvS Fail in Stochastic Environments*, NeurIPS 2022 (arXiv:2205.15967).



Фигура 70: Експлоатируемост на ARDT в зависимост от параметъра τ на експектилната регресия; по втората ос – средната целева стойност при преетикетирането. Средни стойности от три начални числа (за сравнение: стандартен DT – 0,731, равновесие – 0,017 жетона).

Измерена върху **Кун** с умишлено опростен заместител, който отчита само състоянието, целевата стойност при преетикетирането се изменя монотонно с τ , както изисква теорията – но експлоатируемостта е *най-ниска* от оптимистичната страна. Най-вероятната причина е, че действието липсва като аргумент на оценката; вариант с $\tilde{Q}(s, a)$ още не е изпълнен, така че резултатът нито опровергава, нито потвърждава ARDT.¹⁵⁷

12.4 Какво всъщност прави обуславянето по възвръщаемостта в покера

Равновесието и експлоатируемостта на **Кун покер** могат да бъдат изчислени точно (вж. глава 2),¹⁵⁸ поради което въпросът може да бъде решен, а не само обсъждан. Резултатът е, че обуславянето по възвръщаемостта **променя** стратегията съществено, но не я **насочва**: експлоатируемостта остава почти една и съща в целия диапазон на реално постижимите целеви стойности, с изключение на рязък скок (до $\approx 1,9$ жетона) при една конкретна стойност – най-честата печалба, която в Кун е

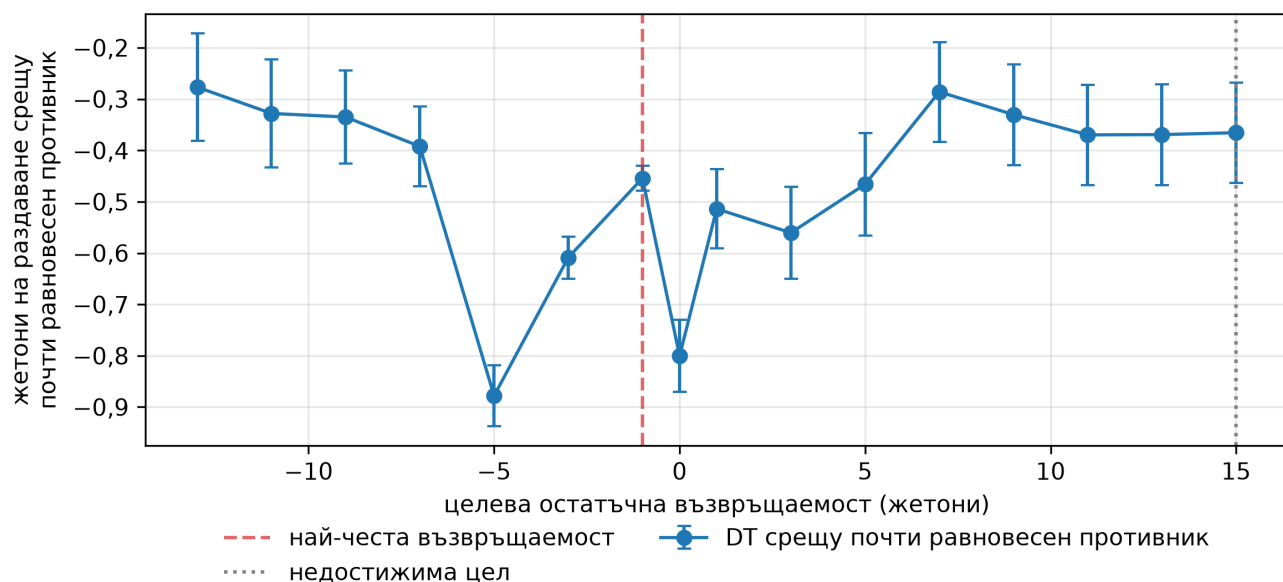
¹⁵⁷Tang, X., Marques, A., Kamalaruban, P. & Bogunovic, I., *Adversarially Robust Decision Transformer*, NeurIPS 2024 (arXiv:2407.18414) – Section 3 и Algorithm 1.

¹⁵⁸Kuhn, H. W. (1950). “A Simplified Two-Person Poker.” In H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the Theory of Games*, Vol. I (Annals of Mathematics Studies 24), pp. 97–103. Princeton University Press. DOI 10.1515/9781400881727-010.

печалбата при пас.

На същите данни обикновеното поведенческо клониране достига експлоатируемост 0,055 жетона – близо до равновесието (0,016) – срещу 0,799 за DT: обуславянето по възвръщаемостта активно унищожава информация (стойността за DT варира между повторни обучения, 0,67–0,80).

Естественото обяснение е, че в игра само с четири възможни стойности на печалбата *големината* на възвръщаемостта кодира коя последователност от залози е изиграна, а *знакът ѝ* – кой е държал по-добрата карта. Следователно обуславянето избира хода на раздаването, а не качеството на играта.



Фигура 71: Резултат на DT в Ледюк Холдем (жетони на раздаване срещу почти равновесен противник) според целевата възвръщаемост, по която е обусловен, със стандартни грешки (Пирсън $r = +0,062$). Прекъсната линия: най-честата възвръщаемост (-1); точкова: недостижима цел (+15).

Ледюк Холдем тества това обяснение с петнадесет стойности на печалбата вместо четири, два рунда на залагане и обща карта. **Спадът при най-честата възвръщаемост изчезва, както предвижда обяснението, но насочване пак не се наблюдава** (Пирсън $r = +0,062$); остават обаче два необяснени спада – при 0 (втората по честота възвръщаемост) и при -5. Следователно обяснението чрез малкия брой възможни печалби обяснява срива, но не и провала. Провалът е по-фундаментален: в игра с нулева сума и непълна информация реализираната възвръщаемост се определя основно от тайната карта и действията на опонента, които агентът не контролира. Колкото и да са възможните стойности на печалбата, сигнал, който агентът не контролира, не става управляем.

Това е основният извод на главата: при обучение върху записани партии DT, обусловен по възвръщаемостта, е неподходящ инструмент, а предимството на преетикетирането при ARDT е, че

заменя целева стойност, която агентът не контролира, с такава, която зависи главно от собствените му действия.

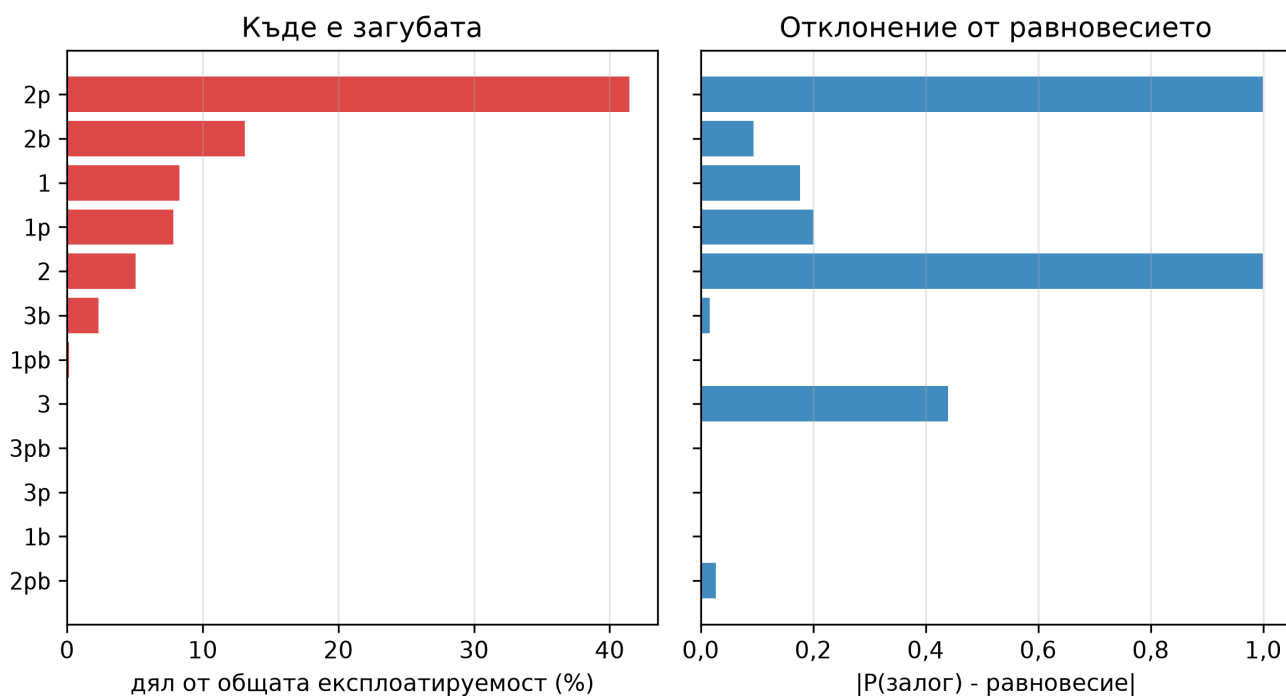
12.5 Езикови модели като стратегически агенти

Втората парадигма изцяло пропуска обучението: опишете правилата на английски и оставете езиков модел да играе. Тествани са три модела с отворени тегла, изпълнявани локално (LM Studio 0.4.20 на RTX 5090, температура 0,7; квантизацията на теглата не е записана): gpt-oss-20b, Qwen2.5-7B-Instruct и OpenThinker3-7B (Qwen2.5-7B-Instruct, дообучен за разсъждения). Ако не е посочено друго, резултатите по-долу са за Qwen2.5-7B-Instruct; всички числа са от собствени измервания. Сходни експерименти в Кун, Ледюк и лимитен холдем показват, че и по-големи модели не се доближават до равновесна игра, а действията им често се разминават със заявените разсъждения¹⁵⁹; подкана с изрично разсъждение за противника позволява на GPT-4 да адаптира играта си.¹⁶⁰ Измерени със същия точен критерий, големите езикови модели се оказват **действително експлоатируеми**, но по-интересно е *къде* грешат.

Те разбират правилно **ранга** на ръцете, но грешат в **честотите** на смесените стратегии. Всеки от тестваните реални модели залага за стойност с поп с вероятност 1,00, докато изчисленото с CFR равновесие смесва с 0,68, и при обикновена подкана нито един не блъфира с вале в извадка от 24 опита на информационно множество. (В Кун равновесието на първия играч образуват еднопараметрично семейство – блъф с вале с вероятност $\alpha \in [0, 1/3]$ и залог с поп с вероятност 3α ,¹⁵⁸ така че поотделно и двете поведения са равновесни; неравновесна е само комбинацията им.) Модел със 7 млрд. параметъра (Qwen2.5-7B) се представя наравно с gpt-oss-20b – модел със смес от експерти с 21 млрд. параметъра общо, но само 3,6 млрд. активни при всеки токен; при три модела и една опростена игра това не позволява извод за ролята на мащаба.

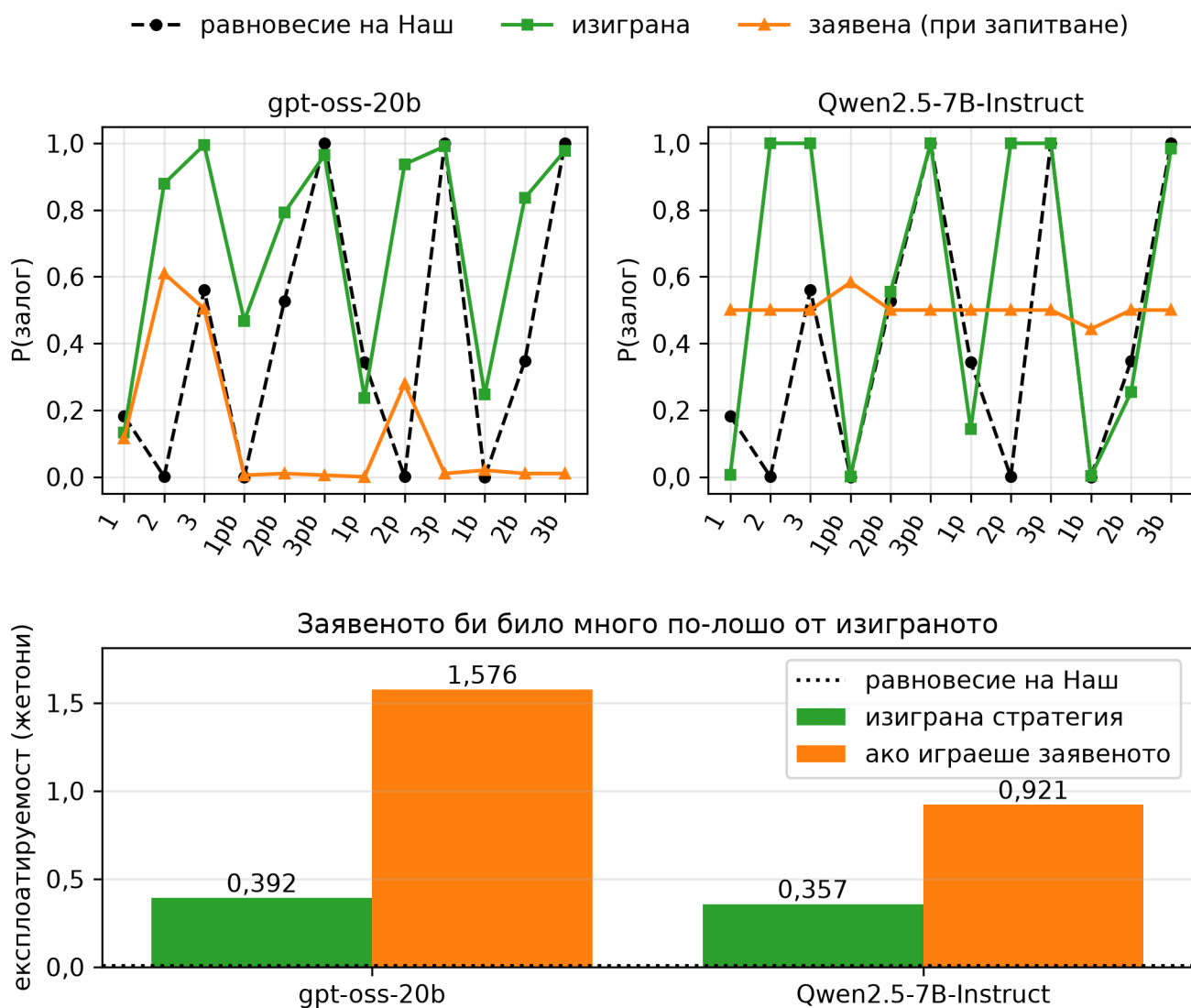
¹⁵⁹Lin, M., Dai, E., Liu, H. et al., *How Far Are LLMs from Professional Poker Players? Revisiting Game-Theoretic Reasoning with Agentic Tool Use*, ICLR 2026 (arXiv:2602.00528).

¹⁶⁰Guo, J., Yang, B., Yoo, P., Lin, B. Y., Iwasawa, Y. & Matsuo, Y., *Suspicion-Agent: Playing Imperfect Information Games with Theory of Mind Aware GPT-4*, 2023 (arXiv:2309.17277).



Фигура 72: Дял на всяко информационно множество в общата експлоатируемост на Qwen2.5-7B-Instruct в Кун покер (0,357 жетона) и отклонението му от равновесието на Наш (1/2/3 – вале/дама/поп; p – пас, b – залог в историята). Двете подредби почти не си съответстват.

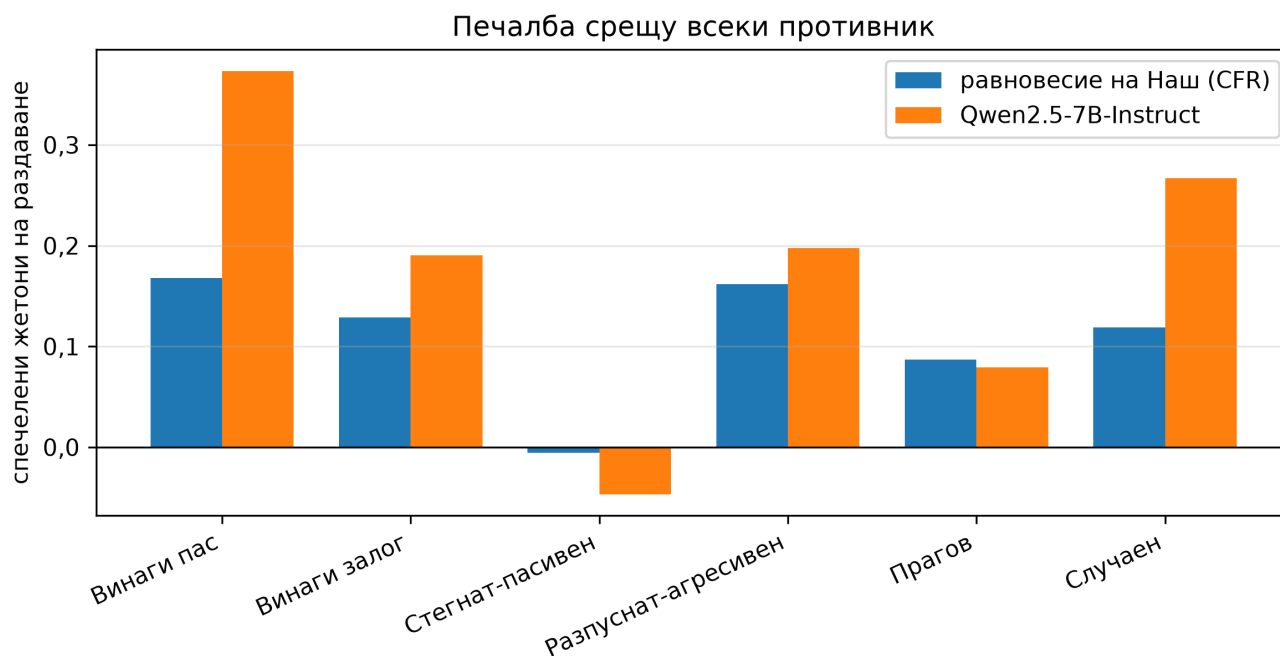
Разлагането на загубата по решение обръща очевидното тълкуване. „Провалът“ с поп – най-голямото видимо отклонение от равновесието – носи само **0,1%** от общата загуба, защото прекомерното залагане с ръка, която никога не е по-слаба, почти не струва нищо. Едно-единствено решение с дама струва **41,4%**. Големината на отклонението и цената му почти не са корелирани – конкретен аргумент при оценяването да се отчита диагностика по отделни решения, а не само едно число за експлоатируемост.



Фигура 73: Заявени (при запитване) и реално изиграни честоти на залагане на gpt-oss-20b и Qwen2.5-7B-Instruct по информационни множества, с равновесието за сравнение (горе; средно абсолютно отклонение от равновесието, заявени срещу изиграни: 0,43 срещу 0,33 и 0,35 срещу 0,25), и експлоатируемост на изиграната стратегия и на стратегия, построена от заявените честоти (долу).

Питането на модел какво възнамерява е *по-лошо* от измерването на това, което прави. Оценени като стратегии, заявените от тези модели честоти са 2,6 и 4,0 пъти по-експлоатируеми в сравнение със стратегиите, които те реално играят. Един модел отговаря „50%“ почти навсякъде, а другият – „почти 0%“ почти навсякъде, включително и когато държи най-силната ръка. Поведенческото проучване тук не е просто по-удобно от самоанализа – то е по-точно (собствено измерване).

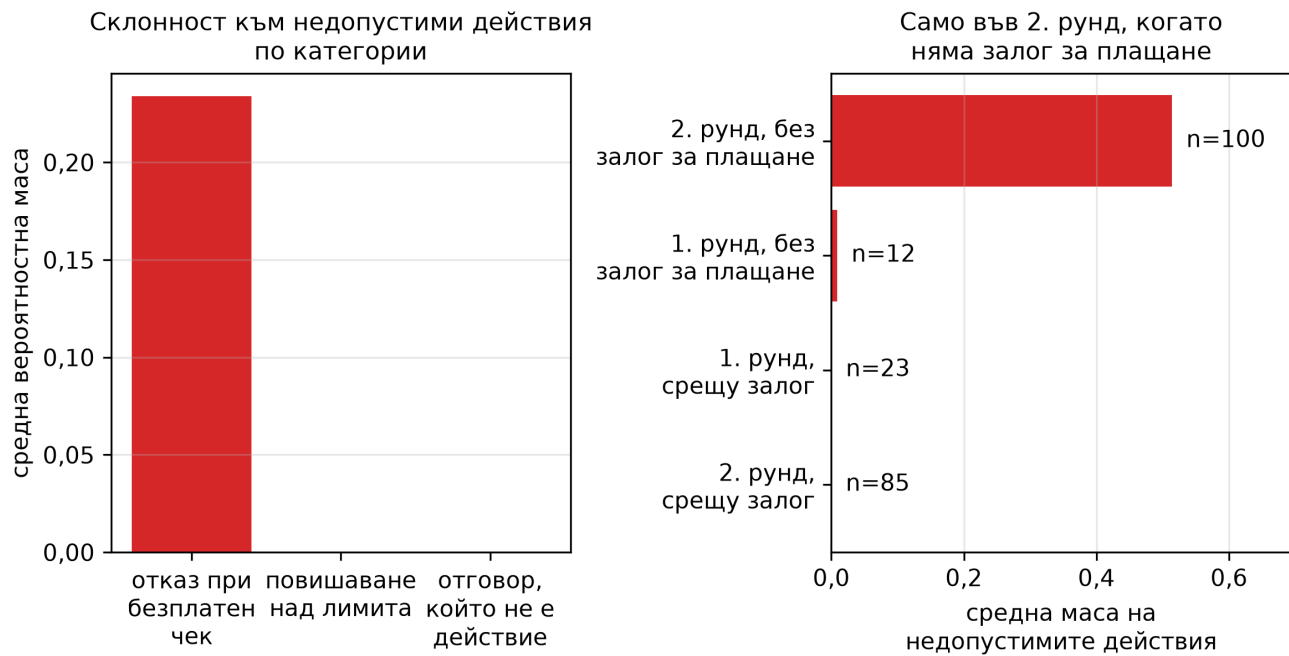
12.6 Експлоатация, адаптация и ограничения на опростената игра



Фигура 74: Печалба срещу всеки противник от набора: Qwen2.5-7B-Instruct печели средно 0,177 жетона на раздаване (равновесието – 0,110) при експлоатируемост 0,357 жетона (равновесието – 0,006).

Мерките за експлоатируемост показват доколко съвършен противник ви наказва; те не казват нищо за това колко добре вие сами наказвате слабостите. Измерено спрямо набор от умишлено експлоатируеми архетипи, езиковият модел (Qwen2.5-7B-Instruct) **печели средно 0,177 жетона на раздаване срещу 0,110 за равновесната игра (с 61% повече), но експлоатируемостта му е 0,357 жетона срещу 0,006** – и близо 90% от предимството идва от пасивния и случайния противник. Срещу двата най-компетентни архетипа той се представя *по-зле* от равновесието. Това е компромисът между безопасност и експлоатация, с който се занимава вторият принос на дисертацията – тук наблюдаван, а не предположен.

Когато историята на сесията (последните 20 раздавания) е подадена в контекста, моделът (Qwen2.5-7B-Instruct) **не** се адаптира: срещу тривиално пасивен противник той улавя 83% от наличната експлоатация още в първата половина на сесията и не се подобрява във втората (0,824 срещу 0,827); при другите два противника резултатът не е статистически надежден. Това, което изглежда като моделиране на противника, всъщност е фиксирано априорно убеждение, благоприятстващо разпуснатото-агресивна игра. Когато обаче типът на противника е *описан* в подканата, моделите променят честотата на блъфиране с до 0,92 – липсва изводът от историята, а не способността да се реагира на описание.



Фигура 75: Склонност към недопустими действия в Ледюк Холдем (Qwen2.5-7B-Instruct) по категории и по ситуации (средно 0,234 от вероятностната маса в 220 информационни множества): почти цялата маса е отказ от ръката, когато чекът е безплатен, във втория рунд на залагане.

Накрая опростената игра ласкае тези модели. С един рунд на залагане и една обща карта повече големият езиков модел вече е статистически неразличим от DT, губи средно срещу набора от слаби противници (печели само срещу двата пасивни типа) и насочва приблизително една четвърт от вероятностната си маса към недопустими действия. Този провал се дължи на *едно* погрешно схващане, а не на обща обърканост: моделът иска **да се откаже от ръката, когато може безплатно да направи чек**, почти изключително със слаби ръце без чифт при висока обща карта, и нито веднъж не се опитва да повиши над лимита за залагане. Всеки оптимизъм, произтичащ от опростената игра, трябва да се възприема като свойство на самата опростена игра.

12.7 Основни изводи за синтеза на дисертацията

- **Обулавянето по величина, която агентът контролира само отчасти, не насочва поведението му.** Наблюдавано е в две опростени игри и съответства на теорема 2.1 на Paster и др.; затова за записани покер партии по-обещаващо е преетикетирането на ARDT, а не самият DT – при условие че се отчетат и случайните ходове (раздаването на картите), при които минимаксната цел на ARDT вече не е точна.¹⁶¹
- **Публикуваният ARDT преетикетира със стойност на двойката състояние-действие.**

¹⁶¹Tang, X., Cheng, Z. & Kumar, S., *Robust Adversarial Reinforcement Learning in Stochastic Games via Sequence Modeling*, Reliable ML Workshop @ NeurIPS 2025 (arXiv:2510.11877) – метод CART; отбелязва, че минимаксната цел на ARDT е точна само при детерминирани преходи.

Възпроизвеждането на ползата от метода най-вероятно изисква $\tilde{Q}(s, a)$, а не $V(s)$ - това е следващата стъпка, която още предстои да се провери.

- **Скаларна оценка класира методите, но не ги диагностицира.** Едно решение носи 41,4% от загубата, докато най-забележимото отклонение носи 0,1%.
- **Измервайте поведението, а не самоотчета.** Вербализираната стратегия е съществено по-лоша от изиграната стратегия за всеки тестван модел.
- **Моделът не изведе противника от историята в контекста** (един модел със 7 млрд. параметъра, прозорец от 20 раздавания), но реагира, когато типът на противника му е съобщен; затова моделът на противника трябва да се изгради изрично и резултатът му да се подава на стратегията – пряко свързано с първия принос на дисертацията.
- **Протоколът за измерване е част от резултата.** При алчно декодиране (температура 0) езиковият модел играе чиста стратегия и всяка измерена честота дегенерира до 0 или 1; четири отделни открития в тази глава се оказаха измервателни артефакти, всеки от които беше открит чрез евтина проверка спрямо точно изчисливо количество.